النظرية الإحتسابية والأتمتة

Med material

إعداد الطالبة:

رغد تايه

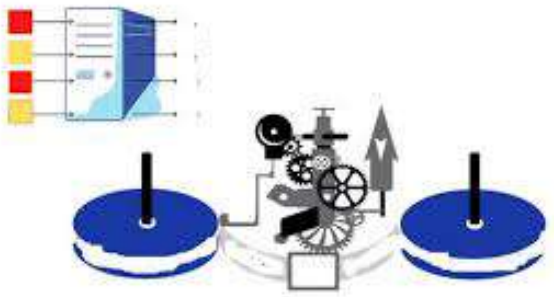
فريق داتا ستريم

Theory of Computation and Automata

Prince Abdullah bin Ghazi Faculty of Information and Communication Technology

Computer Science Department

Dr. Nabeel Al-Milli



Content

Mathematical Preliminaries

Sets

Functions

Relations

Graphs

Proof Techniques

Introduction

الحساب ←

Computation: is simply a **sequence of steps** that performed **by computer.** → سلسلة من الخطوات التي يجريها الحاسوب

الحساب نظرية فرع

Computation Theory: is the branch that deals with how **efficiently problems can be solved** on a model of computation, **using an algorithm.**

هو الفرع الذي يبحث بدراسة مدى كفاءة حل المشكلات باستخدام نموذج حسابي باستخدام خوارزمية معينة

R.
Al-
Milli

Applications of Computation Theory:

1. Design and Analysis of Algorithms →
2. Computational Complexity
3. Logic in Computer Science
4. Compiler
5. Cryptography →
6. Randomness in Computation →
7. Quantum Computation (**New Trend**)

R.
Al-
Milli

SETS → المجموعات

A set is a collection of elements → هي مجموعة من العناصر

من شروط تكون مجموعة

We write

R.
Al-
Milli

Set Representations

$C = \{ a, b, c, d, e, f, g, h, i, j, k \}$ → curly braces

$C = \{ a, b, \ldots, k \}$ ⟶ *finite set* → يعرفا نهايتها أو آخر عنصر فيها أو عدد العناصر التي فيها

$S = \{ 2, 4, 6, \ldots \}$ ⟶ *infinite set* → غير محدودة أو متنهية → ما يعرفا آخر عنصر فيها أو عدد العناصر التي بتحويها

أعداد موجبة — even

$S = \{ j : j > 0, \text{ and } j = 2k \text{ for some } k>0 \}$

$S = \{ j : j \text{ is nonnegative and even} \}$

R.
Al-
Milli

Basic Operations on Sets:

- **Complement:** $\bar{A}$ or $\overline{A}$ or A^c

  $\overline{A} = \{ x : x \notin A, x \in U \}$

  Contain all elements in universal set which are not in A.

- **Union:** consist of all elements in either A or B

  $A \cup B = \{ x : x \in A \text{ or } x \in B \}$

- **Intersection:** consist of all elements in both A or B

  $A \cap B = \{ x : x \in A \text{ and } x \in B \}$

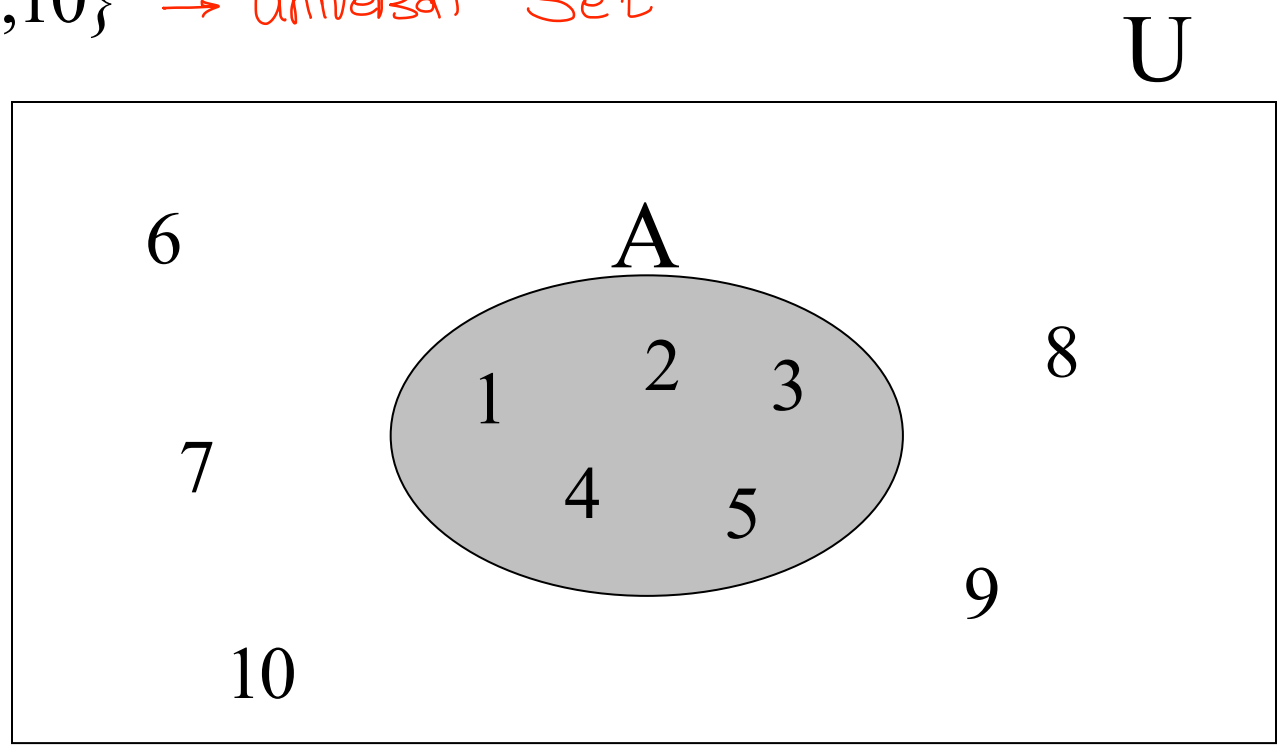
- **Difference (/):** consist of all elements in A but not in B

  $A / B = \{ x : x \in A \text{ but } x \notin B \}$

R.
Al-
Milli

$A = \{ 1, 2, 3, 4, 5 \}$

$U = \{1, 2, \ldots, 10\}$ → Universal set



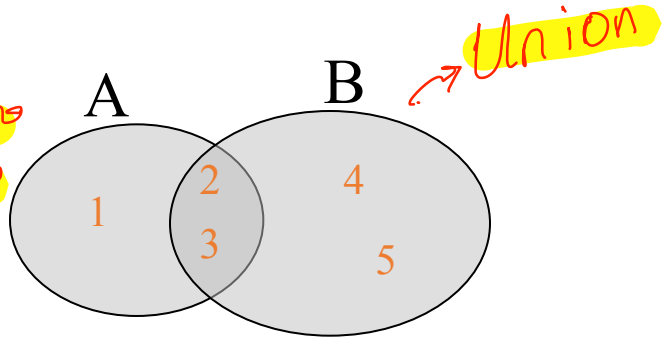
R.
Al-
Milli

Set Operations

$A = \{ 1, 2, 3 \}\ B = \{ 2, 3, 4, 5 \}$

- Union

$A \cup B = \{ 1, 2, 3, 4, 5 \}$ → 2, 3 → مكررين بكتبهم مرة وحدة



- Intersection

$A \cap B = \{ 2, 3 \}$

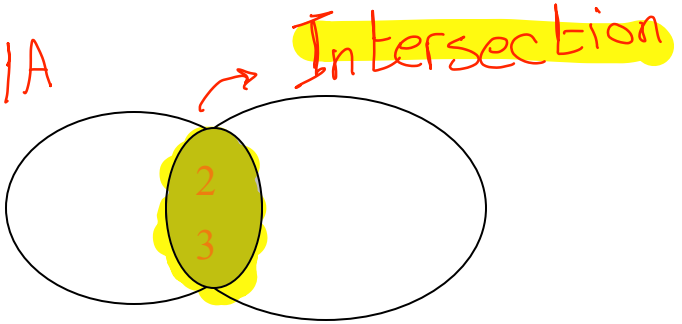
- Difference → العملية مش تبادلية

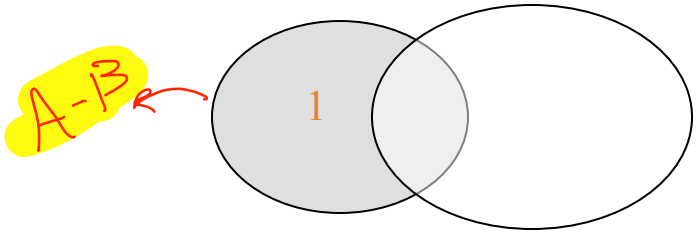
↳ $A - B \neq B - A$

$A - B = \{ 1 \}$

$B - A = \{ 4, 5 \}$

العناصر الموجودة في B ومش موجودة في A



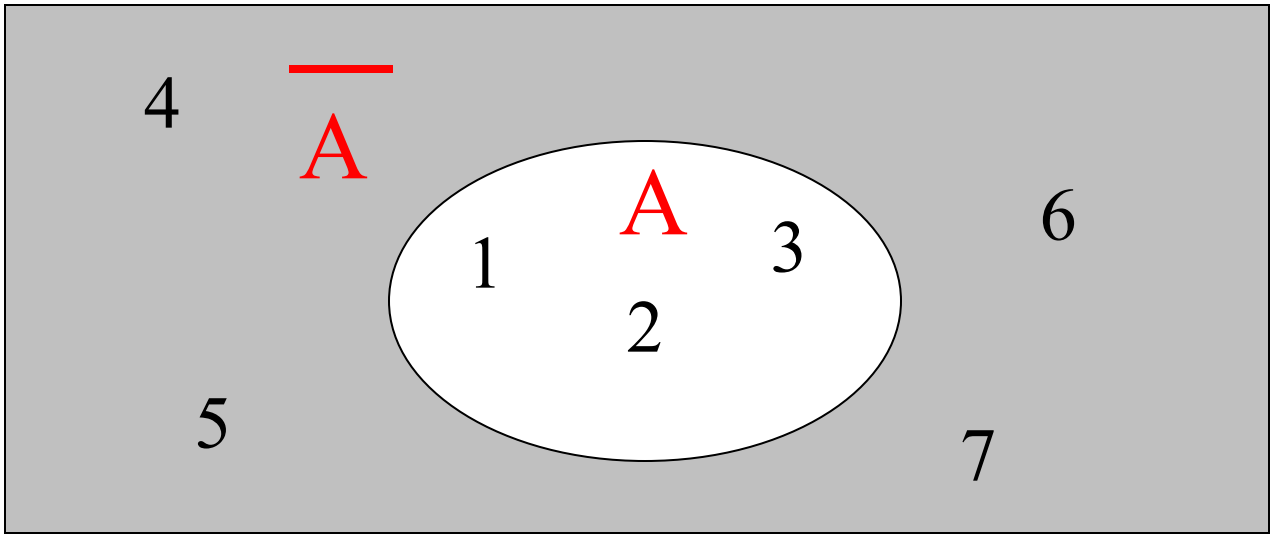


Venn diagrams

- Complement → المكملة

Universal set = {1, …, 7} ← المجموعة الكبيرة [الرئيسية]

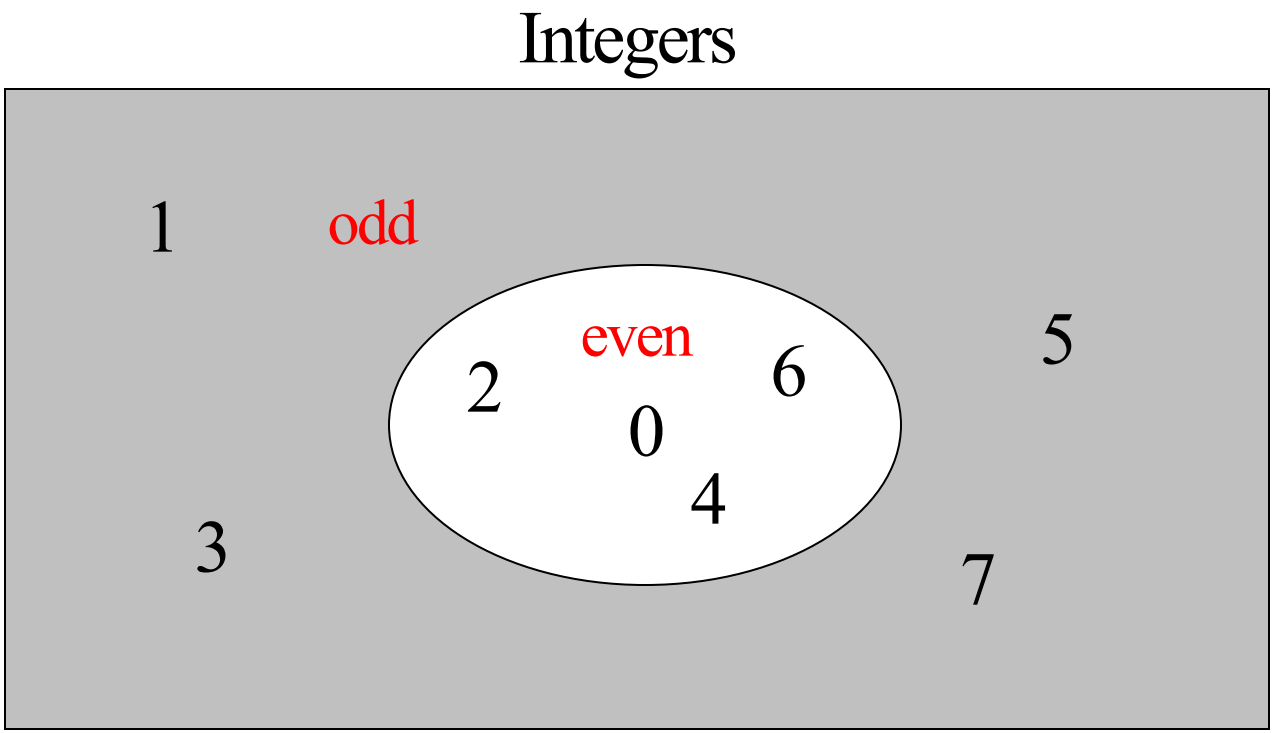
$A = \{1, 2, 3\}$ $\bar{A} = \{4, 5, 6, 7\}$



$A = A$ ← وكل شيء الشيء يساوي نفسه ← يرجع للمجموعة الأصلية

$$\overline{\{\text{ even integers }\}} = \{\text{ odd integers }\}$$

عكسها



R.
Al-
Milli

Properties of Sets:

Let A, B, and C be subsets of the universal set U.

- **Distributive properties**

$$A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$$
$$A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$$

- **Idempotent properties**

$$A \cap A = A.$$
$$A \cup A = A.$$

- **Double Complement property**

$$(A^{\sim})^{\sim} = A.$$

- **De Morgan's laws**

$$(A \cup B)^{\sim} = A^{\sim} \cap B^{\sim}$$
$$(A \cap B)^{\sim} = A^{\sim} \cup B^{\sim}$$

- **Commutative properties**

$$A \cap B = B \cap A.$$
$$A \cup B = B \cup A.$$

- **Associative laws**

$$A \cap (B \cap C) = (A \cap B) \cap C$$
$$A \cup (B \cup C) = (A \cup B) \cup C$$

يرجع ويوزع بالآخر

- **Identity properties**

$$A \cup \emptyset = A.$$
$$A \cap U = A.$$

$\emptyset$ → Empty set → مجموعة فاضية بدون عناصر

- **Complement properties**

complement

$$A \cup A^{\sim} = U.$$
$$A \cap A^{\sim} = \emptyset.$$

R.
Al-
Milli

Empty, Null Set: $\emptyset$

$\emptyset = \{\}$

$S \cup \emptyset$ → $S \cup \emptyset = S$

$S \cap \emptyset$ $\emptyset$ → $S \cap \emptyset = \emptyset$

$S - \emptyset$ → $S - \emptyset = S$

$\emptyset - S =$ $\emptyset$ → $\emptyset - S = \emptyset$

$\overline{\emptyset}$ = Universal Set

U

R.
Al-
Milli

Subset

$A = \{1, 2, 3\}\ B = \{1, 2, 3, 4, 5\}$

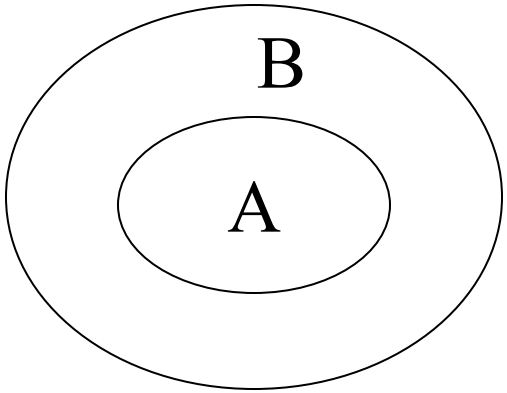
كل عناصر A موجودين في B

$A \subseteq B$ → $A \subseteq B$

A هنا تعني إنه مجموعة جزئية من B

NOTE → B → هي المجموعة الأكبر

Proper Subset: $A \subset B$ → $A \subset B$



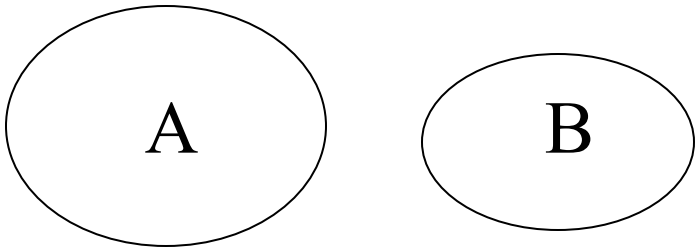
R.
Al-
Milli

Disjoint Sets

$A = \{ 1, 2, 3 \}$ $B = \{ 5, 6 \}$

$A \cap B = \emptyset$

يعني ما فيه اي شيء مشترك بين A و B



R.
Al-
Milli

Set Cardinality

- For finite sets

$$A = \{ 2, 5, 7 \}$$

$$|A| = 3$$

(set size)

R.
Al-
Milli

Powersets

A powerset is a set of sets → يعني كم مجموعة بتقدر اعمل من المجموعة الأصلية

$S = \{ a, b, c \}$

Powerset of S = the set of all the subsets of S

$2S = \{ \emptyset, \{a\}, \{b\}, \{c\}, \{a, b\}, \{a, c\}, \{b, c\}, \{a, b, c\} \}$

Observation: $|2S| = 2^{|S|}$ $(8 = 2^3)$

$2^{|S|} \rightarrow |S| \rightarrow$ set size

$2^S \rightarrow S = s \rightarrow 2^3 = 8$ تعديل

→ عدد المجموعات الي ممكن اعملها من المجموعة الاصلية

R.
Al-
Milli

Cartesian Product

$A = \{2, 4\}$ $B = \{2, 3, 5\}$

$A \times B = \{ (2, 2), (2, 3), (2, 5), (4, 2), (4, 3), (4, 5) \}$

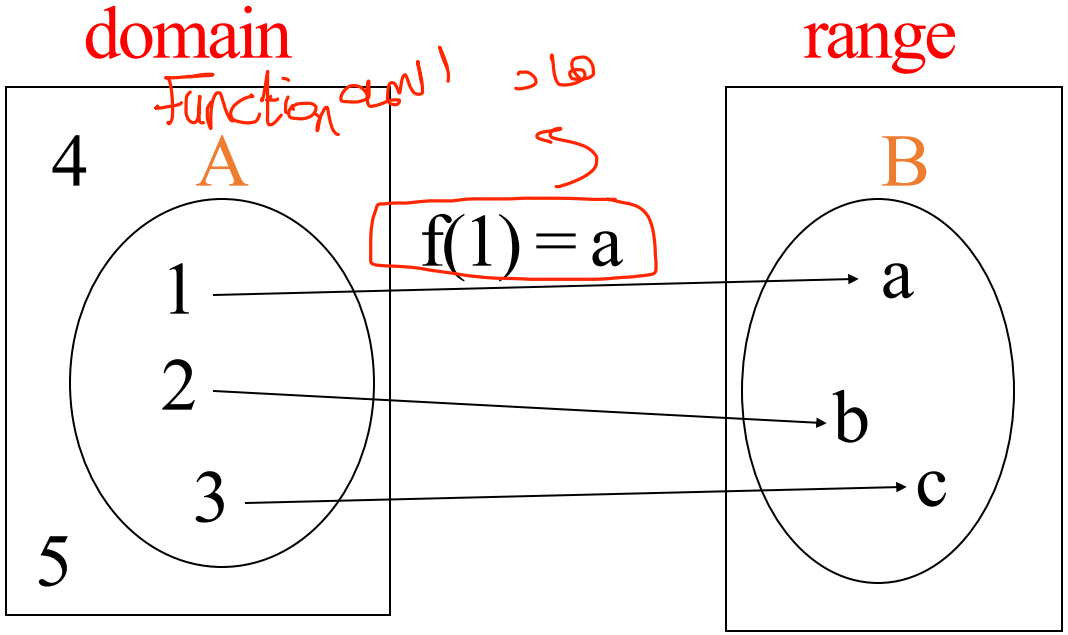
$|A \times B| = |A| \, |B|$

Generalizes to more than two sets

$A \times B \times \ldots \times Z$

R.
Al-
Milli

FUNCTIONS



f : A -> B

* لو عوضت قيمة الـ (1) الموجودة بالـ domain

لشو رح تكون صورتك او القيمة ← (a)

R.
Al-
Milli

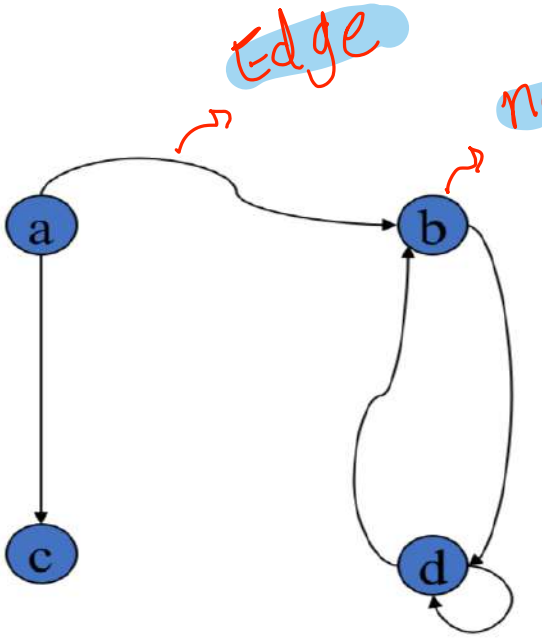
Relations and functions

A k-ary relation R on $A1, \ldots, Ak$ is a subset of $A1 \times A2 \times \ldots \times Ak$

A **binary relation** on A is a subset of $A \times A$

Example of a Relation

Example of a Relation



$$R = \{(a, b), (a, c), (b, d), (d, b), (d, d)\}$$

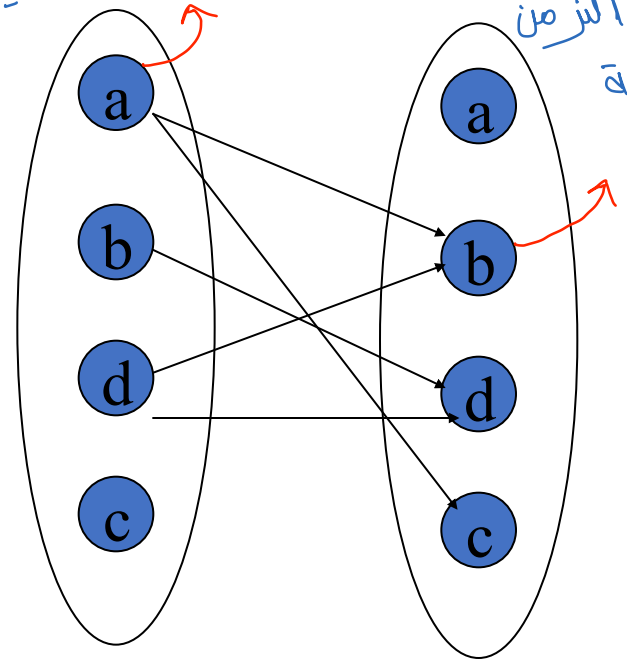
R.
Al-
Milli

Example of a relation

Many to Many Relation

مرتبطة مع أكثر من node بالجهة الثانية

مرتبطة مع أكثر من node بالجهة الأولى



$$R = \{(a, b), (a, c), (b, d), (d, b), (d, d)\}$$

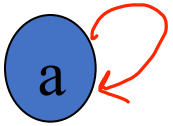
* كل node مرتبطة مع من هو بالجهة الأولى بينهم علاقة أو لا

R.
Al-
Milli

Properties of binary relations

Reflexive

(a,a) ∈ R for each a ∈ A



R.
Al-
Milli

Properties of binary relations

في كل الـ design لازم حالة وحدة على الأقل تكون Symmetric عشان السمي Symmetric

Note

Symmetric

If (a,b) ∈ R then (b,a) ∈ R

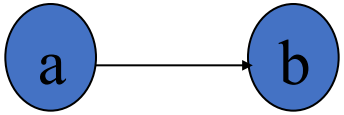
+ لازم من a ا b و لازم واجد من b ا a بكون Symmetric

Properties of binary relations

مش ضروري
كل الـ node
Transitive يبنهم علاقة

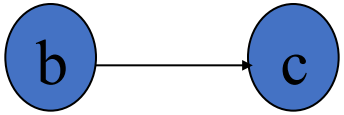
Transitive

If (a,b) ∈ R and (b,c) ∈ R then (a,c) ∈ R

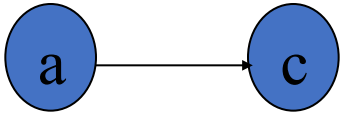


If

And



Then



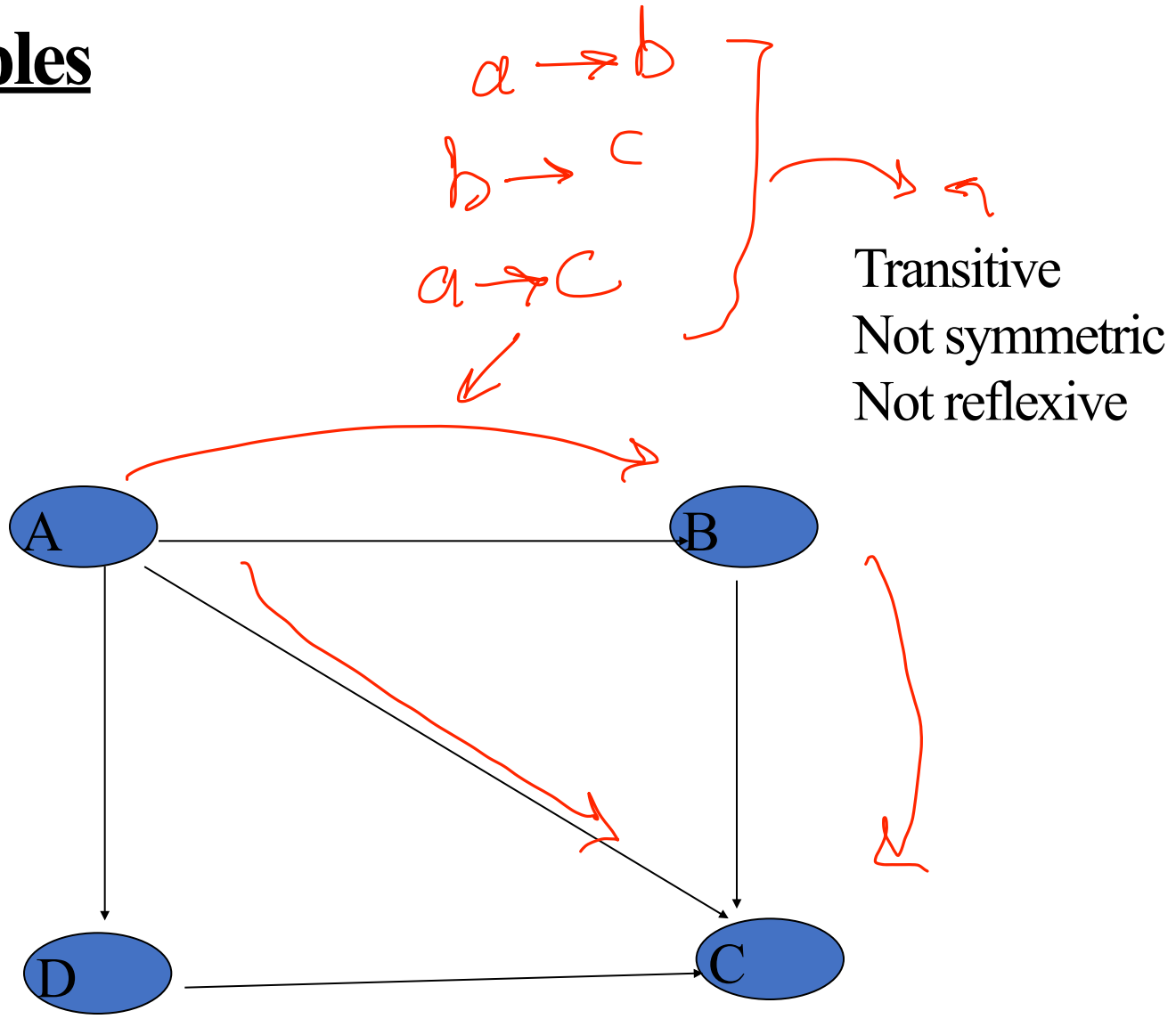
إذا
ب هو إليها علاقة مع أ
و أ إليها علاقة مع ج
متتالية
ب مع تكون a مع ج
=

Equivalence relation

An **equivalence relation** is a relation that is

- Reflexive
- Symmetric
- Transitive

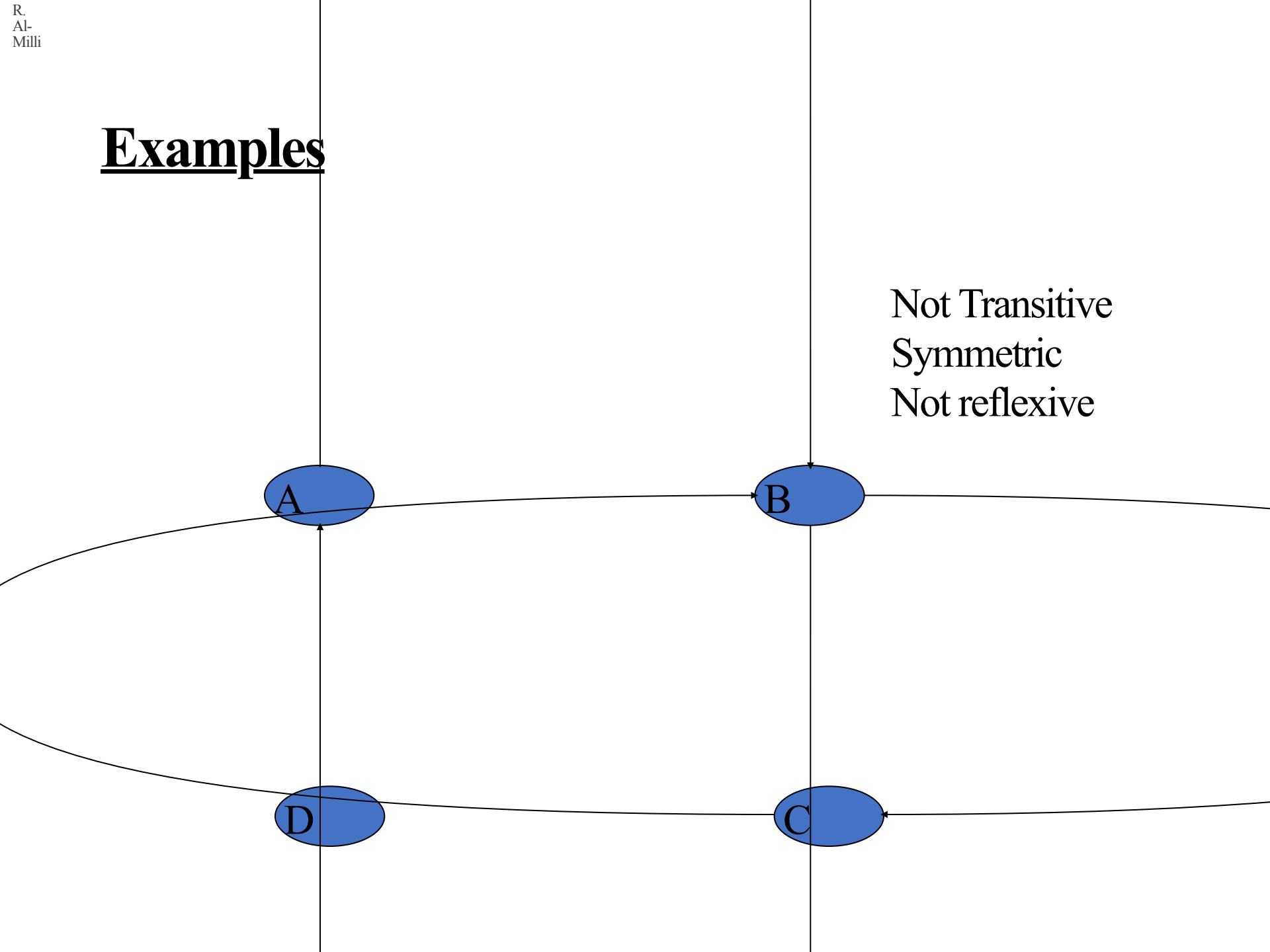
Examples



R.
Al-
Milli

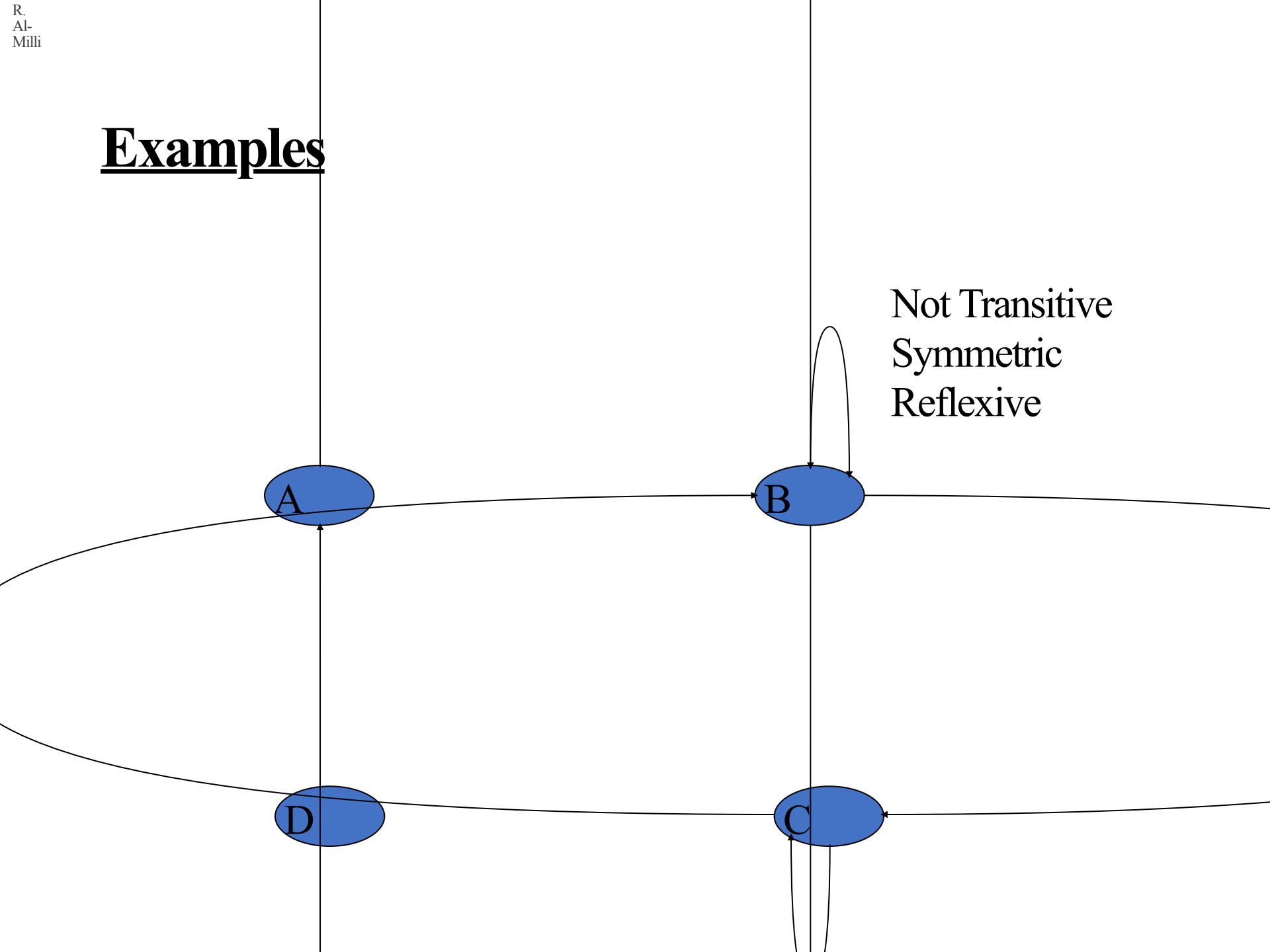
Examples

Not Transitive
Symmetric
Not reflexive



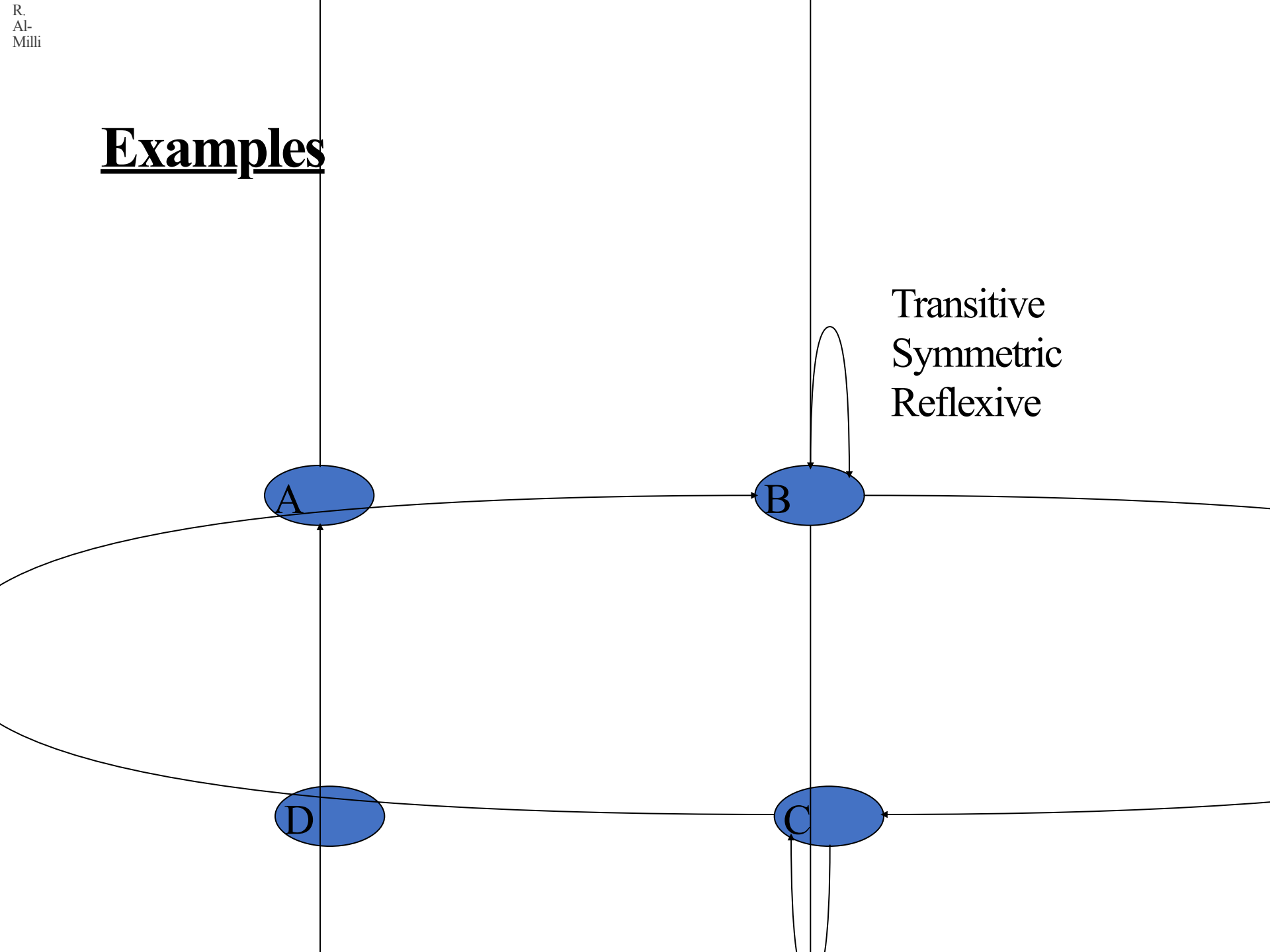
R.
Al-
Milli

Examples



R.
Al-
Milli

Examples



R.
Al-
Milli

Equivalence Classes

For equivalence relation R
equivalence class of $x = \{y : x \, R \, y\}$

Example:

$R = \{ (1, 1), (2, 2), (1, 2), (2, 1), (3, 3), (4, 4), (3, 4), (4, 3) \}$

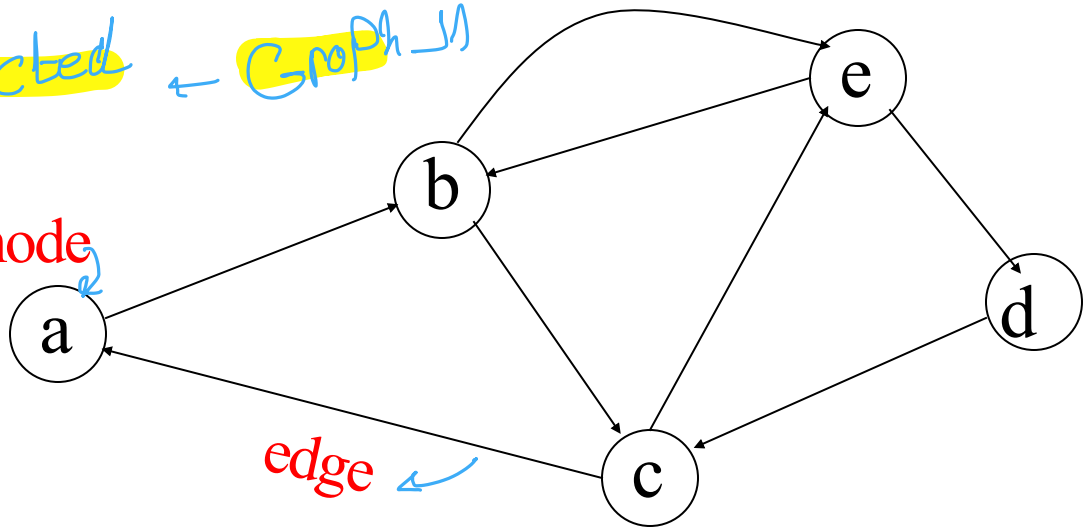
Equivalence class of $1 = \{1, 2\}$

Equivalence class of $3 = \{3, 4\}$

R.
Al-
Milli

GRAPHS

A directed graph



- Nodes (Vertices)

V = { a, b, c, d, e }

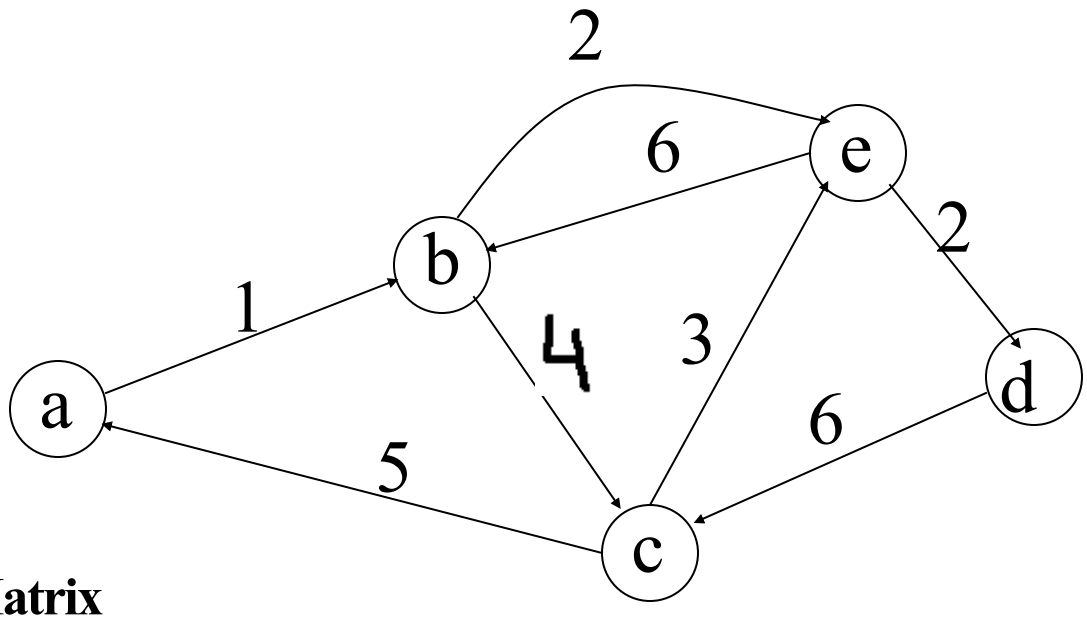
- Edges

E = { (a,b), (b,c), (b,e),(c,a), (c,e), (d,c), (e,b), (e,d) }

- Degree |G|=|V|+|E|= 5+8=13

R.
Al-
Milli

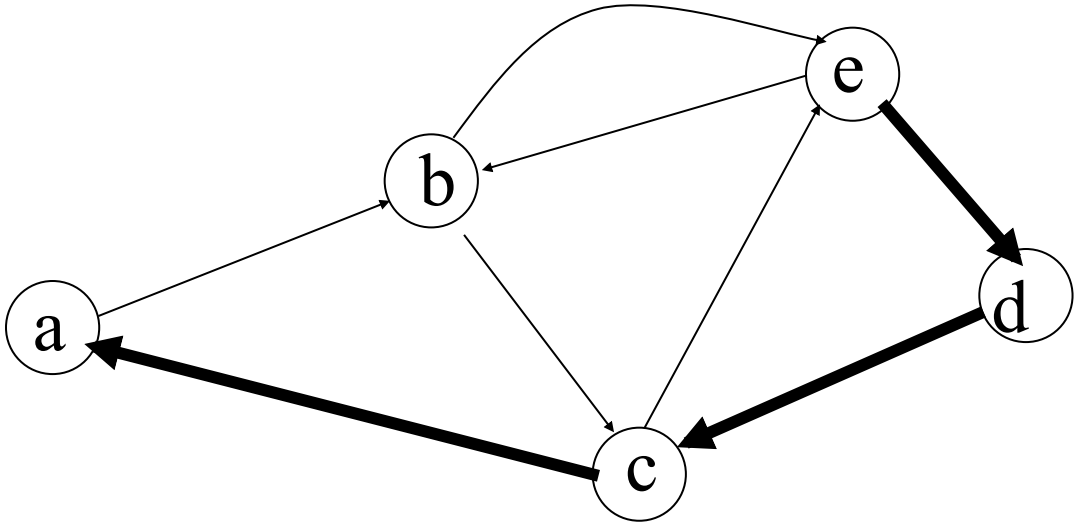
Labeled Graph



Adjacency Matrix

	a	b	c	d	e
a	-	1	-	-	-
b	-	-	4	-	2
c	5	-	-	-	3
d	-	-	6	-	-
e	-	6	-	2	-

Walk

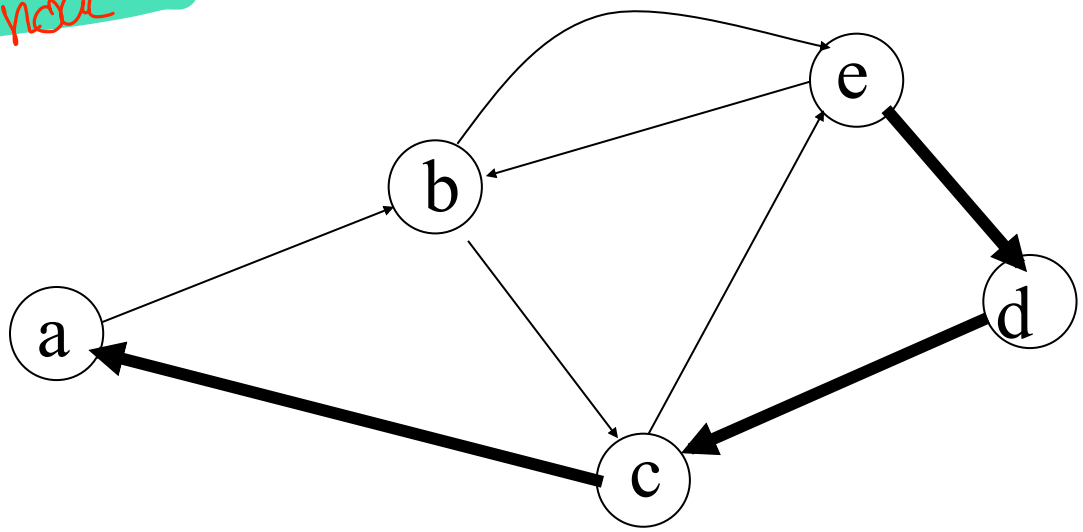


Walk is a sequence of adjacent edges

(e, d), (d, c), (c, a)

R.
Al-
Milli

Path

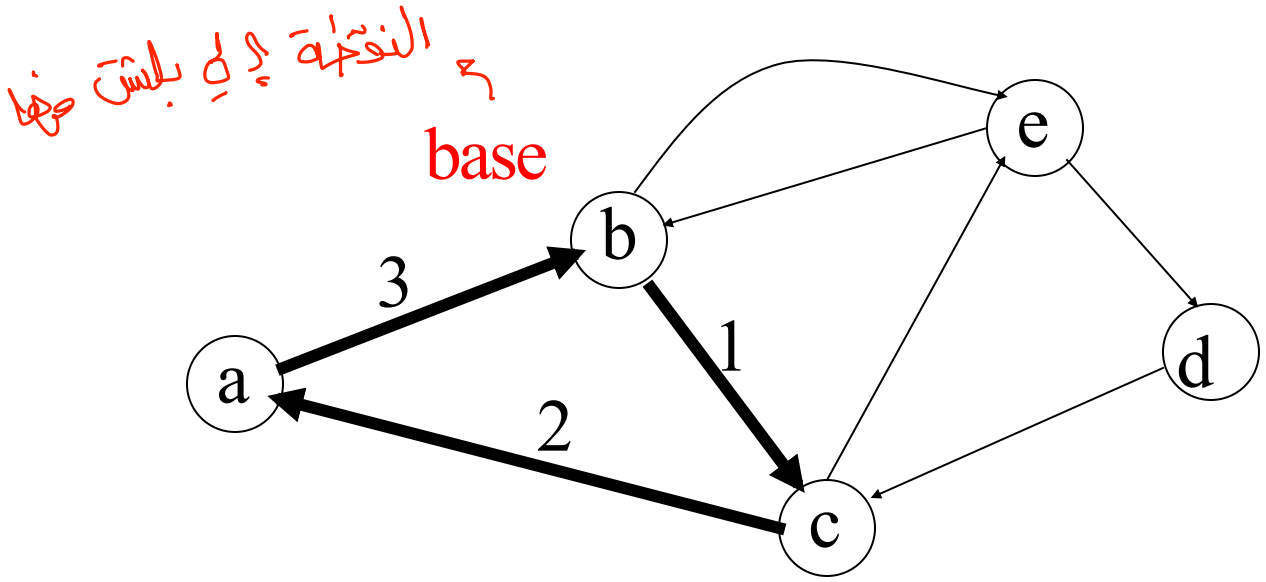


Path is a walk where no edge is repeated

Simple path: no node is repeated

R.
Al-
Milli

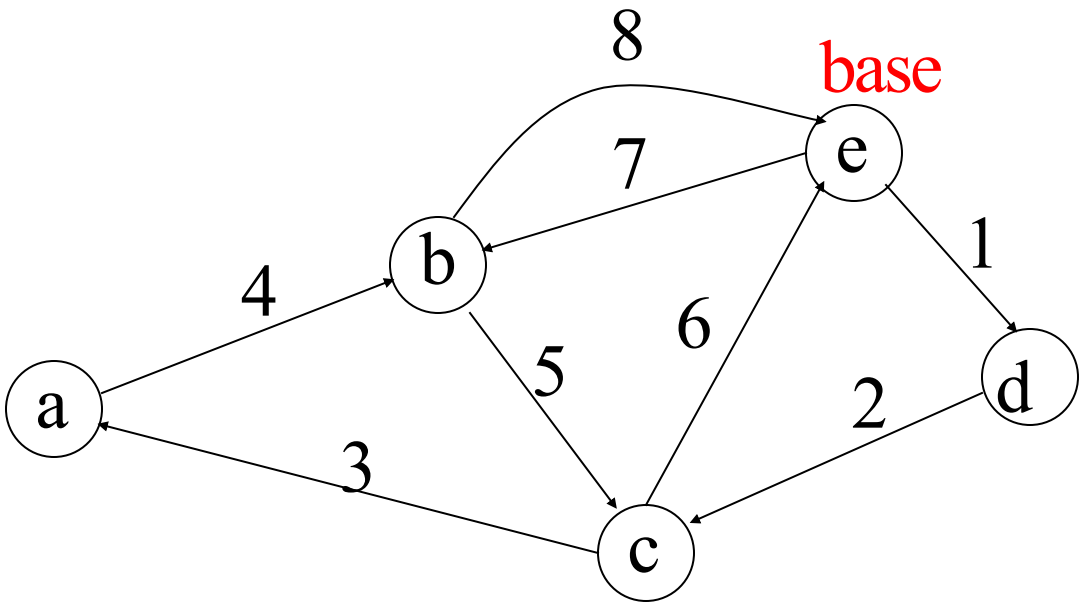
Cycle



Cycle: a walk from a node (base) to itself

Simple cycle: only the base node is repeated

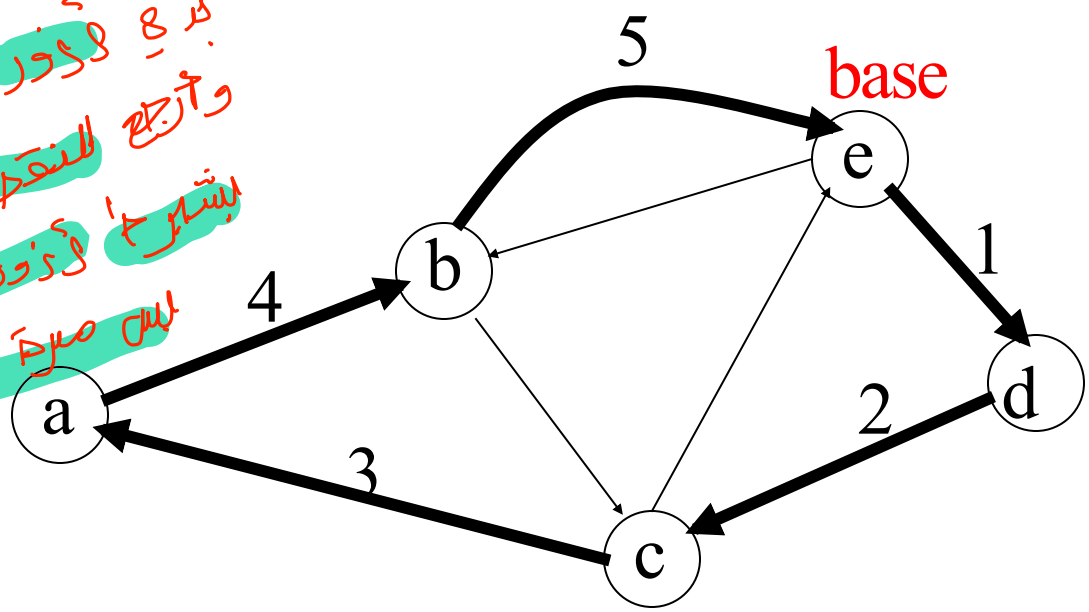
Euler Tour



A cycle that contains each edge once,
Back to the start point

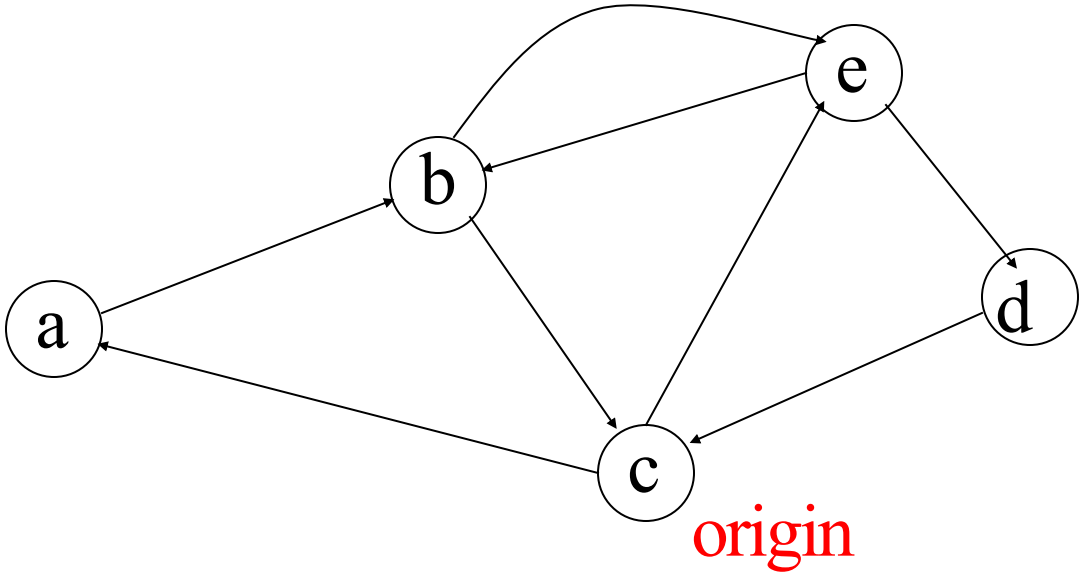
R.
Al-
Milli

Hamiltonian Cycle



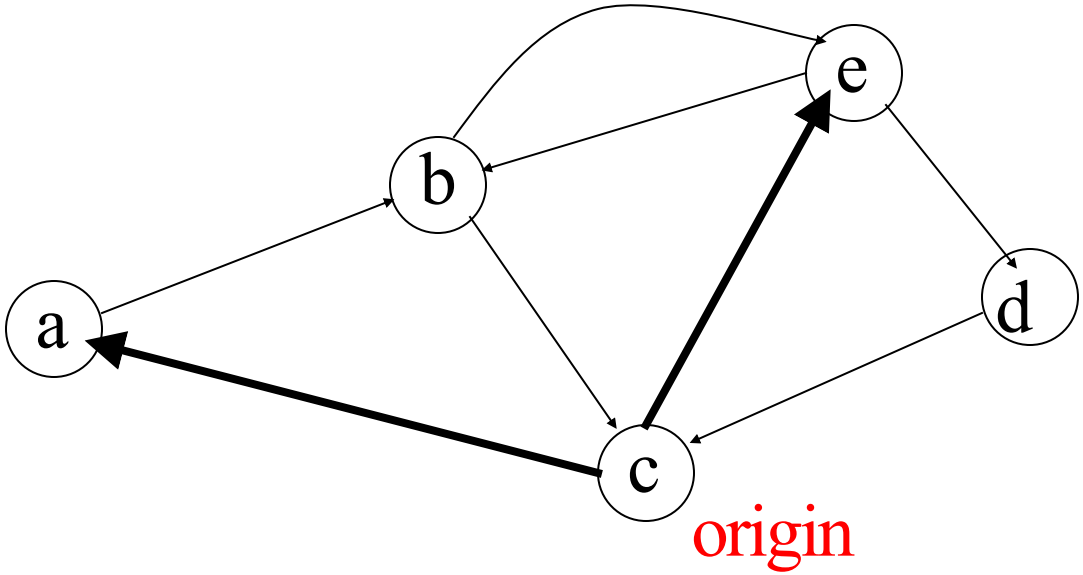
A cycle that contains each node once,
Back to the start point

Finding All Simple Paths



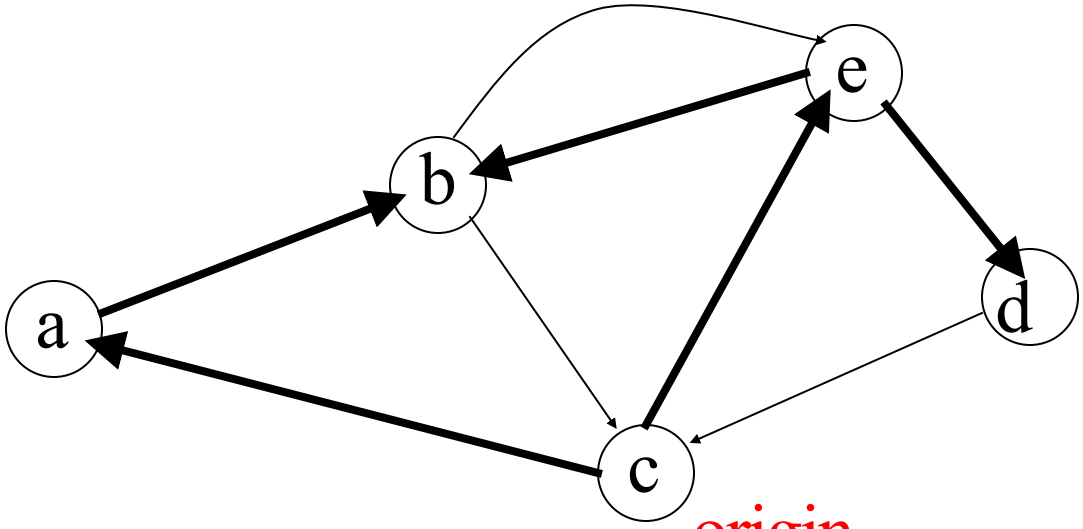
R.
Al-
Milli

Step 1



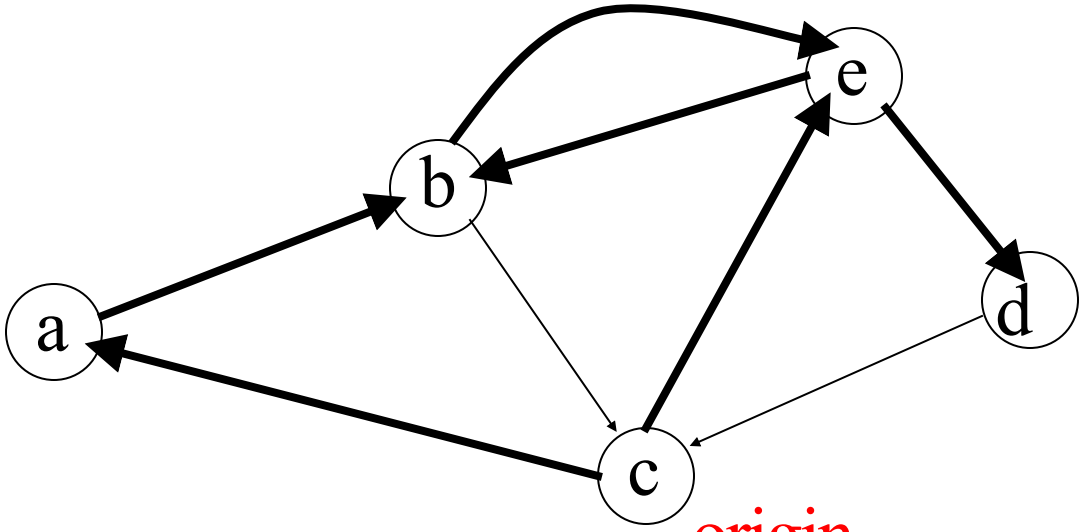
(c, a)
(c, e)

Step 2



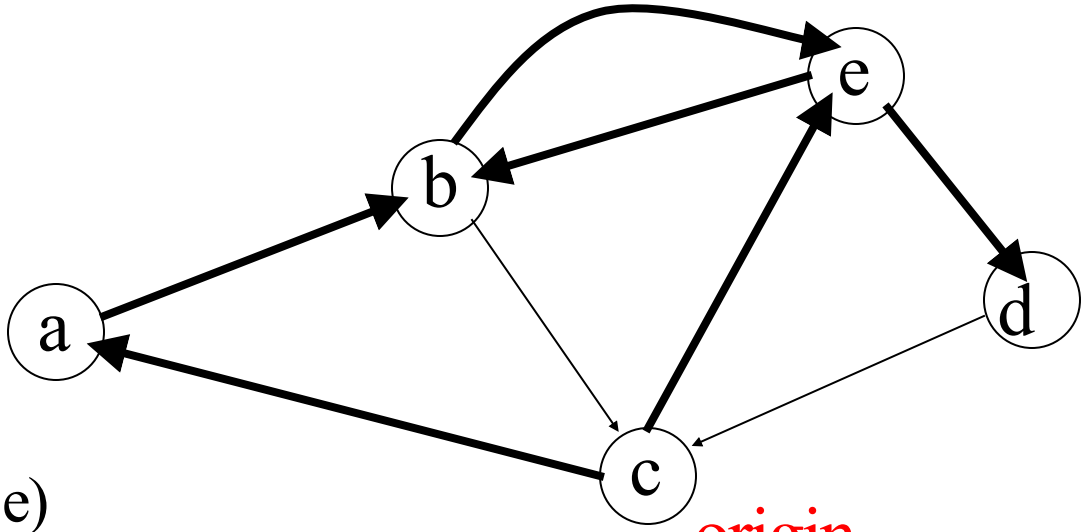
(c, a)
(c, a), (a, b)
(c, e)
(c, e), (e, b)
(c, e), (e, d)

Step 3



(c, a)
(c, a), (a, b)
(c, a), (a, b), (b, e)
(c, e)
(c, e), (e, b)
(c, e), (e, d)

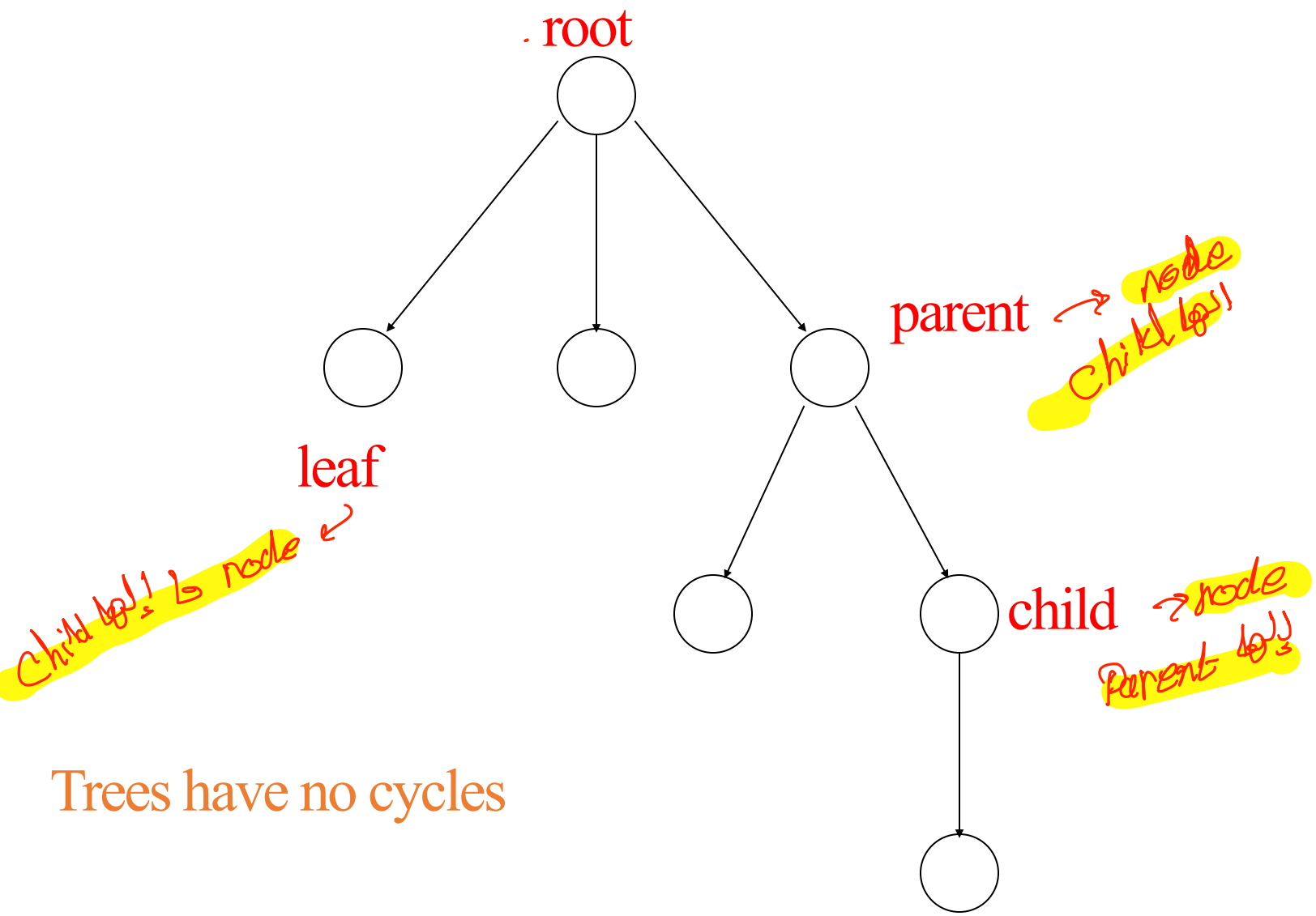
Step 4



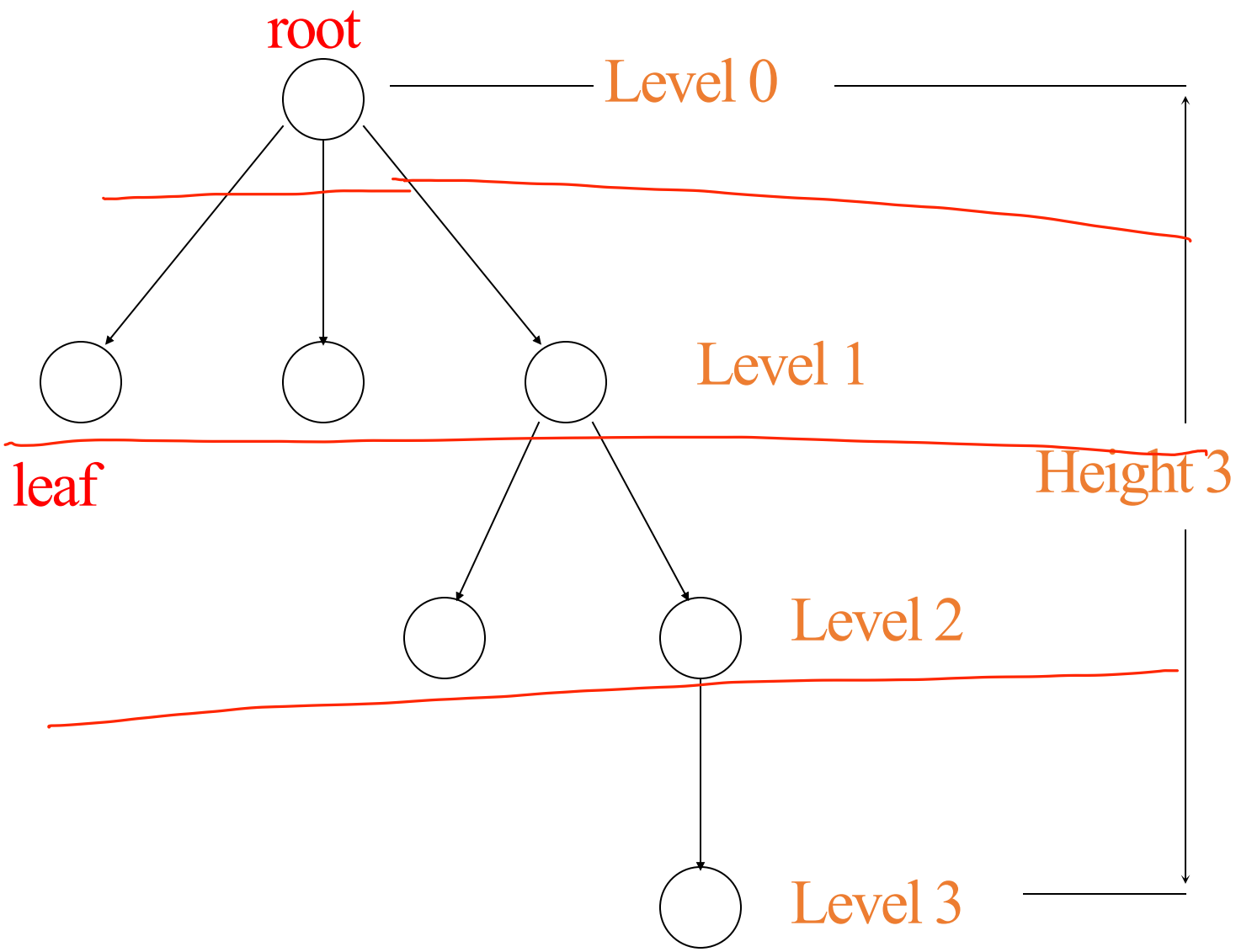
(c, a)
(c, a), (a, b)
(c, a), (a, b), (b, e)
(c, a), (a, b), (b, e), (e,d)
(c, e)
(c, e), (e, b)
(c, e), (e, d)

R.
Al-
Milli

Trees

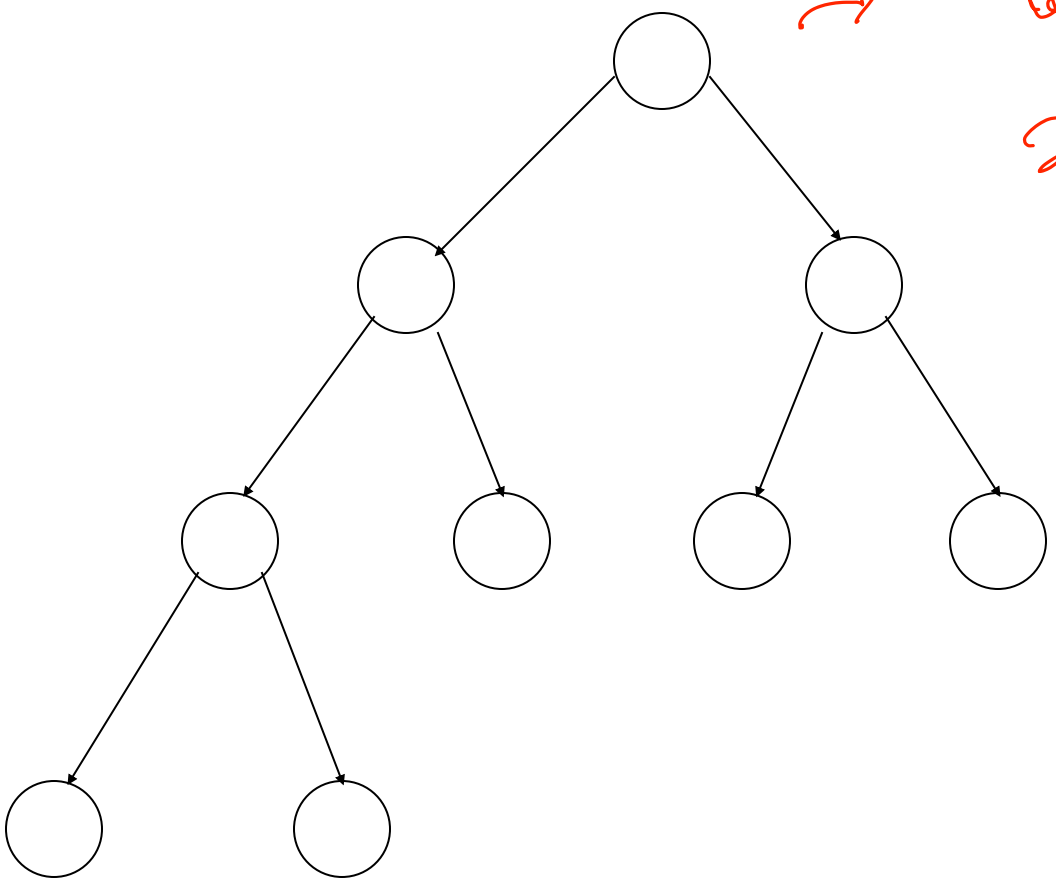


Trees have no cycles


root
Level 0
Level 1
leaf
Height 3
Level 2
Level 3

R.
Al-
Milli

Binary Trees



Language → a set of string

string → a seq of symbols from alphabet

→ string → cat, dog, house

language → { cat, dog, house }

ای string کو جیسے ایک braces میں

language

→ alphabe → are a finite set of symbols

$$\Sigma = \{a, b, c, \ldots\ldots, d\}$$

Note → Language are used to describe computation

Languages

Language: *a set of strings*

String: *a sequence of symbols from some alphabet*

Example:

Strings: cat, dog, house

Language: {cat, dog, house}

Alphabets: are a finite set of symbols.it is denoted by Σ

Alphabet: $\Sigma = \{a, b, c, \ldots, z\}$

Examples:

$\Sigma = \{a, b\}$

$\Sigma = \{A, B, C, D\}$

$\Sigma = \{0, 1, 2\}$

$\Sigma = \{\#, \beta, \Delta\}$

* Note :-

Binary alphabet

$\Sigma = \{0, 1\}$

Languages are used to describe computation problems:

$$PRIMES = \{2,3,5,7,11,13,17,\ldots\}$$

$$EVEN = \{0,2,4,6,\ldots\}$$

Alphabet: $$\Sigma = \{0,1,2,\ldots,9\}$$

- (set membership) * لحل نحول الحساب لمسألة انتماء لمجموعة

Computation is translated to **set membership**

إشارة ينتمي ← $(\in)$

Example computation problem:

Is number x prime?

هل الرقم x ينتمي لهاي المجموعة أو لا

Equivalent set membership problem:

$$x \in PRIMES = \{2,3,5,7,11,13,17,\ldots\}?$$

Decimal numbers alphabet $\Sigma = \{0,1,2,\ldots,9\}$

102345 567463386

Binary numbers alphabet $\Sigma = \{0,1\}$

100010001 101101111

Unary numbers alphabet $\Sigma = \{1\}$ → رقم واحد

Unary number: 1 11 111 1111 11111

Example:

Let $\sum = \{x\}$ be set of alphabet of one letter x. we can write this in form:

$L_1 = \{x, xx, xxx, \ldots\}$

or write this in an alternate form:

$L_1 = \{x^n \text{ for } n = 1, 2, 3, \ldots\}$

String Operations:

Concatenation

$$wv = a_1 a_2 \cdots a_n b_1 b_2 \cdots b_m$$

If W=abba V= bbbaaa

V V=bbbaaabbbaaa

W W=abba abba

WV= $abbaabbbaaa$

Reverse → عكس ترتيب الرموز

$X=abc$

$x^R=cba$

Length → عدد الرموز أو الحروف

$|x|=|abc|=3$

$|\varepsilon|=0$ $|\lambda|=0$ $|\wedge|=0$

λ:lambda ε:Epsilon

Length of Concatenation

$$|uv| = |u| + |v|$$

Example:

$$u = aab, \quad |u| = 3$$

$$v = abaab, \quad |v| = 5$$

$$|uv| = |aababaaab| = 8$$

$$|uv| = |u| + |v| = 3 + 5 = 8$$

Empty String

A string with no letters is denoted: λ or ε

Acts as a neutral element

Observations: $|\lambda| = 0$

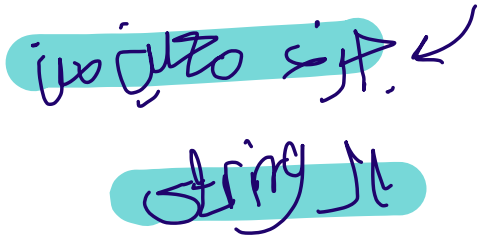
$$\lambda w = w\lambda = w$$

$$\lambda abba = abba\lambda = ab\lambda ba = abba$$

λaλaλ=aa

Substring

Substring of string: *a subsequence of consecutive characters*



String

Substring

String	Substring
$abbab$	ab
$abbab$	$abba$
$abbab$	b
$abbab$	$bbab$

Prefix and Suffix

string $abbab$

Prefixes	Suffixes
λ	$abbab$
a	$bbab$
ab	bab
abb	ab
$abba$	b
$abbab$	λ

$w = uv$

prefix

suffix

Exponent Operation

$$w^n = \underbrace{ww\cdots w}_{n}$$

Example:

$$(abba)^2 = abbaabba$$

Definition:

$$w^0 = \lambda$$

$$(abba)^0 = \lambda$$

$(ab)^0=\lambda$

$(ab)^1=ab$

$(ab)^3=ababab$

$Ab^3=abbb$ → هون بدي انتبه إنه الـ b لا تتكرر بس الـ A

$(a(ab)^2)^3=(aabab)^3=aababaababaabab$

abab

aabab

aababaababaabab

$N_a(w)$ – number of symbol a in string w

$$W = aabbcaaba$$

$N_a(w)=5$

$N_b(w)=3$

$N_c(w)=1$

$N_d(w)=0$

The * Operation

Σ^*: the set of all possible strings from alphabet Σ

$$\Sigma = \{a, b\}$$

$$\Sigma^* = \{\lambda, a, b, aa, ab, ba, bb, aaa, aab, \ldots\}$$

Example:

If $\Sigma = \{\ \}$, then

$$\Sigma^* = \{\lambda\}$$

Suppose $\sum$ ={1,2,3}

$\sum^0$ ={λ}

$\sum^1$ ={1,2,3} , $|\sum^1|$ =3^1=3 , 3 Elements in set, each element consists of one symbol

$\sum^2$ ={11,12,13,21,22,23,31,32,33} , $|\sum^2|$ = 3^2=9 , 9 Elements in set, each element consists of two symbols

$\sum^3$ ={111,112,113,121,122,123,131,132,133,
211,212,213,221,222,223,231,232,233,
311,312,313,321,322,323,331,332,333} , $|\sum^3|$=3^3=27 ,27 elements in set, each element consists of 3 symbols.

Note :

$|\sum^n|=|\sum|^n$

$\sum^* = \sum^0 \cup \sum^1 \cup \sum^2 \cup \sum^3 \cup$ .....

Example:

If $\sum$ ={0,1}, then

$\sum^*$ ={λ ,0, 1, 00,01,10,11,000,001,.....}

HINT:

Suppose $\sum$ ={1,2,3}

$\sum^0$ ={λ}

$\sum^1$ ={1,2,3} , $|\sum^1|$ =3^1=3 , 3 Elements in set, each element consists of one symbol

$\sum^2$ ={11,12,13,21,22,23,31,32,33} , $|\sum^2|$ = 3^2 =9 , 9 Elements in set, each element consists of two symbols

$\sum^3$ ={111,112,113,121,122,123,131,132,133,
211,212,213,221,222,223,231,232,233,
311,312,313,321,322,323,331,332,333} , $|\sum^3|$ =3^3 =27 ,27 elements in set, each element consists of 3 symbols.

Note :

$|\sum^n| = |\sum|^n$

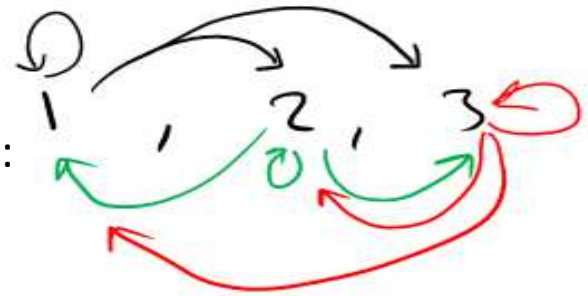
$\sum^* = \sum^0 \cup \sum^1 \cup \sum^2 \cup \sum^3 \cup \ldots\ldots$

Example:

If $\sum$ ={0,1}, then

$\sum^*$ ={λ ,0, 1, 00,01,10,11,000,001,.....}

HINT:



The + Operation

Σ^+ : the set of all possible strings from Σ alphabet except λ

$$\Sigma = \{a, b\}$$

$$\Sigma^* = \{\lambda, a, b, aa, ab, ba, bb, aaa, aab, \ldots\}$$

$$\Sigma^+ = \Sigma^* - \lambda \rightarrow$$

يعني اي حل بس مدنوع تكون λ موجوده

$$\Sigma^+ = \{a, b, aa, ab, ba, bb, aaa, aab, \ldots\}$$

Notes:

- **Any Language start with *, first element is λ**
- **(* , +) called Kleene Closure**

Note that:

Sets $\emptyset = \{\} \neq \{\lambda\}$

Set size $|\{\}| = |\emptyset| = 0$

Set size $|\{\lambda\}| = 1$ → Language

String length $|\lambda| = 0$ → string

Regular Expression (RE)

Regular Expression (RE)

هي طريقة رياضية وضمنية يستخدم لوصف اللغات بطريقة مختصرة نكتب فيها جمل اللغة

- Regular Expression is a set of symbols, thus if alphabet= {a, b}, then aab, a, baba, bbbbb, and baaaaa would all be strings of symbols of alphabet.

* يعني بدل ما أكتب ← ... ، aaa ، aa ، a ، λ ← a* بكتب

* ← عدد من التكرارات لما فيه الحرف

- an empty string denoted by Λ which has no symbols in it.

- The simple expression x* will be used to indicate some sequence of x's.

$$x^* = \{ \Lambda, x, x^2, x^3, x^4, \ldots \}$$

$$= \{x^n \text{ for } n = 0, 1, 2, 3, 4, \ldots\}$$

Example: Write the language for each of the following regular expressions, $\sum = \{a,b\}$.

1- $(ab)^* = \{\Lambda, ab, abab, ababab, \ldots\}$

2- $ab^*a = \{aa, aba, abba, abbba, abbbba, \ldots\}$

3- $a^*b^* = \{\Lambda, a, b, aa, ab, bb, aaa, aab, abb, bbb, aaaa, \ldots\}$

Example: Write a regular expression for the language containing odd number of 1s, $\sum = \{0,1\}$.

The language will contain at least one 1. It may contain any number of 0s anywhere in the string. So the language we have to write a regular expression for is 1, 01, 01101, 0111, 111, … This language can be represented by the following regular expression: L={1,01,10,010,001,100,111,0111,1110,101010

0*(10*10*)*10*

0*(10*10*)*10*

Example: Write the language for each of the following regular expressions, $\sum = \{x\}$.

1- $L_1 = \{x^{odd}\} = x(xx)^*$ **or** $(xx)^*x = \{x, xxx, xxxxx, \ldots\}$

2- $L_2 = \{x^{even}\} = (xx)^*$ **or** $(xx)^*xx$ **or** $xx(xx)^* = \{\Lambda, xx, xxxx, \ldots\}$

Star-Closure (Kleene *)

All strings that can be constructed from L

Definition:

$$L^* = L^0 \cup L^1 \cup L^2 \cdots$$

Example:

$$\{a, bb\}^* = \begin{Bmatrix} \lambda, \\ a, bb, \\ aa, abb, bba, bbbb, \\ aaa, aabb, abba, abbbb, \ldots \end{Bmatrix}$$

يعمل تكرارات بأي عدد يدي ياه ←
حتى الصفر
يستخدمها لما ألون بدي اعمل
أو امثل Language غير منتهية
تتكرر بنفس الشكل

Positive Closure

Definition:

$$L^{+} = L^{1} \cup L^{2} \cup \cdots$$

Example:

$$\{a, bb\}^{+} = \begin{Bmatrix} a, bb, \\ aa, abb, bba, bbbb, \\ aaa, aabb, abba, abbbb, \ldots \end{Bmatrix}$$

Example of regular sets/regular languages

Language in a Set Builder Notations	Regular language/regular expression	Regular set
$L = \emptyset$	$\emptyset$	$\{\}$
$L = \{\varepsilon\}, or\ L = \{\lambda\},$	ε or λ	$\{\varepsilon\}$ or $\{\lambda\}$
$L = \{a\}$	a	$\{a\}$
$L = \{a, b\}$	$a + b$	$\{a, b\}$
$L = \{a^n : n \geq 0\}$	$a*$	$\{\lambda, a, aa, aaa \ldots..\}$
$L = \{ab\}$	$a.b$	$\{ab\}$
$L = \{a^n : n \geq 0\} \cup \{ba\}$	$a* + ba$	$\{\lambda, a, aa, aaa, \ldots\ldots, ba\}$
$L = \{x^2 : x \in \{a, b\}\}$	$(a+b)^2$	$\{aa, ab, ba, bb\}$

39

Deterministic Finite Automaton

(DFA)

What is a Language?

- A language is a set of strings over an alphabet.
- For example, alphabet {0, 1}
- Language L1 = {1, 0, 11, 00}
- Language L2 = {01, 111, 000}
- Language L3 = {ε} the empty string
- Language L4 = ∅, the empty language, with no strings

Deterministic Finite Automaton (DFA)

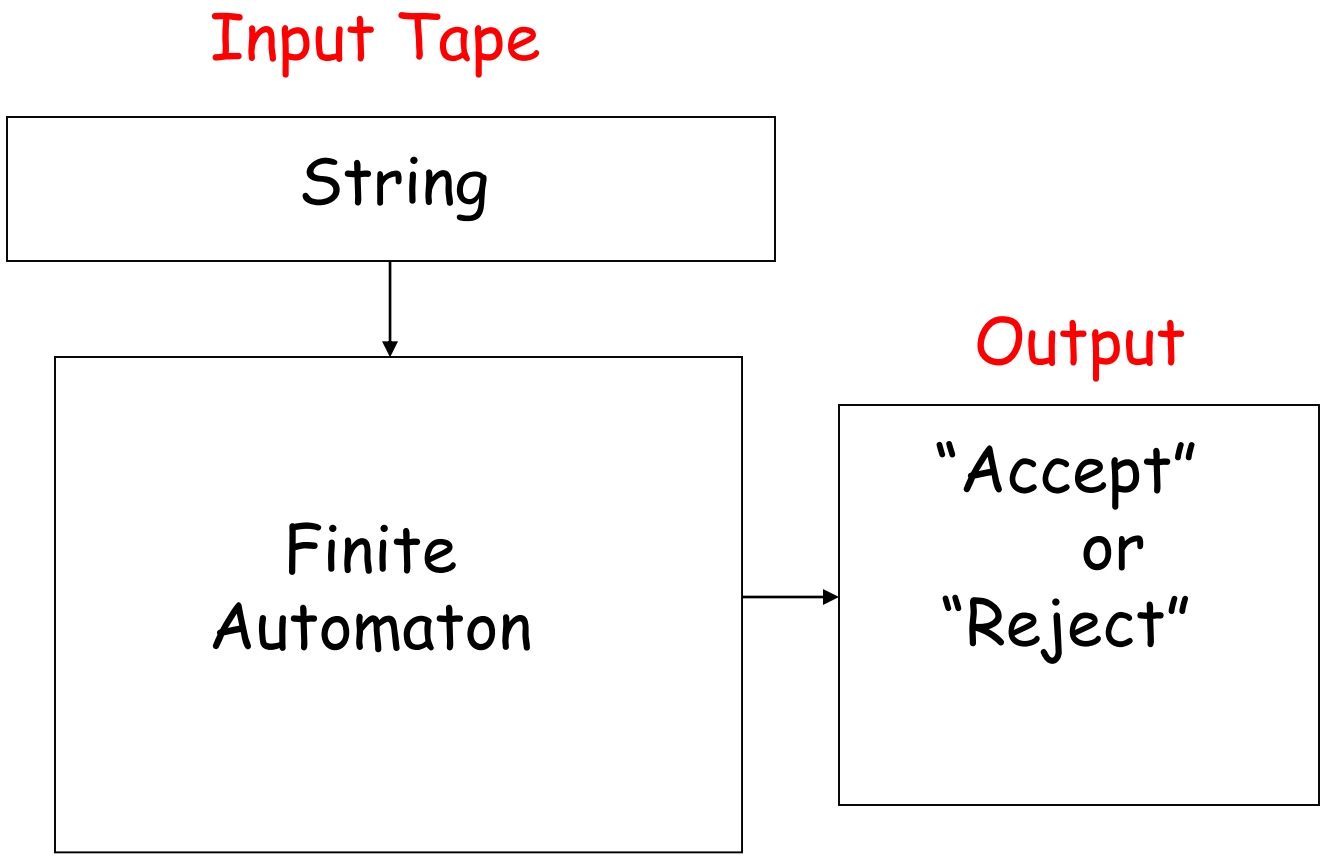
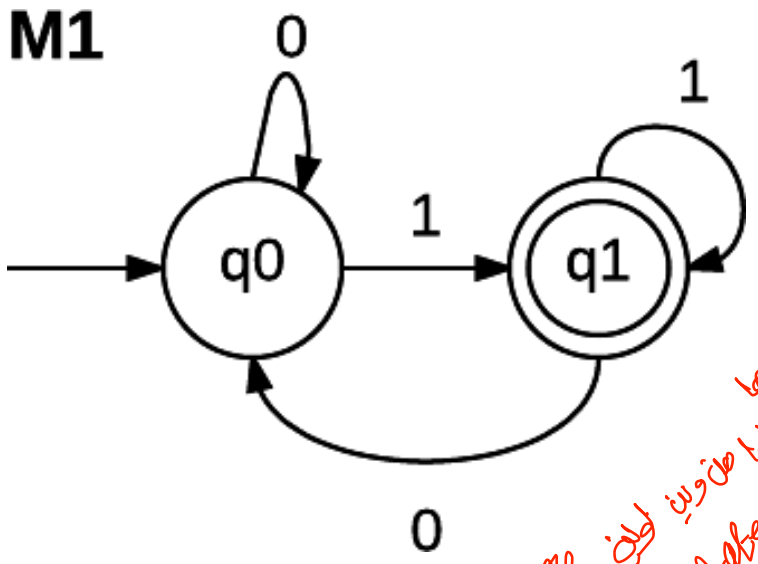


table → جدول state diagram → شكل دوائر وأسهم

Deterministic Finite Automata (DFA)

Example 1: M1



What is the formal description of M1?

- This DFA is a 5-tuple: $(Q, \Sigma, \delta, q_0, F)$

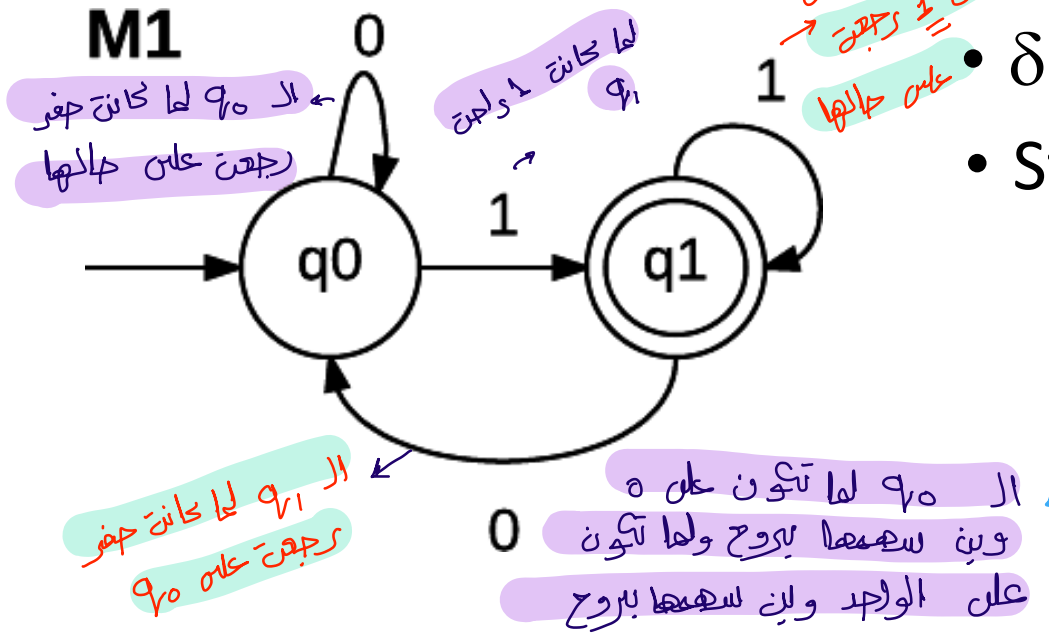
كل المراحل إلي ممكن يمر فيها ←

1. Q = set of states = {q0, q1}
2. Σ = alphabet = {0, 1} → الـ String إلي مسموح لاستخدمها
3. δ = transition function = (state diagram/table) ← الخريطة إلي رح لاستخدمها عشان أعرف فيها طريقي لحتى رح لاتنقل بكل state
4. q_0 = first state = q0 → المرحلة الأولى
5. F = set of final states = {q1}.

آخر مرحلة ← وداعيا بتنلها بـ اثنين جوا بعض

* كل state بتنلها بدائرة وحدة إلا الـ F
* كل state لازم يطلع أسهم فيها على عدد الـ alphabet المسموح استخدمها

Deterministic Finite Automata (DFA)

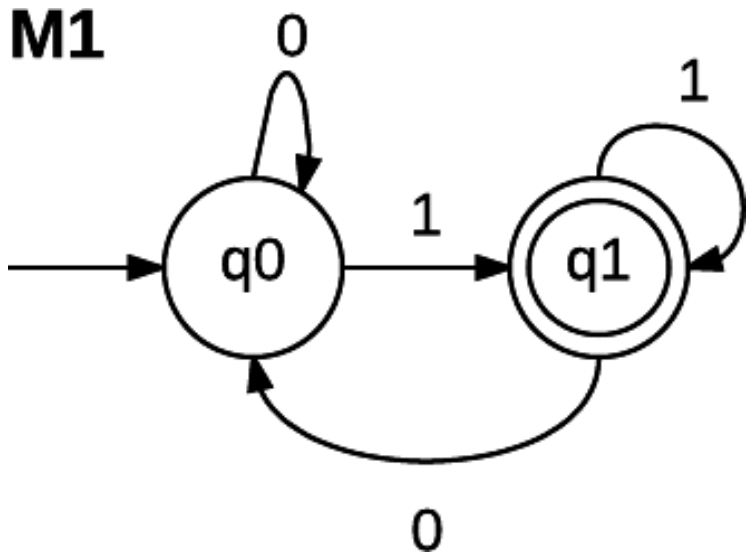


- *Example 1: M1*
- δ = transition function
- State Diagram (figure)
- Transition Table

	0	1
q0	q0	q1
***q1**	q0	q1

Dr. Nabeel Al-Milli

Deterministic Finite Automata (DFA)



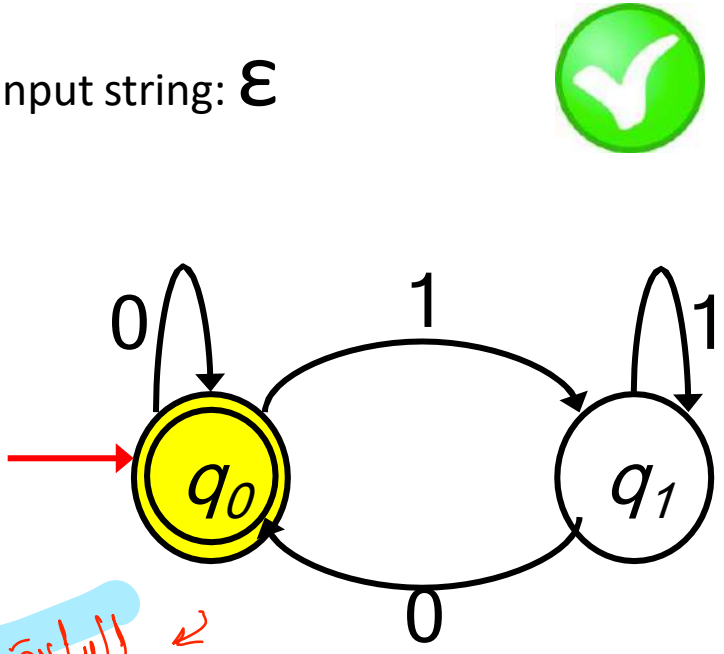
- Transition Table

	0	1
q0	q0	q1
***q1**	q0	q1

- What is the language recognized/accepted by M1?
- e.g {01, 011, 0101, 01011, 1, 11, 101, 1001…}
- L1 = {over (0, 1)|strings that end with 1}

Machine 2

Input string: ε



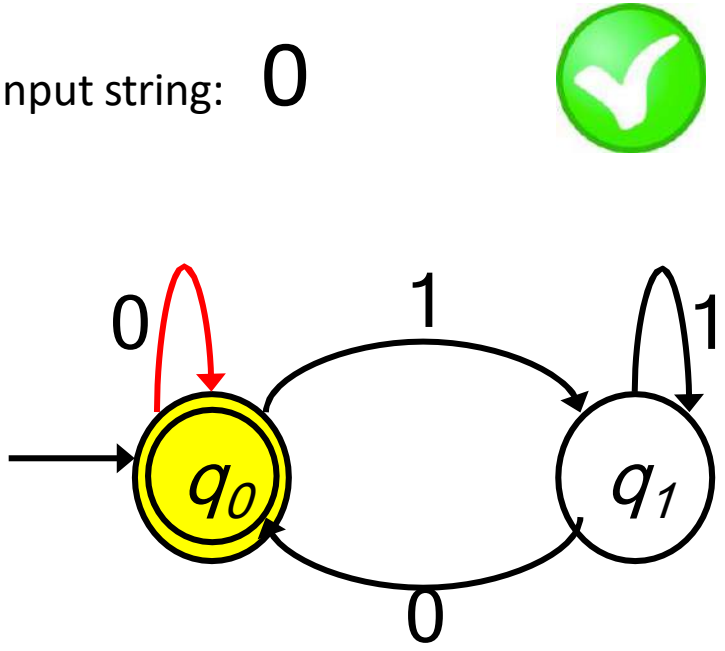
- Example2: M2
- What is the language recognized by M2?
- e.g. {ε,

Note:

If the start queue 0 is the first and final state then Epsilon is accepted String

Checking some accepted strings

Input string: 0

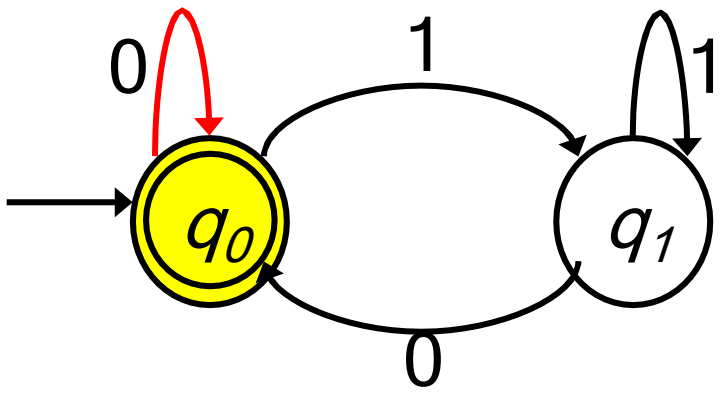


State Diagram for M_2

- Example2: M2
- What is the language recognized by M2?
- e.g. {ε, 0,

Checking some accepted strings

Input string: 00

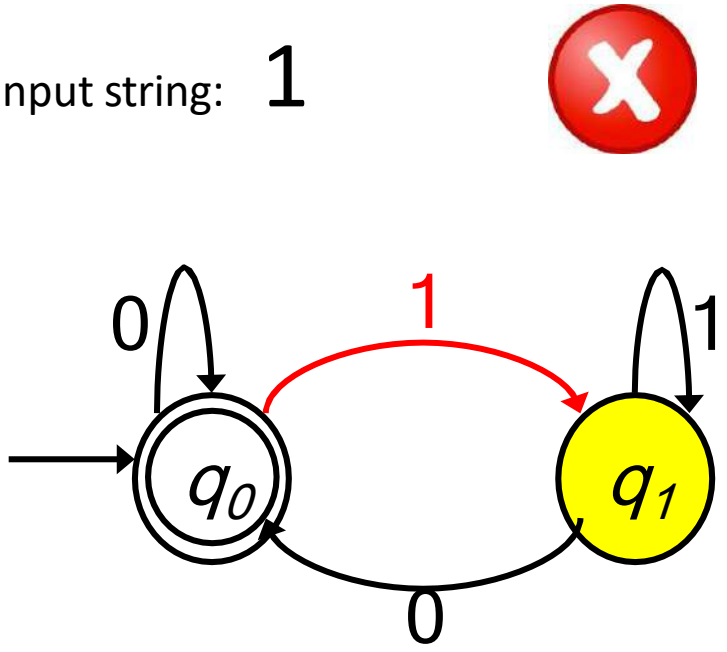


State Diagram for M_2

- Example2: M2
- What is the language recognized by M2?
- e.g. {ε, 0, 00,

Checking some accepted strings

Input string: 1

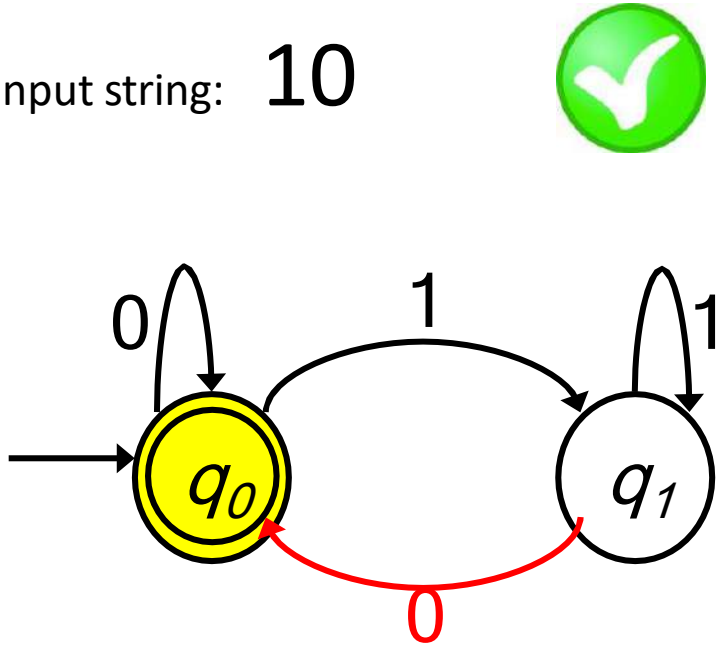


State Diagram for M_2

- Example2: M2
- What is the language recognized by M2?
- e.g. {ε, 0, 00,

Machine 2

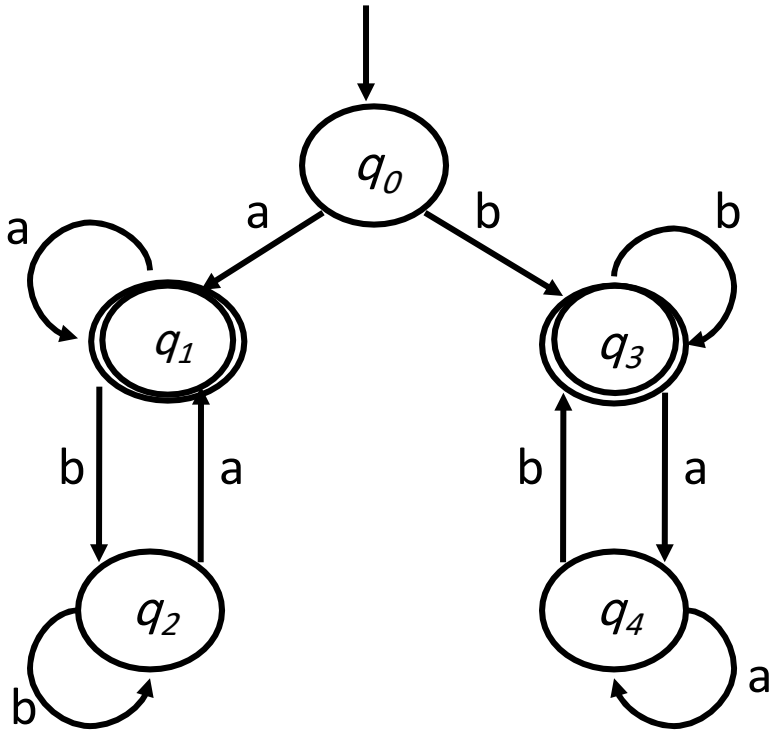
Input string: 10



State Diagram for M_2

- Example2: M_2
- What is the language recognized by M_2?
  - e.g {ε, 0, 00, 10, …}
- $L(M_2)$ = { over (0, 1)| strings that end with 0 or ε, empty string}

Machine 3(Assignment)∞



State Diagram for M_3

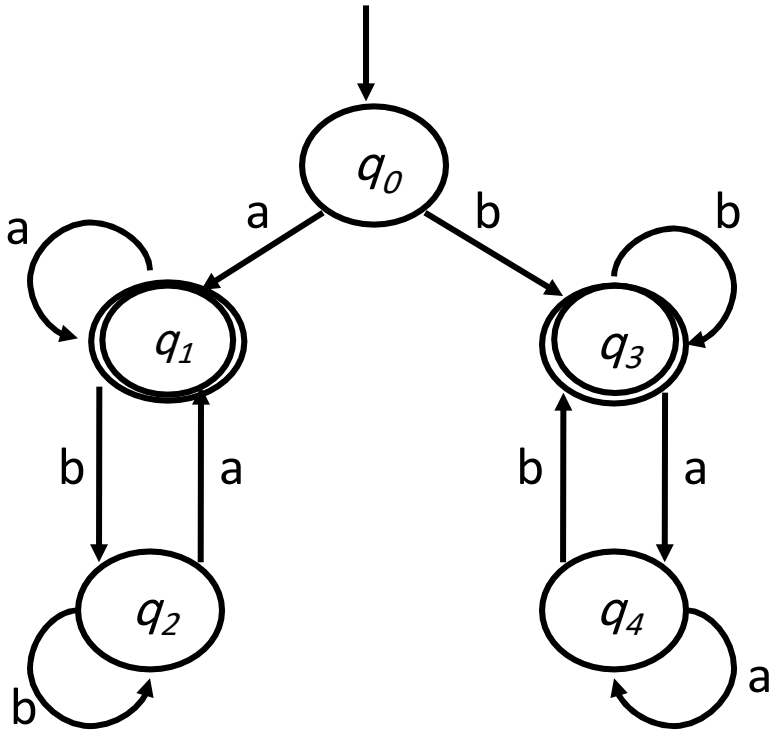
-What is the formal description of M_3?

-What is the language recognized by M_3?

-Show Transition Table

-Formulate Language

Machine 3 (Solution)



State Diagram for M_3

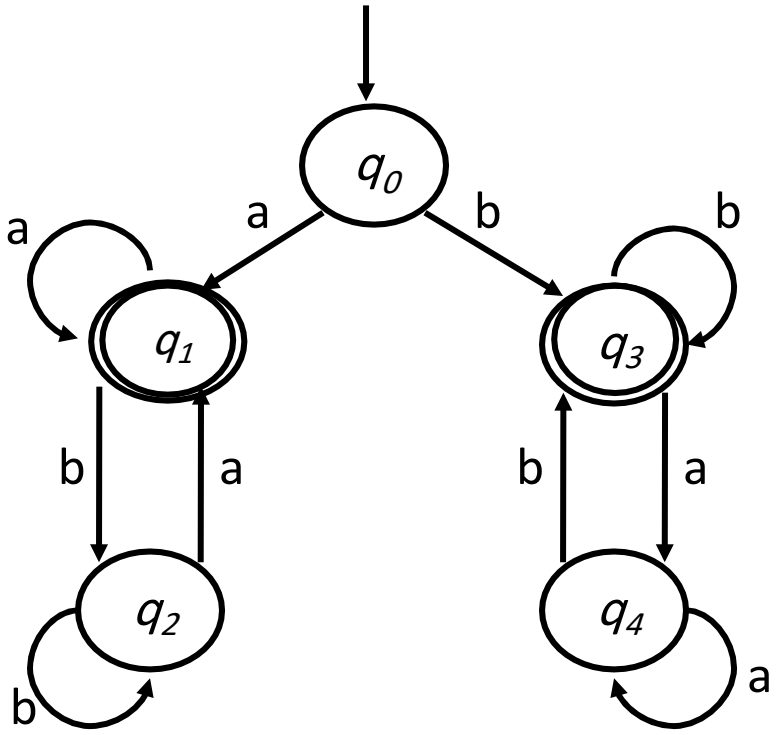
What is the formal description of M_3?

- Example3: M_3

- This DFA is a 5-tuple: $(Q, \Sigma, \delta, q_0, F)$

1. $Q = \{q_0, q_1, q_2, q_3, q_4\}$
2. $\Sigma = \{a, b\}$
3. δ = (state diagram/transition table)
4. $q_0 = q_0$
5. $F = \{q_1, q_3\}$.

Machine 3



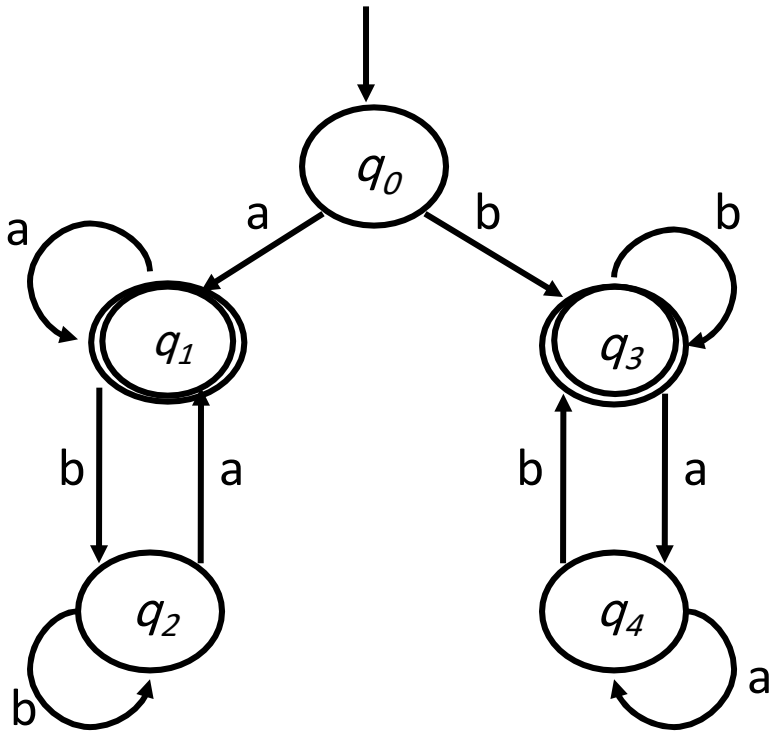
State Diagram for M_3

- Example3: M_3
- δ = transition function = (state diagram/table)

	a	b
q0	q1	q3
***q1**	q1	q2
q2	q1	q2
***q3**	q4	q3
q4	q4	q3

Transition table for M_3

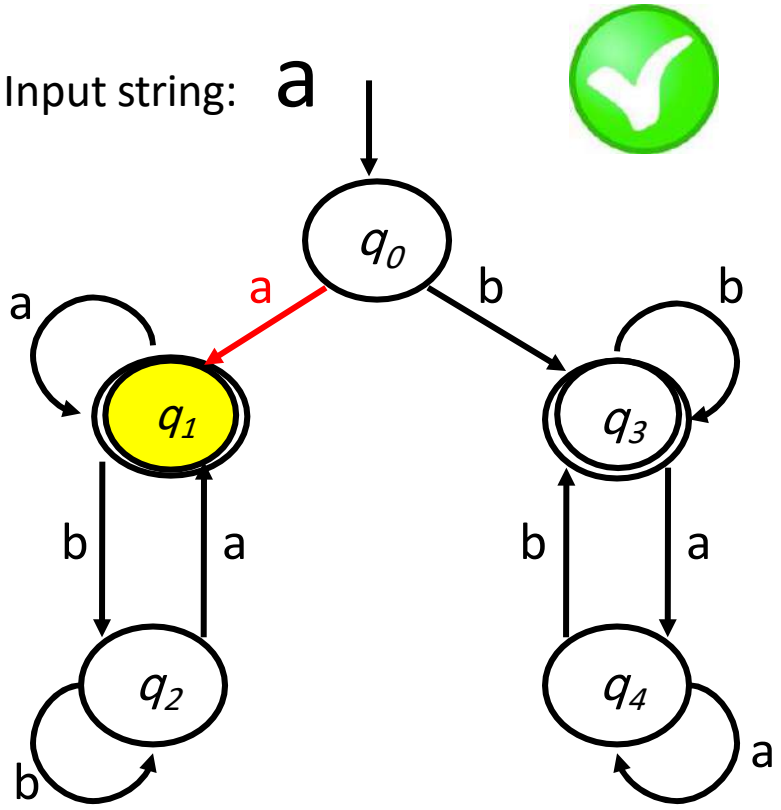
Machine 3



State Diagram for M_3

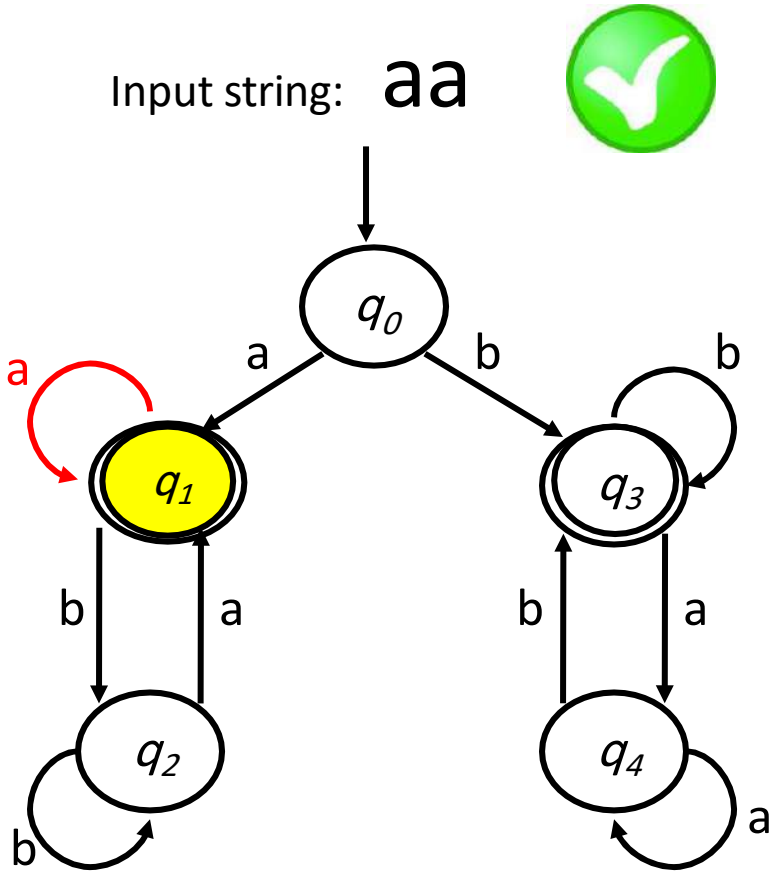
- Example3: M_3
- What is the language recognized by M_3?

Machine 3



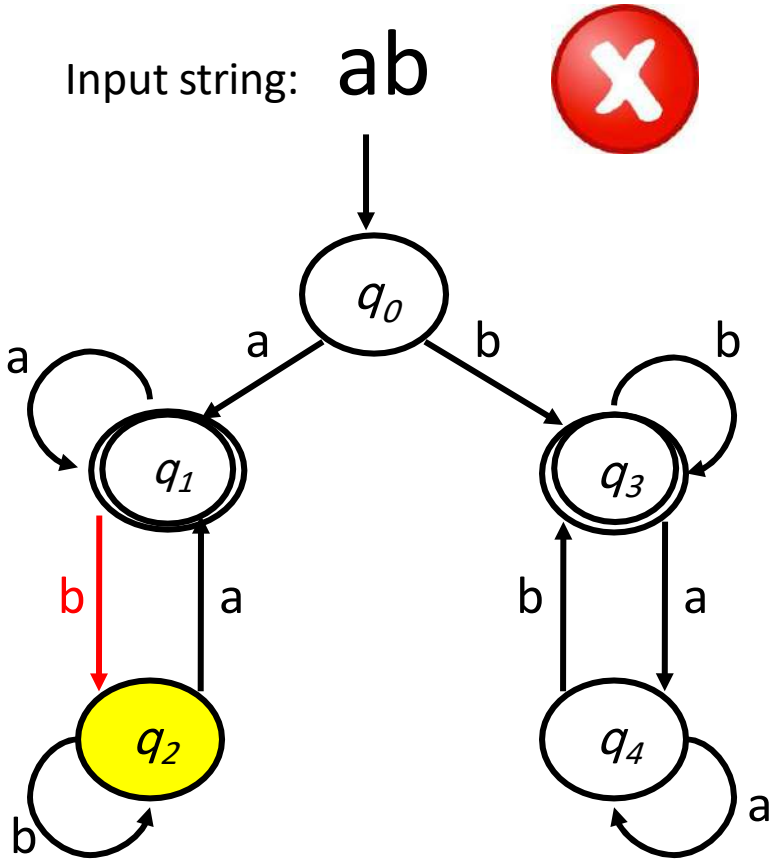
State Diagram for M_3

- Example3: M_3
- What is the language recognized by M_3?
- Some accepted strings = {a,



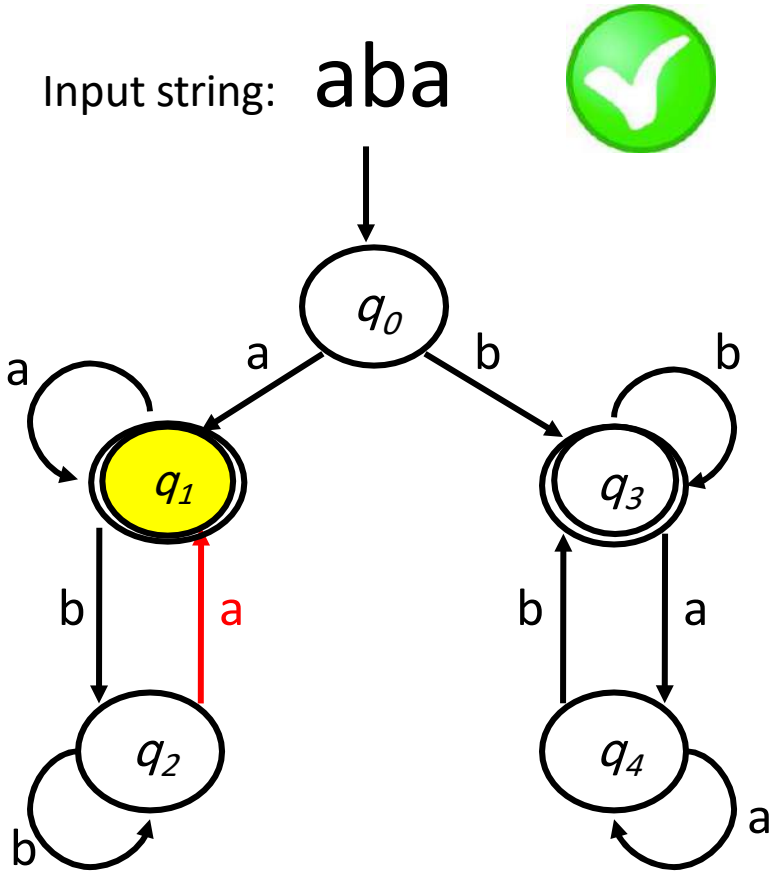
State Diagram for M_3

- Example3: M_3
- What is the language recognized by M_3?
- Some accepted strings = {a, aa



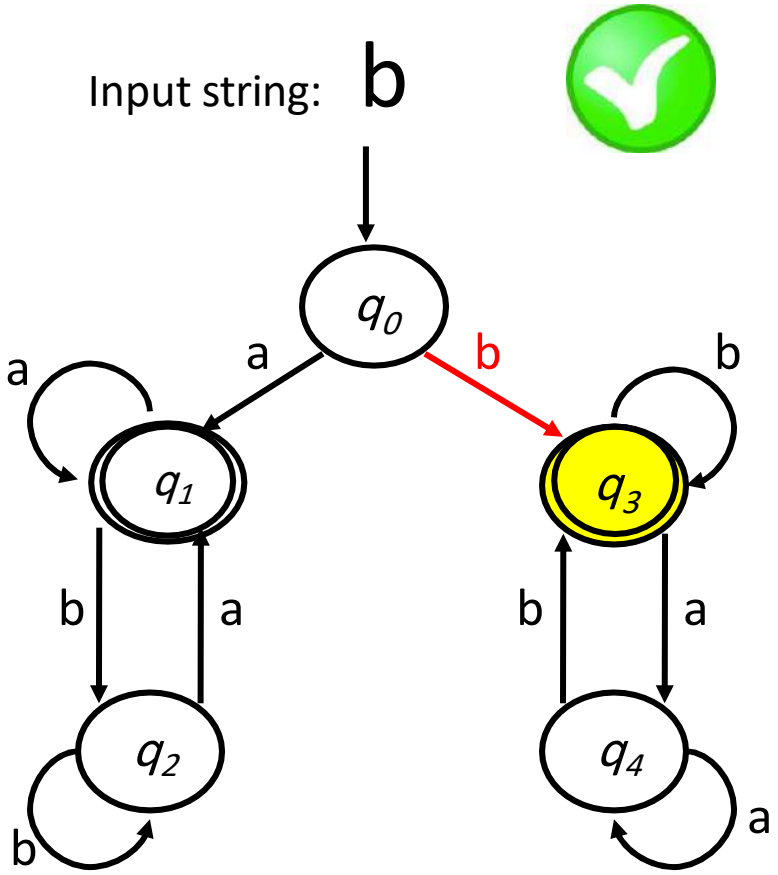
State Diagram for M_3

- Example3: M_3
- What is the language recognized by M_3?
- Some accepted strings = {a, aa



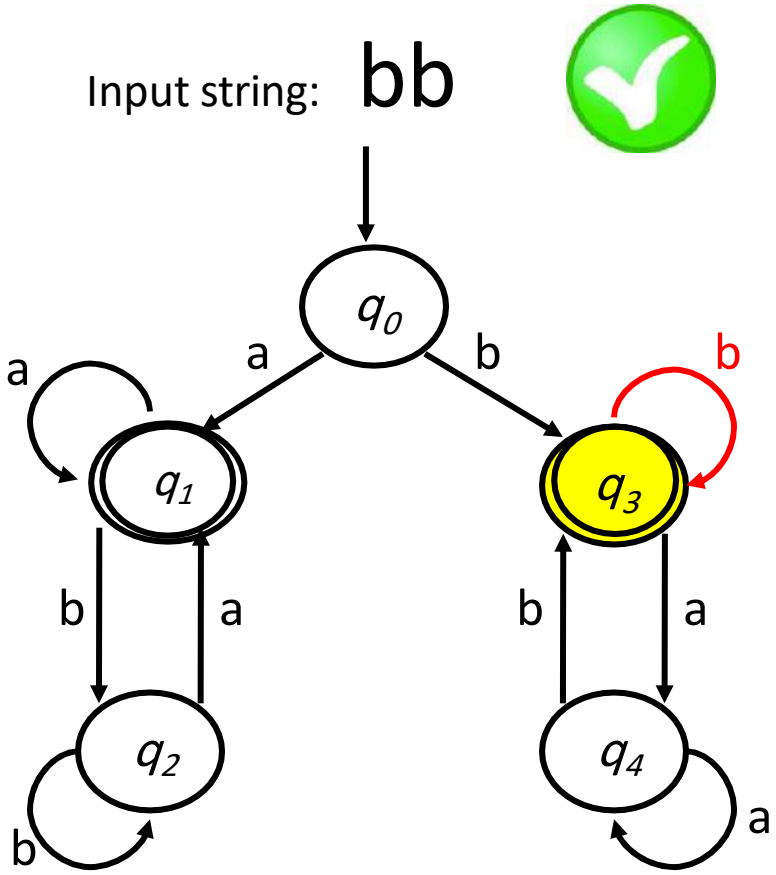
State Diagram for M_3

- Example3: M_3
- What is the language recognized by M_3?
- Some accepted strings = {a, aa, aba, ..



State Diagram for M_3

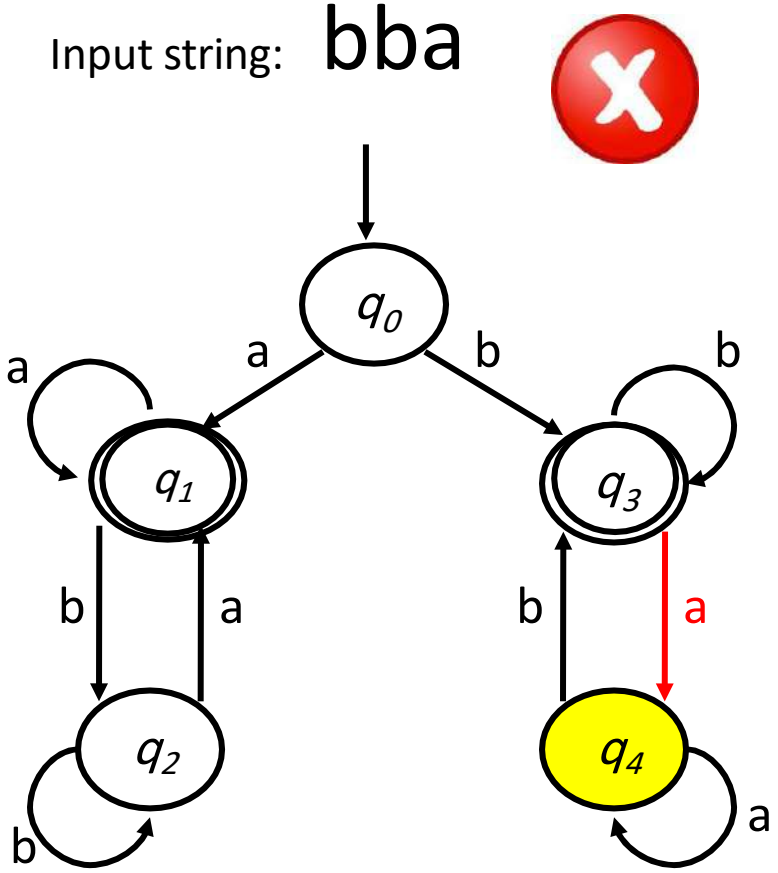
- Example3: M_3
- What is the language recognized by M_3?
- Some accepted strings = {a, aa, aba, .., b,



State Diagram for M_3

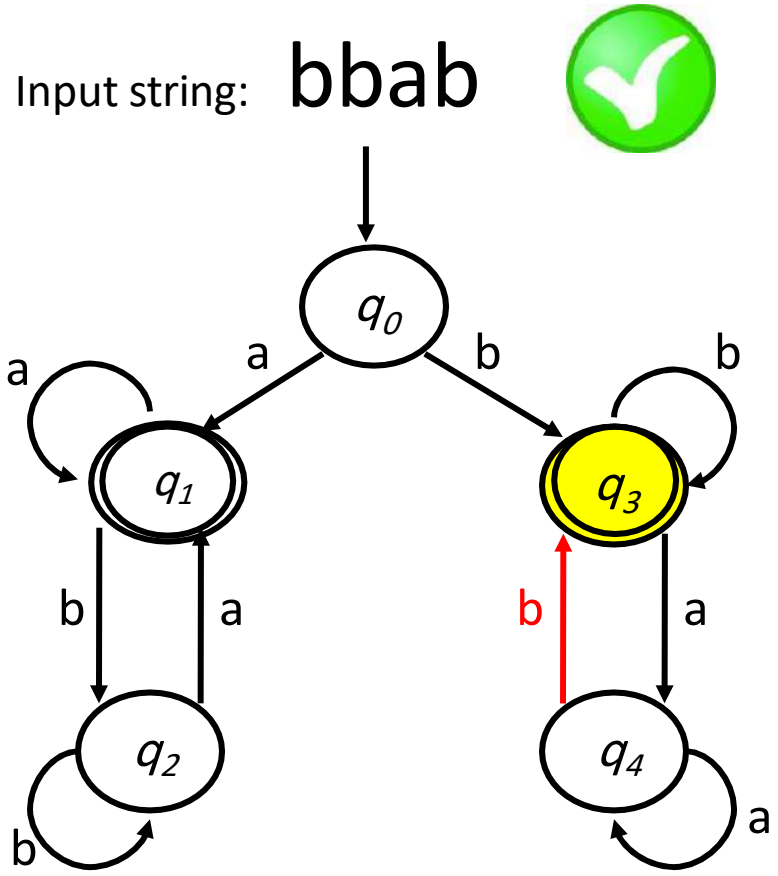
- Example3: M_3
- What is the language recognized by M_3?
- Some accepted strings = {a, aa, aba, .., b, bb,

Input string: bba



State Diagram for M_3

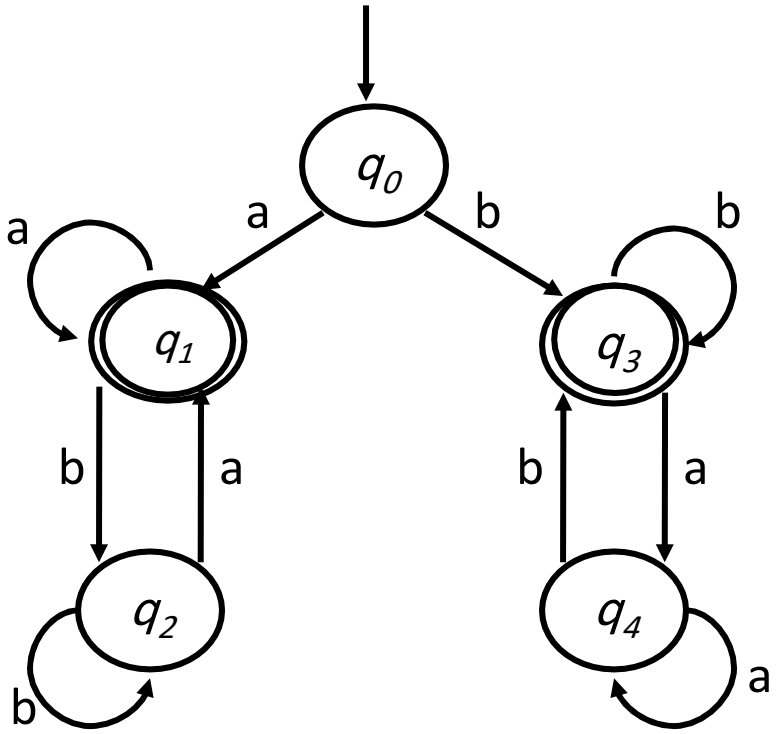
- Example3: M_3
- What is the language recognized by M_3?
- Some accepted strings = {a, aa, aba, .., b, bb,



State Diagram for M_3

- Example3: M_3
- What is the language recognized by M_3?
- Some accepted strings = {a, aa, aba, .., b, bb, bbab

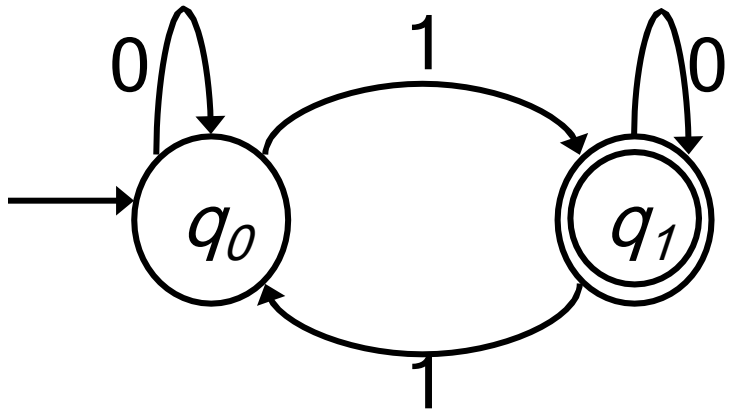
Machine 3



State Diagram for M_3

- Example3: M_3
- What is the language recognized by M_3?
- Some accepted strings = {a, aa, aba, .., b, bb, bbab, …}
- M_3 accepts strings that start and end with a, or start and end with b.
- $L(M_3)$ = {over (a, b)| strings that start and end with the same symbol}

Machine 4:

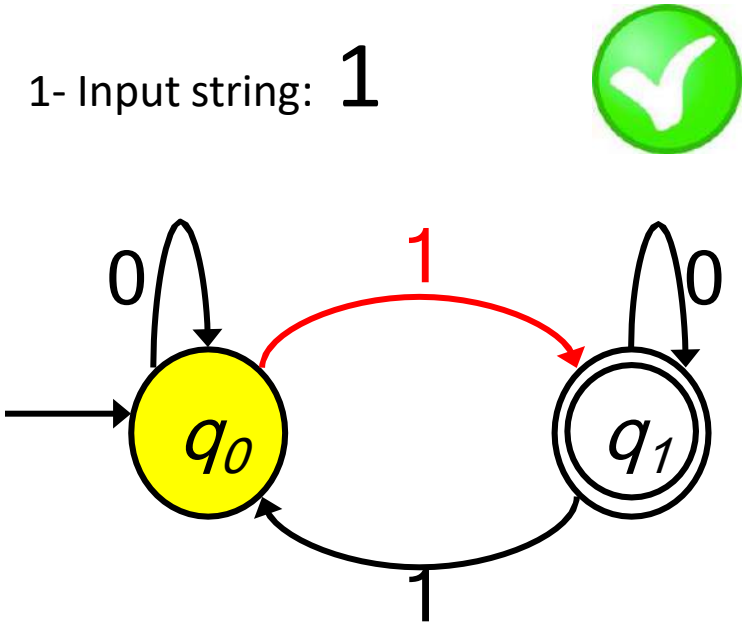


State Diagram for M_4

- What is the language recognized/accepted by M_4?

Machine 4:

1- Input string: 1

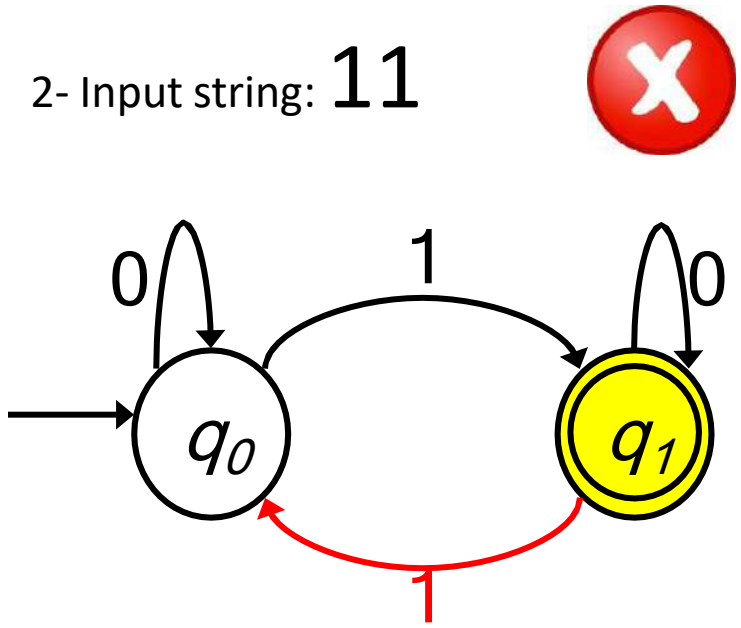


State Diagram for M_4

- What is the language recognized/accepted by M_4?
  - Accepted strings = {1,

Machine 4:

2- Input string: 11

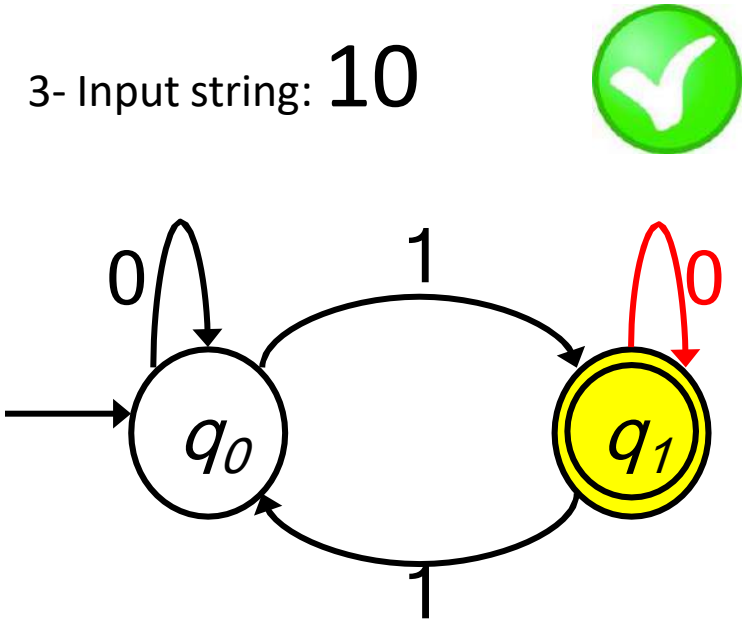


State Diagram for M_4

- What is the language recognized/accepted by M_4?
  - Accepted strings = {1,

Machine 4:

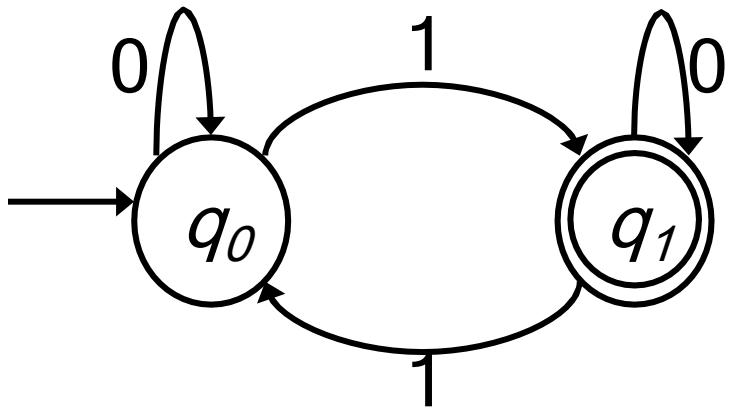
3- Input string: 10



State Diagram for M_4

- What is the language recognized/accepted by M_4?
  - Accepted strings = {1, 10

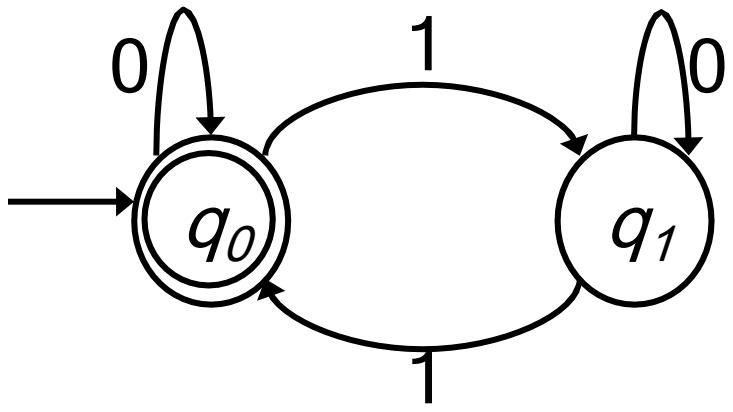
Machine 4:



State Diagram for M_4

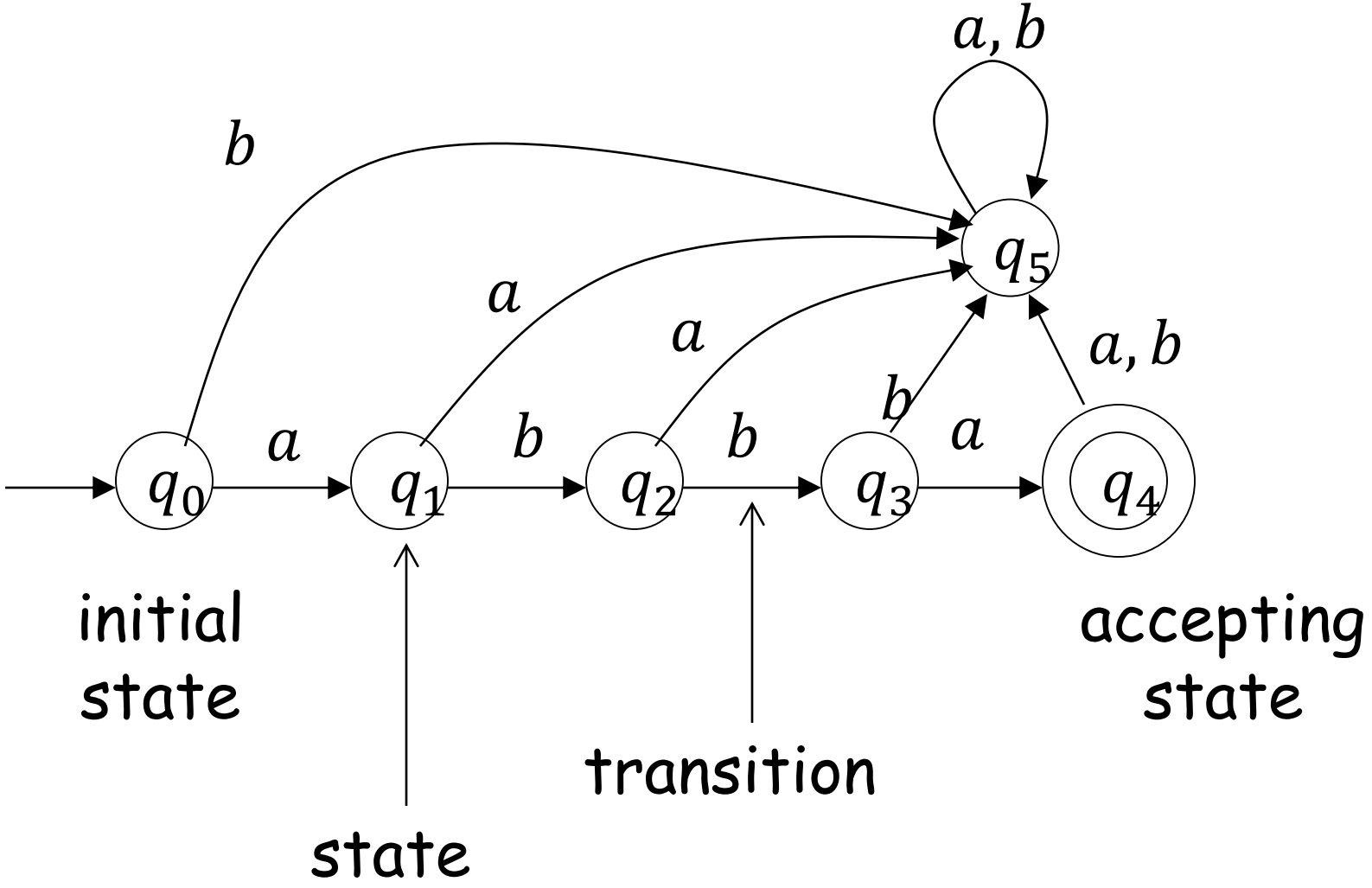
- What is the language recognized/accepted by M_4?
- Accepted strings = {1, 10, 111, 1011, 11111, 01, 010, …}
- Notice that all strings have odd number of 1's.
  - $L(M_4)$ = {strings with odd number of 1's}

Machine 5:

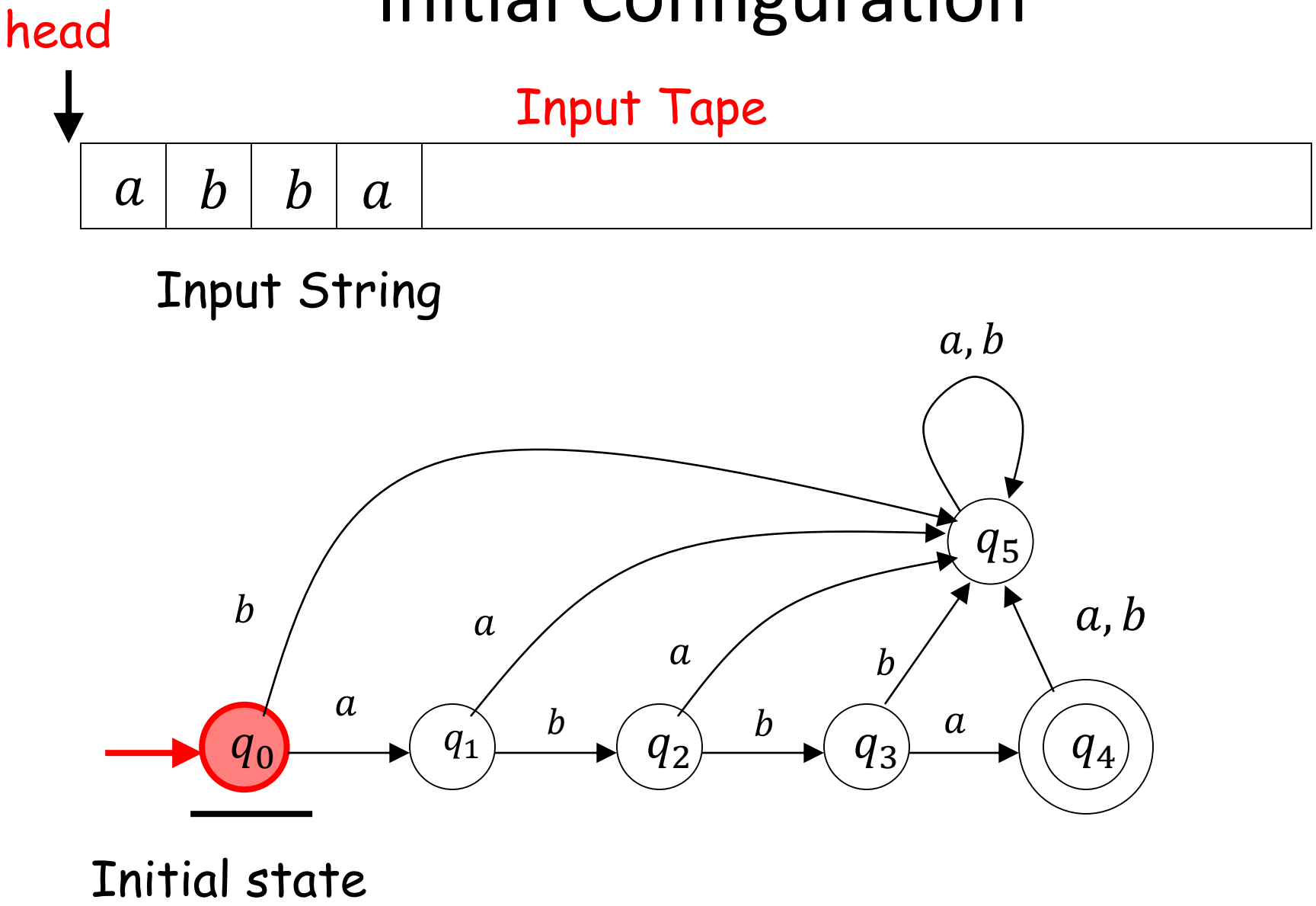


- What is the language recognized/accepted by M_5?
- Accepted strings = {ε, 0, 11, 101, 1111, 011,
- 0110, …}
- Notice that all strings have even number of 1's.
- L(M5) = {strings with even number of 1's, or the empty string}

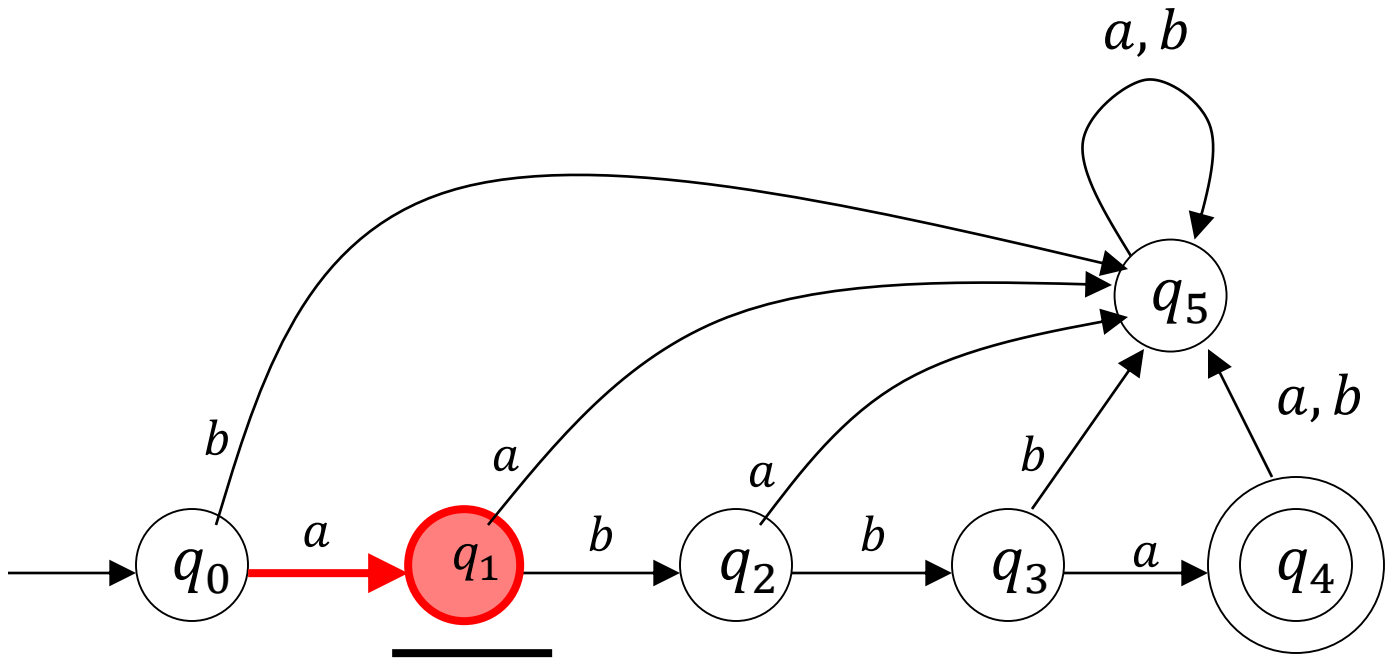
Transition Graph

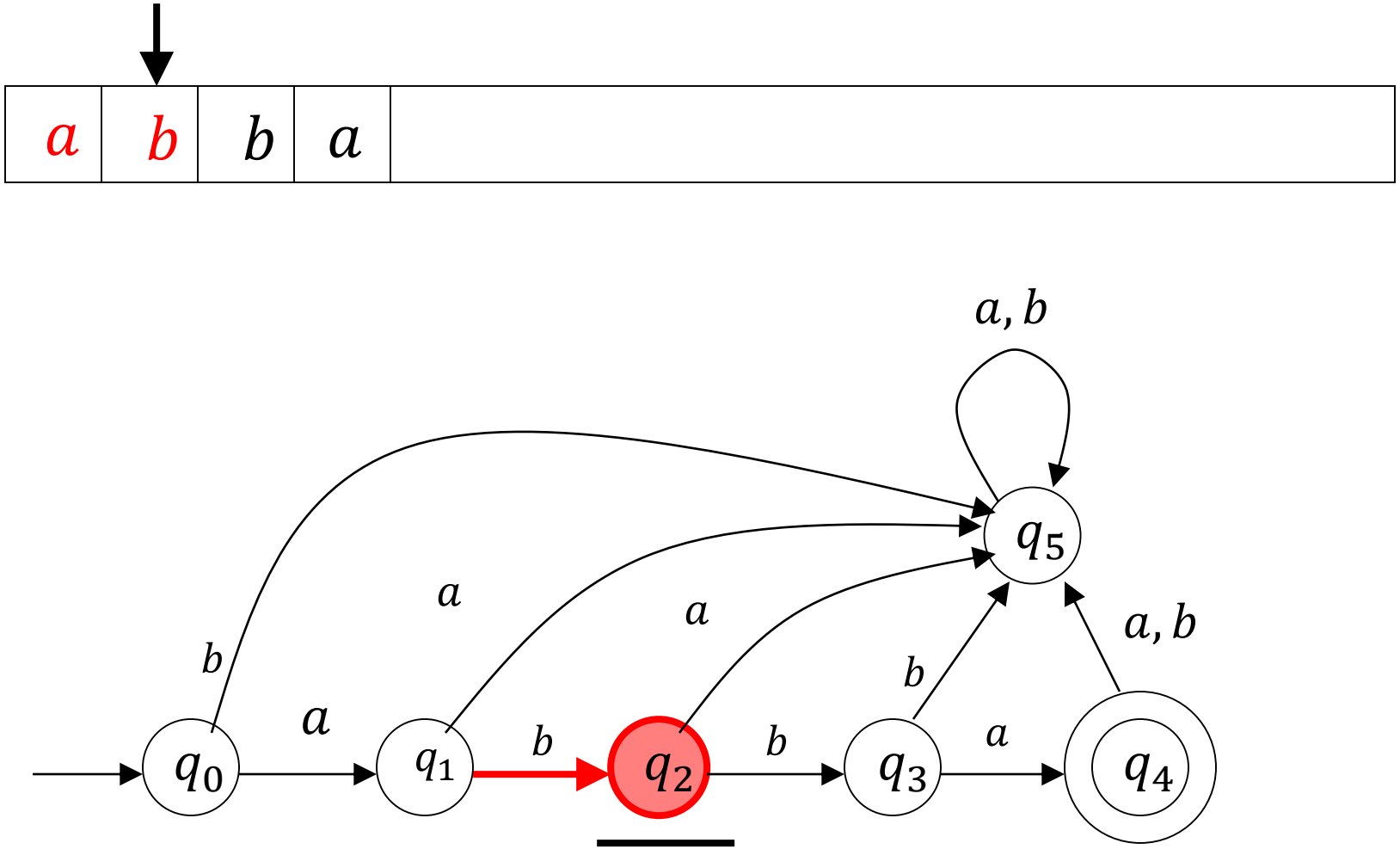


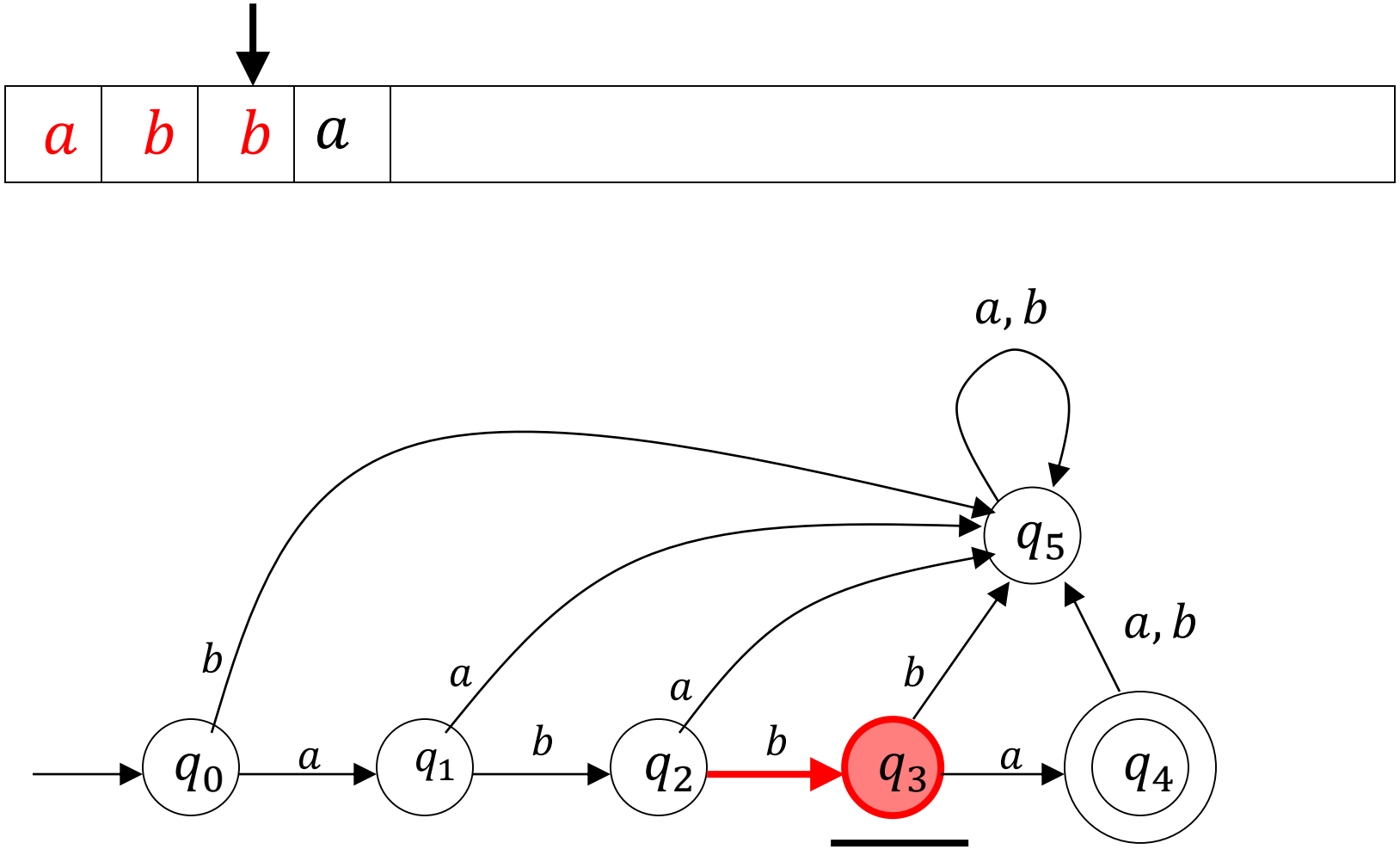
Initial Configuration

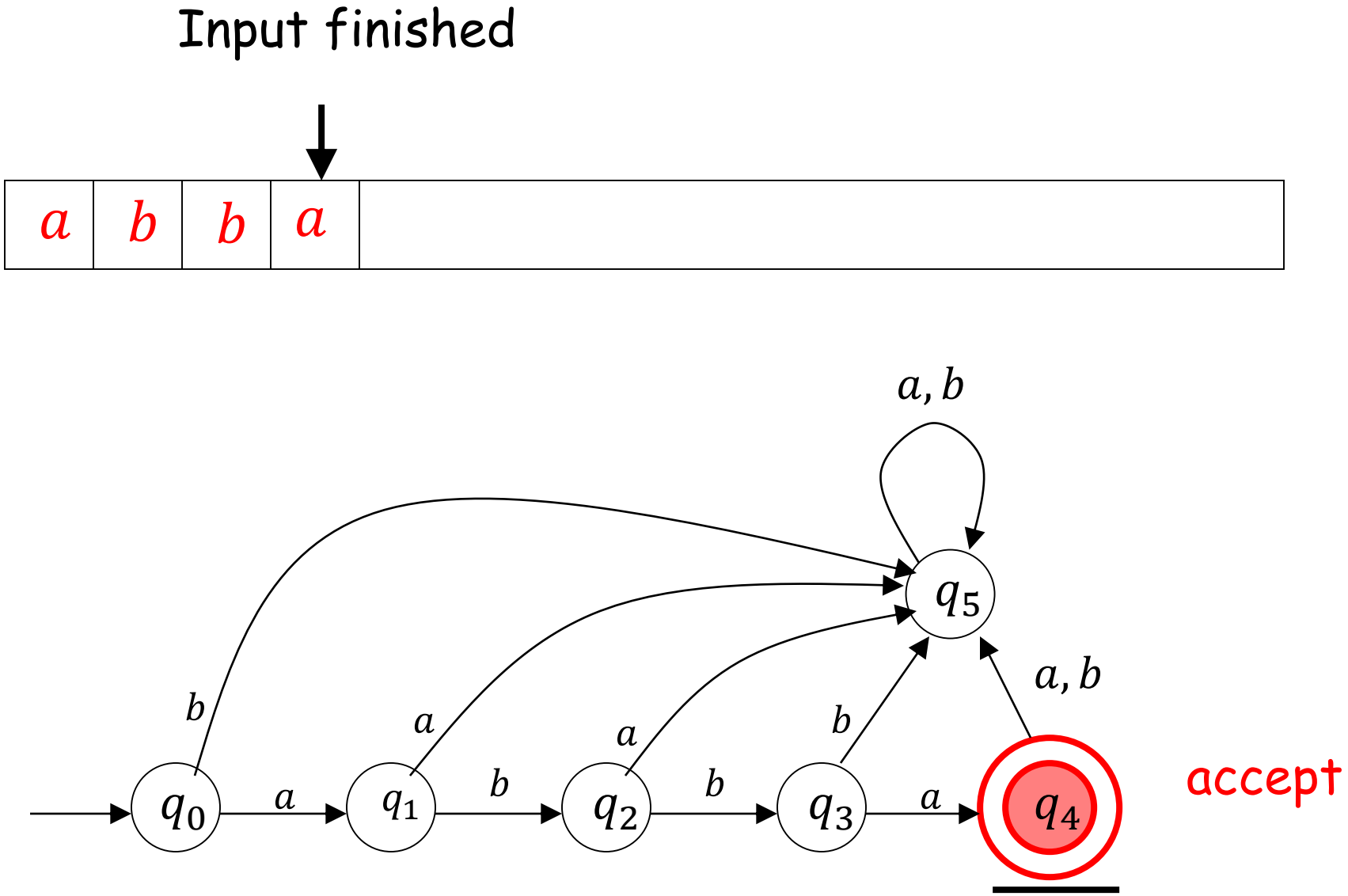


Scanning the Input



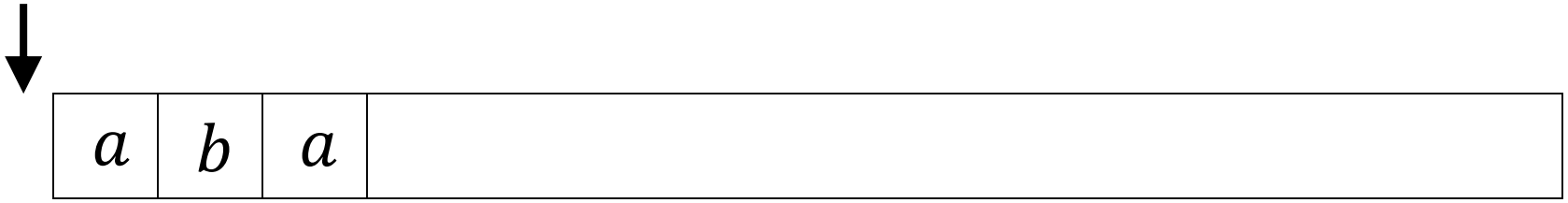

a
b
b
a
a, b
q5
a
a
b
a, b
b
a
b
b
a
q0
q1
q2
q3
q4


a
b
b
a
a, b
q5
b
a
a
b
a, b
q0
a
q1
b
q2
b
q3
a
q4

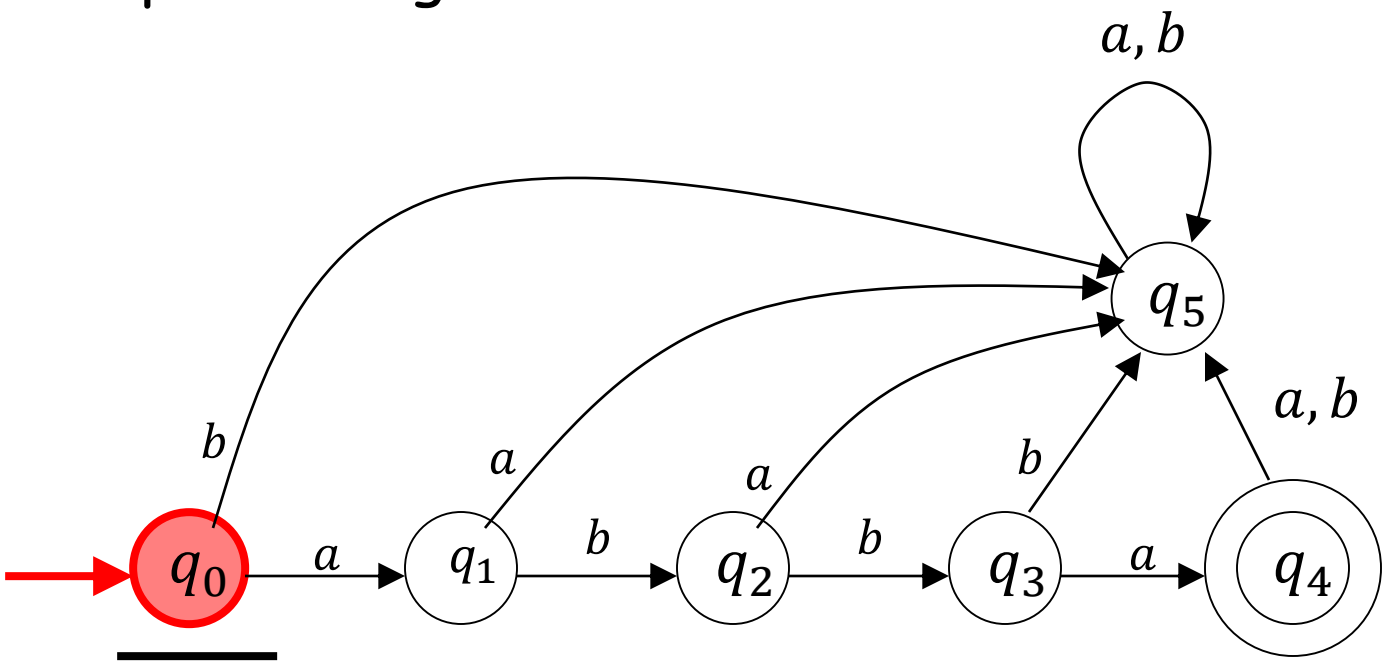


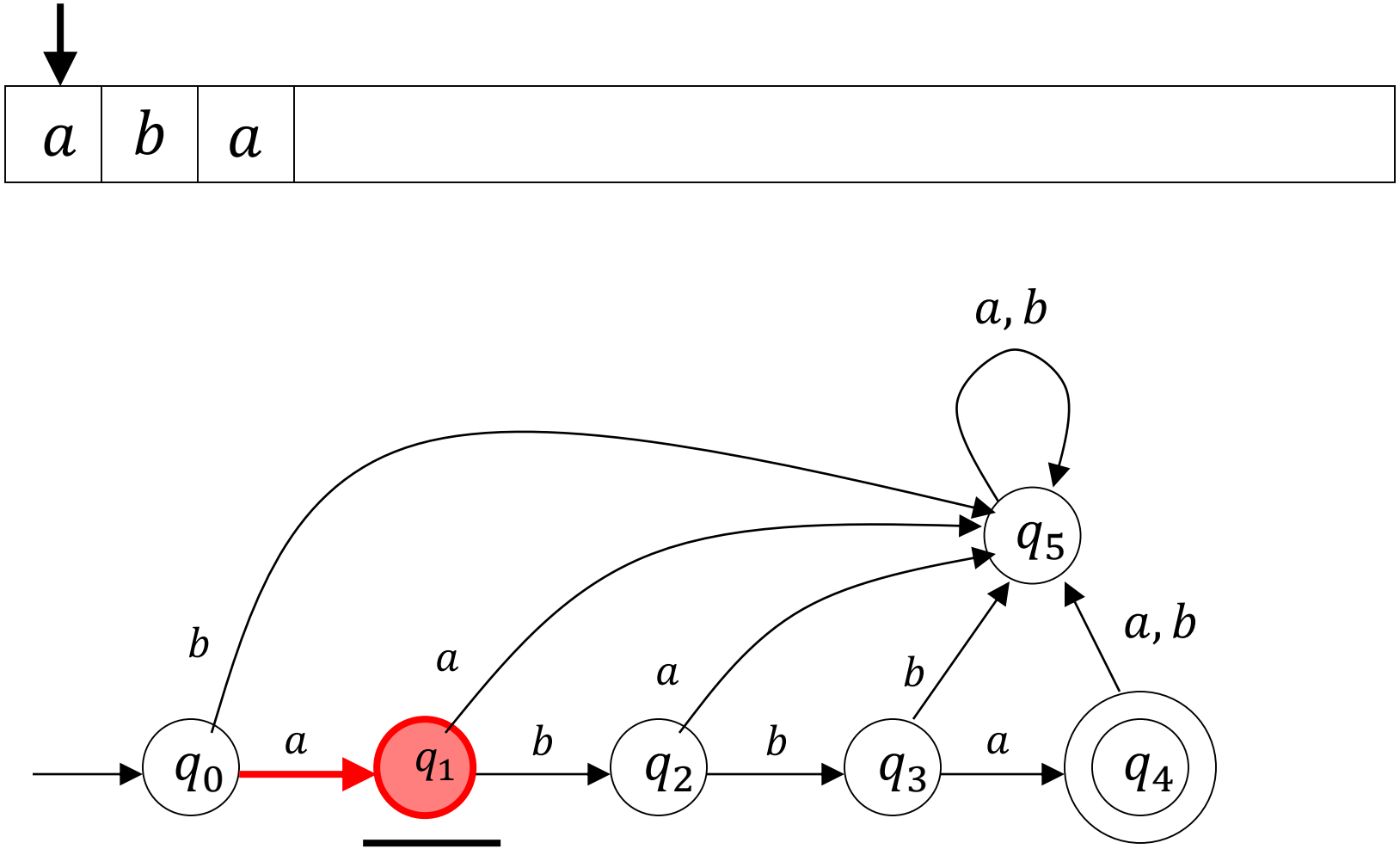
The last state determines the outcome

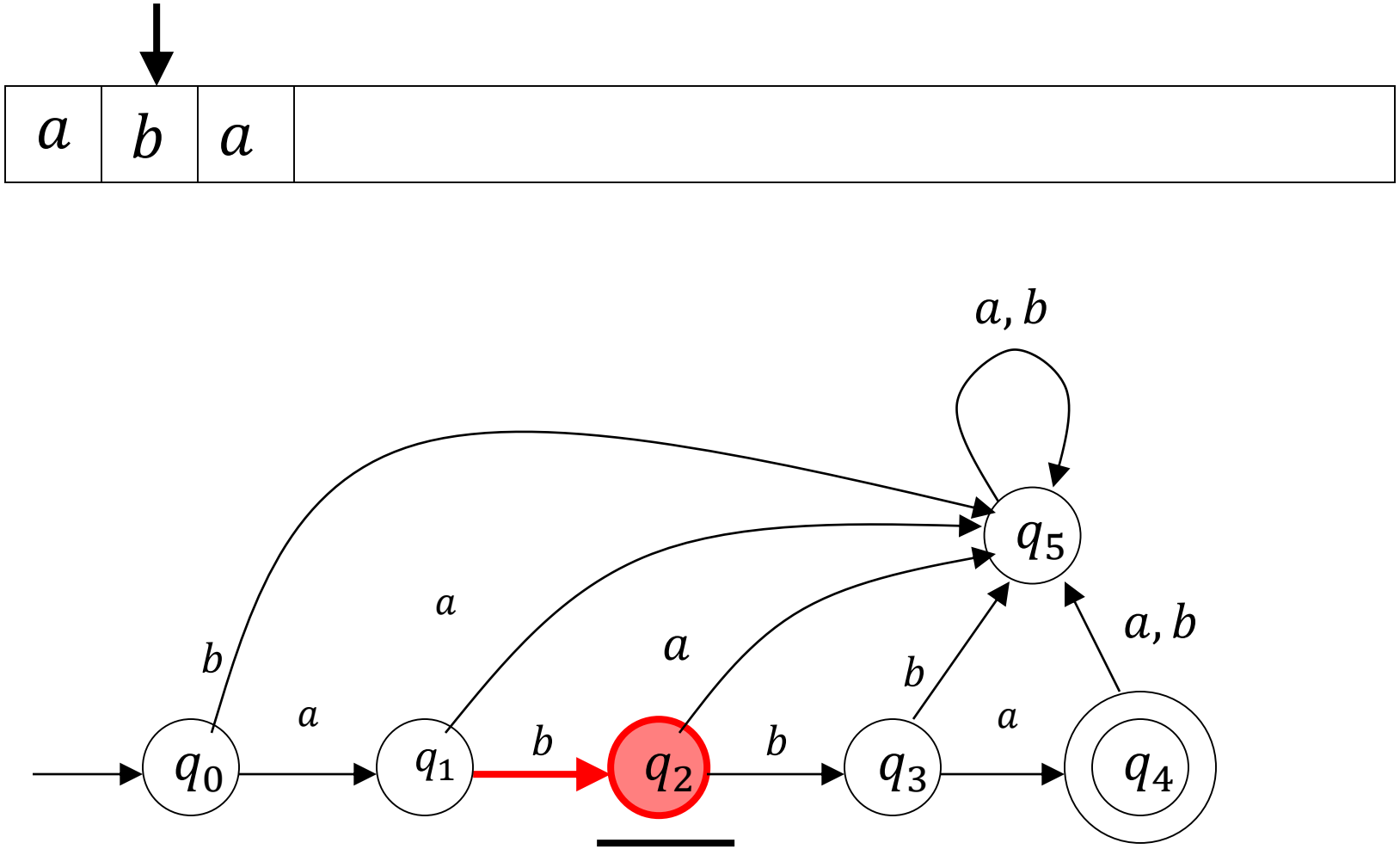
A Rejection Case



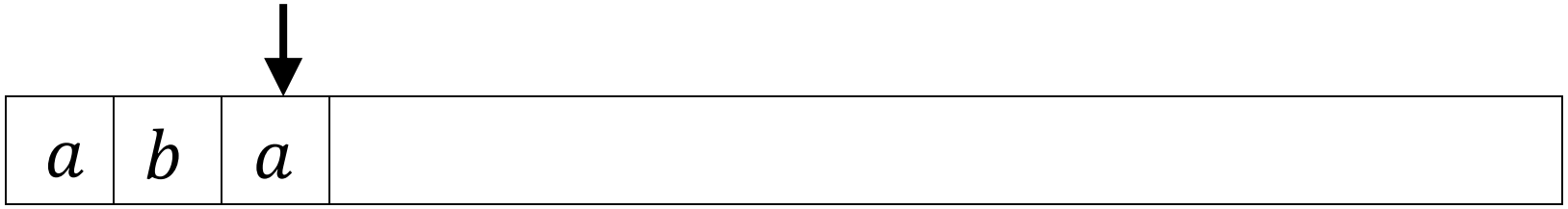
Input String

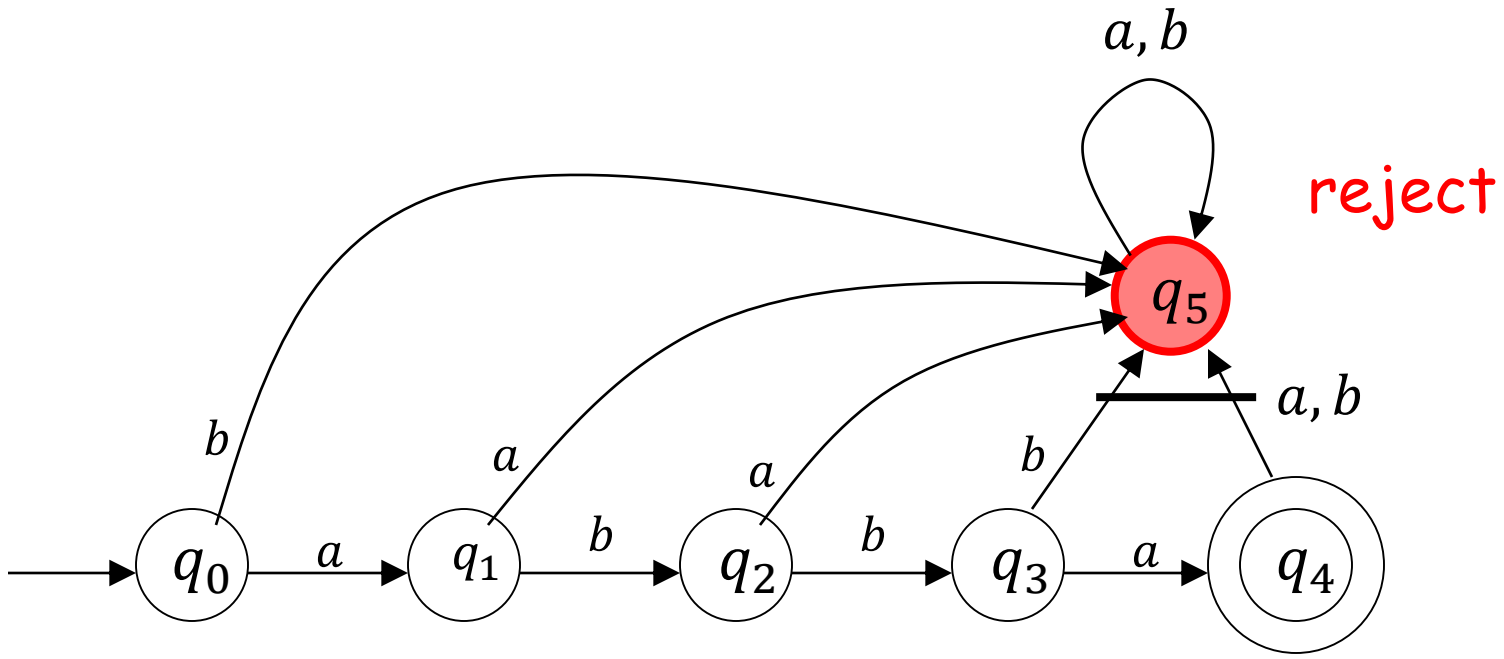



a
b
a
a, b
q5
b
a
a
b
a, b
a
b
b
a
q0
q1
q2
q3
q4


a
b
a
a, b
q5
b
a
a
a, b
b
a
q0
q1
b
q2
b
q3
a
q4

Input finished





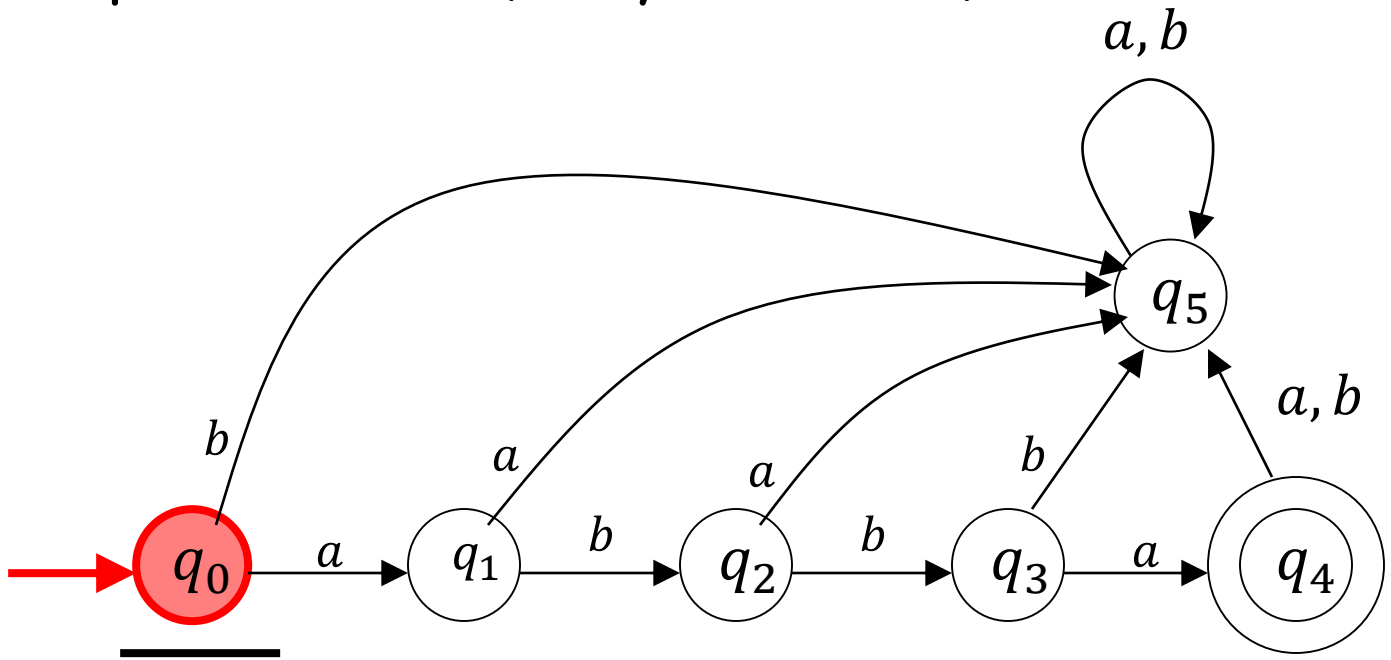
Last state determines the outcome

Another Rejection Case

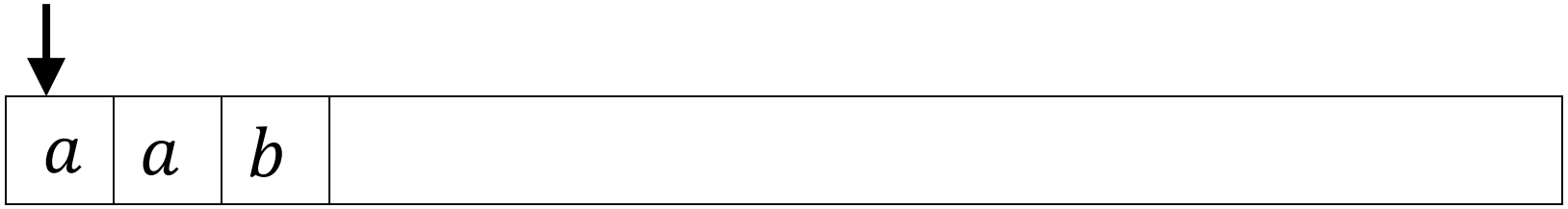
Tape is empty

(λ)

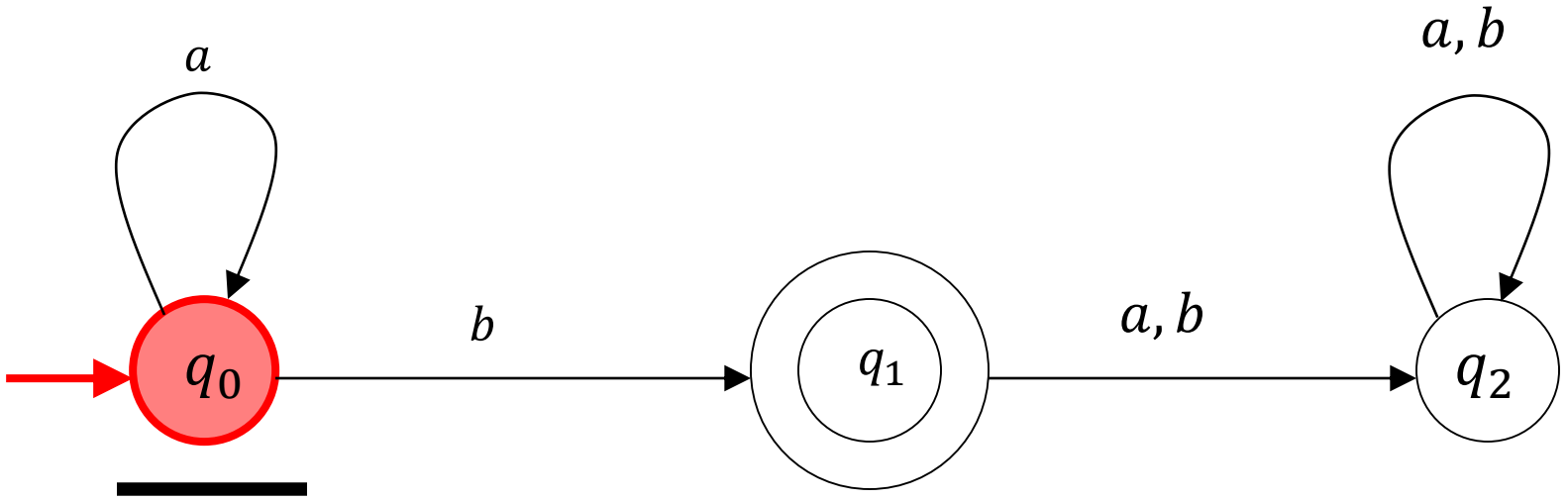
Input Finished (no symbol read)

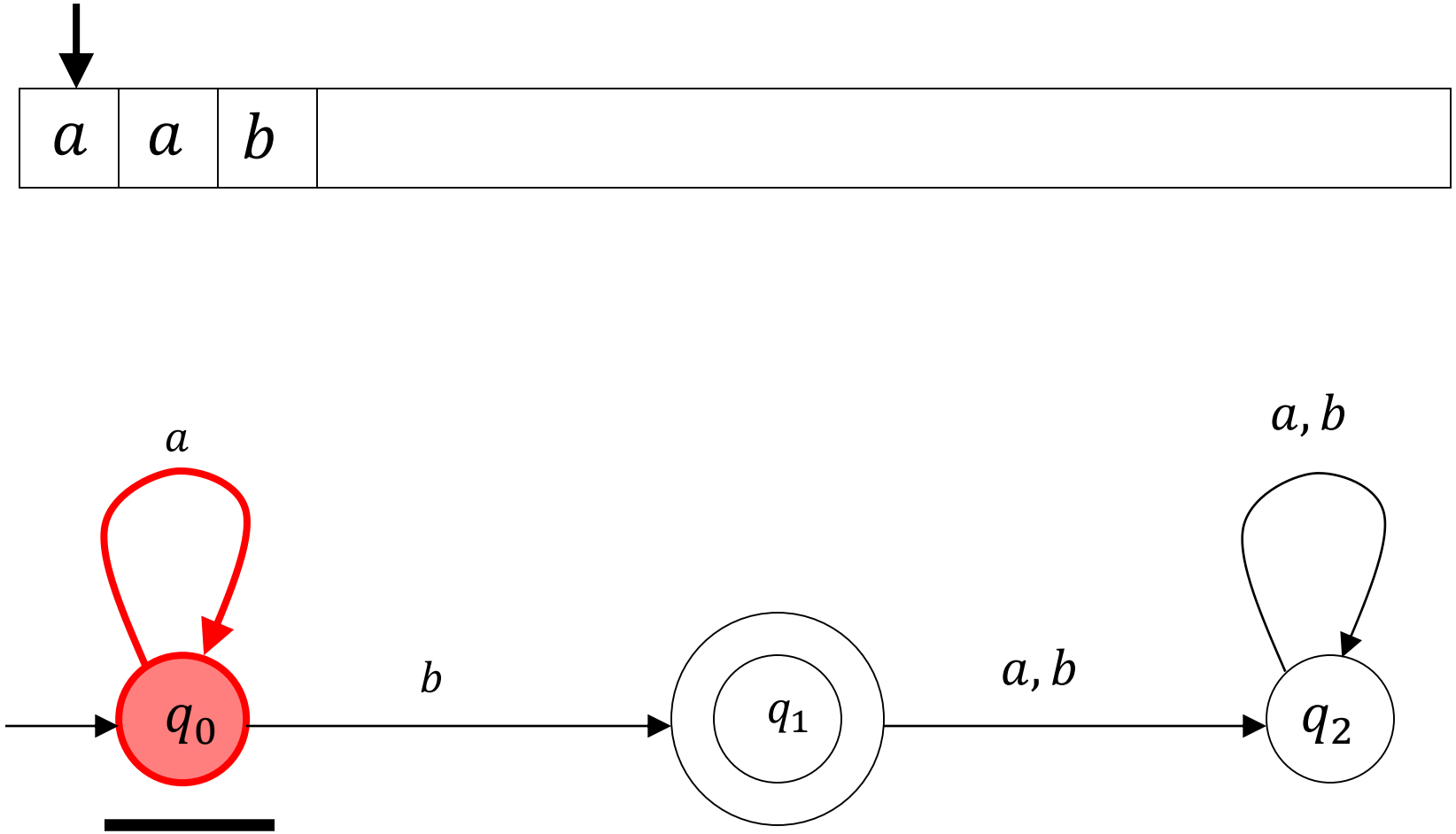


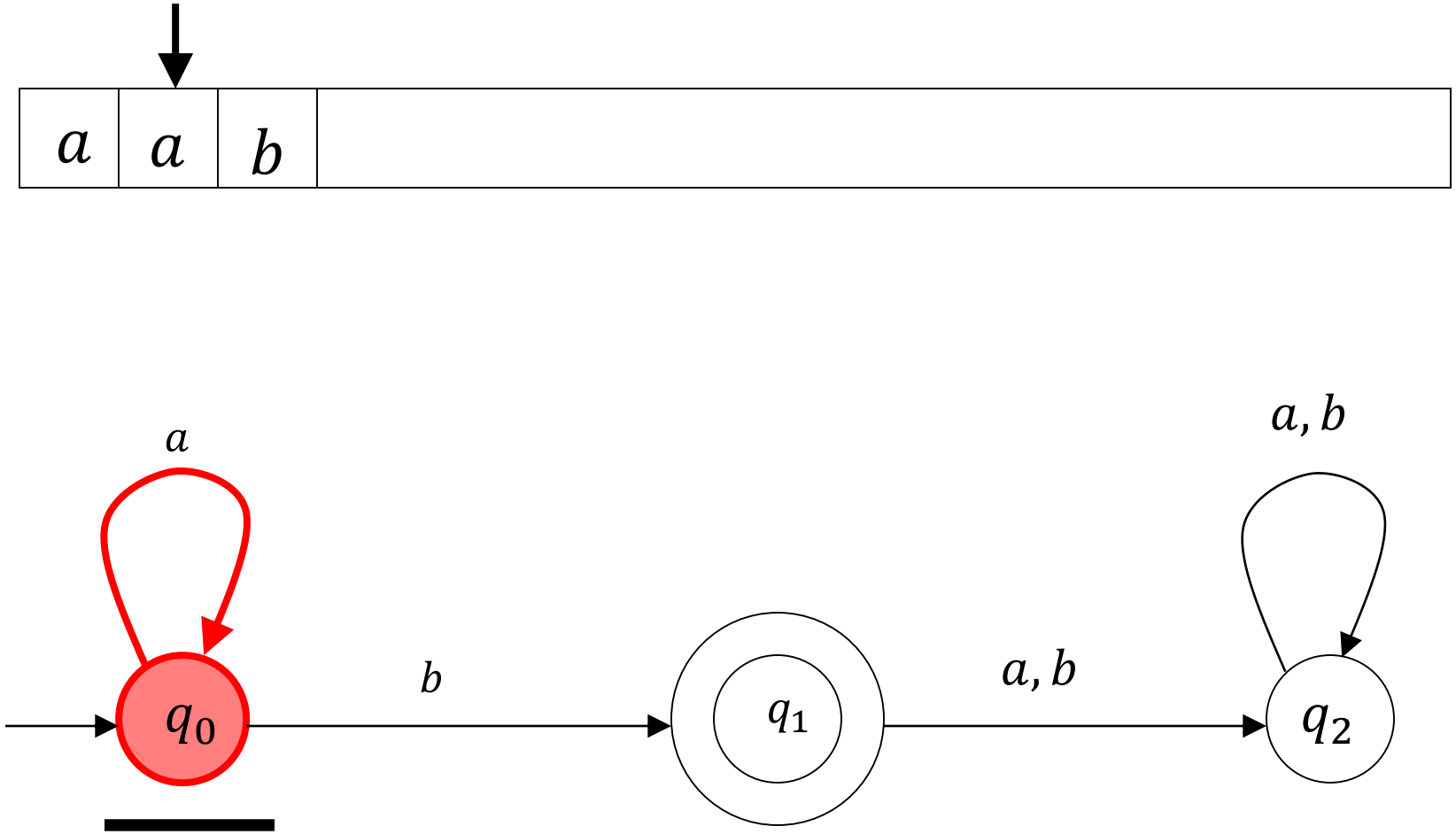
reject

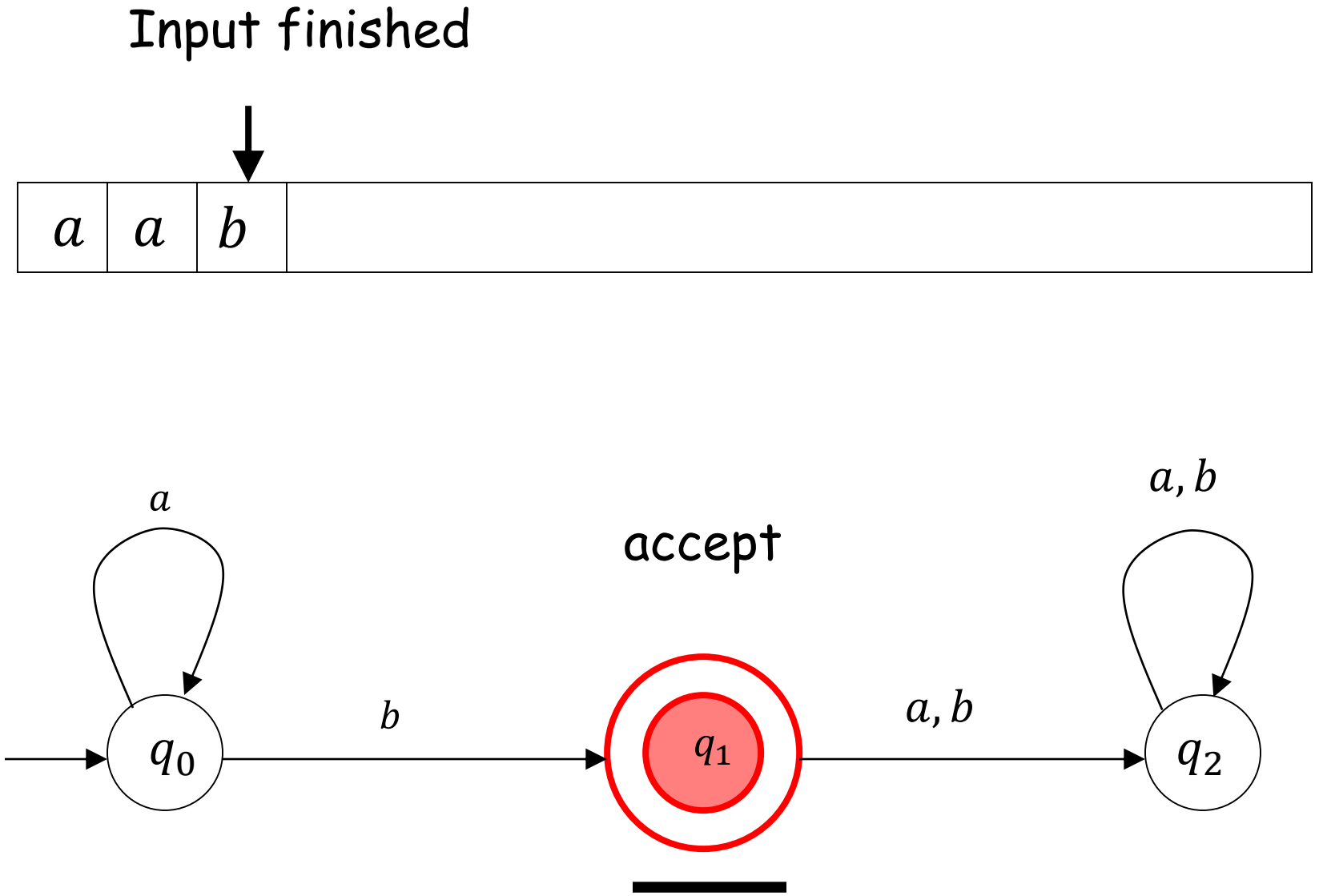


Input String

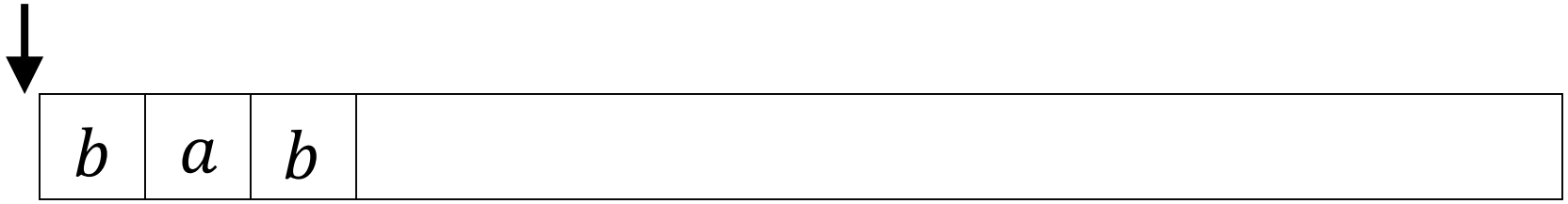



a
a
b
a
a, b
b
a, b
q_0
q_1
q_2

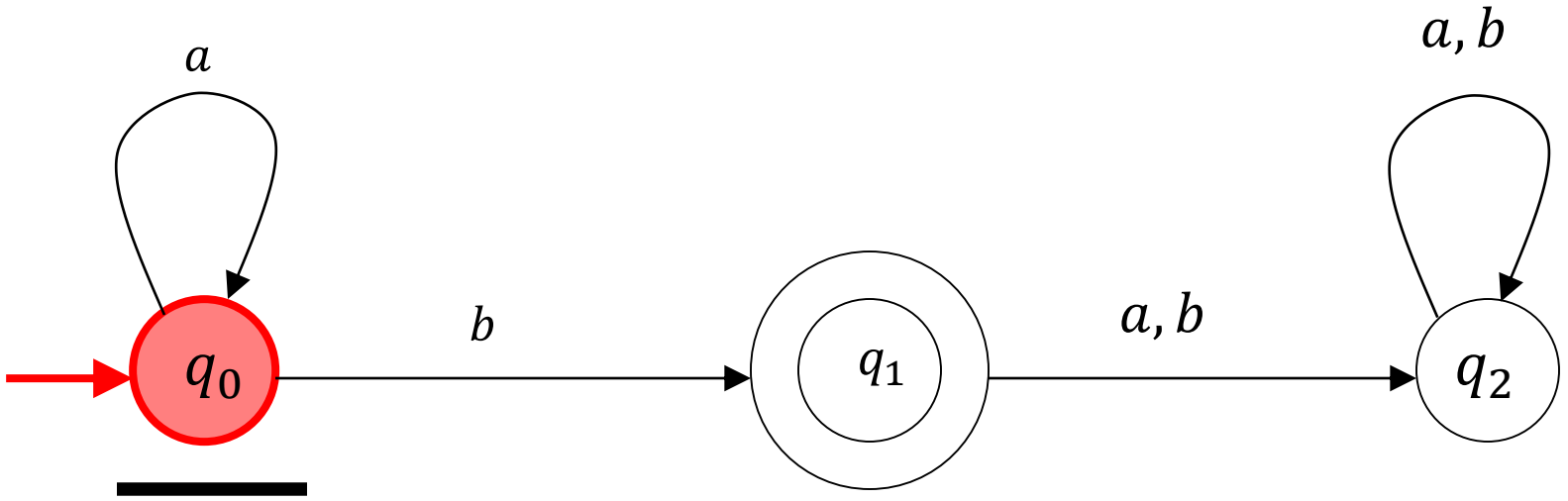

a
a
b
a
a, b
b
a, b
q_0
q_1
q_2

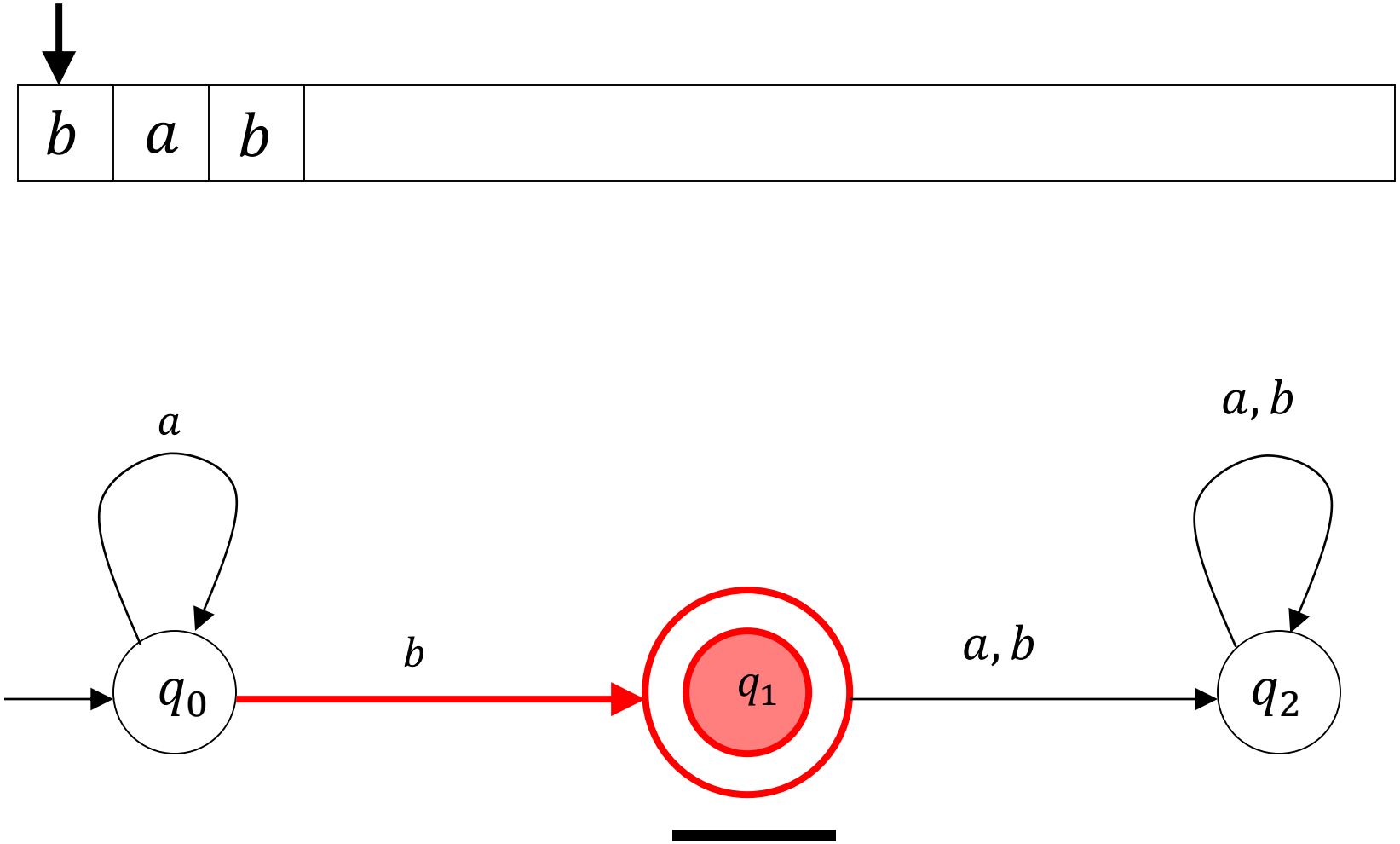

Input finished
a
a
b
a
accept
a, b
b
a, b
q0
q1
q2

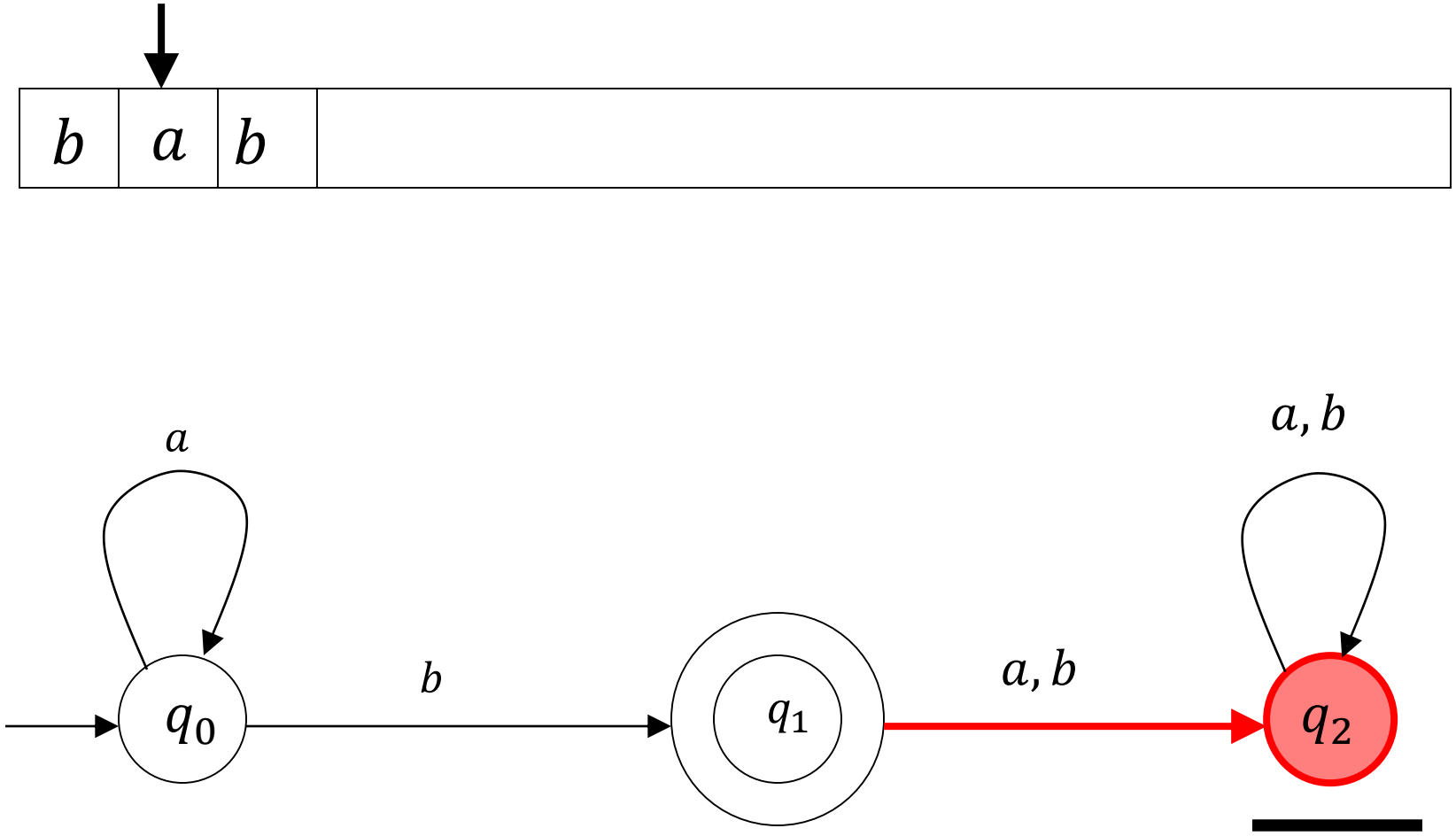
A rejection case

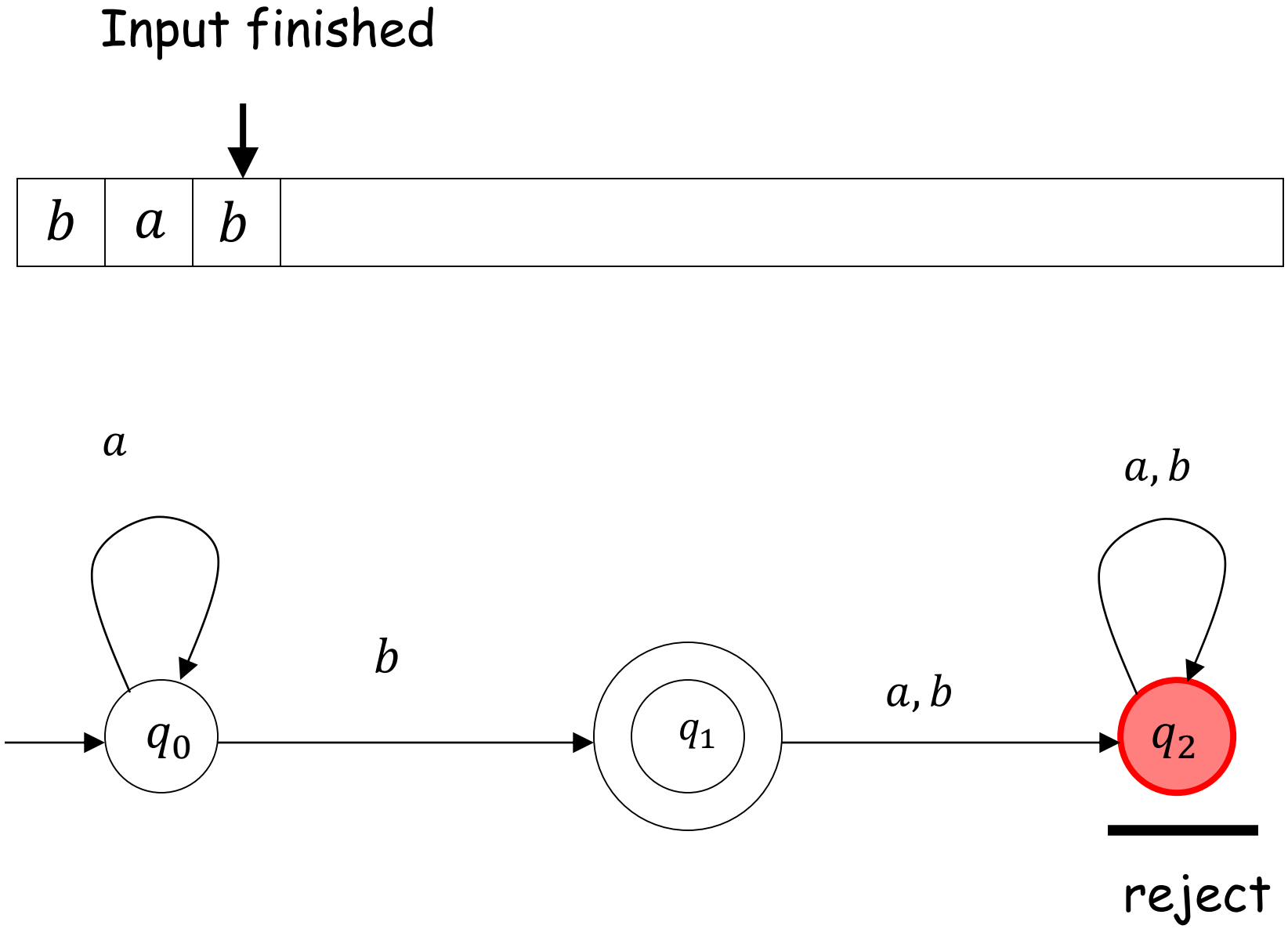


Input String

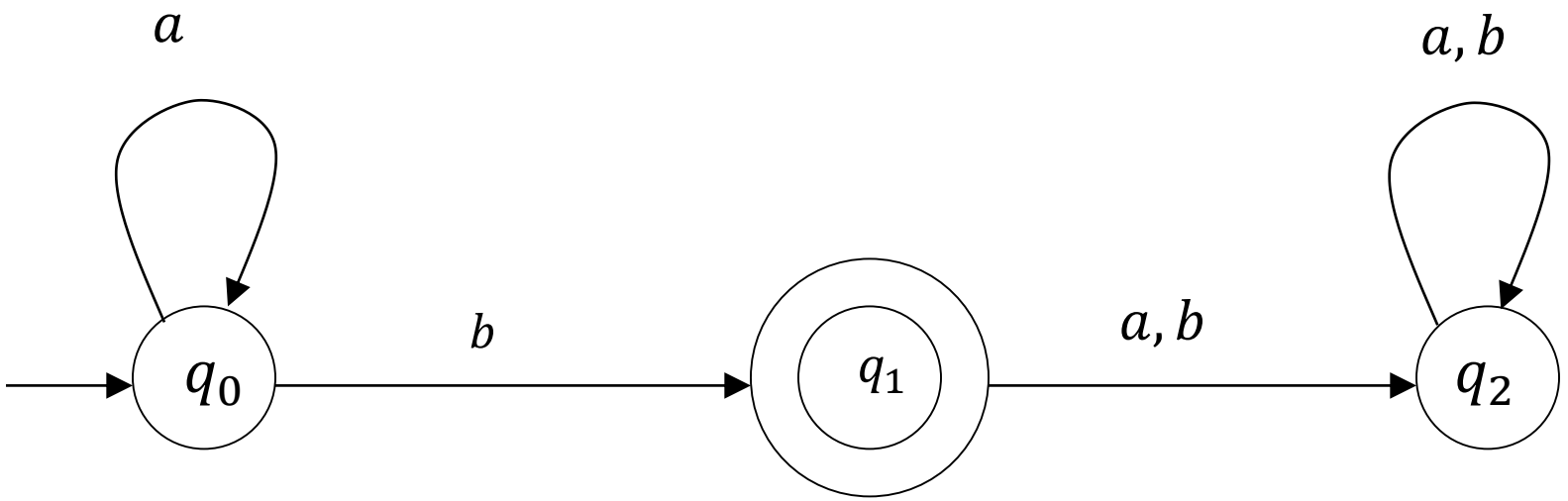



b
a
b
a
a, b
b
a, b
q_0
q_1
q_2


b
a
b
a
a, b
b
a, b
q_0
q_1
q_2

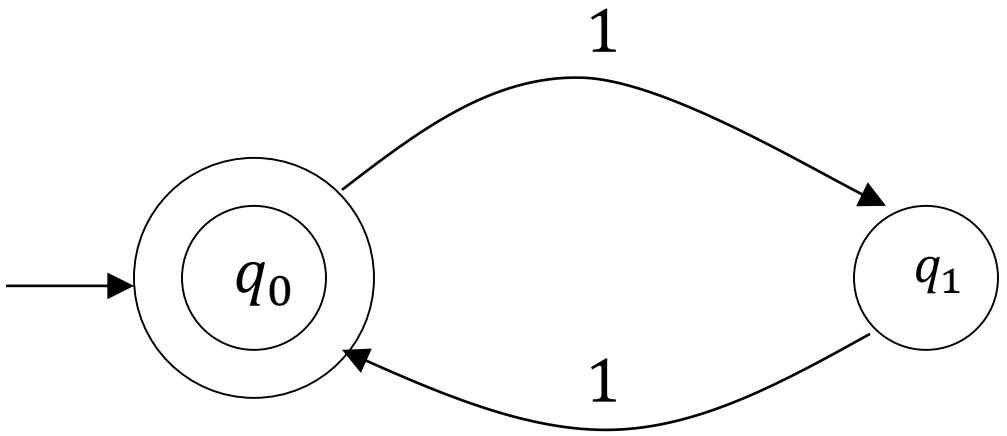

Input finished
b
a
b
a
a, b
b
a, b
q_0
q_1
q_2
reject

Language Accepted: $L = \{a^n b : n \geq 0\}$



Another Example

Alphabet: $\Sigma = \{1\}$



Language Accepted:

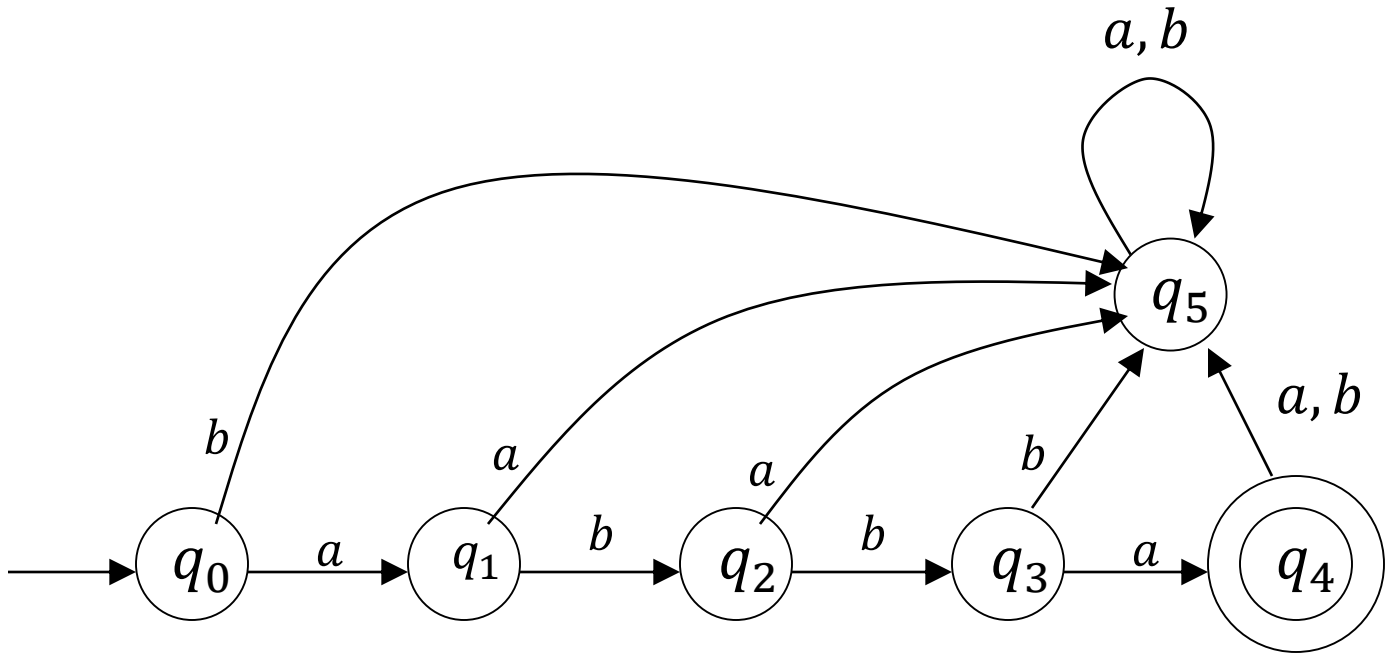
$$EVEN = \{x : x \in \Sigma^* \text{ and } x \text{ is even}\}$$

$$= \{\lambda,\ 11,\ 1111,\ 111111,\ \ldots\}$$

Set of States Q

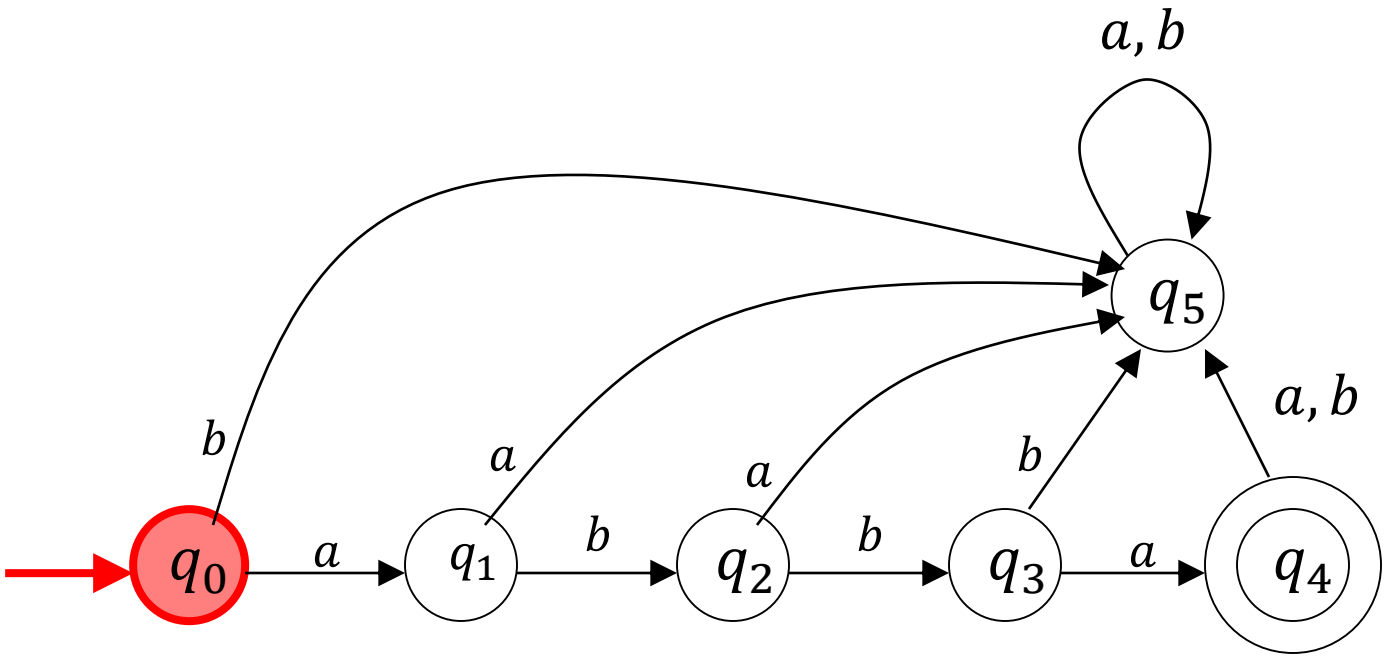
Example

$$Q = \{q_0, q_1, q_2, q_3, q_4, q_5\}$$



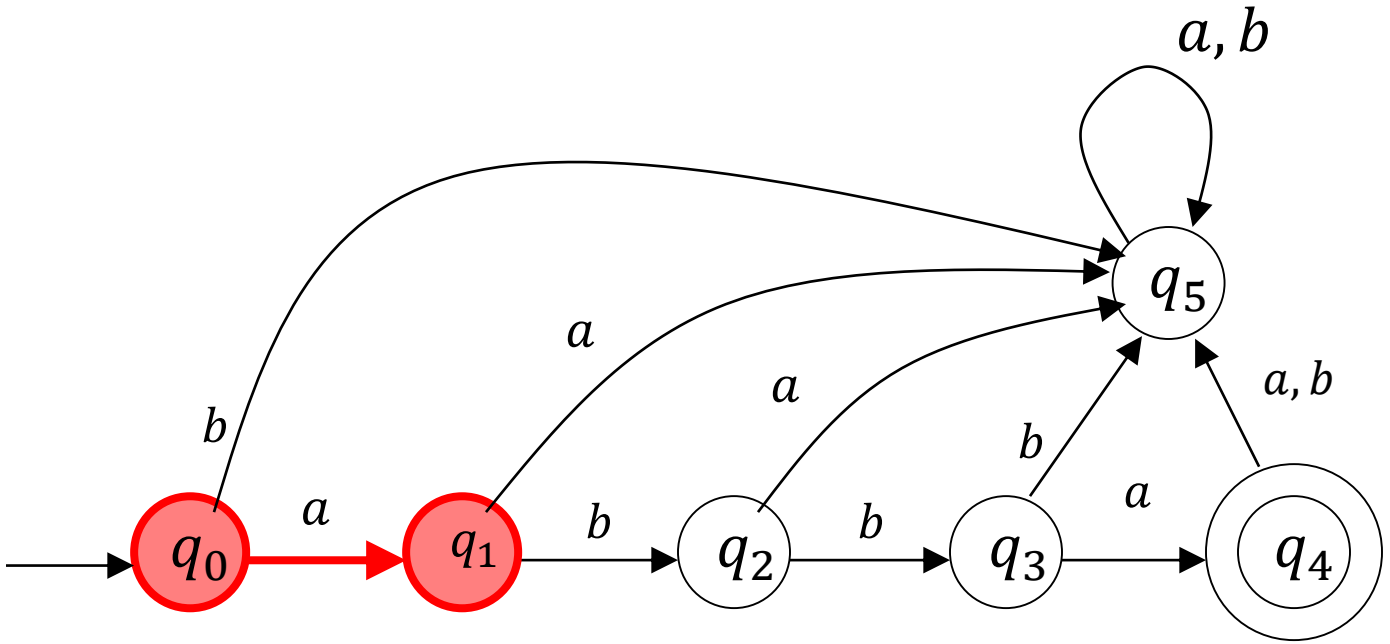
Initial State q_0

Example



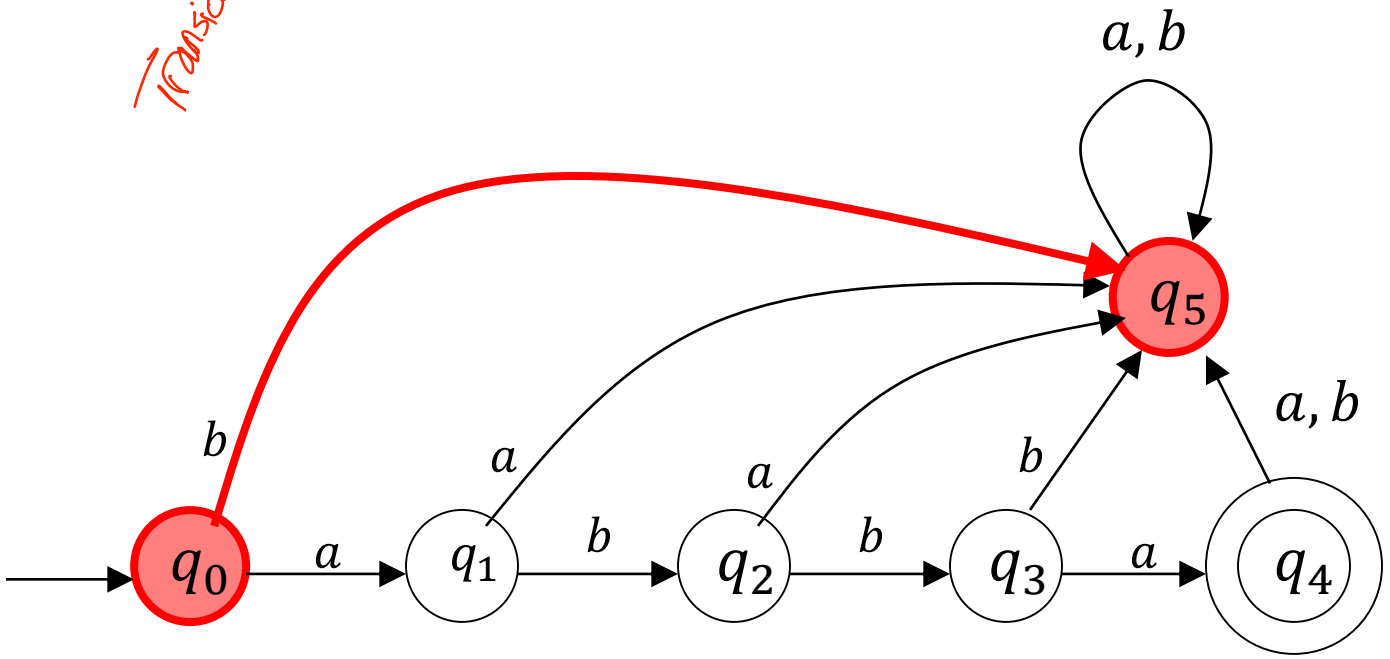
Example:

$$\delta(q_0, a) = q_1$$

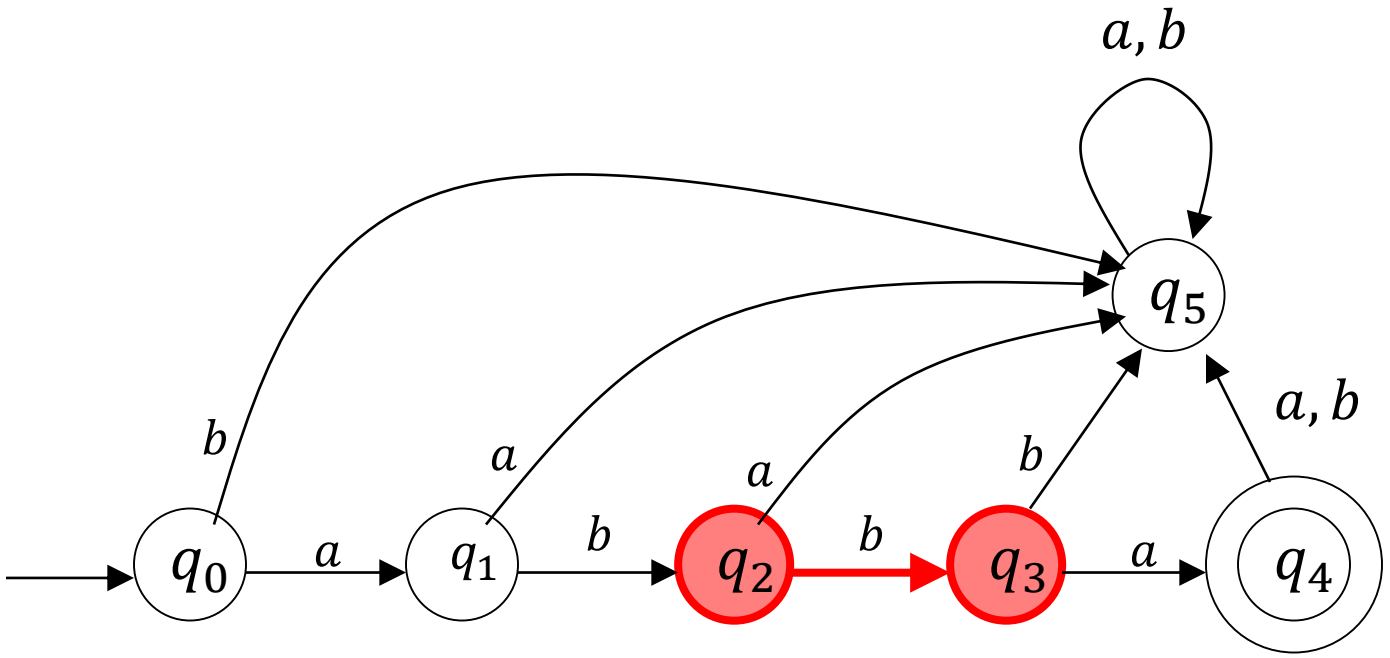


$$\delta(q_0, b) = q_5$$

Transition ← تنشيط



$$\delta(q_2, b) = q_3$$

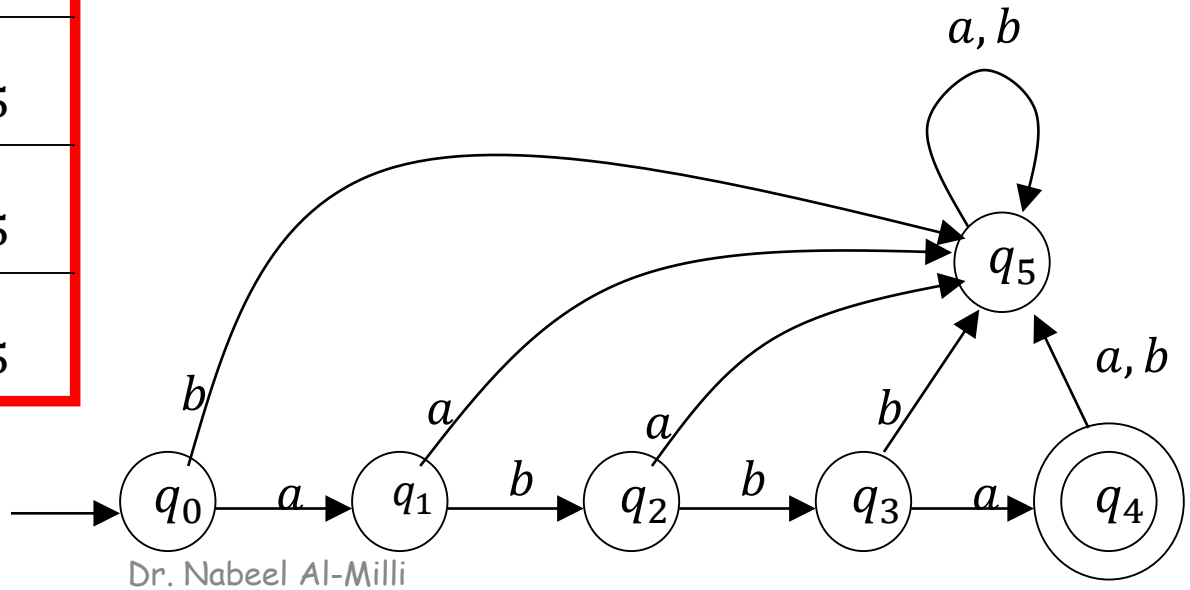


Transition Table for δ

symbols

states

δ	a	b
q_0	q_1	q_5
q_1	q_5	q_2
q_2	q_5	q_3
q_3	q_4	q_5
q_4	q_5	q_5
q_5	q_5	q_5



Extended Transition Function

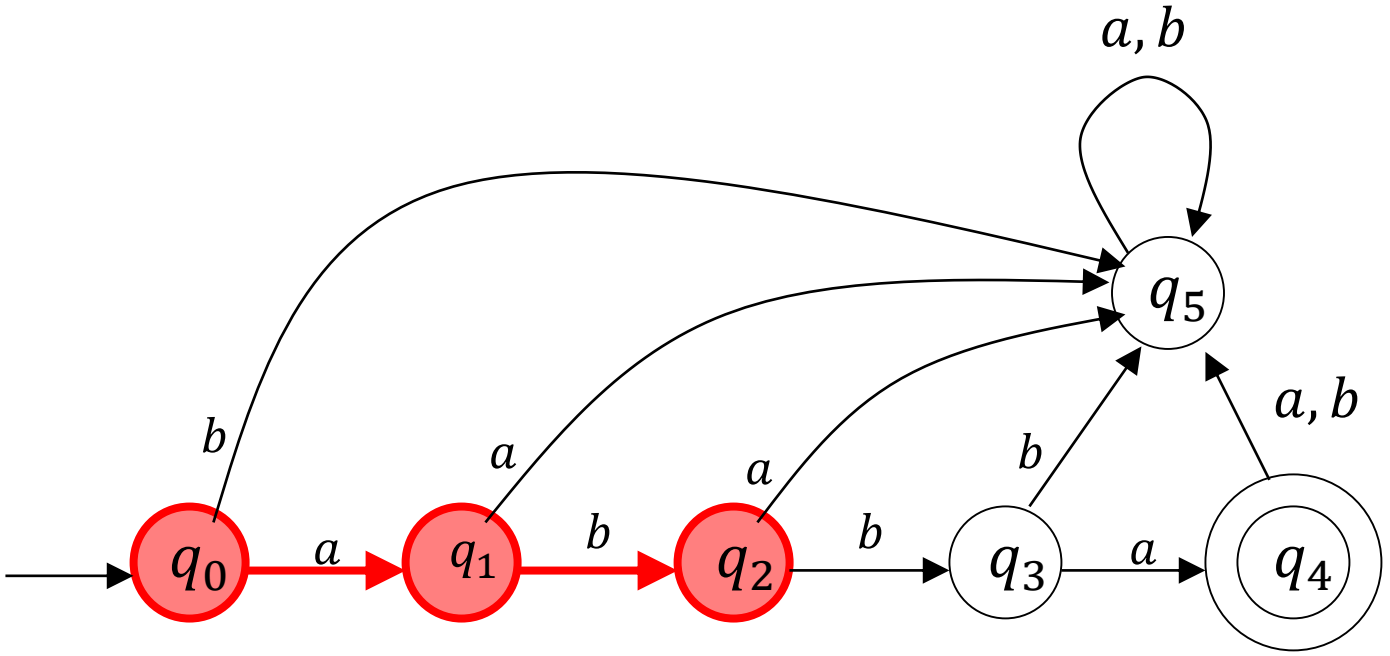
$$\delta^*: Q \times \Sigma^* \rightarrow Q$$

$$\delta^*(q, w) = q'$$

Describes the resulting state
after scanning string w from state q

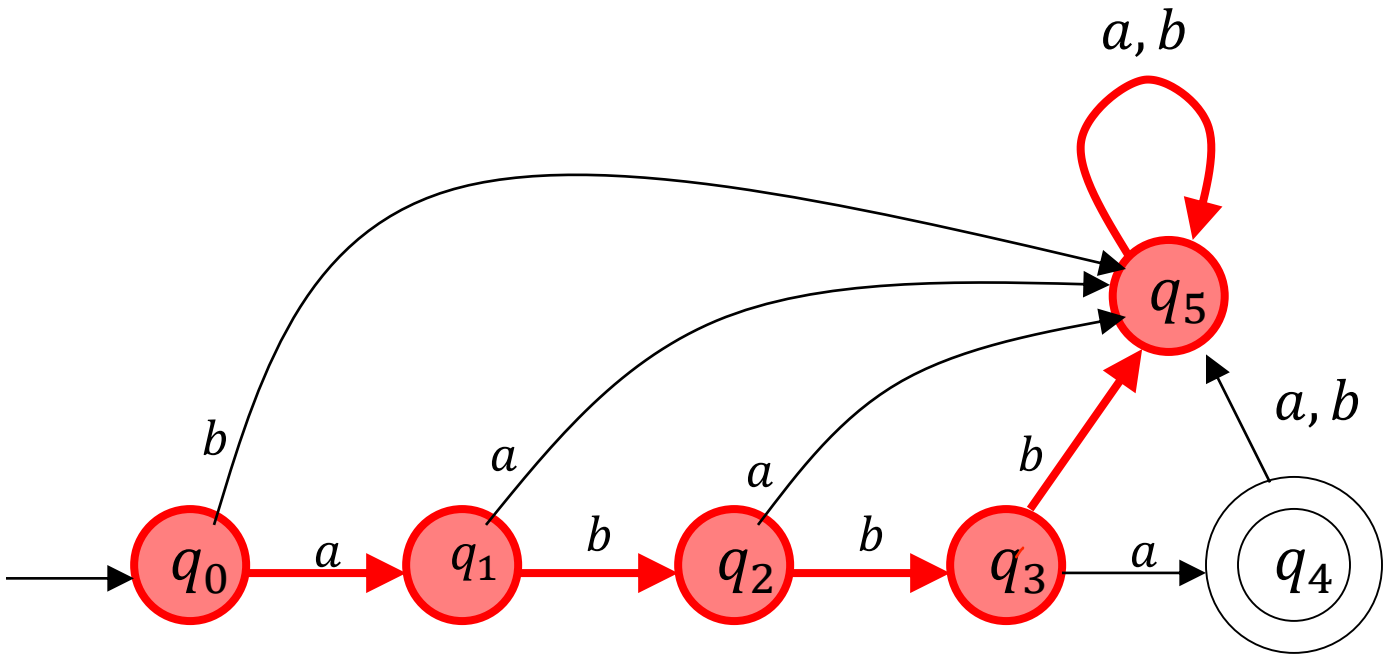
Example:

$$\delta^*(q_0, ab) = q_2$$

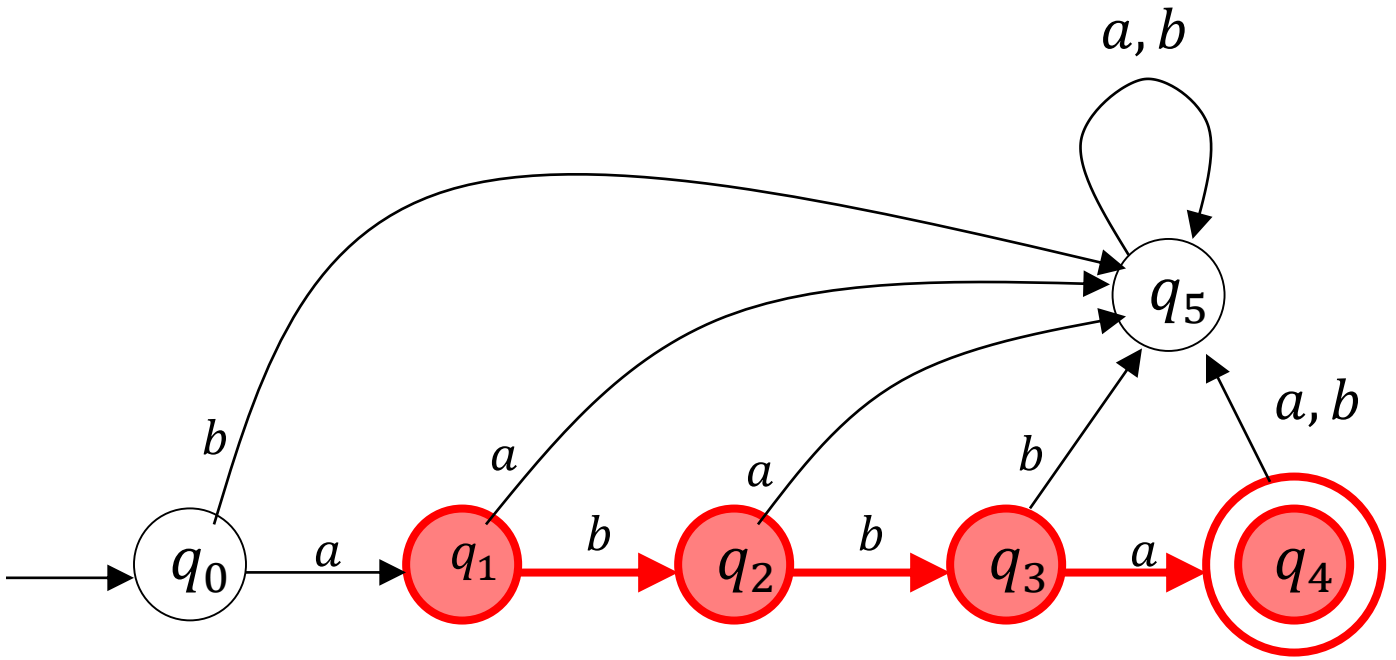


يجب تدريس التتبع

$$\delta^*(q_0, abbbaa) = q_5$$



$$\delta^*(q_1, bba) = q_4$$



Special case:

for any state q

يعني ما أتحرك ولا خطوة وأظل على نفس الـ state

$$\delta^*(q, \lambda) = q$$

Language Accepted by DFA

$L(M)$ **or** L_M

Accepted Language ← Language of Machine

Language accepted by DFA : M

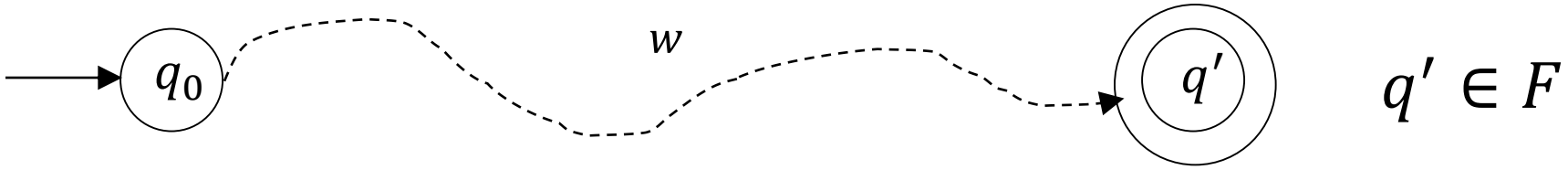
- it is denoted as $L(M)$ and contains all the strings accepted by M

- We also say that M recognizes $L(M)$

For a DFA $M = (Q, \Sigma, \delta, q_0, F)$

Language accepted by M :

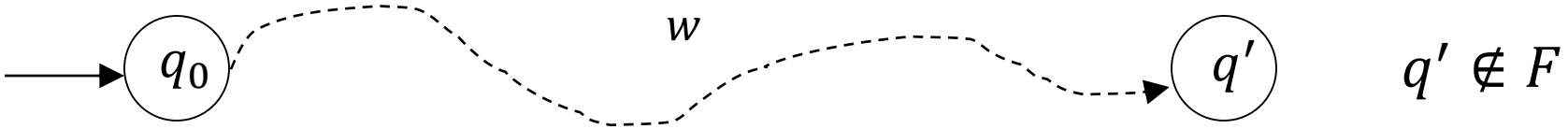
$$\boldsymbol{L(M) = \{w \in \Sigma^* : \delta^*(q_0, w) \in F\}}$$



$q' \in F$

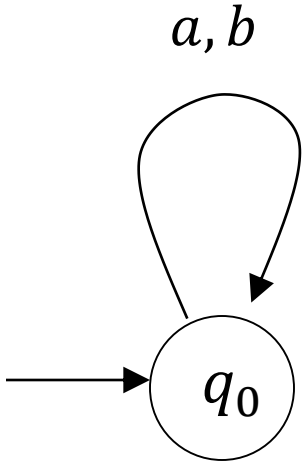
Language rejected by M :

$$\overline{L(M)} = \{w \in \Sigma^* : \delta^*(q_0, w) \notin F\}$$



More DFA Examples

$$\Sigma = \{a, b\}$$

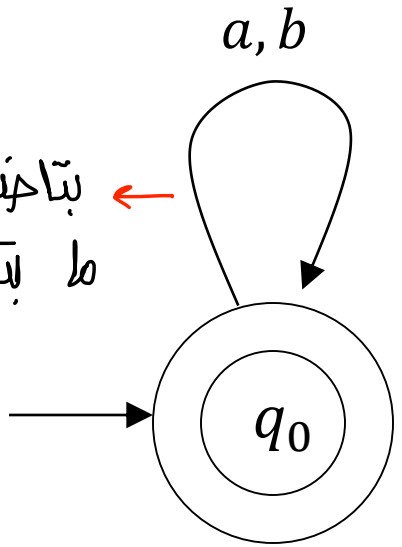


$\emptyset$

F عند ما تيجي ← $L(M) = \{\ \}$

F → Final state

Empty language

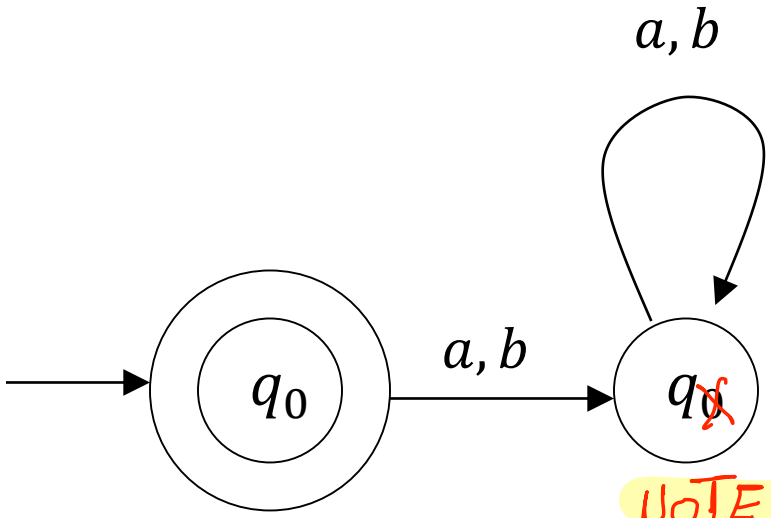


لأنه البداية نفسها النهاية

$L(M) = \Sigma^*$ ={λ,a,b,aa,ab,...}

All strings

$$\Sigma = \{a, b\}$$



NOTE → عدلو الـ q_0 هون
خلوها تصير q_1

البداية من نفسها النهاية ← لأنه ما في اي شي رح يرجعني على F

$$L(M) = \{\lambda\}$$

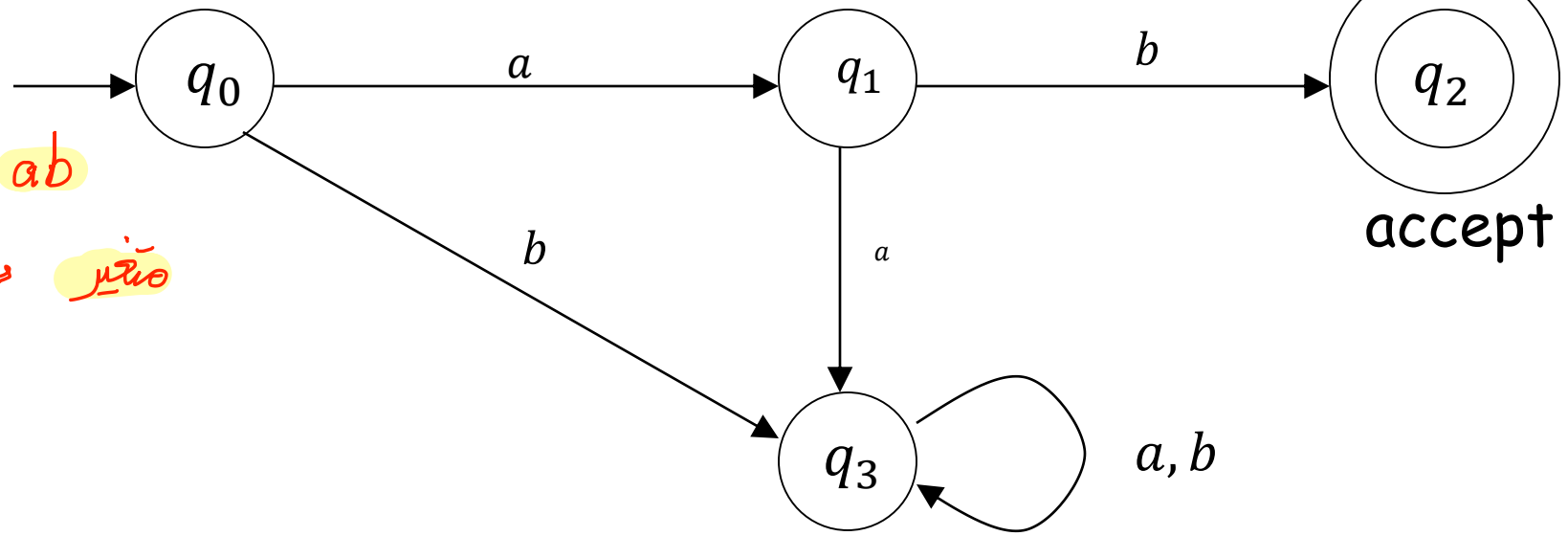
Language of the empty string

$$\Sigma = \{a, b\}$$

$L(M)$ = { all strings with **prefix** ab }

L(M)={abw: w∈Σ*, Σ={a,b} }

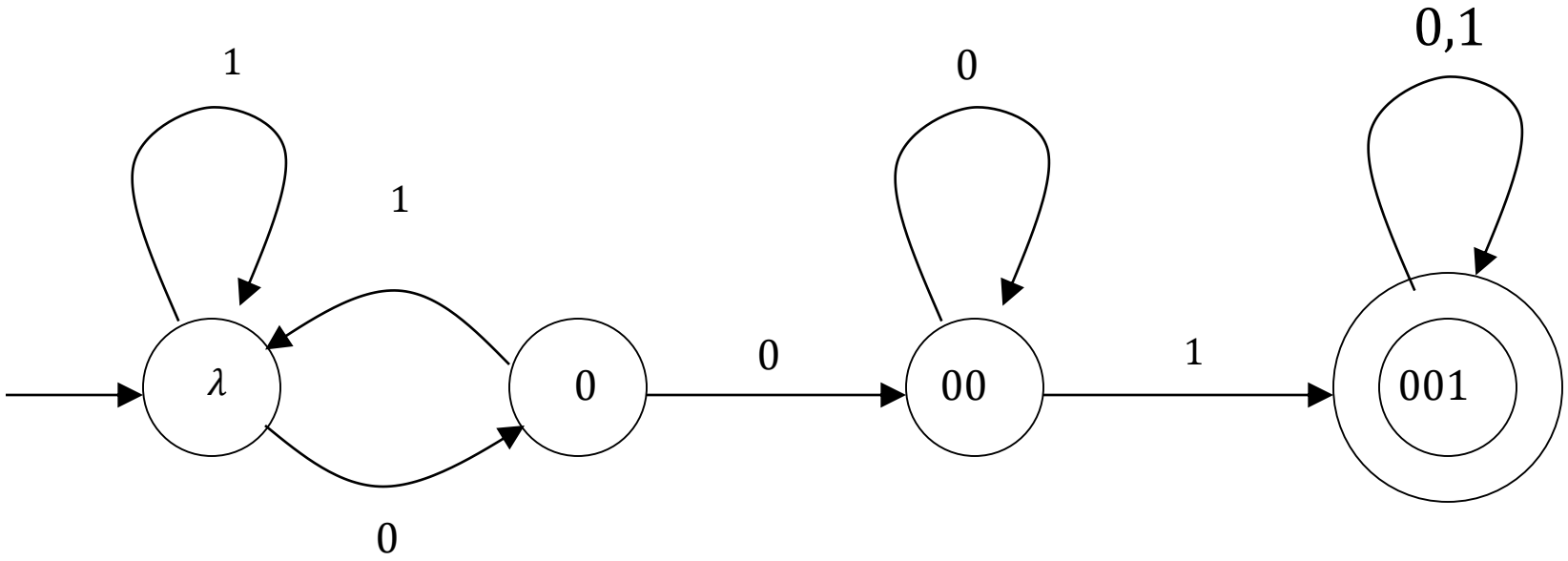
Where Σ*={λ,a,b,aa,ab,bb,ba,aa,...}



L(M)= { all binary strings containing **substring** [001] }

$$L(M)=\{w_1 001 w_2 : w_1 \in \{0,1\}^* , w_2 \in \{0,1\}^* \}$$

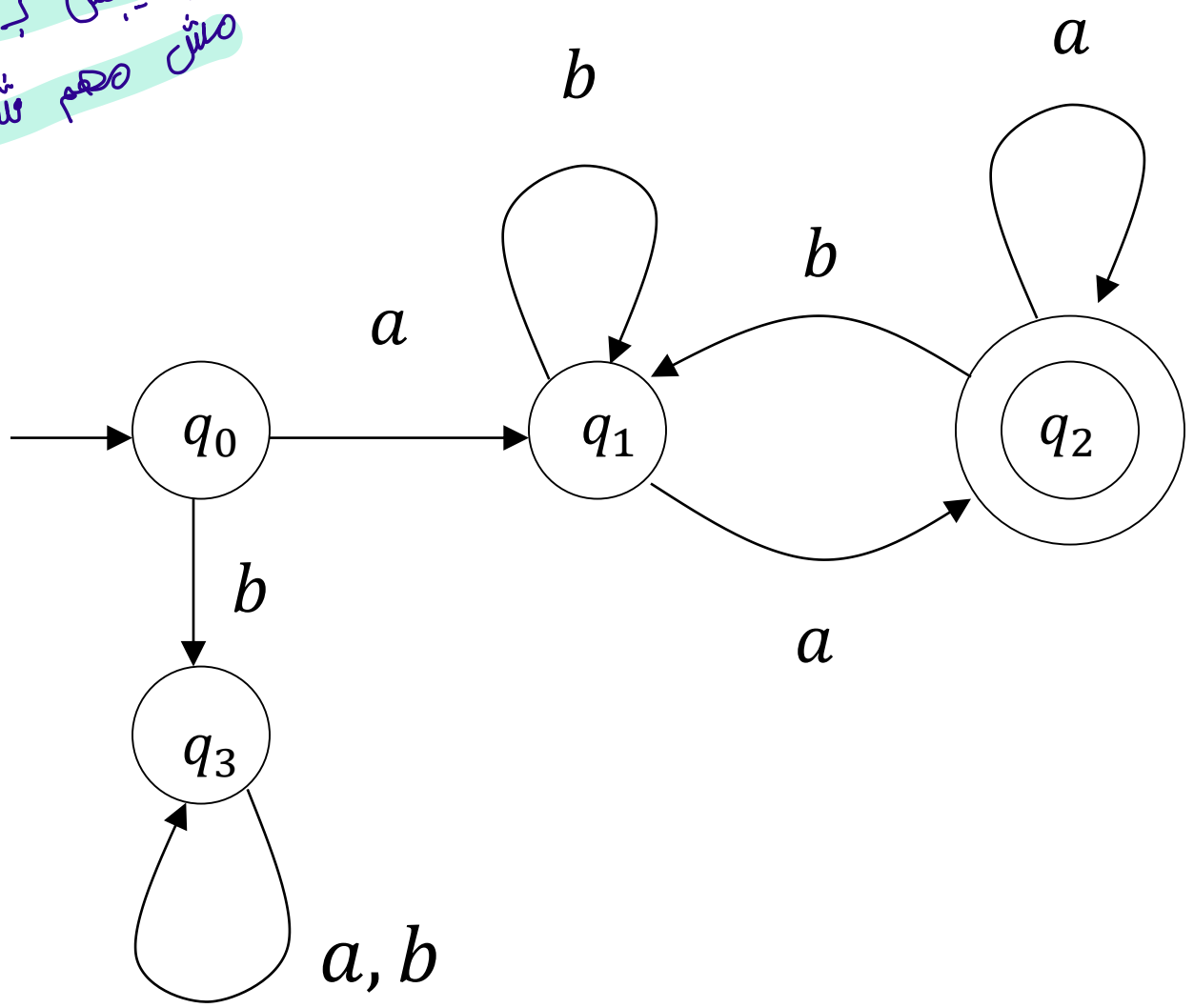
Accepted Strings:{001,1001,0001,0010,0011,11010001 01, …}



كرر قد ما بدك من a,b

$$L(M) = \{awa : w \in \{a,b\}^*\}$$

بدي يبلش بـ a وينتهي بـ a
مش مهم شو بينهم



DFA Design

- Design is a creative process, no simple rule for all DFA
  - A method: imagine you're a machine
  - You see only one symbol at a time of the string
  - After each symbol, you must decide to ACCEPT or NOT.
  - How to do it?
  - What information you need to remember about the string so far.

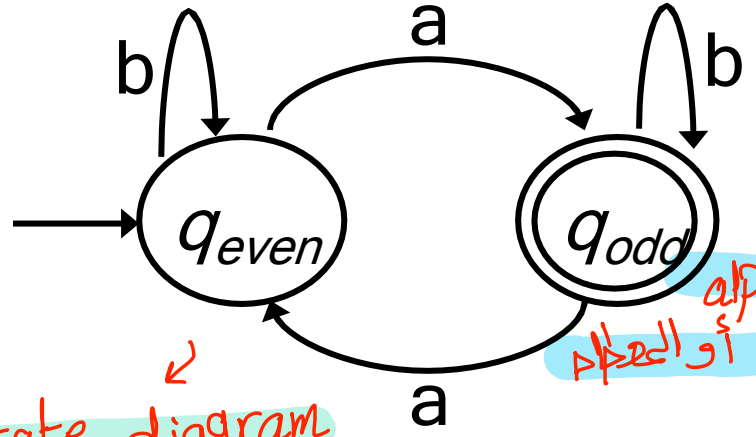
Design DFA for strings with odd # of a's

Example:

- Design a DFA, M6
- Alphabet = {a, b}
- L(M6) = {all strings with odd number of a's}
- Let's see some examples:
- Accepted strings [odd a's] = {a, aaa, ab, ba, abaa, bbbba, ...}
- Rejected strings [even a's] = {aa, b, bb, baba, baa, aaaa, ..., ε}

Design DFA for strings with odd # of a's

- Accepted strings [odd a's] = {a, aaa, ab, ba, abaa, bbbba, …}
- Rejected strings [even a's] = {aa, b, bb, baba, baa, aaaa, …, ε}



state diagram

	a	b
q_even	q_odd	q_even
***q_odd**	q_even	q_odd

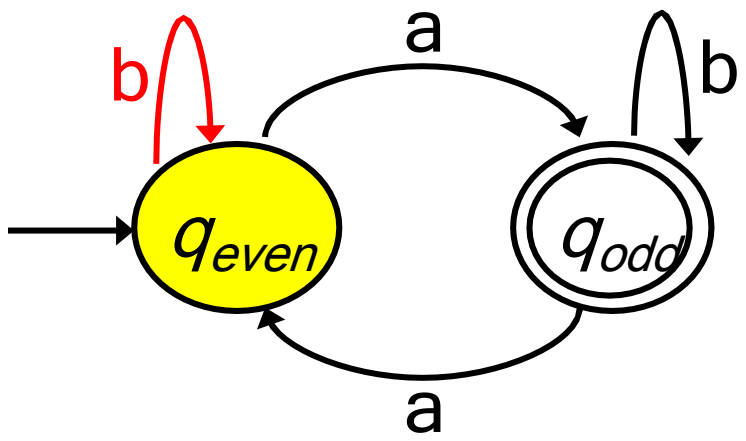
state table

Imp. Note:

Because of Deterministic machine so every state has to contains Two strings a and b

Design DFA for strings with odd # of a's

- Accepted strings [odd a's] = {a, aaa, ab, ba, abaa, bbbba, …}
- Rejected strings [even a's] = {aa, b, bb, baba, baa, aaaa, ..., ε}

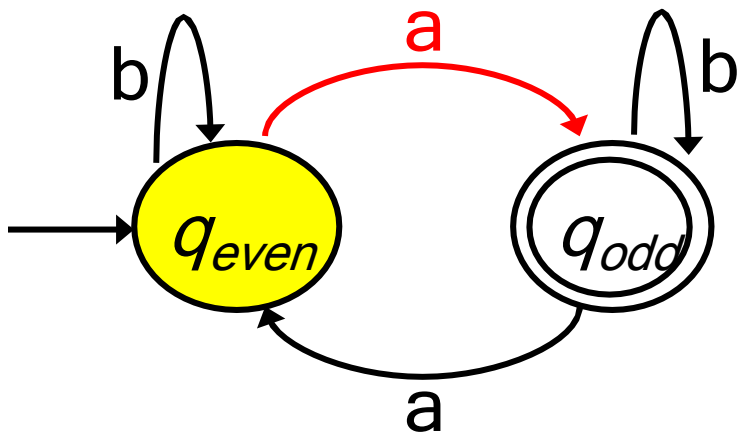


Testing our DFA

Input string: baba

Design DFA for strings with odd # of a's

- Accepted strings [odd a's] = {a, aaa, ab, ba, abaa, bbbba, …}
- Rejected strings [even a's] = {aa, b, bb, baba, baa, aaaa, ..., ε}

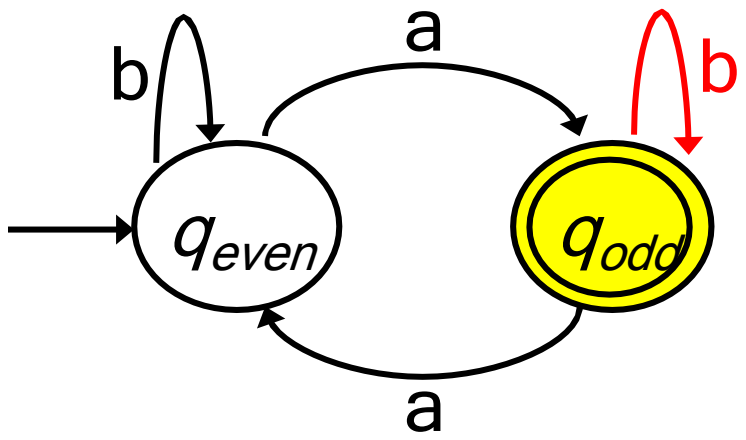


Testing our DFA

Input string: baba

Design DFA for strings with odd # of a's

- Accepted strings [odd a's] = {a, aaa, ab, ba, abaa, bbbba, …}
- Rejected strings [even a's] = {aa, b, bb, baba, baa, aaaa, ..., ε}

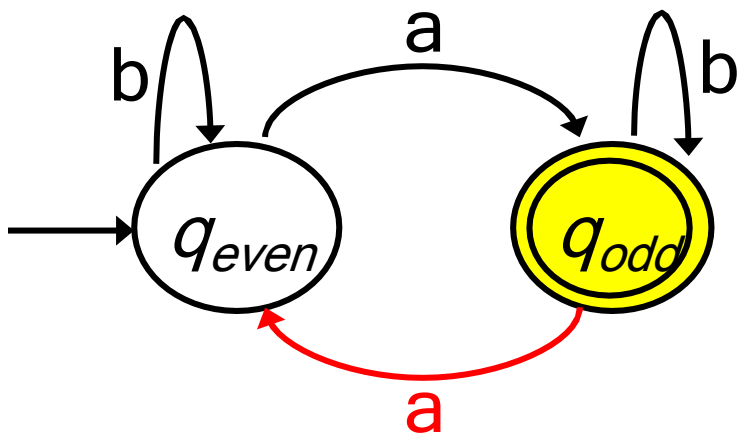


Testing our DFA

Input string: baba

Design DFA for strings with odd # of a's

- Accepted strings [odd a's] = {a, aaa, ab, ba, abaa, bbbba, …}
- Rejected strings [even a's] = {aa, b, bb, baba, baa, aaaa, ..., ε}

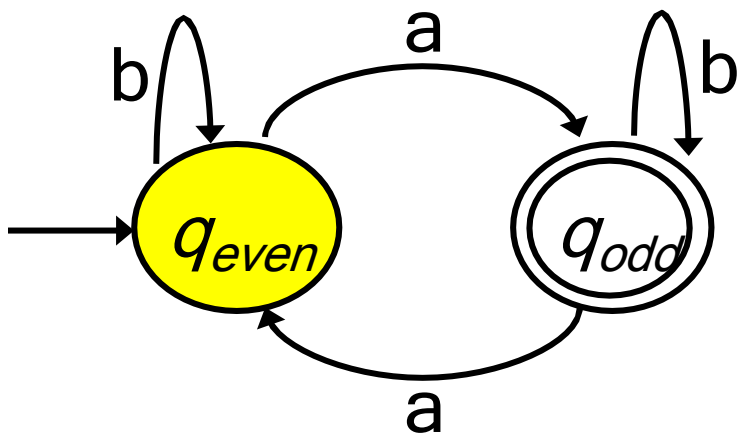


Testing our DFA

Input string: baba

Design DFA for strings with odd # of a's

- Accepted strings [odd a's] = {a, aaa, ab, ba, abaa, bbbba, …}
- Rejected strings [even a's] = {aa, b, bb, baba, baa, aaaa, ..., ε}

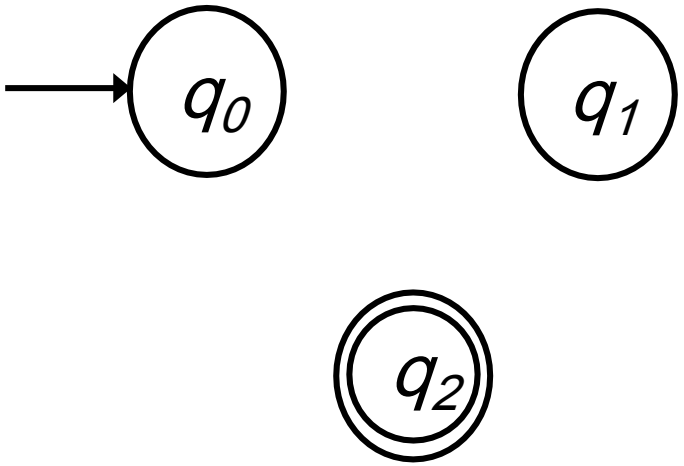


Testing our DFA

Input string: baba

Rejected

Machine 7: strings that end with 01

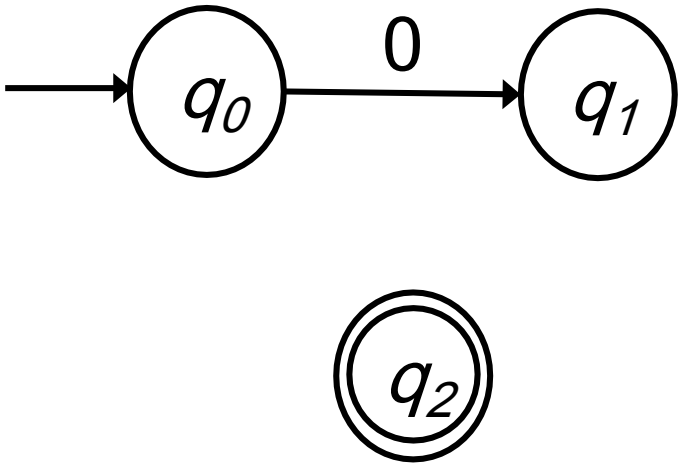


State Diagram for M_7

- Problem: Design a DFA for :
- $L(M_7)$ = {over (0,1)| strings that end with 01}
- E.g (01, 101, 1101, 0001, 10101)
- Information to remember:
- If string ends with 0,
- There is 1 after the 0

	0	1
q0 [end with 1]		
q1 [ends with 0]		
*q2 [ends with 01]		

Machine 7: strings that end with 01



State Diagram for M_7

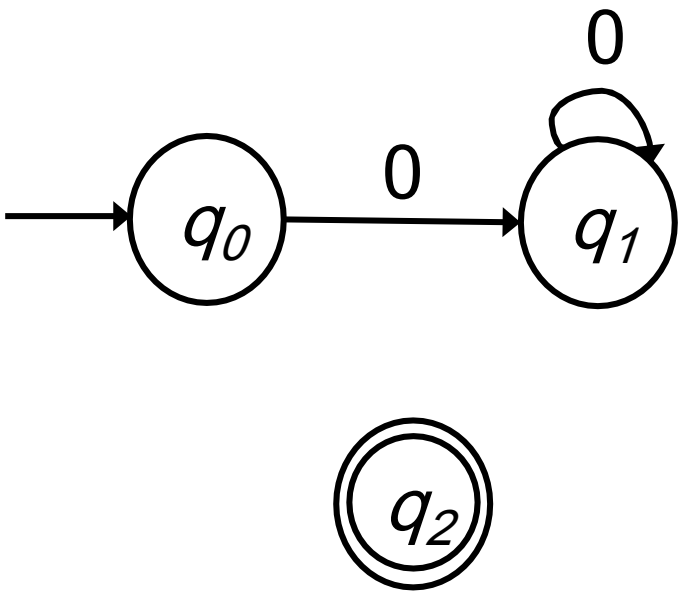
- Problem: Design a DFA for :
- $L(M_7)$ = {over (0,1)| strings that end with 01}
- E.g (01, 101, 1101, 0001, 10101)
- Information to remember:

1- If string ends with 0,There is 1 after the 0

2- Because of Deterministic machine so every state has to contains Two strings a and b

	0	1
q0 [doesn't end with 0]	q1	
q1 [ends with 0]		
*q2 [ends with 01]		

Machine 7: strings that end with 01

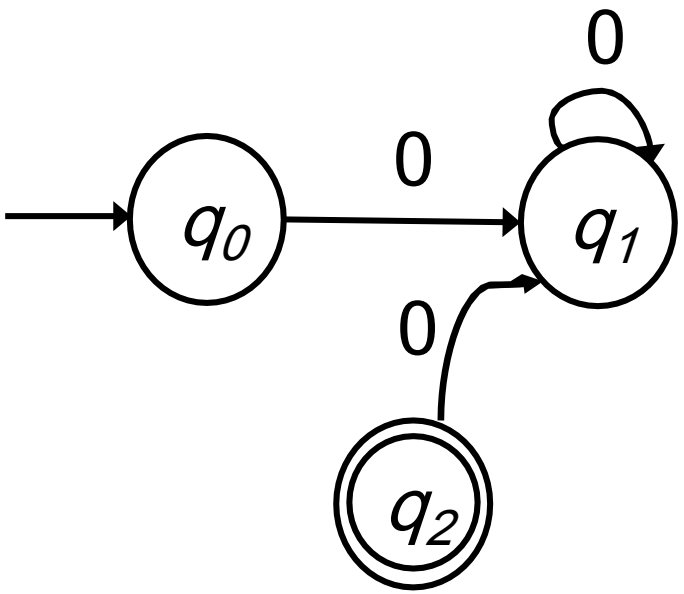


State Diagram for M_7

- Problem: Design a DFA for :
- $L(M_7)$ = {over (0,1)| strings that end with 01}
- E.g (01, 101, 1101, 0001, 10101)

	0	1
q0 [doesn't end with 0]	q1	
q1 [ends with 0]	q1	
*q2 [ends with 01]		

Machine 7: strings that end with 01

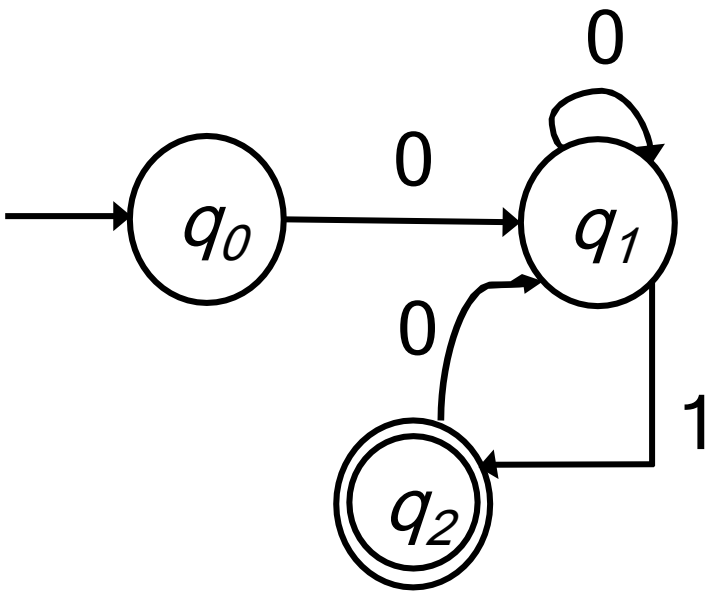


State Diagram for M_7

- Problem: Design a DFA for :
- $L(M_7)$ = {over (0,1)| strings that end with 01}
  - E.g (01, 101, 1101, 0001, 10101)

	0	1
q0 [doesn't end with 0]	q1	
q1 [ends with 0]	q1	
*q2 [ends with 01]	q1	

Machine 7: strings that end with 01

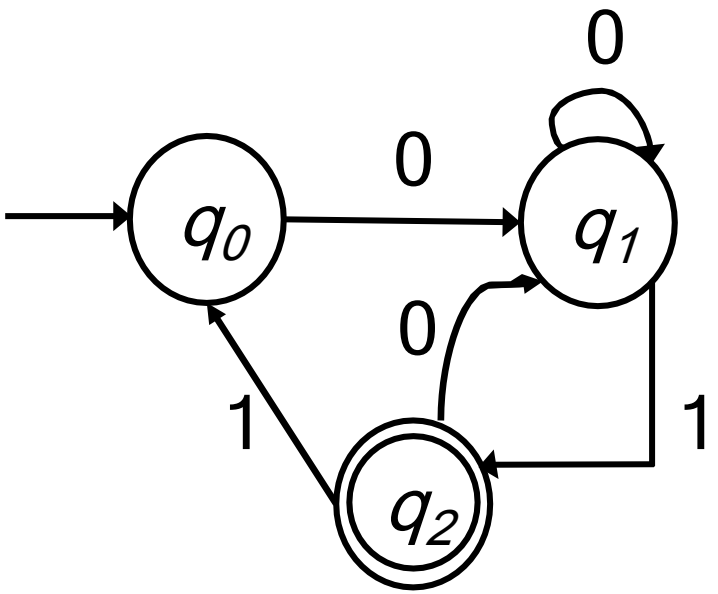


State Diagram for M_7

- Problem: Design a DFA for :
- $L(M_7)$ = {over (0,1)| strings that end with 01}
  - E.g (01, 101, 1101, 0001, 10101)

	0	1
q0 [doesn't end with 0]	q1	
q1 [ends with 0]	q1	q2
*q2 [ends with 01]	q1	

Machine 7: strings that end with 01

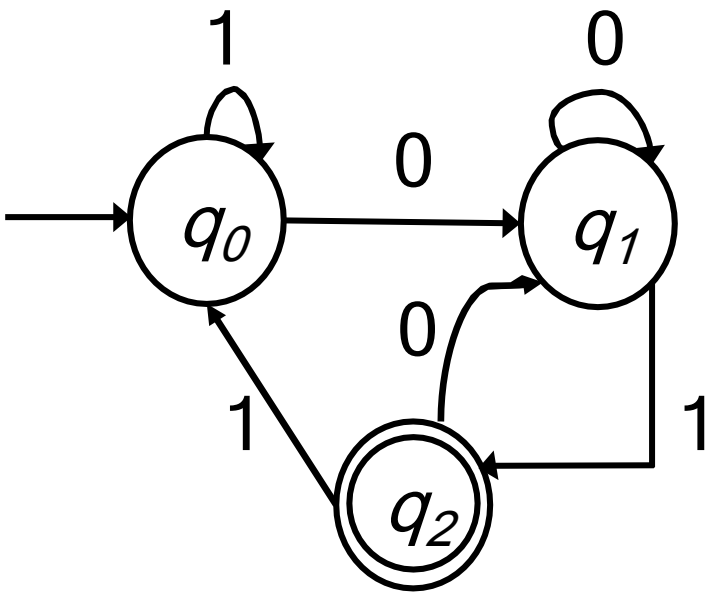


State Diagram for M_7

- Problem: Design a DFA for :
- $L(M_7)$ = {over (0,1)| strings that end with 01}
  - E.g (01, 101, 1101, 0001, 10101)

	0	1
q0 [doesn't end with 0]	q1	
q1 [ends with 0]	q1	q2
*q2 [ends with 01]	q1	q0

Machine 7: strings that end with 01



State Diagram for M_7

- Problem: Design a DFA for :
- $L(M_7)$ = {over (0,1)| strings that end with 01}
  - E.g (01, 101, 1101, 0001, 10101)

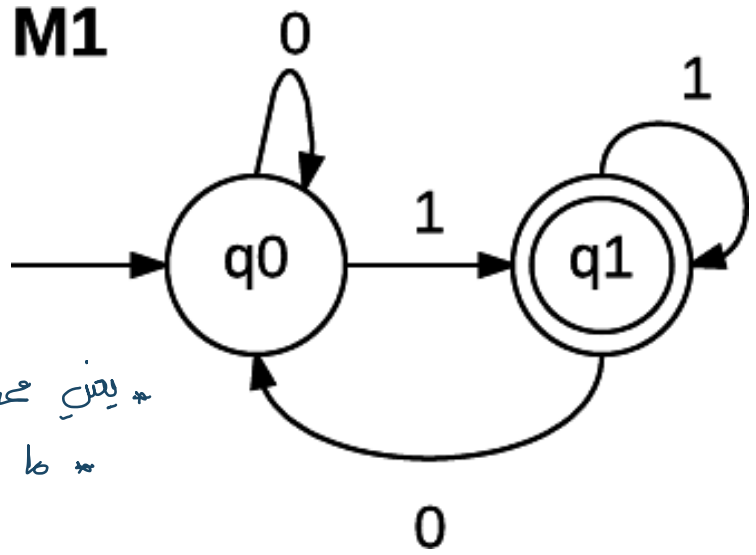
	0	1
q0 [doesn't end with 0]	q1	q0
q1 [ends with 0]	q1	q2
*q2 [ends with 01]	q1	q0

THE END

Nondeterministic Finite Automata (NFA)

What is a Deterministic Machine (e.g. *DFA*)

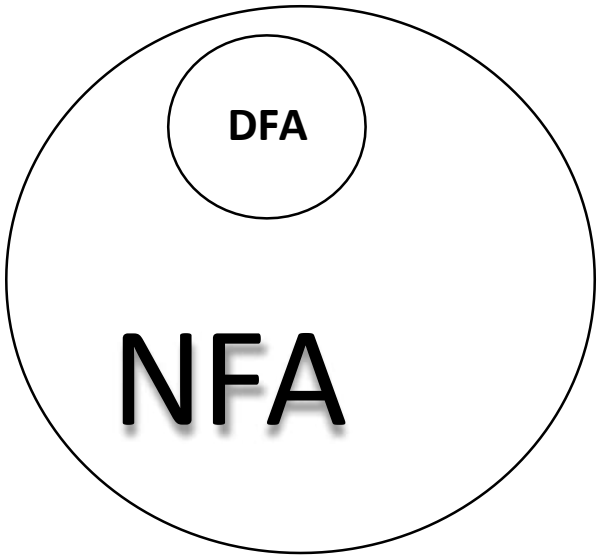
- Deterministic machine
  - When the machine is at a given state, for a specific input, it gives one output at each time.
  - For example:



* يعني ممكن أنتقل من q_0 عن طريق 1 لـ q_1 فقط
* ما عندي حالة ثالثة أو alphabet ثاني

Nondeterministic Machine (NFA)

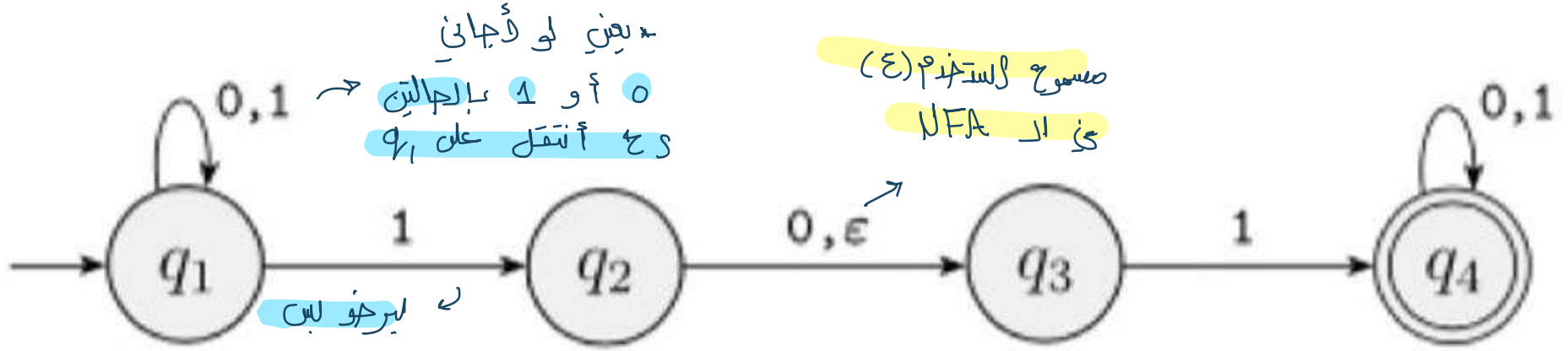
- Nondeterministic machine
  - When the machine is at a given state, for a specific input, it may give more than one output at each time.
  - Every deterministic finite automata (DFA), is considered a nondeterministic finite automata (NFA)



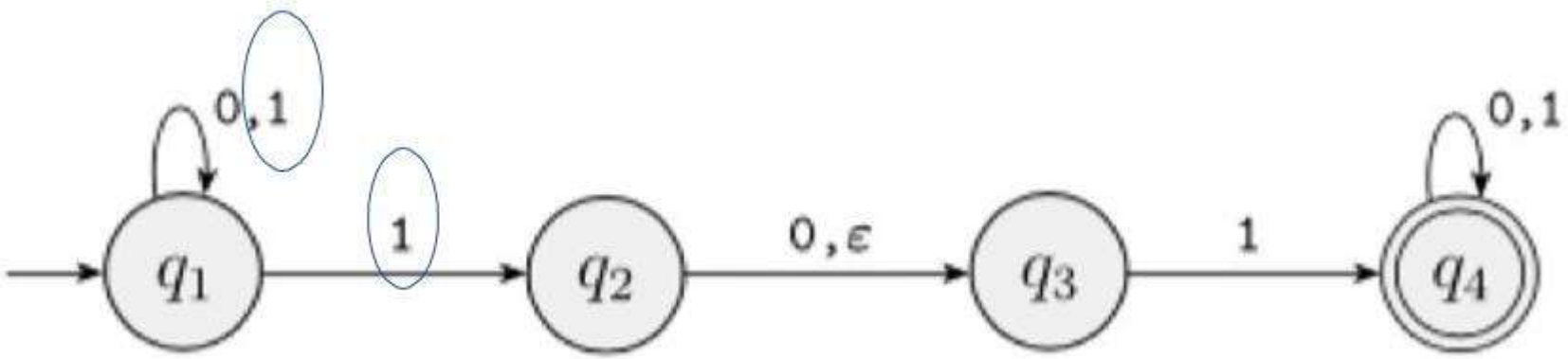
* الـ DFA هي حالة خاصة من الـ NFA
يعني الـ NFA أعم

Nondeterministic Machine (NFA)

- Nondeterministic machine
  - When the machine in a given state, for a specific input, it may gives more than one output at each time.

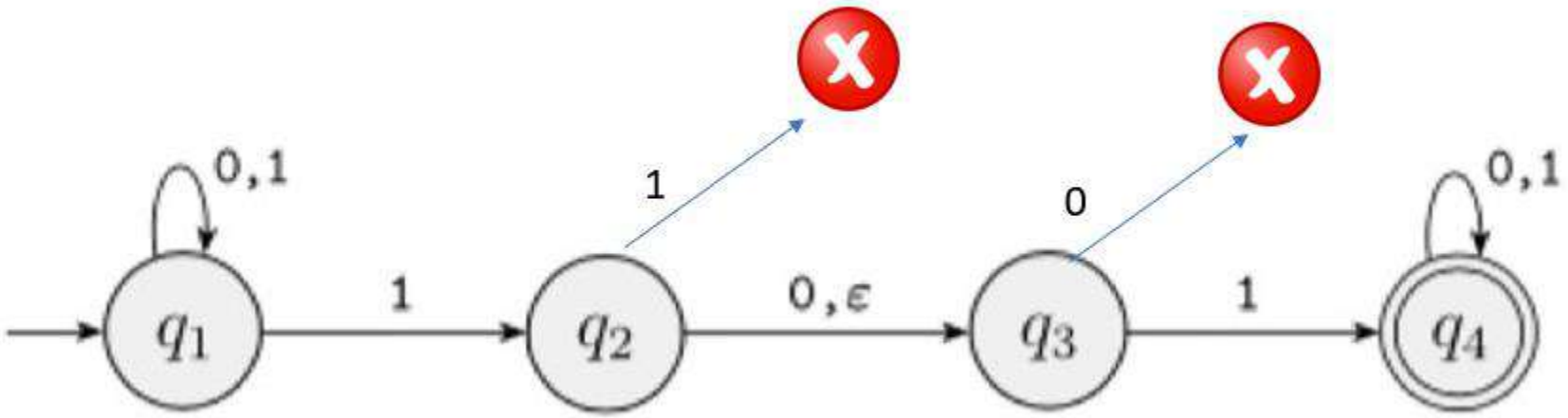


More about NFA state diagram



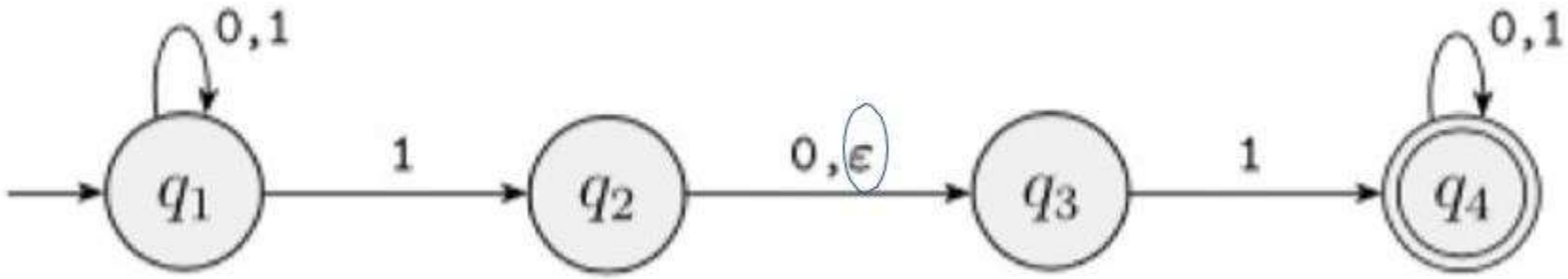
1) from state q1 to state q2, we have two arrow with label 1, but in DFA, we can only have one arrow for each label.

More about NFA state diagram



2) In state q2, we don't have an arrow with label 1, but in DFA, we must have an arrow for each label. Also in state q3 with label 0.

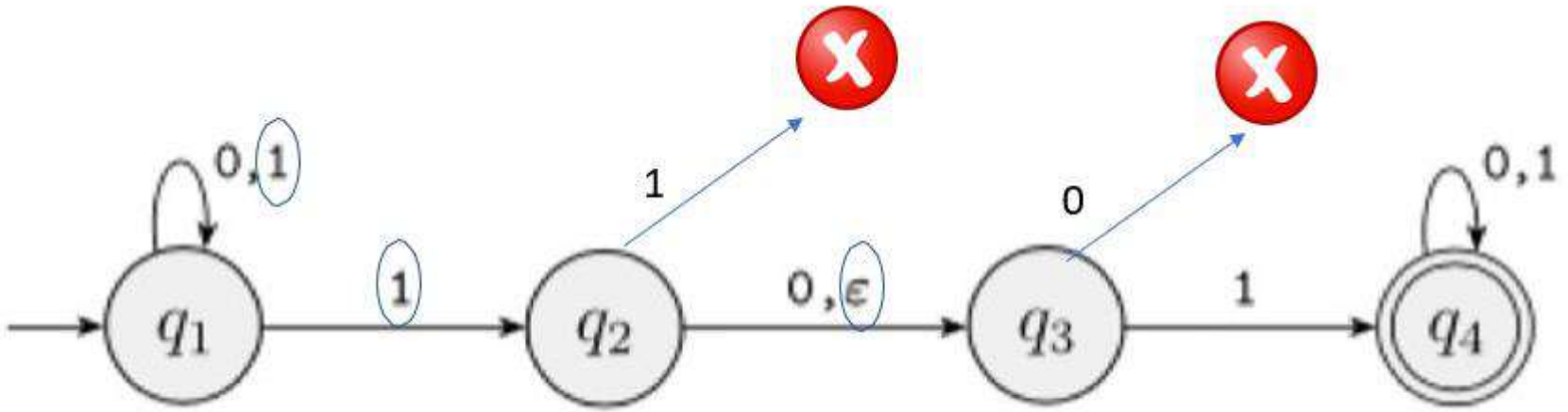
More about NFA state diagram



ε = empty string

3) In state q2, we have an arrow with label ε, but in DFA, we can't have an arrow with label ε.
This arrow means, at state q2, we can move to state q3 without reading any input.

What is the difference between DFA and NFA?



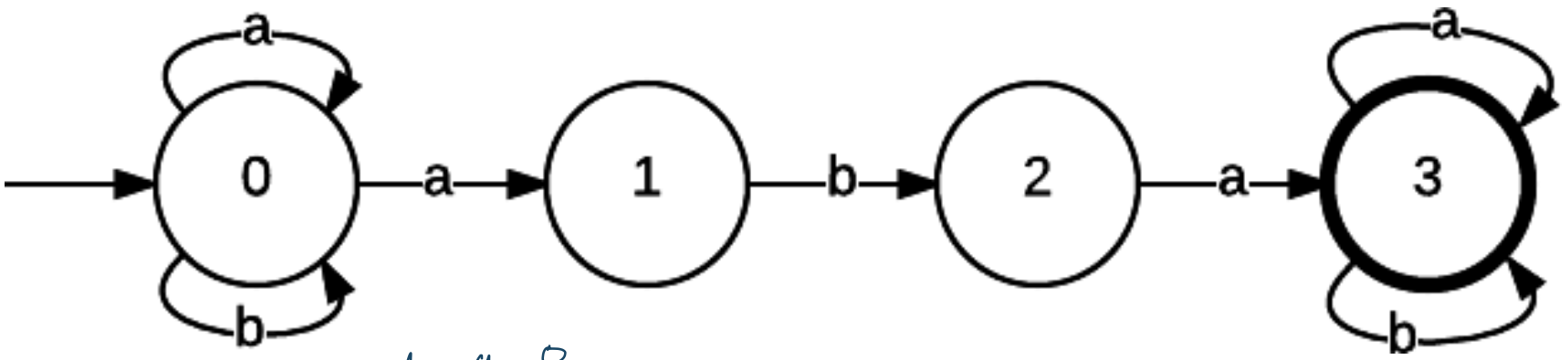
ε = empty string

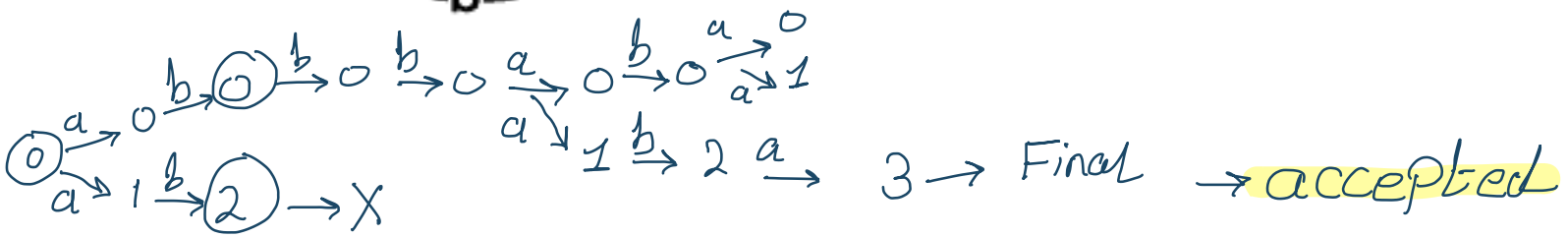
1) In DFA, we must have one arrow for each label. In NFA, we can have zero, one, two, or more arrows for each label.
2) In NFA, we have an arrow with label ε (empty string). But in DFA, we can't have an arrow with label ε. All arrow in DFA have labels from the alphabet.

How does an NFA work?

- Nondeterministic Machine, NFA2
- Input string = *abbbaba*

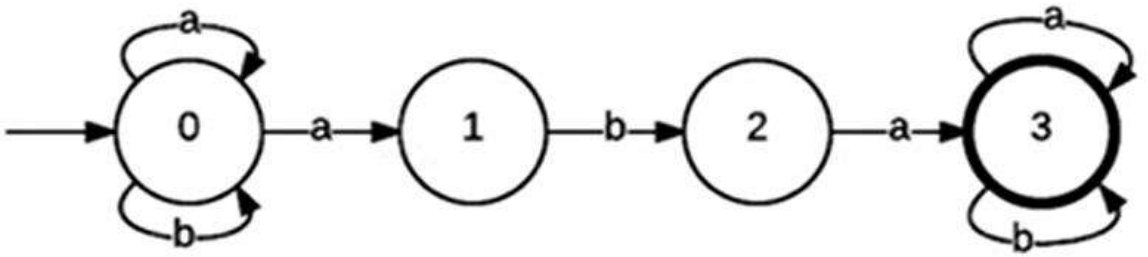
→ string بدي أشوف هاي الـ
rejected أو accepted

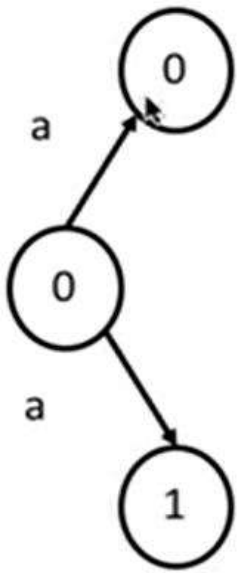




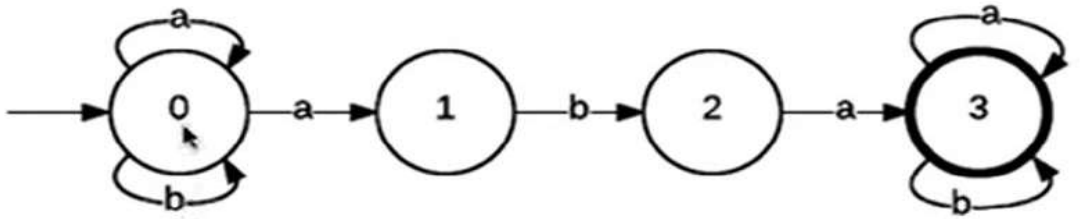
تتبع ↙

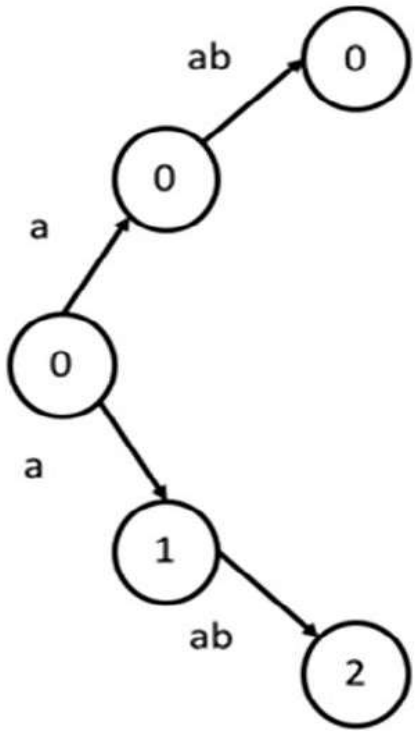
- Nondeterministic Machine, NFA2
- Input string = *abbbaba*



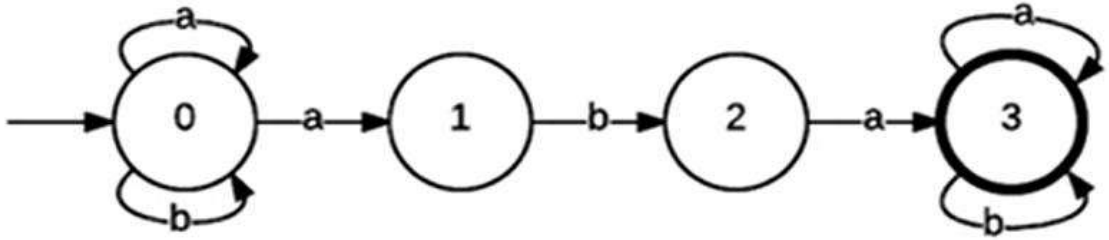


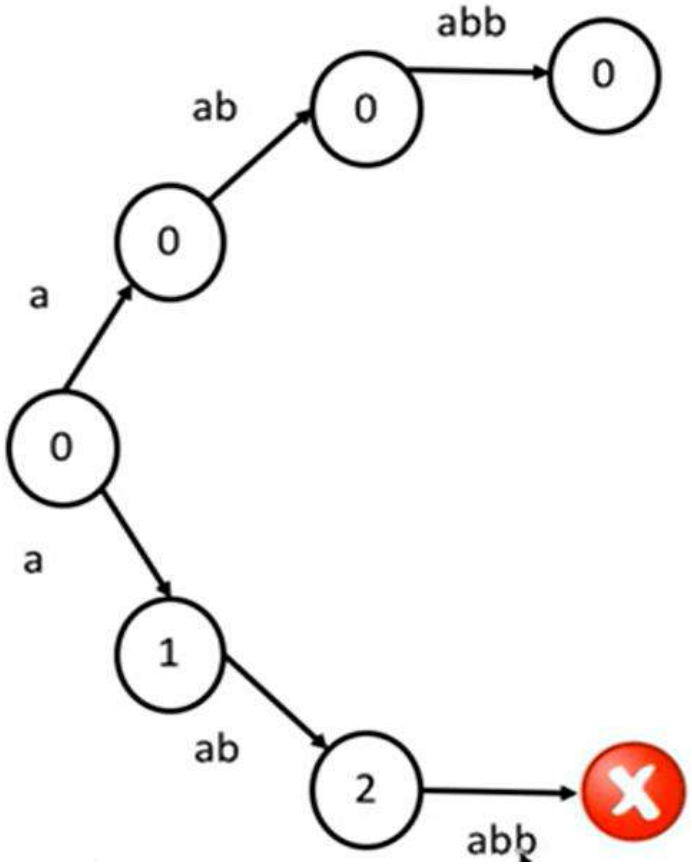
- Nondeterministic Machine, NFA2
- Input string = *abbbaba*





- Nondeterministic Machine, NFA2
- Input string = *abbbaba*

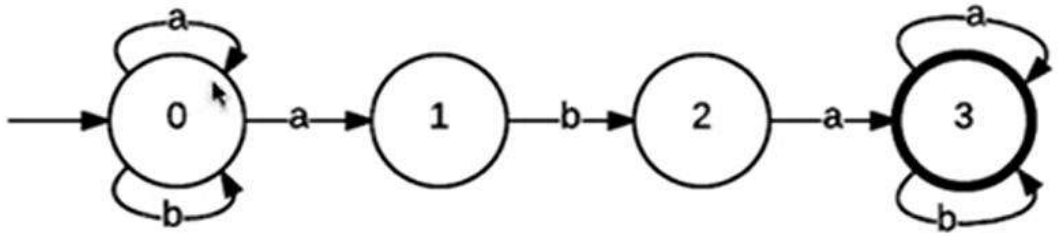


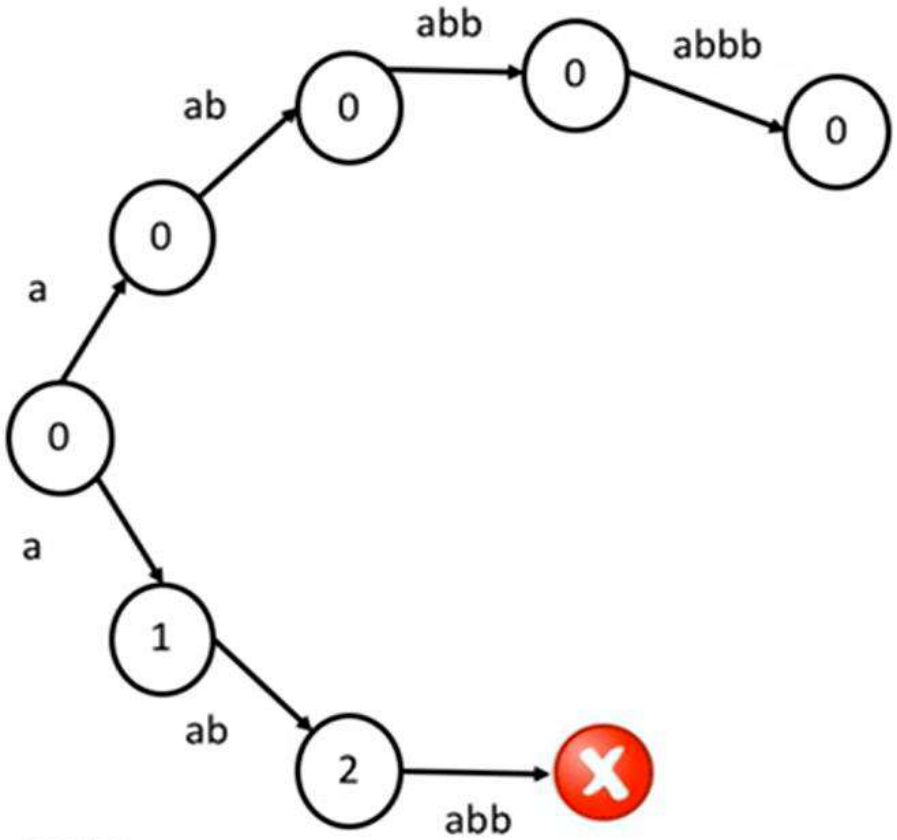


→ block يعمل

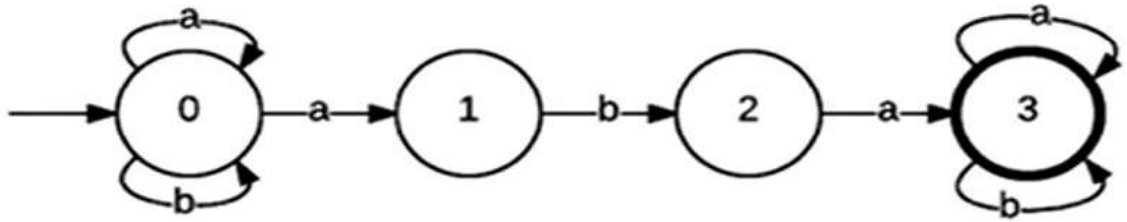
لأنه ما في حركة يوصلني بعدها لأي شي من 2 لأي شي عن حركة b

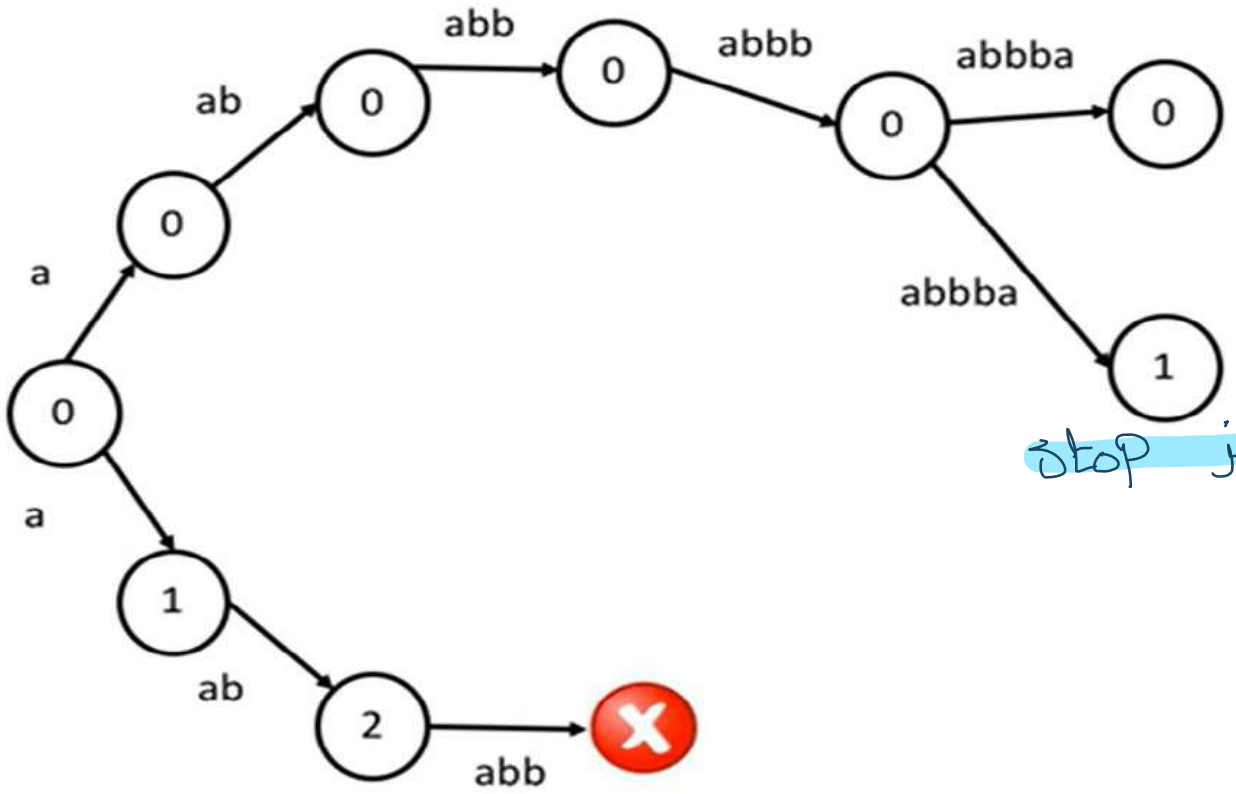
- Nondeterministic Machine, NFA2
- Input string = *abbbaba*



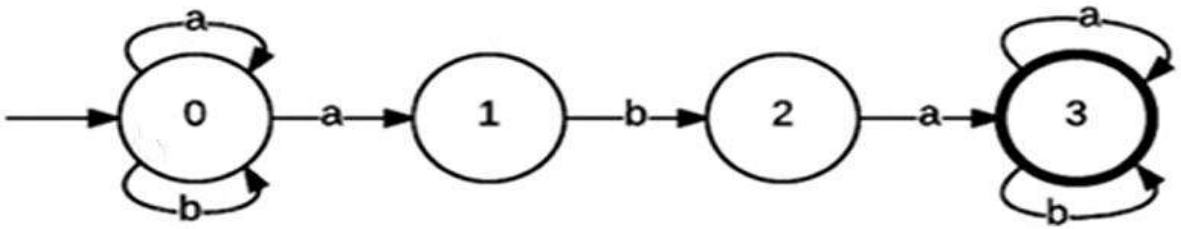


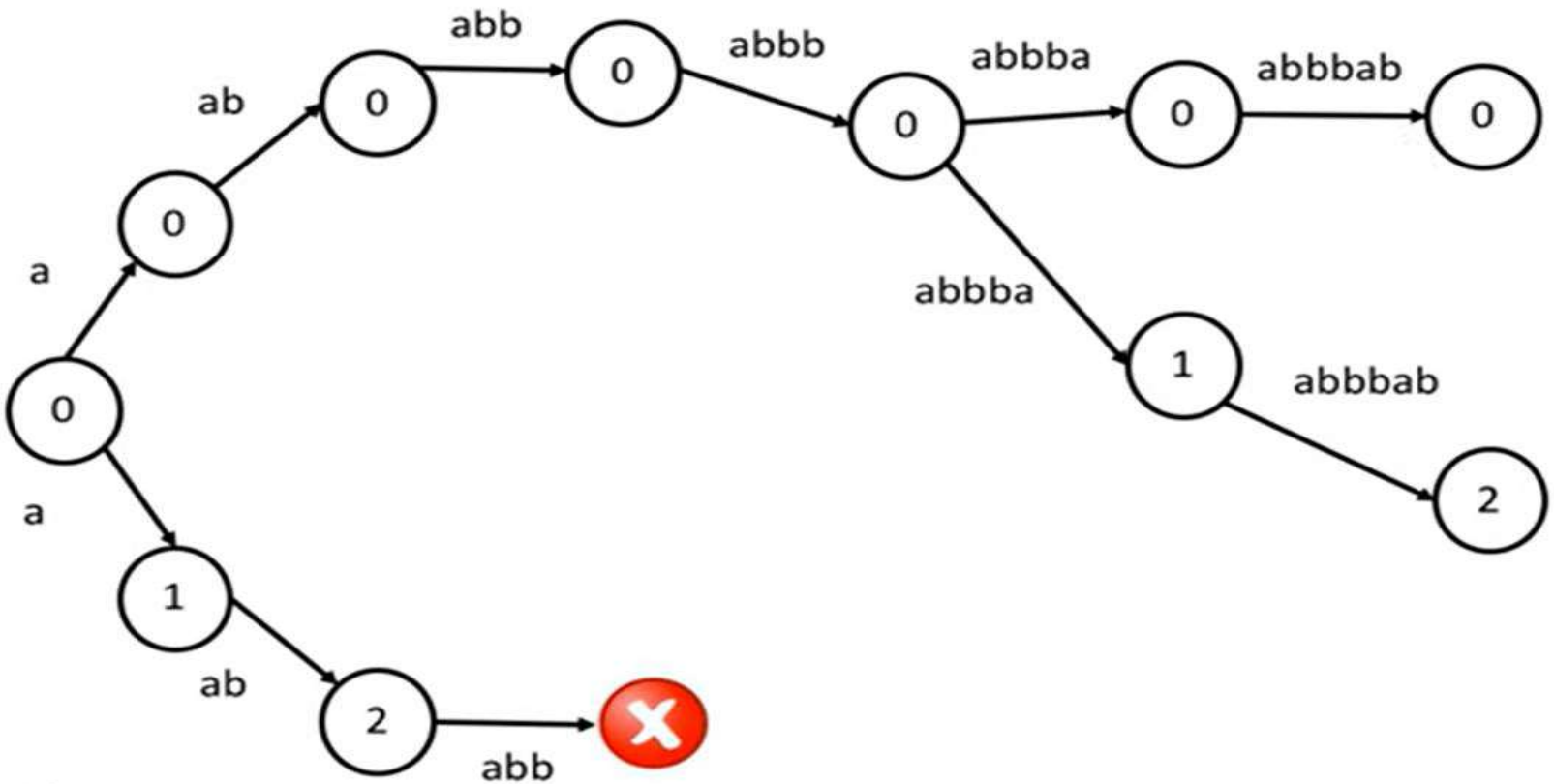
- Nondeterministic Machine, NFA2
- Input string = *abbbaba*



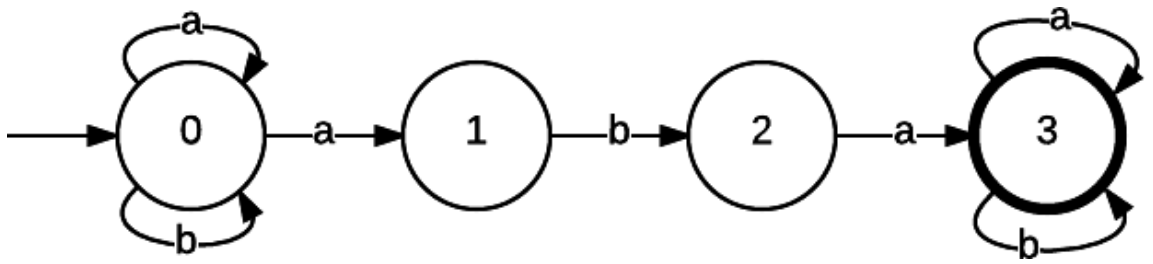


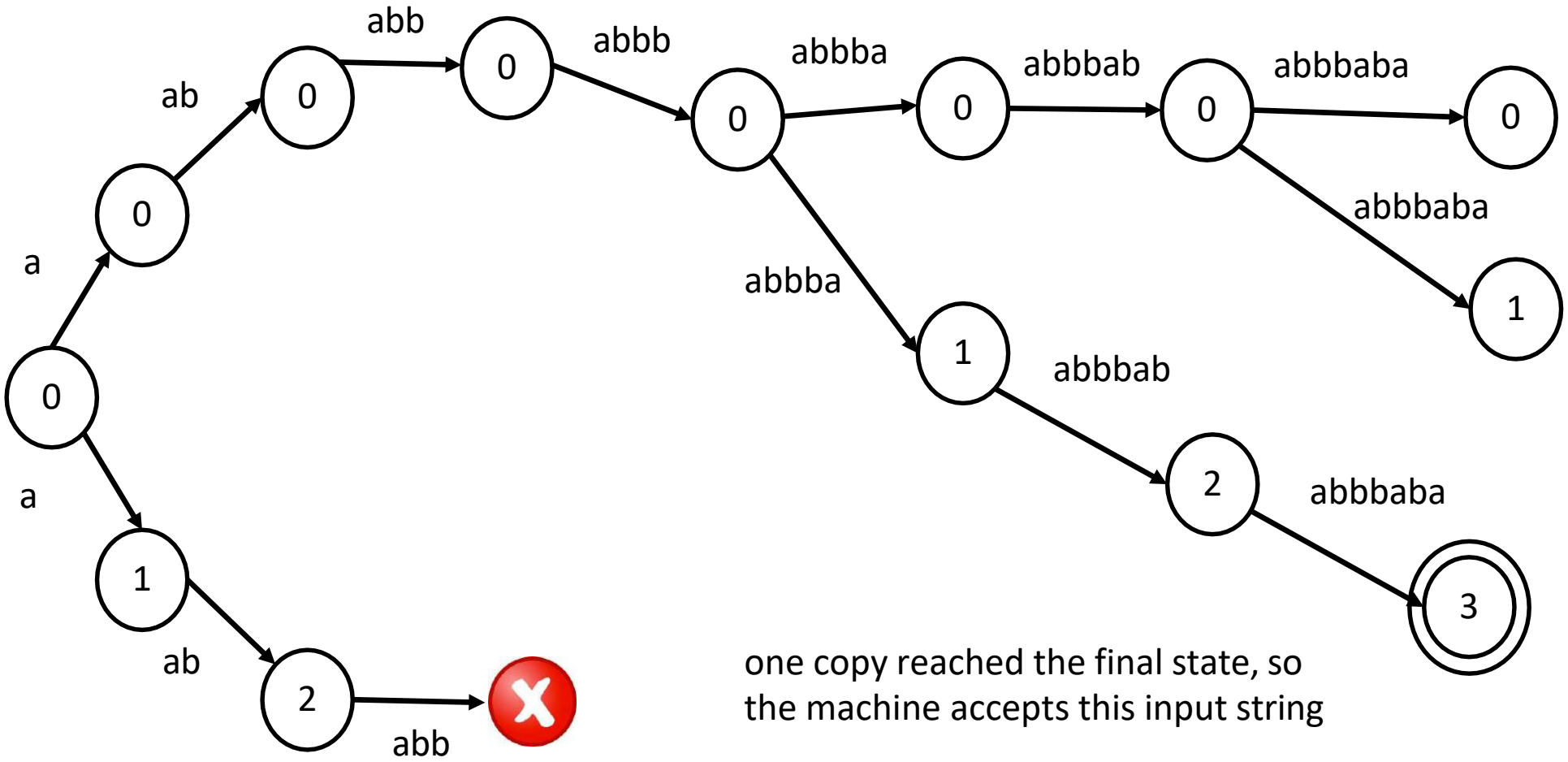
- Nondeterministic Machine, NFA2
- Input string = *abbbaba*





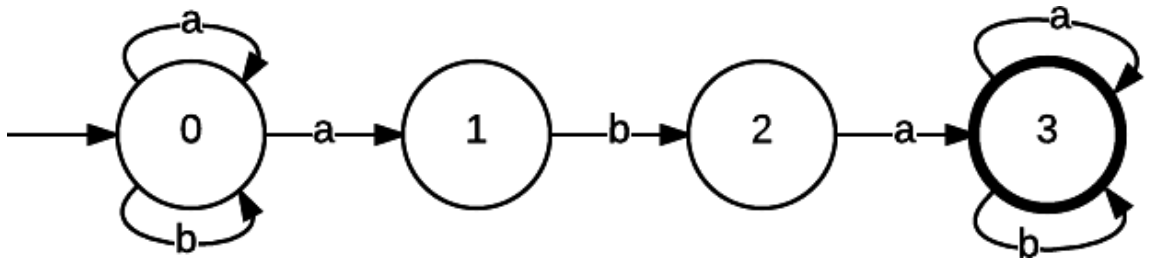
- Nondeterministic Machine, NFA2
- Input string = *abbbaba*

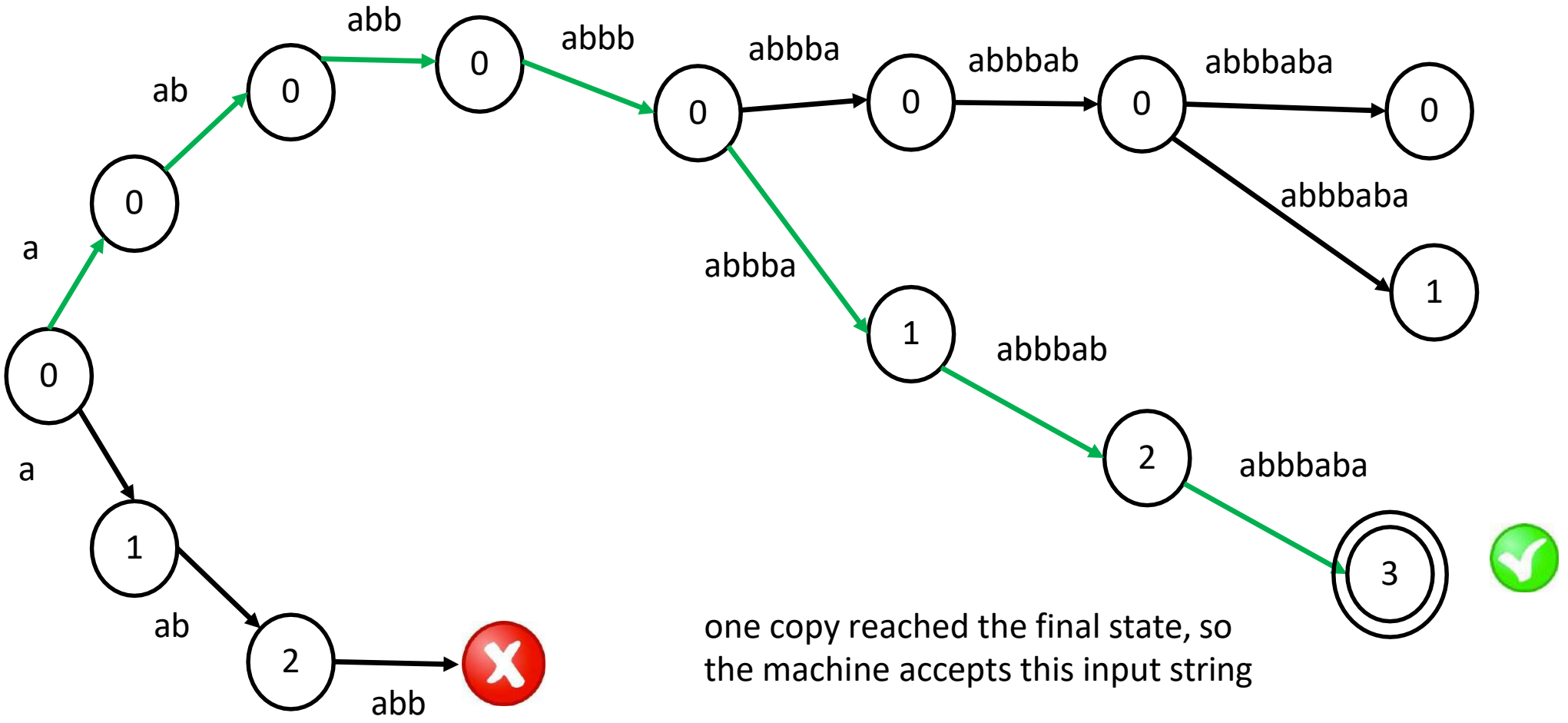




one copy reached the final state, so the machine accepts this input string

- Nondeterministic Machine, NFA2
- Input string = *abbbaba*

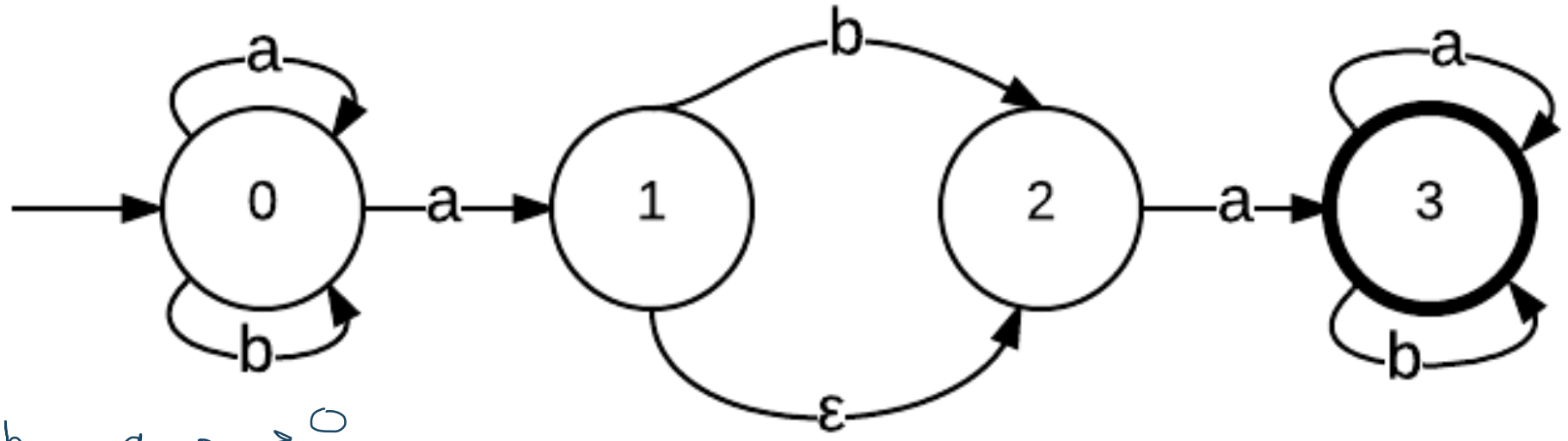


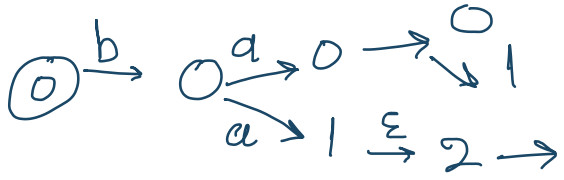


one copy reached the final state, so the machine accepts this input string

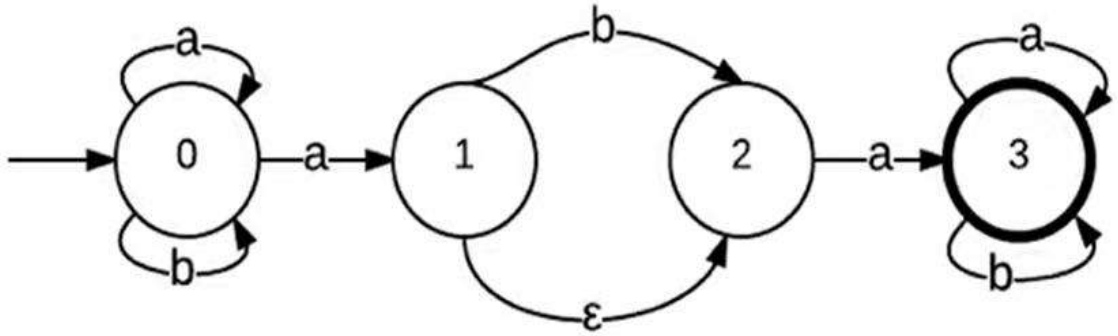
Tracing NFA with empty string

- Nondeterministic Machine, NFA_3
- Does this NFA accept string = *baa?*

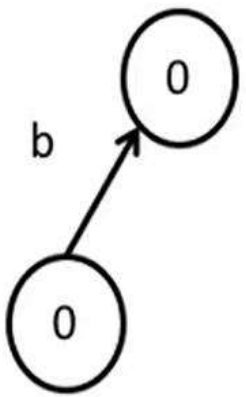




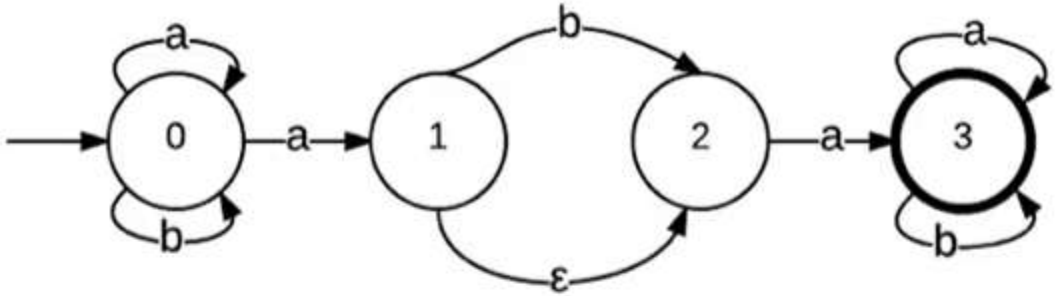
- Does this NFA accept string = *baa?*



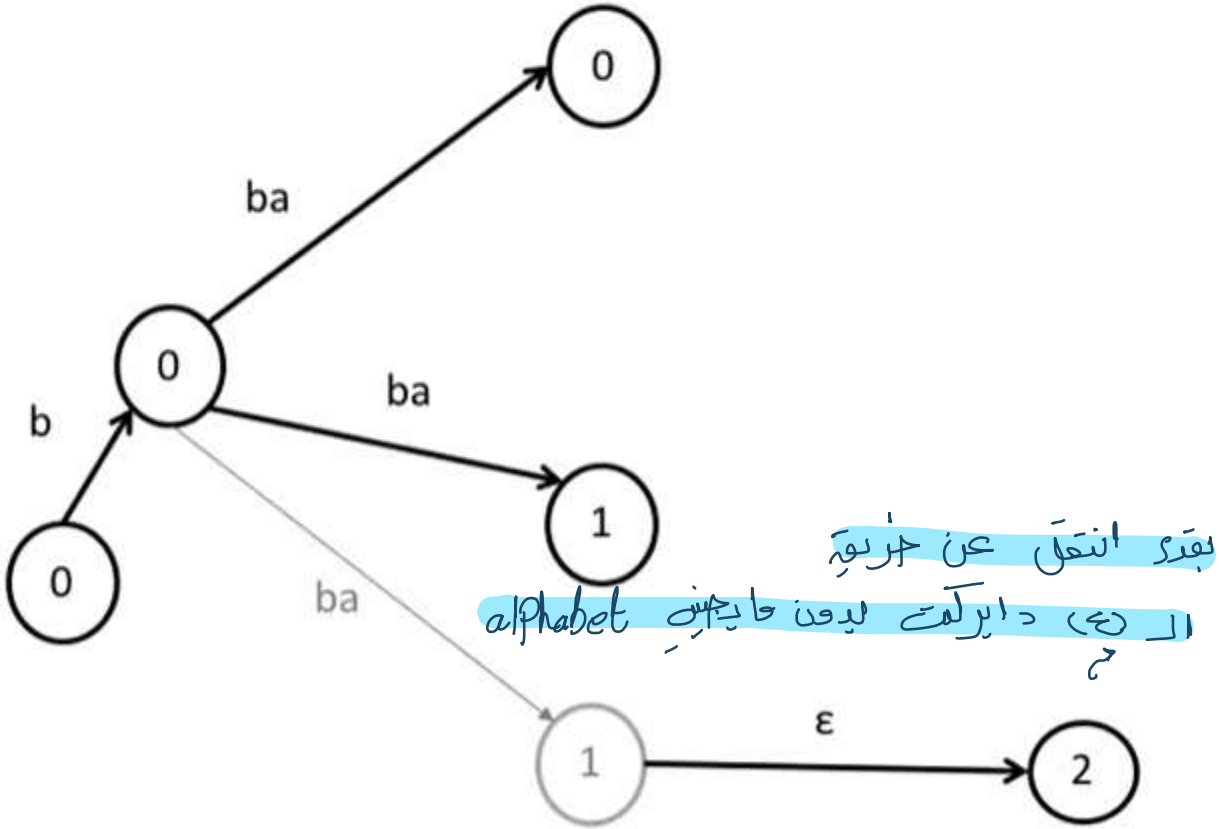
NFA



- Does this NFA accept string = *baa*?

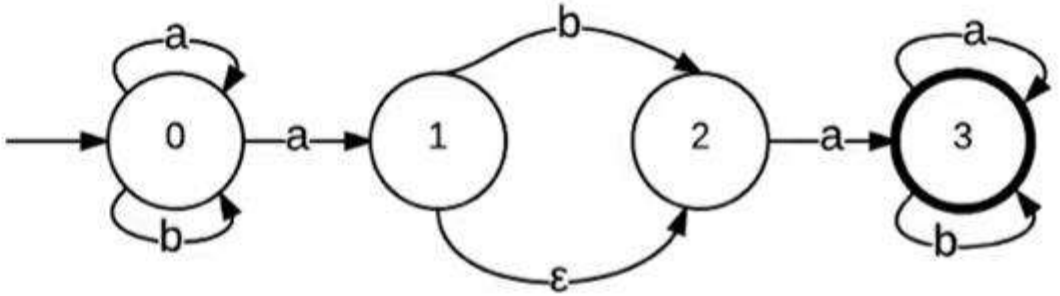


NFA



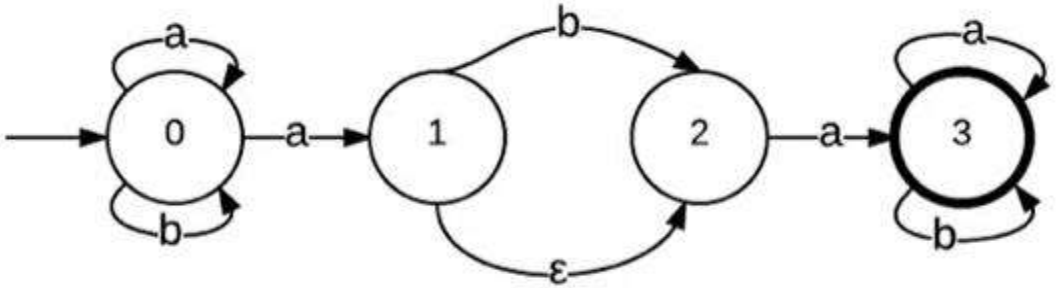
بقدر انتقل عن طريق
alphabet الـ (ε) دايركت بدون ما يجي

- Does this NFA accept string = *baa*?

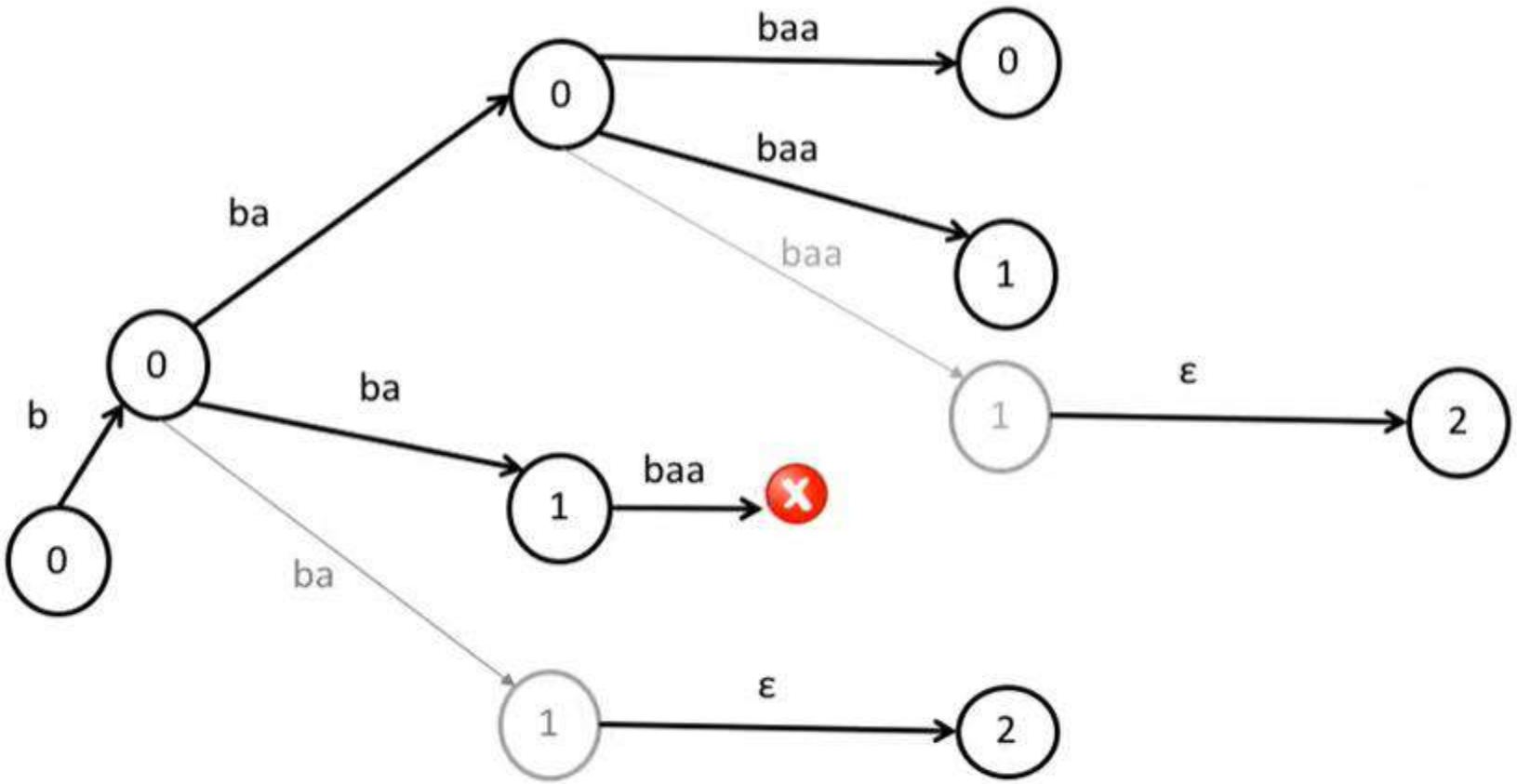


NFA

- Does this NFA accept string = *baa*?

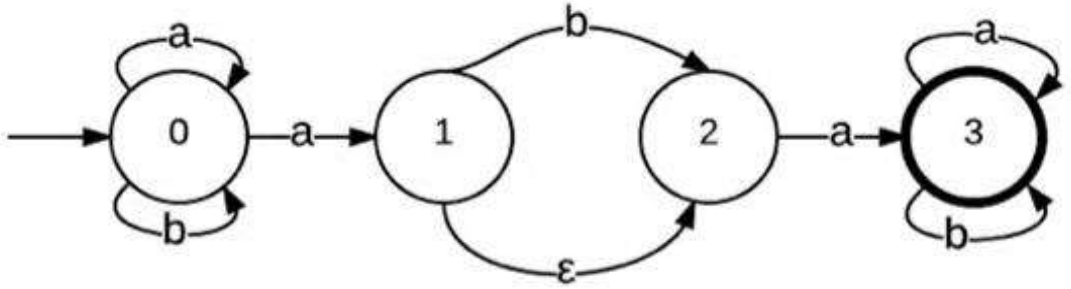


NFA

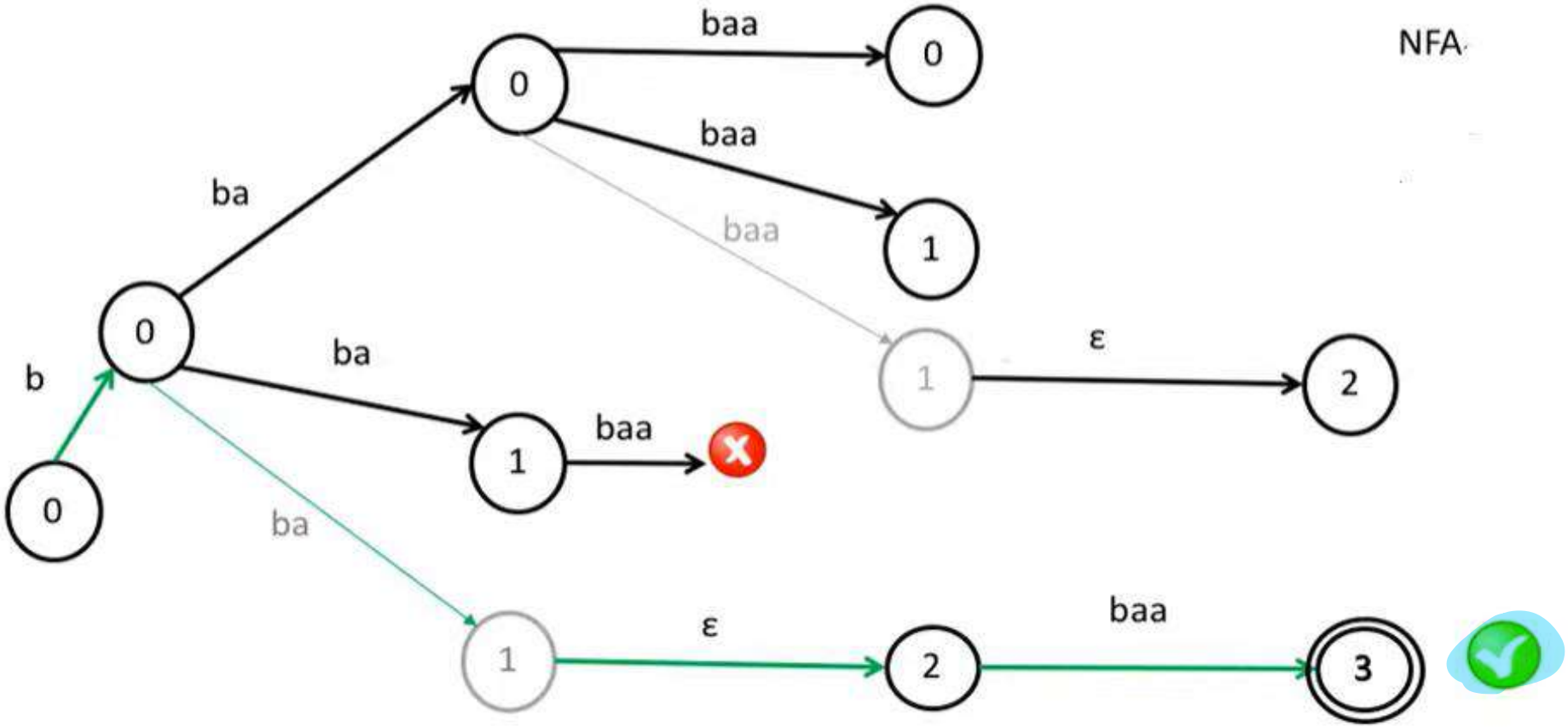


- Does this NFA accept string = *baa*?

one copy reached the final state, so
the machine accepts this input string



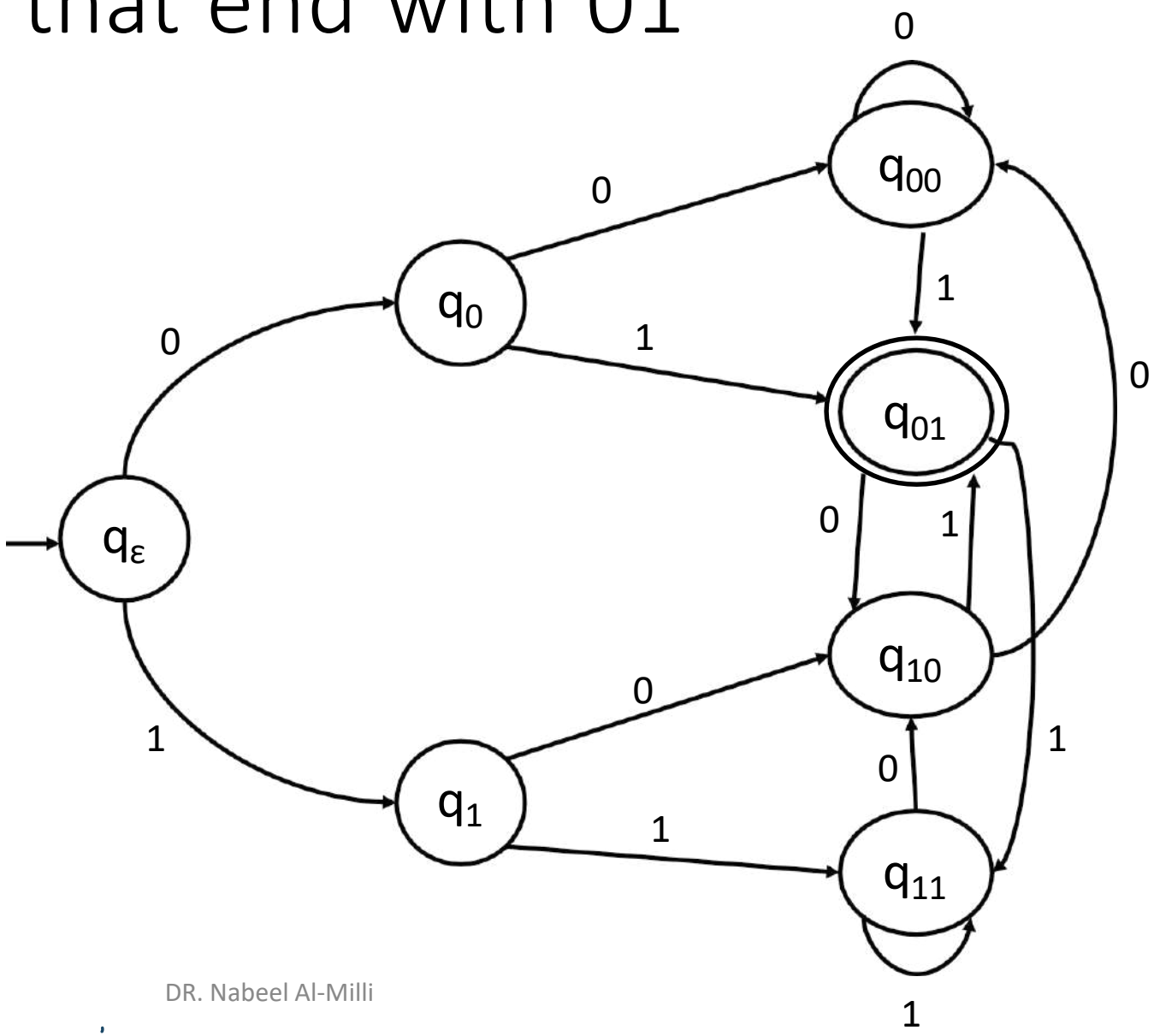
NFA



Minimum DFA

Machine 1: strings that end with 01

states	0	1
q_ε	q_0	q1
q_0	q_{00}	q_{01}
q1	q_{10}	q_{11}
q_{00}	q_{00}	q_{01}
$*q_{01}$	q_{10}	q_{11}
q_{10}	q_{00}	q_{01}
q_{11}	q_{10}	q_{11}



How to minimize a DFA

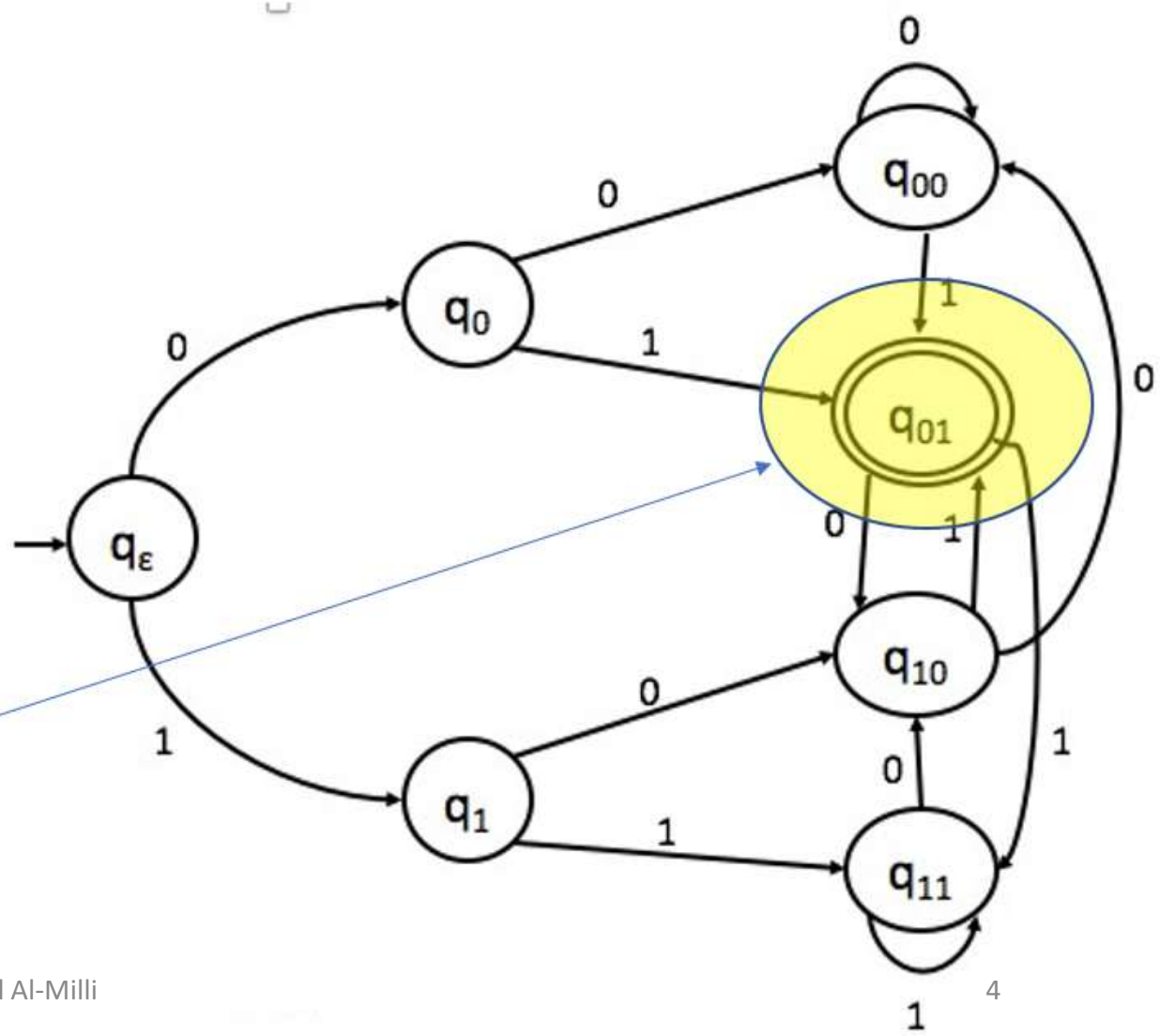
تقليل ← عدد الـ States

- Most exam questions need a DFA of minimum size.
- Algorithm to minimize a DFA
  1. Partition all states into two groups:
     a. Group 1 (G1) contains all final states.
     b. Group 2 (G2) contains all non-final states.

How to minimize a DFA

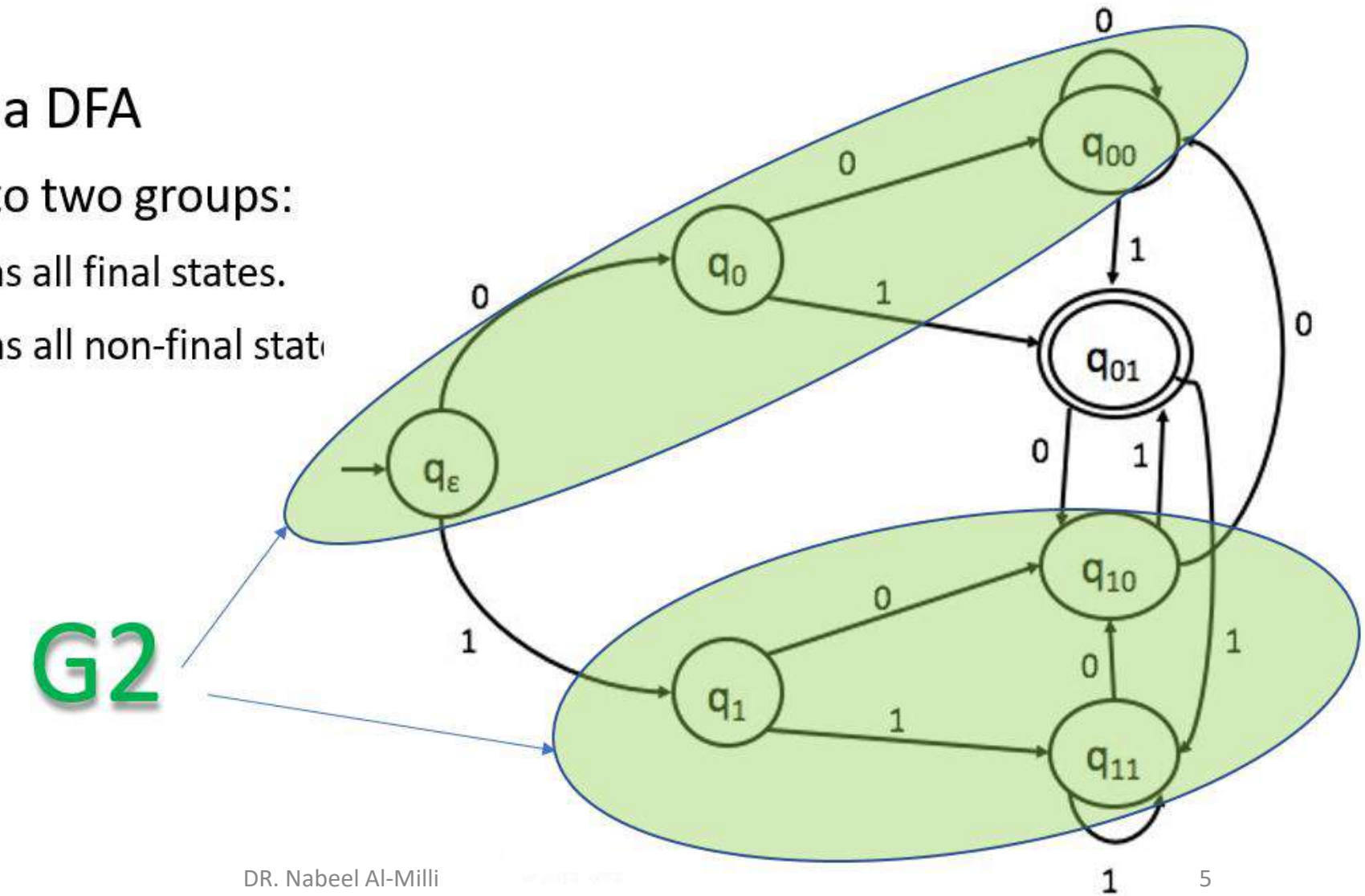
- Algorithm to minimize a DFA
  1. partition all states into two groups:
     a. Group 1 (G1) contains all final states.
     b. Group 2 (G2) contains all non-final stat

G1



How to minimize a DFA

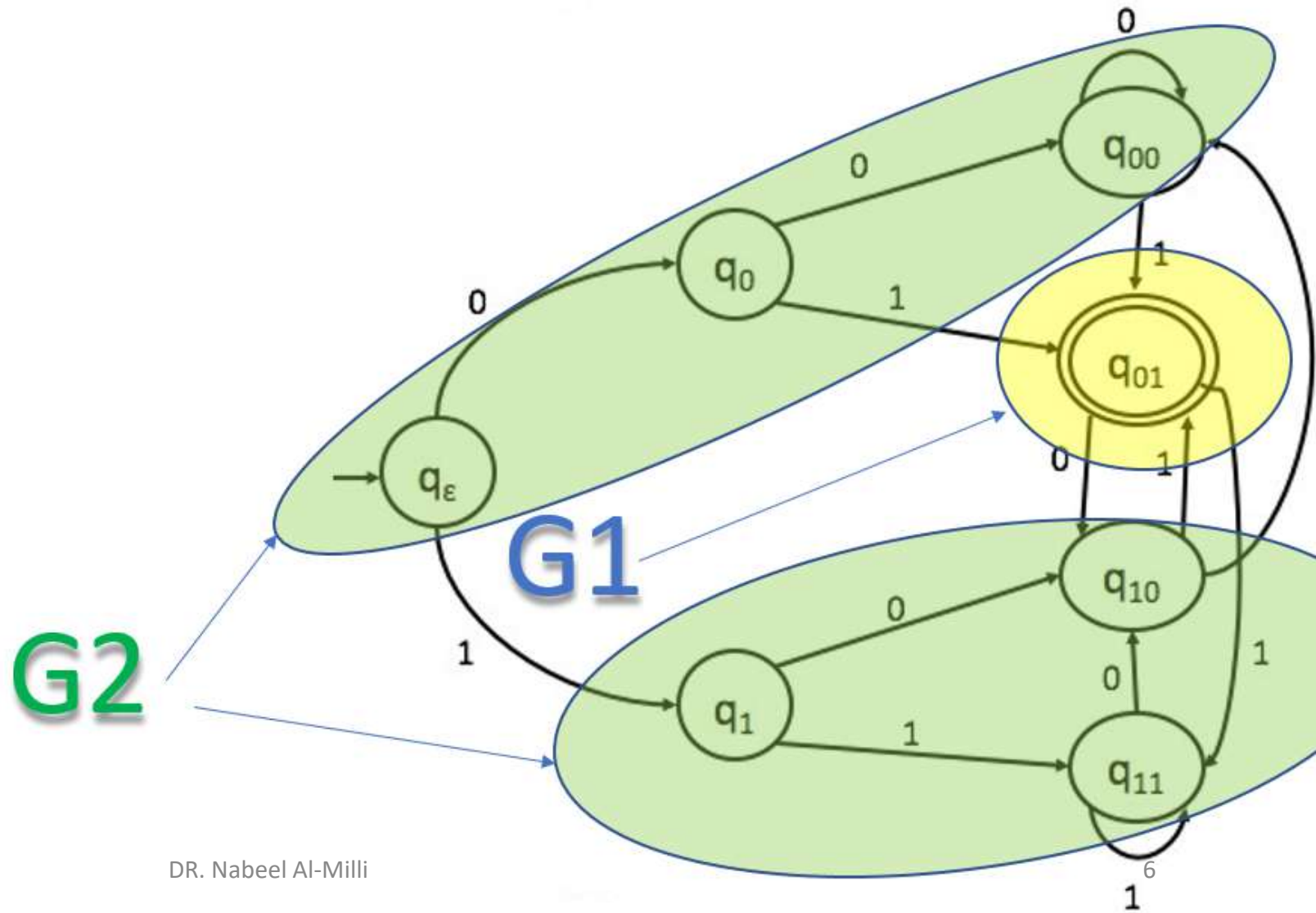
- Algorithm to minimize a DFA
  1. Partition all states into two groups:
     a. Group 1 (G1) contains all final states.
     b. Group 2 (G2) contains all non-final stat



How to minimize a DFA

2. for each group, check whether all states in the group go to the same next group for each input symbol.

- If not, partition it into smaller groups so that all states in each smaller group go to the same next group for each input symbol.
- Repeat step 2 until no more partitions.

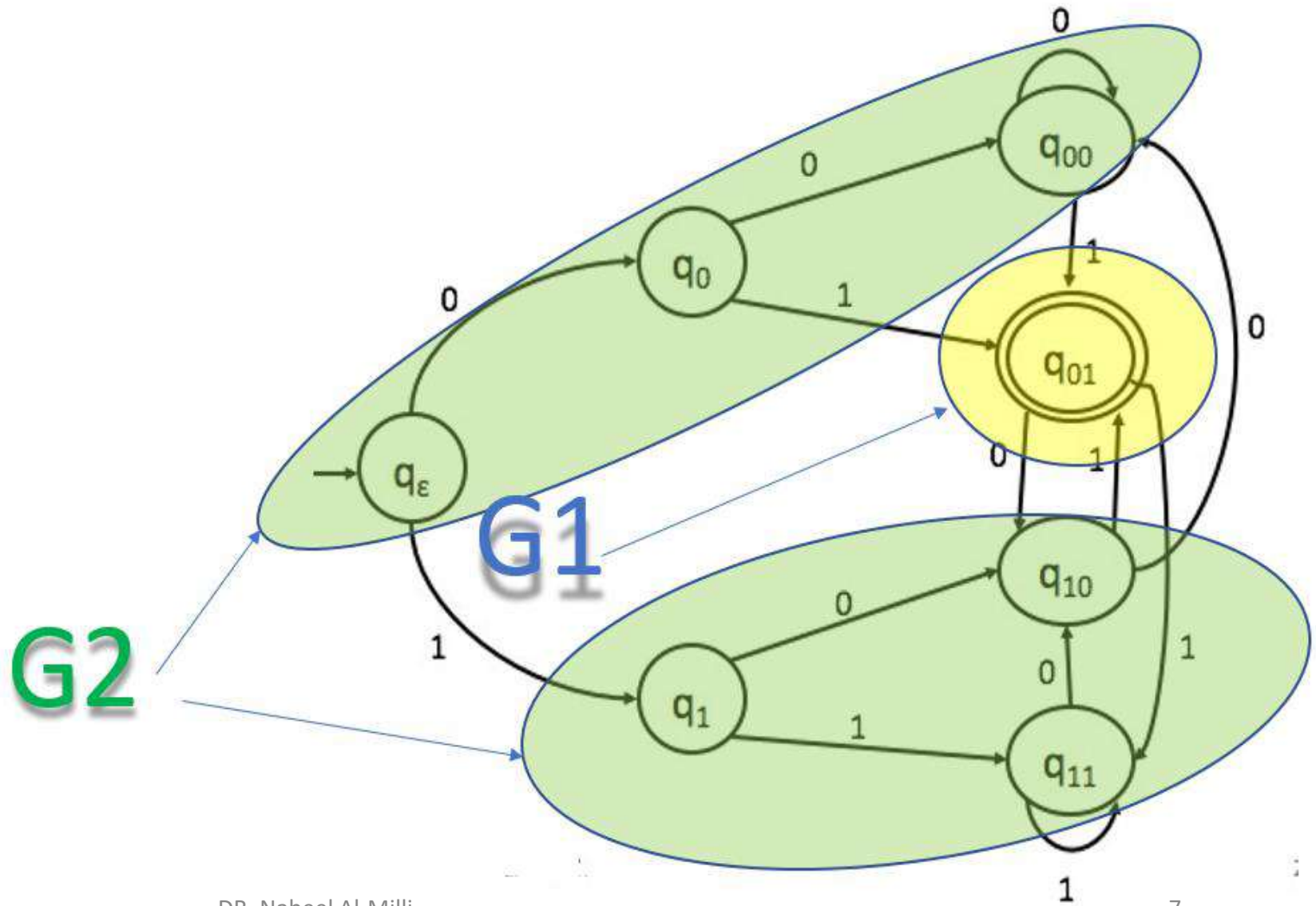


G1 (final states)

	0	1
q01		

G2 (non-final states)

	0	1
q_ε		
q0		
q1		
q00		
q10		
q11		



* بالأول بقسم الـ State إلي عندي

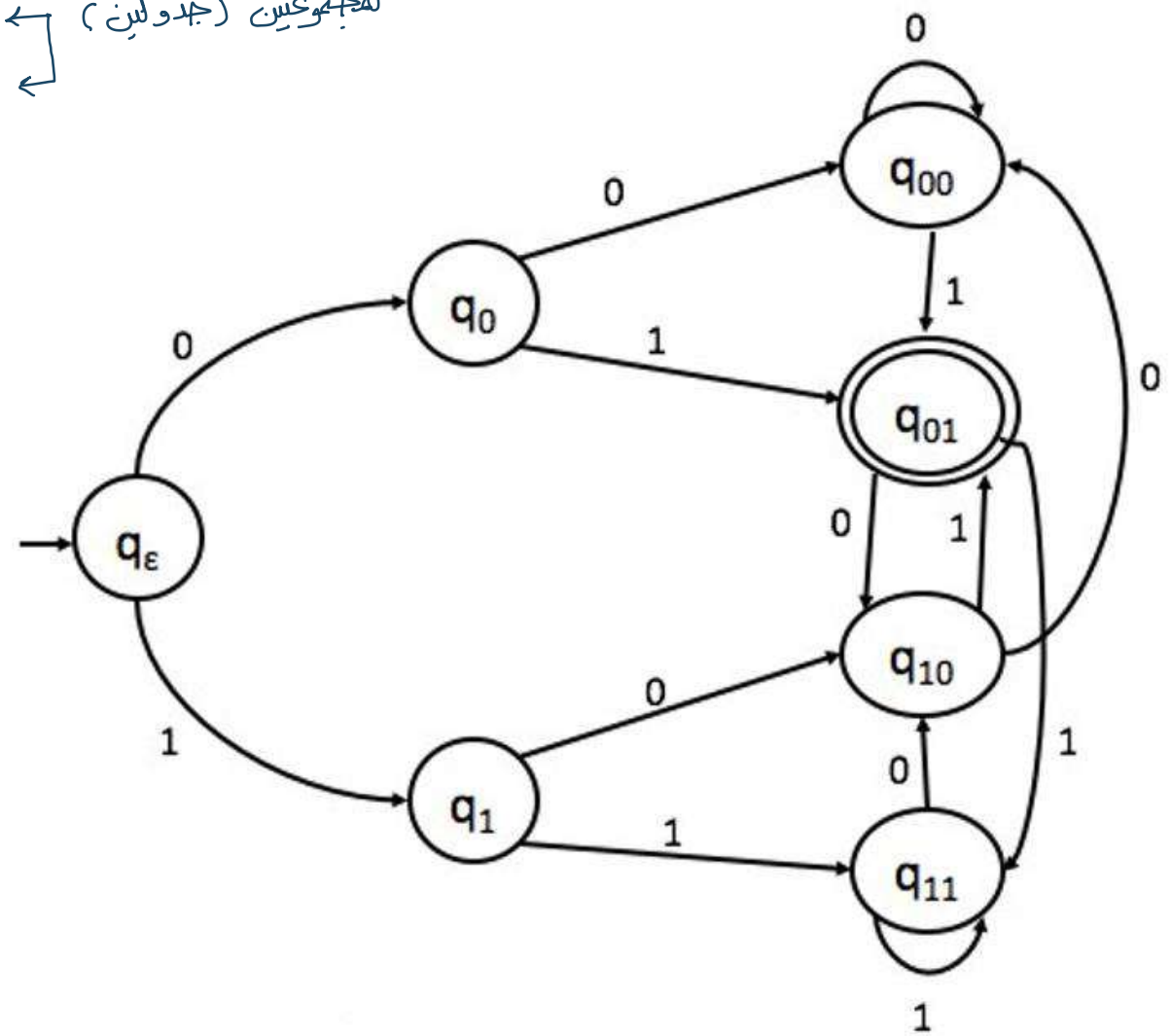
Final ← لمجموعتين (جدولين)
non-Final ←

G1 (final states)

	0	1
q01		

G2 (non-final states)

	0	1
q_ε		
q0		
q1		
q00		
q10		
q11		

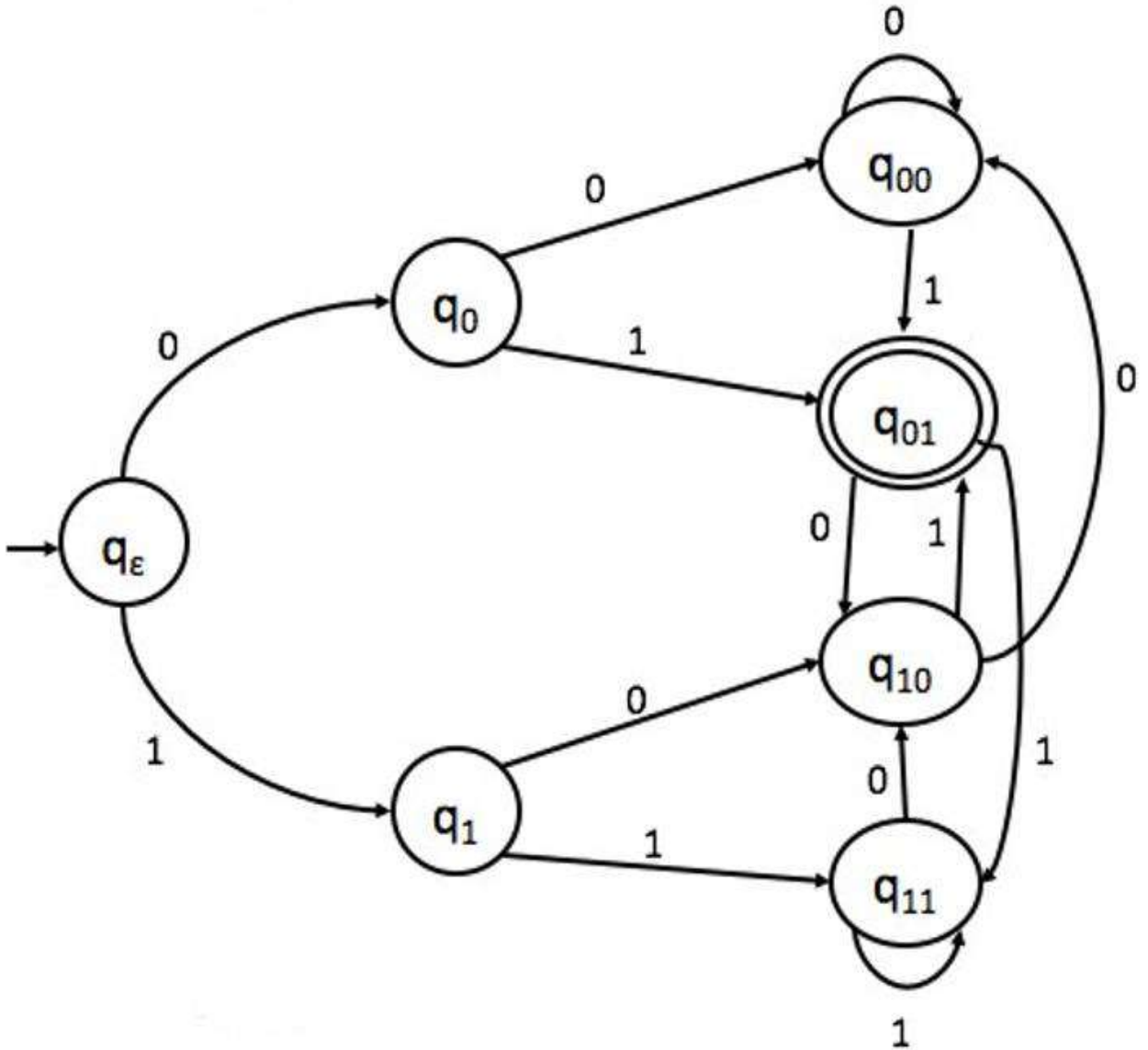


G1 (final states)

	0	1
q01	G2(q10)	

G2 (non-final states)

	0	1
q_ε		
q0		
q1		
q00		
q10		
q11		

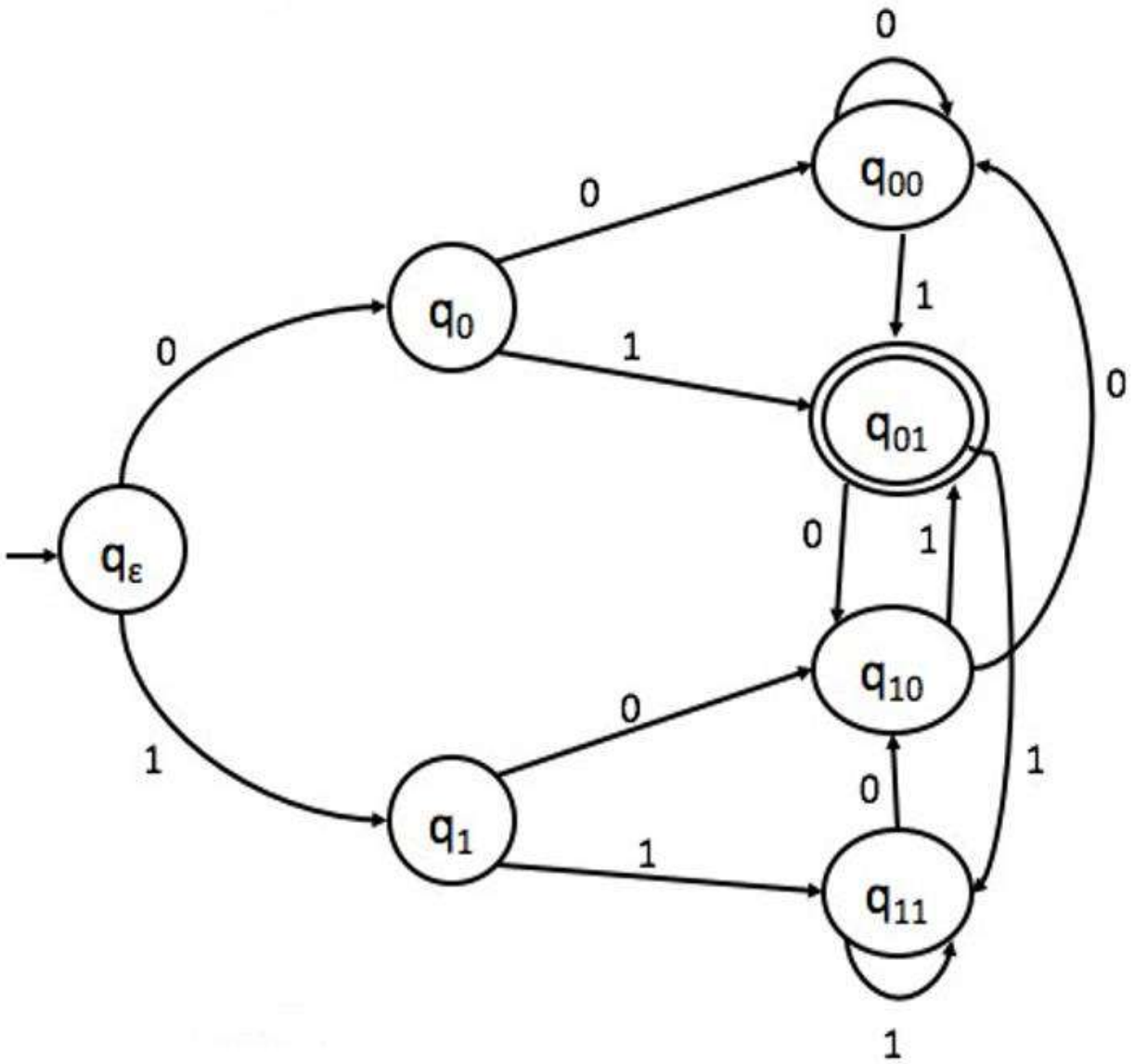


G1 (final states)

	0	1
q01	G2(q10)	G2 (q11)

G2 (non-final states)

	0	1
q_ε	G2(q0)	
q0		
q1		
q00		
q10		
q11		



لما أجاها ٥ راحت على q_{10} الموجود بـ G_2

لما أجاها ← يرجع راحت على q_{11} الموجود بـ G_2

G1 (final states) ←

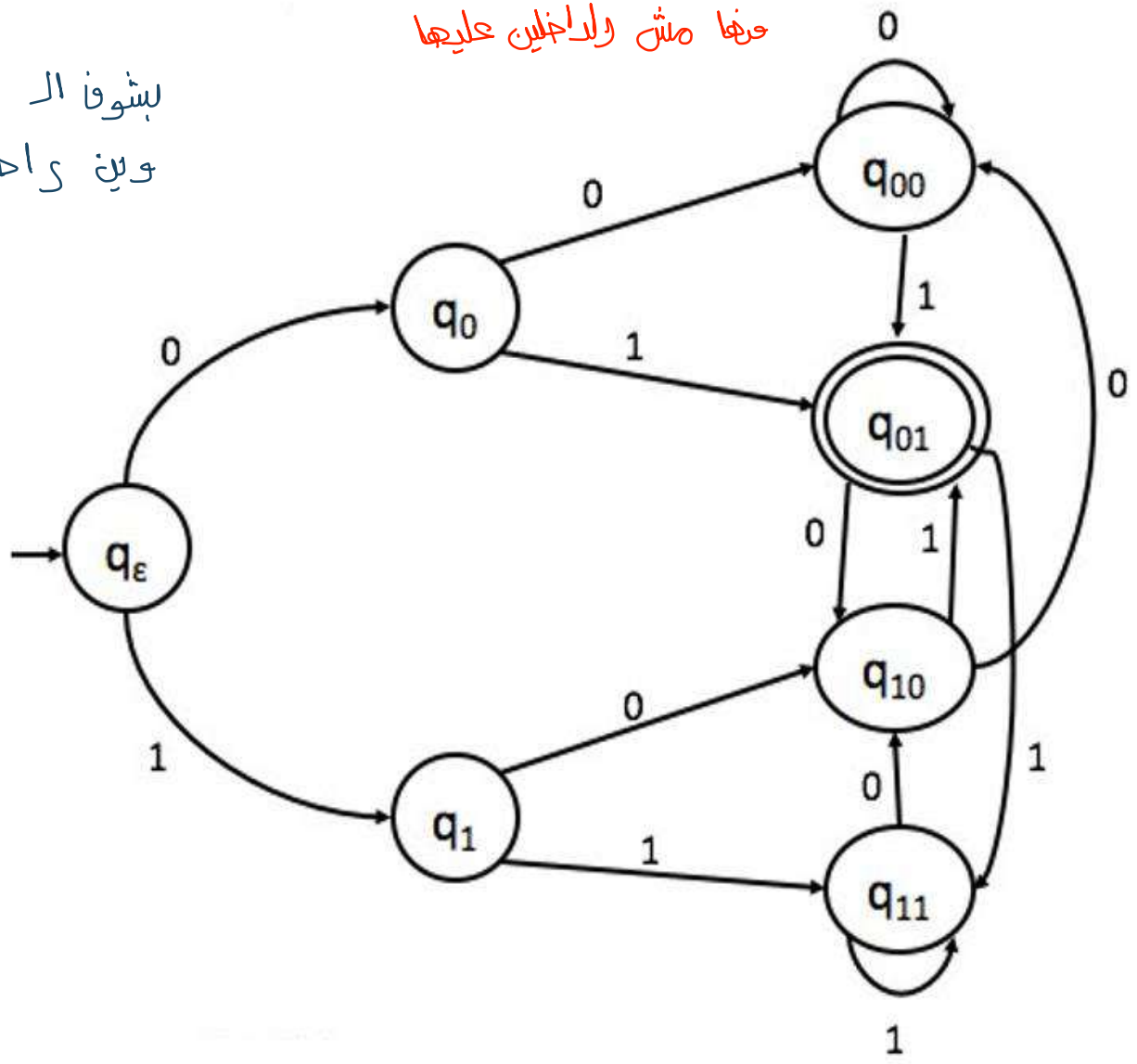
	0	1
q01	G2(q10)	G2 (q11)

→ بشوف الـ q_{01} لما أجاها ٥ وين راحت ← على أي state ← يرجع اسم الـ Group إلي انتقلت على الـ state

G2 (non-final states)

	0	1	
q_ε	G2(q0)	G2(q0)	→22
q0	$G_2(q_{00})$	$G_1(q_{01})$	→21
q1	$G_2(q_{10})$	$G_2(q_{11})$	→22
q00	$G_2(q_{00})$	$G_1(q_{01})$	→21
q10	$G_2(q_{00})$	$G_1(q_{01})$	→21
q11	$G_2(q_{do})$	$G_2(q_{11})$	→22

← إلي بهمني بال state هو عدد الأسهم إلي خارجين منها مش والداخلين عليها

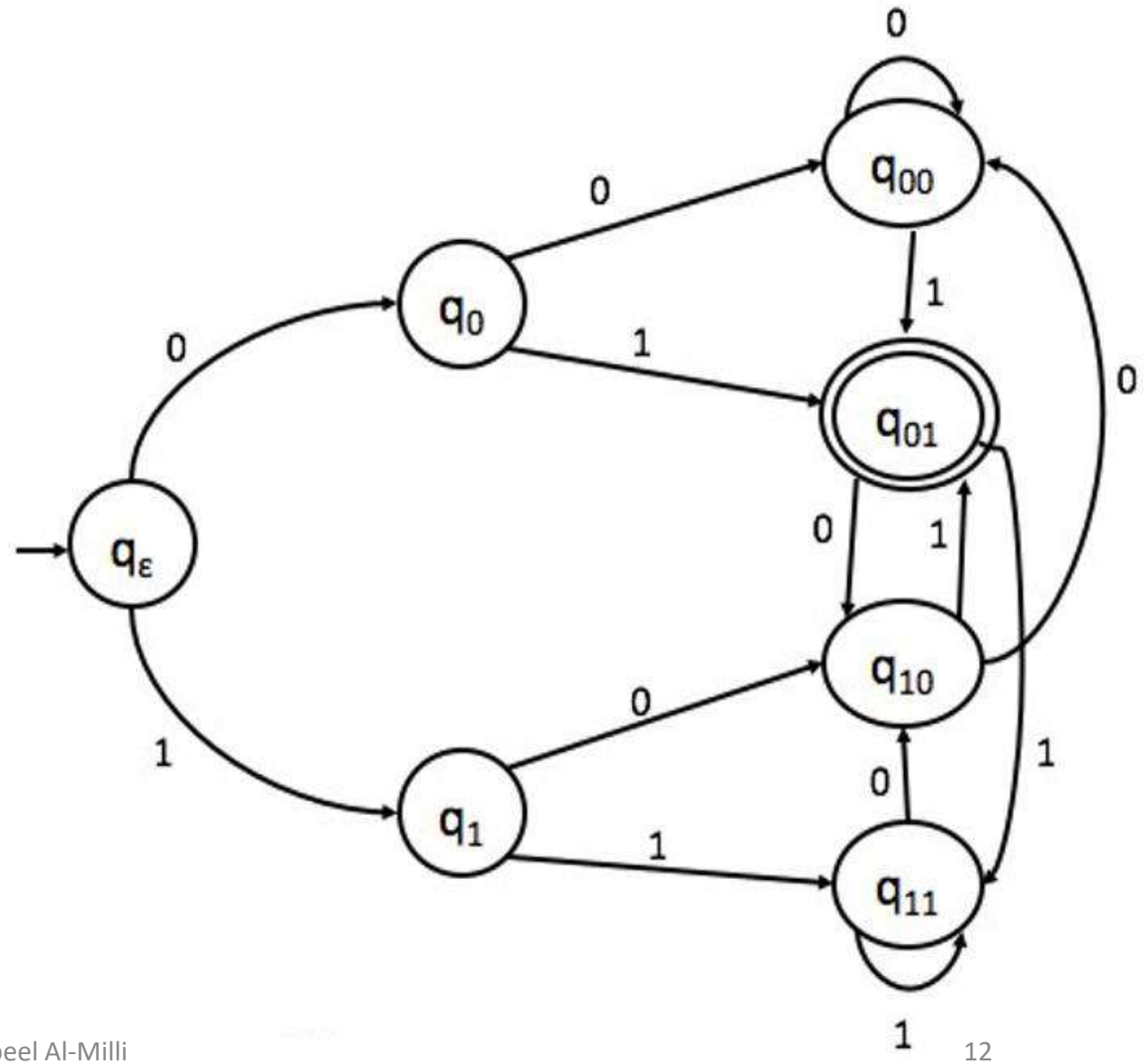


G1 (final states)

	0	1
q01	G2(q10)	G2 (q11)

G2 (non-final states)

	0	1
q_ε	G2(q0)	G2(q0)
q0	G2(q00)	G1(q01)
q1		
q00		
q10		
q11		



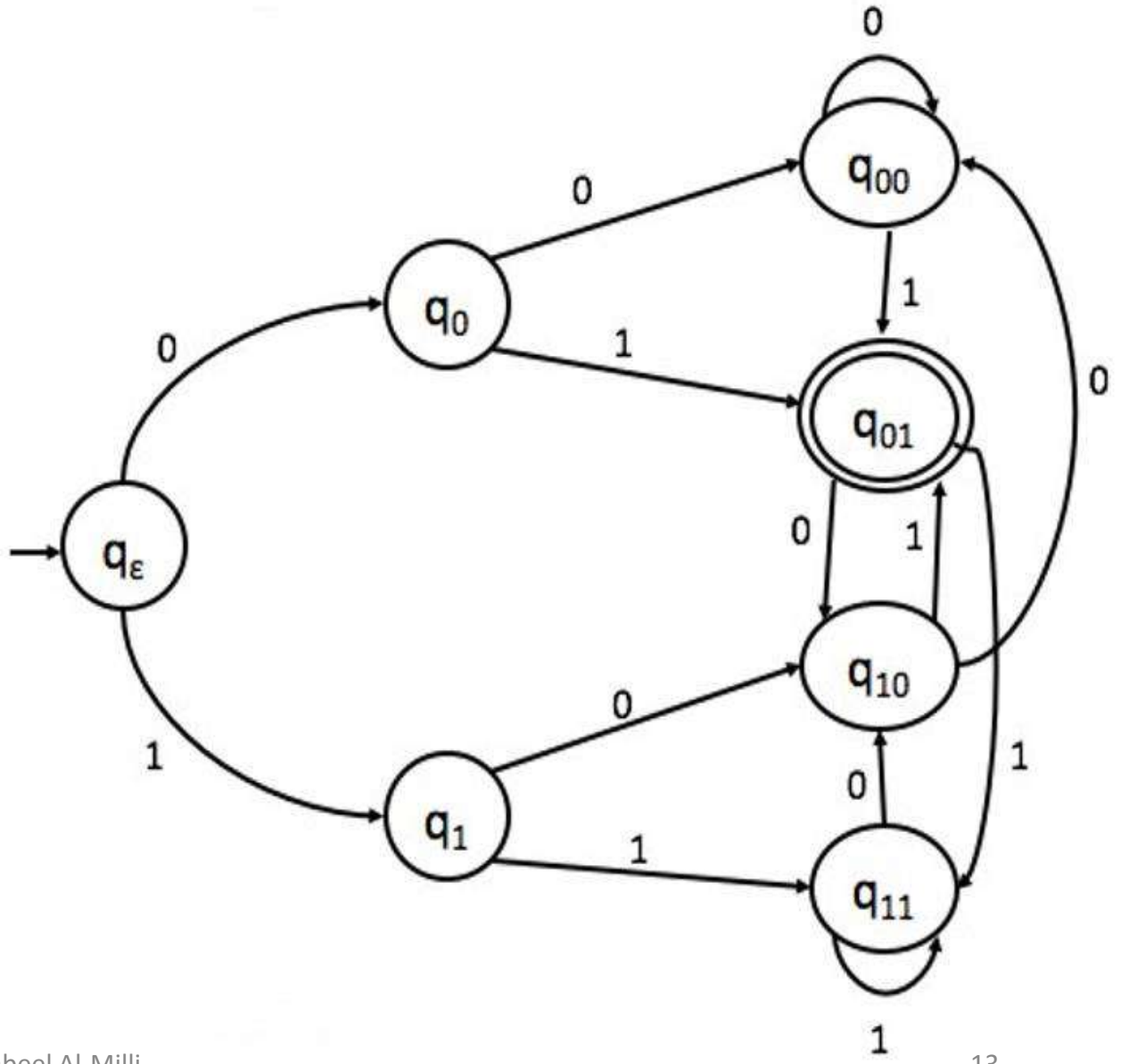
G1 (final states)

	0	1
q01	G2(q10)	G2 (q11)

G2 (non-final states)

	0	1
q_ε	G2(q0)	G2(q0)
q0	G2(q00)	G1(q01)
q1	G2(q10)	G2(q11)
q00	G2(q00)	G1(q01)
q10	G2(q00)	G1(q01)
q11	G2(q10)	G2(q11)

→ block بقارن ك وحدة ← كل مرة now زج واحد

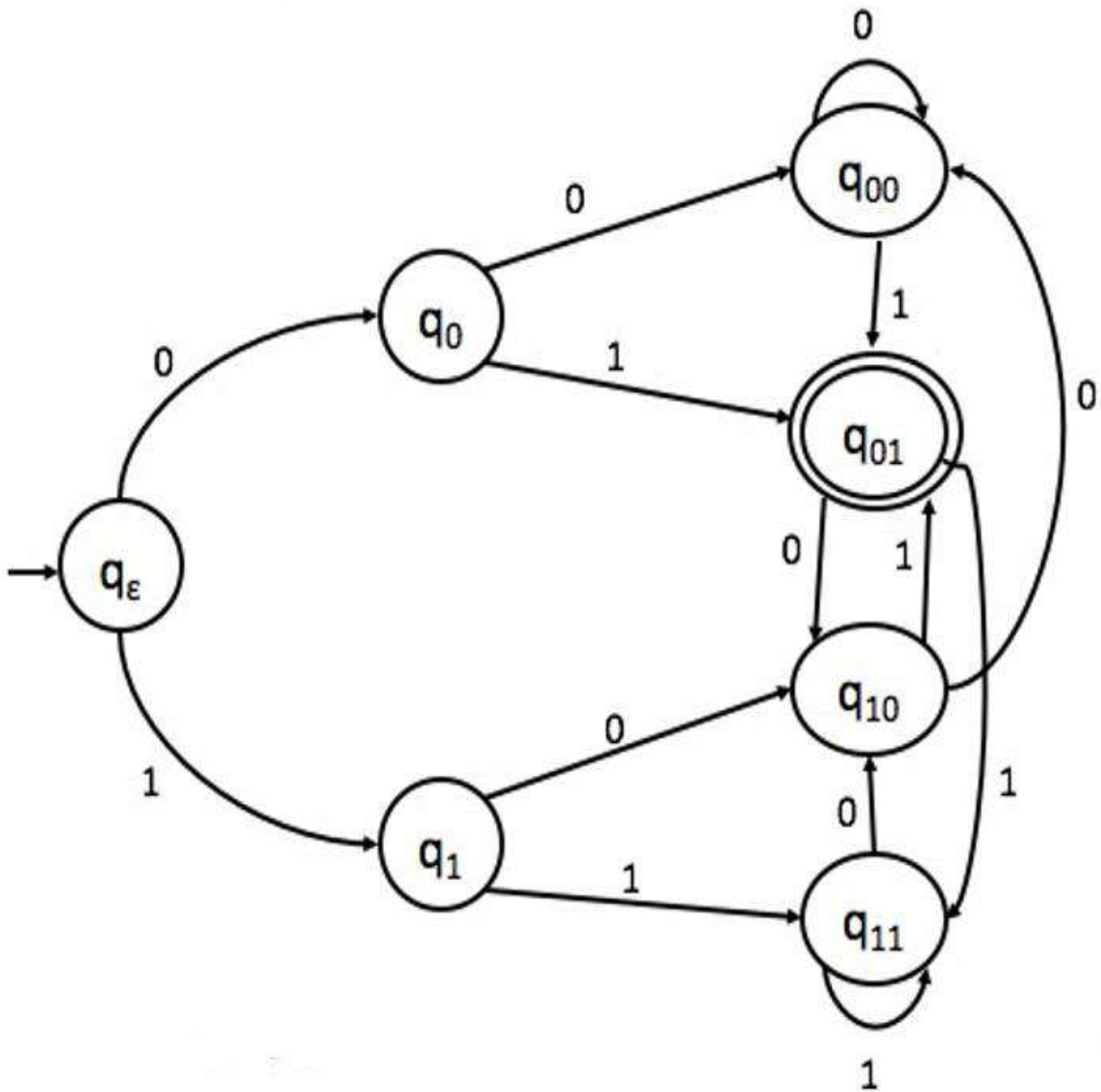


G1 (final states)

	0	1
q01	G2(q10)	G2 (q11)

G2 (non-final states)

	0	1
q_ε	G2(q0)	G2(q0)
q0	G2(q00)	G1(q01)
q1	G2(q10)	G2(q11)
q00	G2(q00)	G1(q01)
q10	G2(q00)	G1(q01)
q11	G2(q10)	G2(q11)



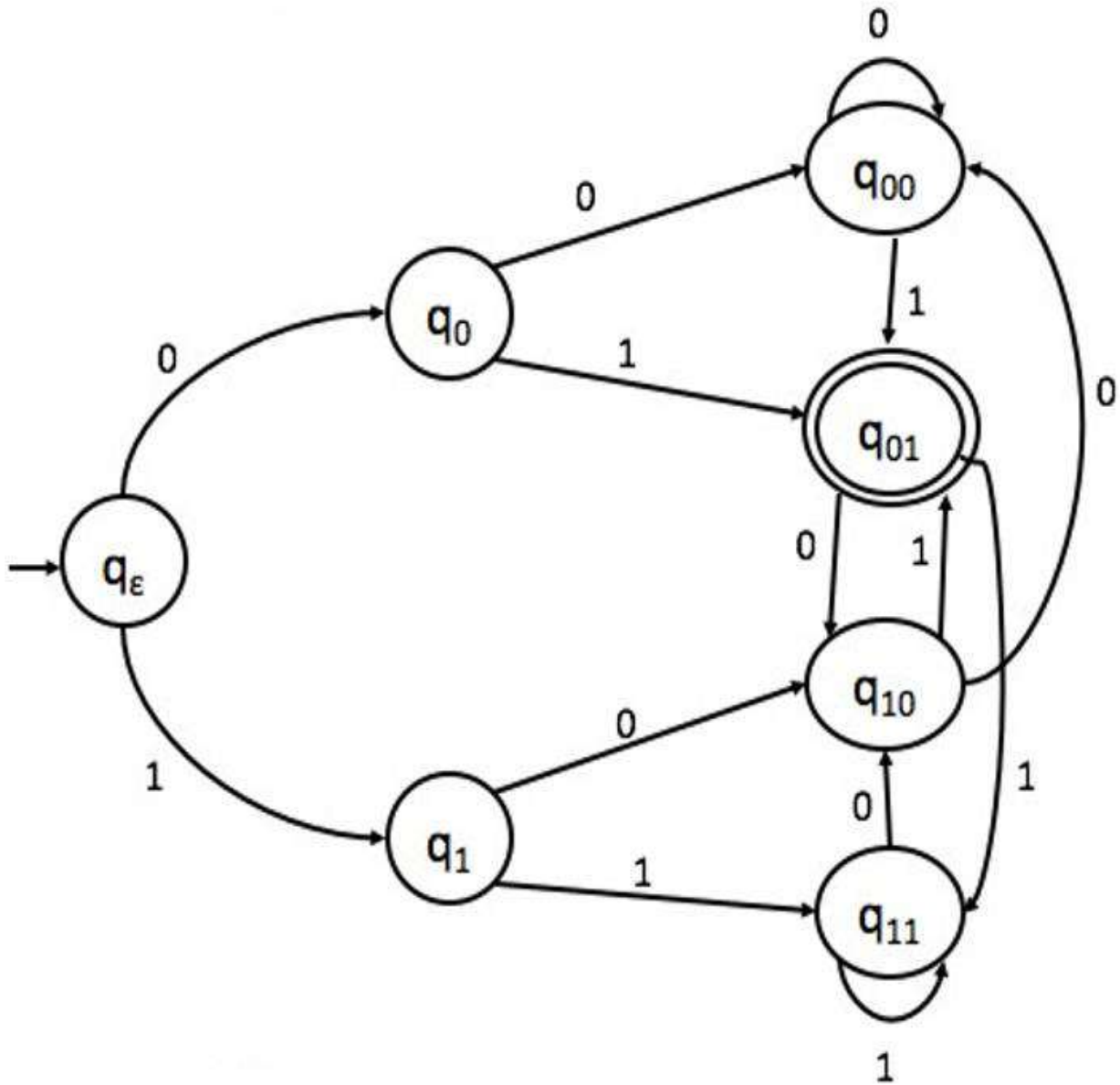
G1 (final states)

	0	1
q01	G2(q10)	G2 (q11)

G2 (non-final states)

	0	1
q_ε	G2(q0)	G2(q0)
q0	G2(q00)	G1(q01)
q1	G2(q10)	G2(q11)
q00	G2(q00)	G1(q01)
q10	G2(q00)	G1(q01)
q11	G2(q10)	G2(q11)

G21



G1 (final states)

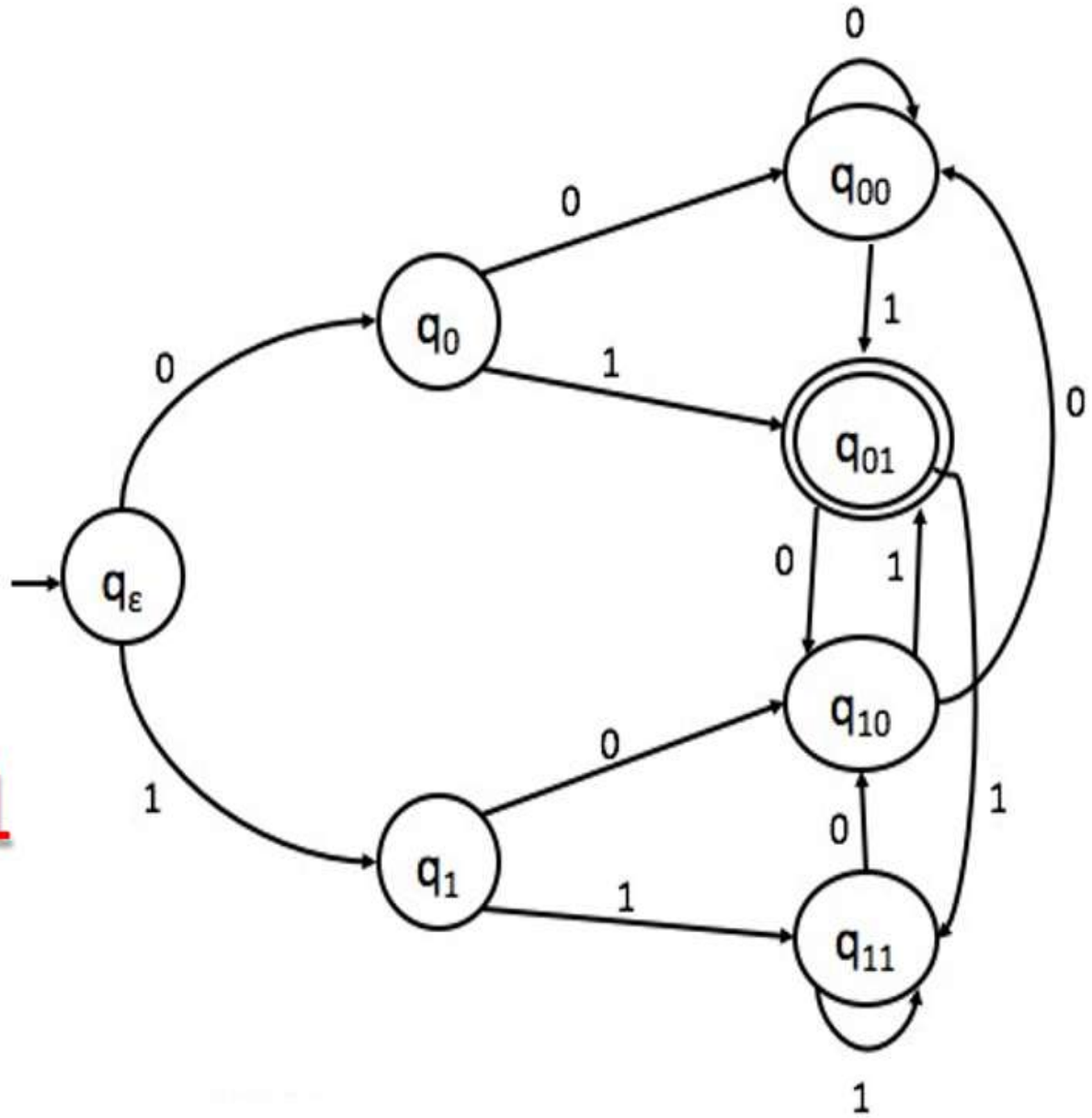
	0	1
q01	G2(q10)	G2 (q11)

G2 (non-final states)

	0	1
q_ε	G2(q0)	G2(q0)
q0	G2(q00)	G1(q01)
q1	G2(q10)	G2(q11)
q00	G2(q00)	G1(q01)
q10	G2(q00)	G1(q01)
q11	G2(q10)	G2(q11)

G22

G21



G1 (final states)

	0	1
q01	G21(q10)	G22(q11)

G21 (non-final states)

	0	1
q0	G21 (q00)	G1 (q01)
q00	G21(q00)	G1 (q01)
q10	G21 (q00)	G1 (q01)

G22 (non-final states)

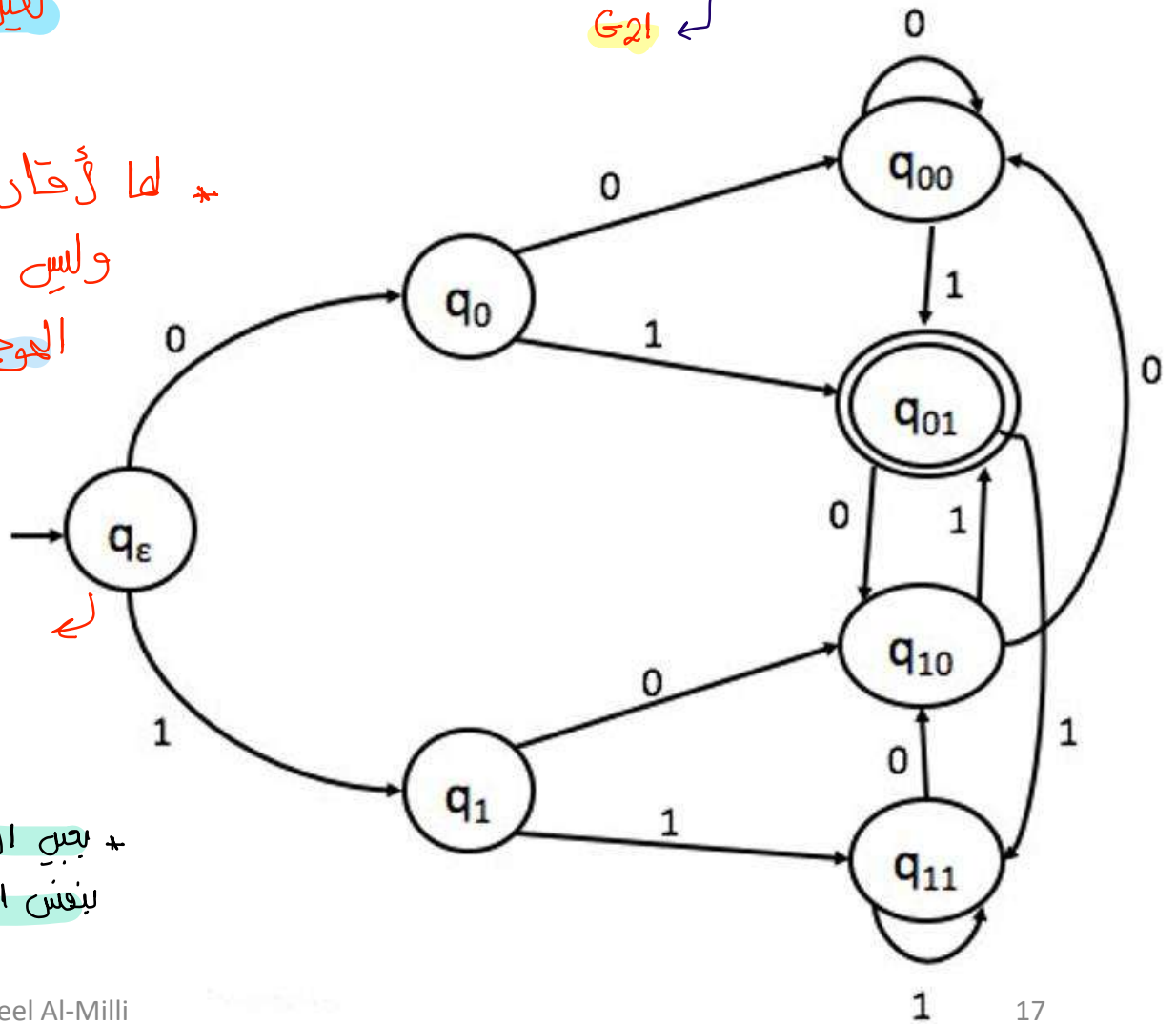
	0	1
q_ε	G21(q0)	G22(q1)
q1	G21(q10)	G22(q11)
q11	G21(q10)	G22 (q11)

* مهم أنقل الـ state بدون إجاباتهم لأنه أسماء الجداول تتغير

* لما أقارن بقارن بأسماء الـ Group وليس بأسماء الـ state الموجودة داخل القوس

* بقسم الجداول على أساس الحالات إلي عندي

Final ← G1
G22
G21



ناقص state

* بجيب الجداول كل مرة بنفس الطريقة *

G1 (final states)

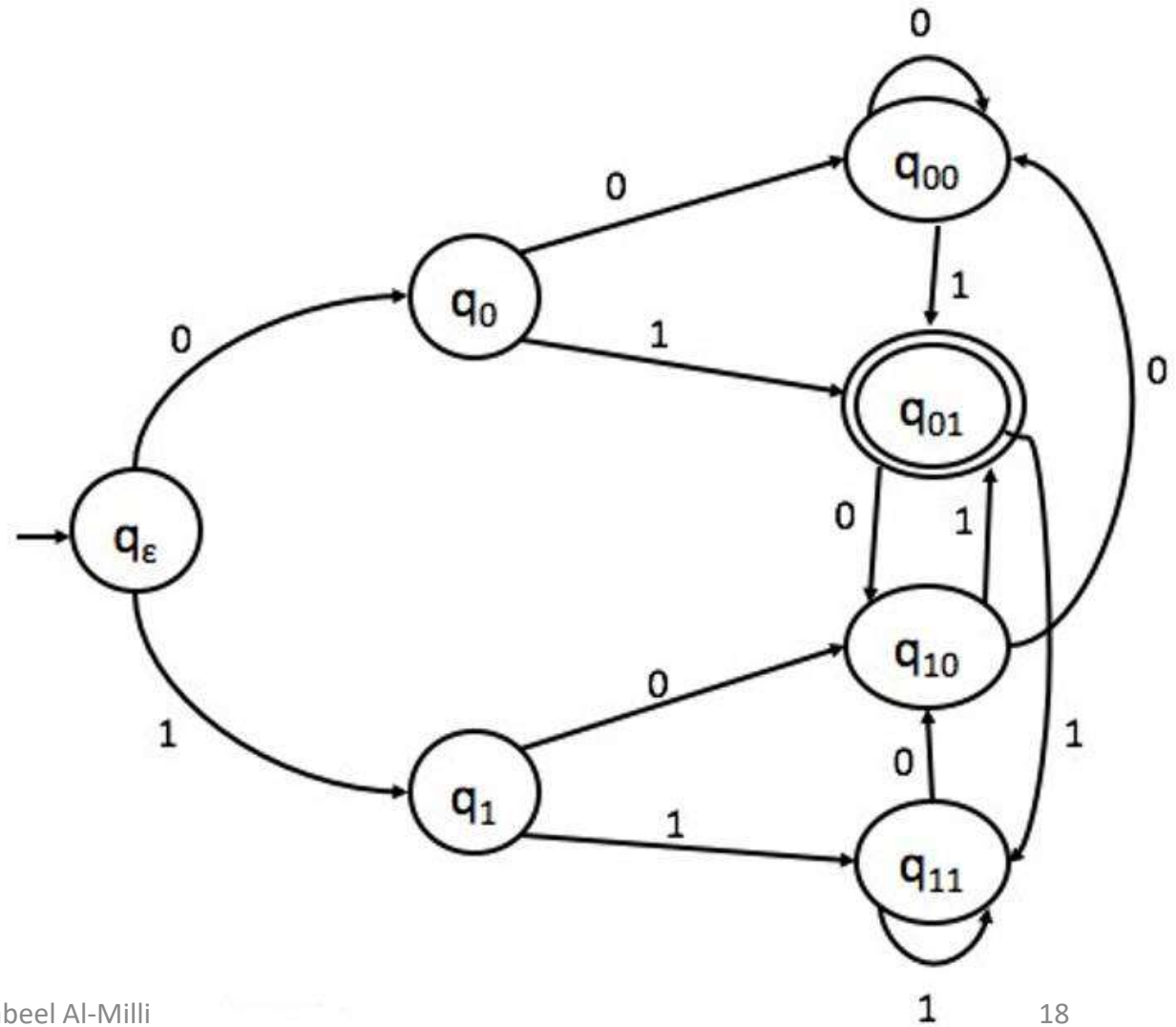
	0	1
q01	G21	G22

G21 (non-final states)

	0	1
q0	G21	G1
q00	G21	G1
q10	G21	G1

G22 (non-final states)

	0	1
q_ε	G21	G22
q1	G21	G22
q11	G21	G22



G1 (final states)

	0	1
q01	G21	G22

ما يصير عليه لأنه زوج واحد

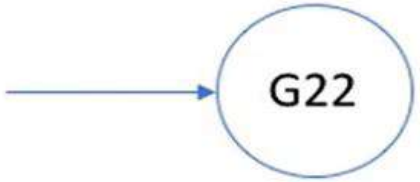
G21 (non-final states)

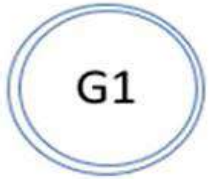
	0	1
q0	G21	G1
q00	G21	G1
q10	G21	G1

G22 (non-final states)

	0	1
q_ε	G21	G22
q1	G21	G22
q11	G21	G22

* ما يرجع يقسم لأنه كلهم أزواج مرتبة متشابهة





G1 (final states)

	0	1
q01	G21	G22

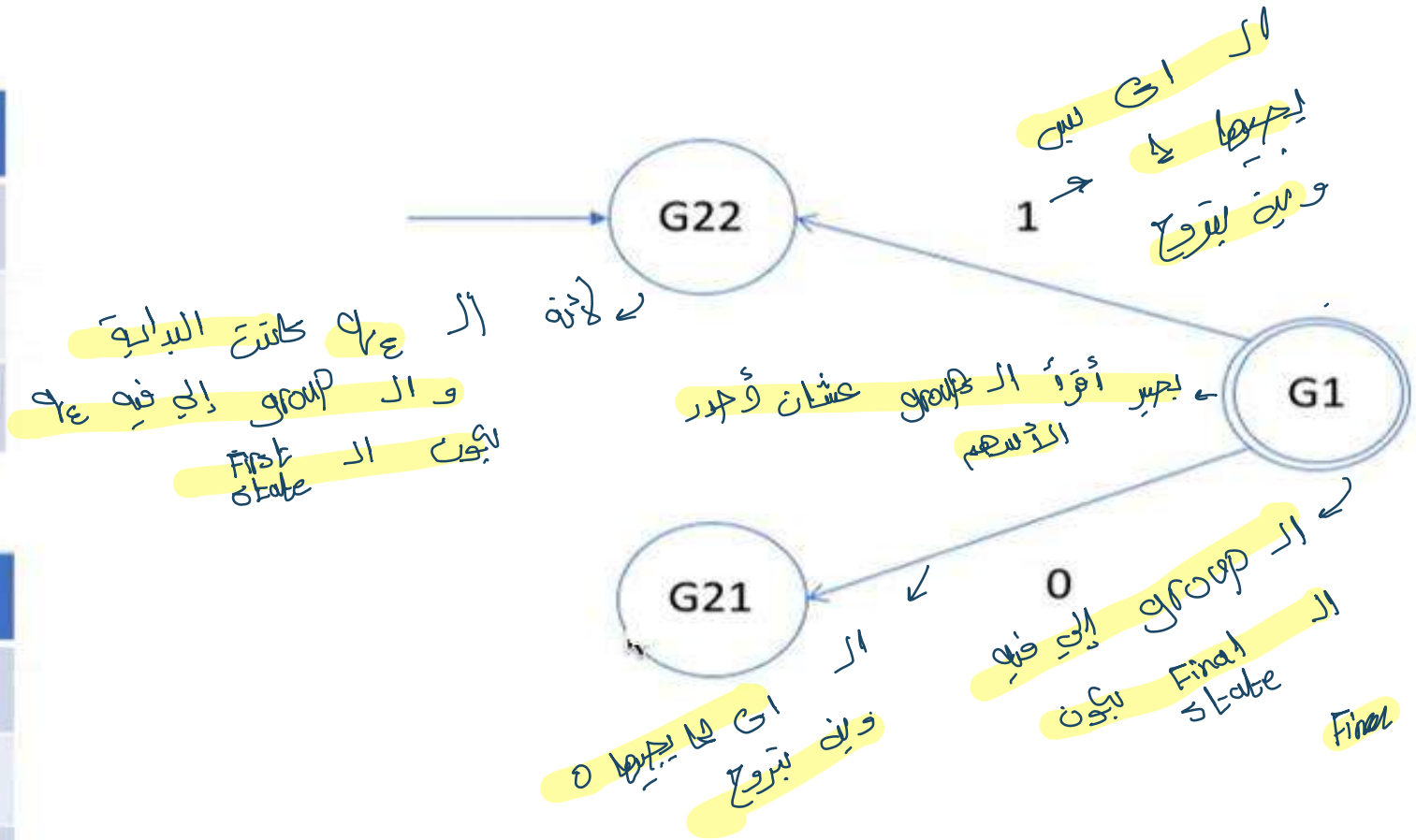
G21 (non-final states)

	0	1
q0	G21	G1
q00	G21	G1
q10	G21	G1

G22 (non-final states)

	0	1
q_ε	G21	G22
q1	G21	G22
q11	G21	G22

* محل جدول حالة علذي بعد التقسيم
state بتمثل



G1 (final states)

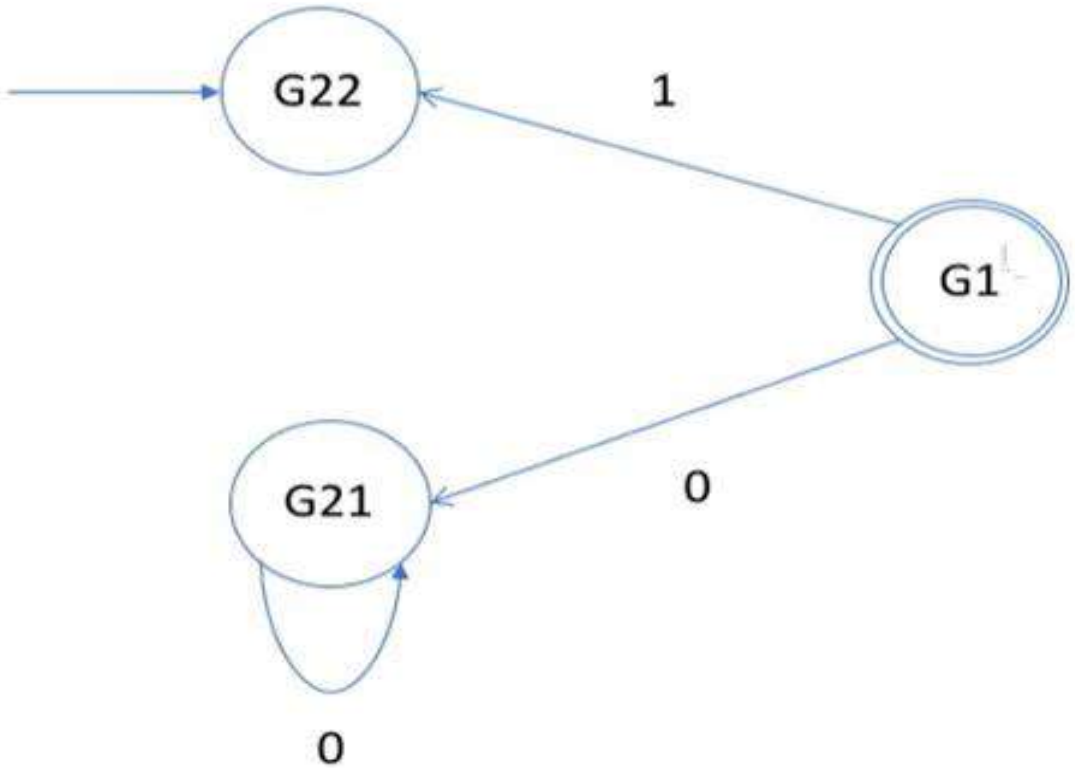
	0	1
q01	G21	G22

G21 (non-final states)

	0	1
q0	G21	G1
q00	G21	G1
q10	G21	G1

G22 (non-final states)

	0	1
q_ε	G21	G22
q1	G21	G22
q11	G21	G22



G1 (final states)

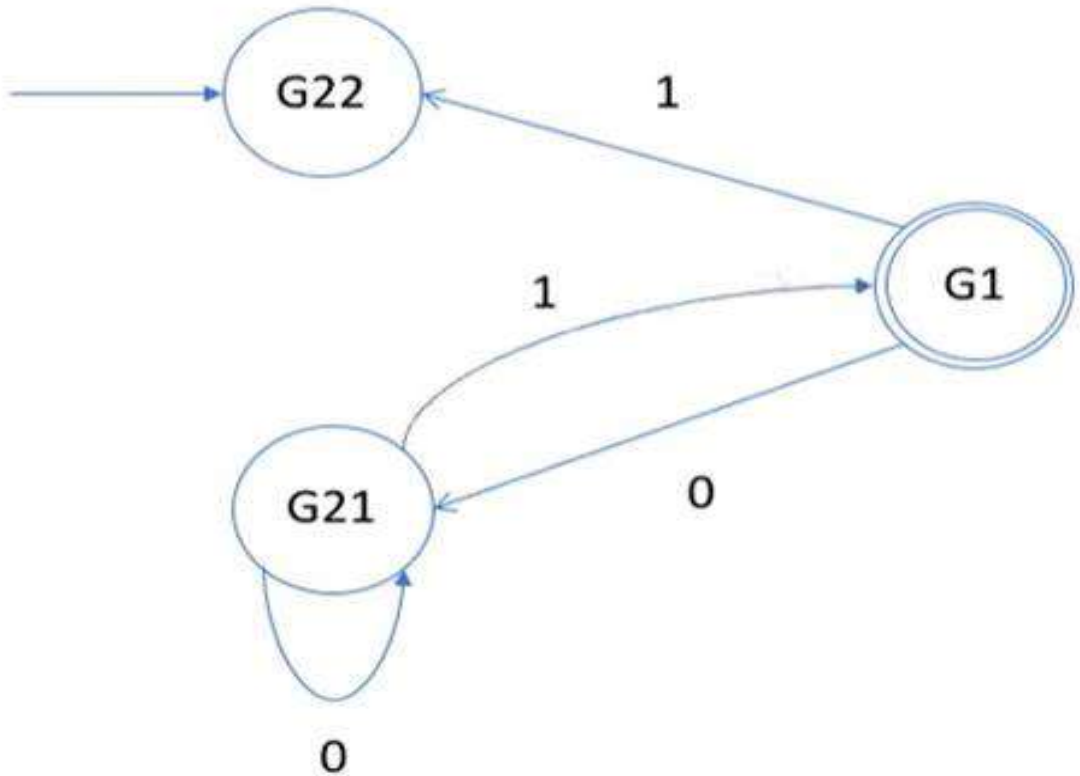
	0	1
q01	G21	G22

G21 (non-final states)

	0	1
q0	G21	G1
q00	G21	G1
q10	G21	G1

G22 (non-final states)

	0	1
q_ε	G21	G22
q1	G21	G22
q11	G21	G22



G1 (final states)

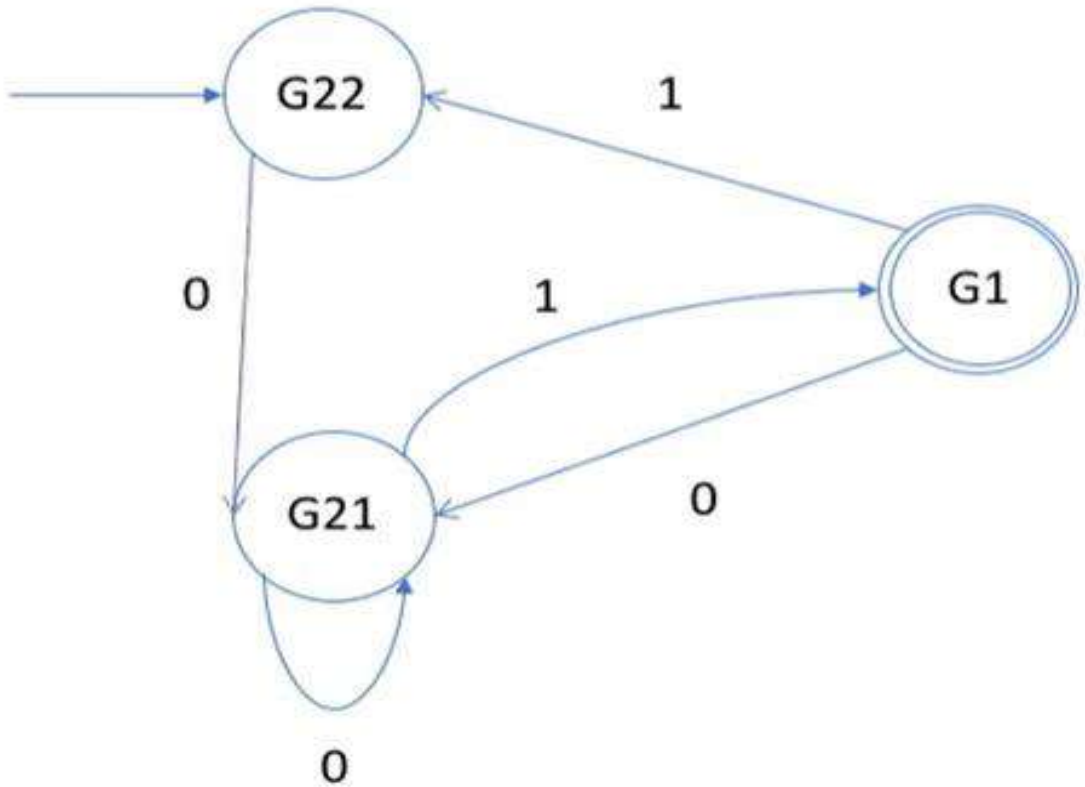
	0	1
q01	G21	G22

G21 (non-final states)

	0	1
q0	G21	G1
q00	G21	G1
q10	G21	G1

G22 (non-final states)

	0	1
q_ε	G21	G22
q1	G21	G22
q11	G21	G22



G1 (final states)

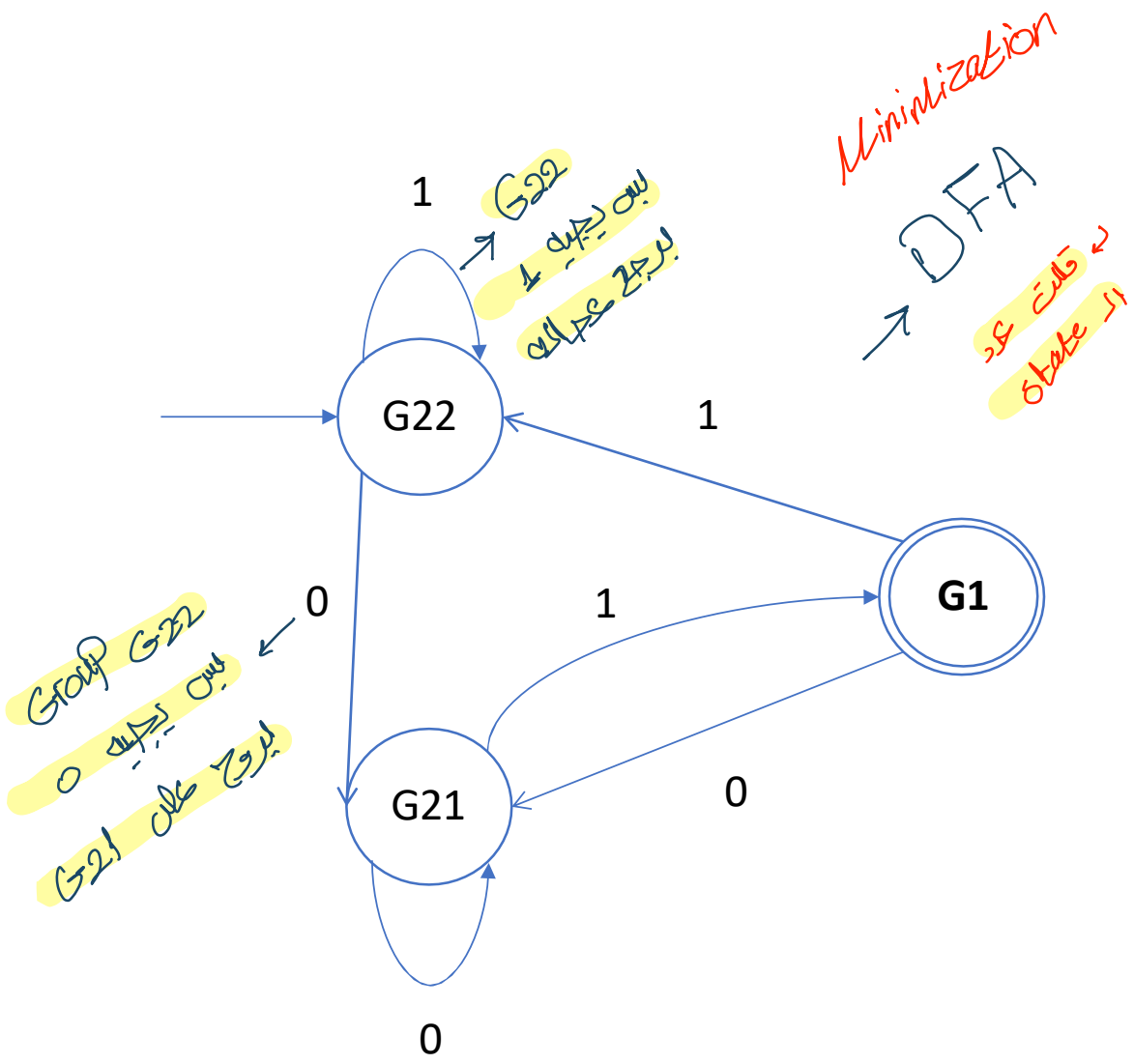
	0	1
q01	G21	G22

G21 (non-final states)

	0	1
q0	G21	G1
q00	G21	G1
q10	G21	G1

G22 (non-final states)

	0	1
q_ε	G21	G22
q1	G21	G22
q11	G21	G22



THE END

Converting NFA to DFA

Which is stronger? D

- NFA might be easier to design and to understand. Because we don't need to worry about all conditions.

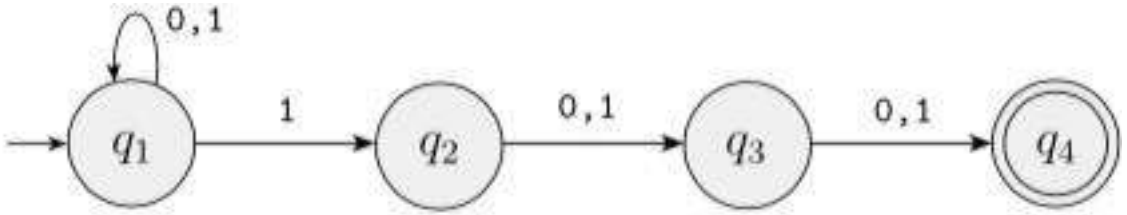


FIGURE 1.31
The NFA N_2 recognizing A

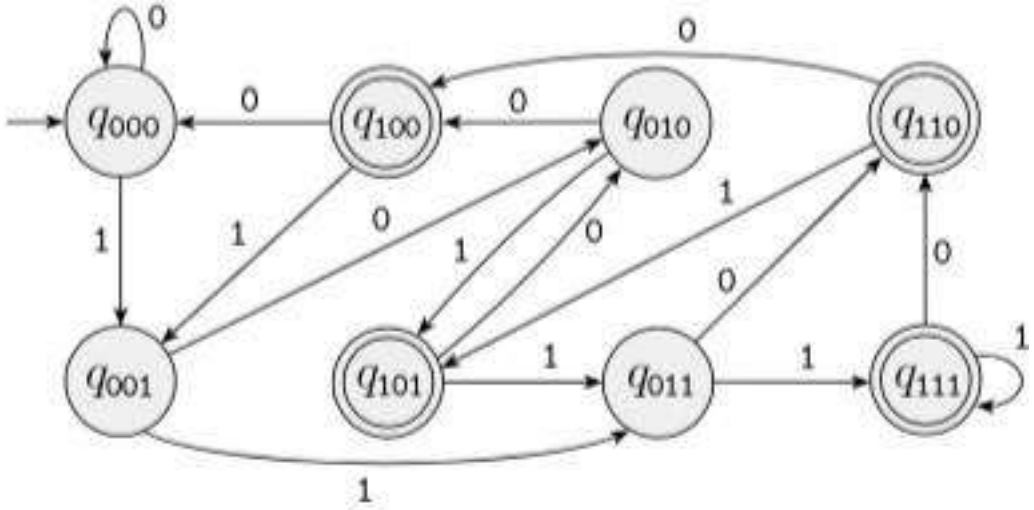


FIGURE 1.32
A DFA recognizing A

Which is stronger? D

- However, NFA and DFA have the same power and can recognize the same class of languages.
- So, every NFA can be converted to DFA.

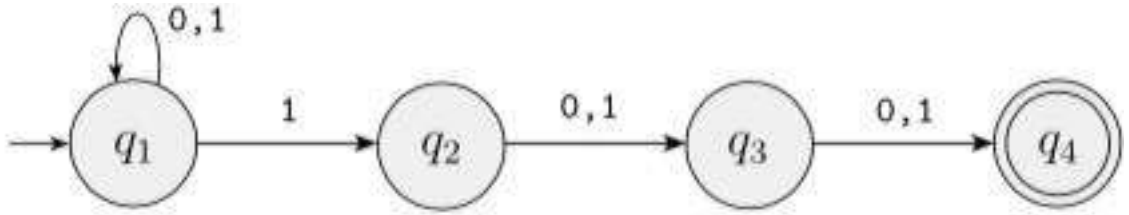


FIGURE 1.31
The NFA N_2 recognizing A

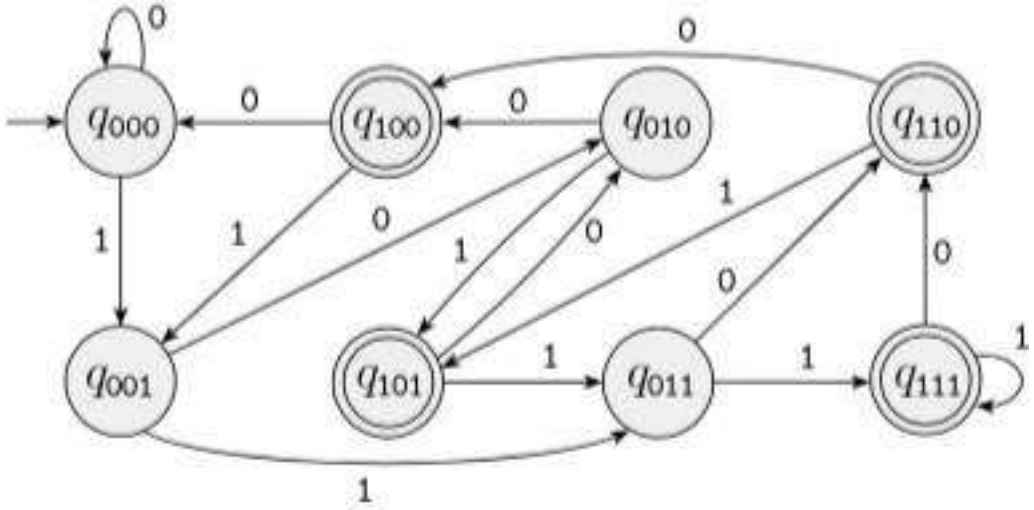
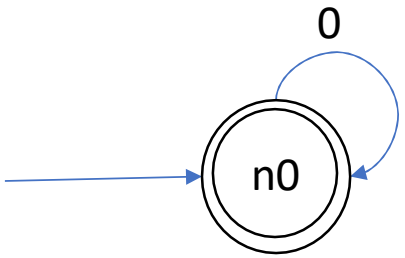


FIGURE 1.32
A DFA recognizing A

Example 1: NFA with one state

NFA



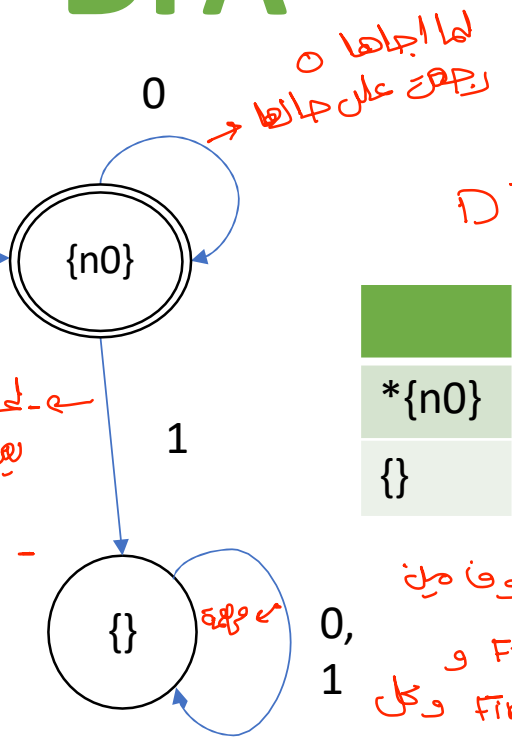
جدول الـ NFA

	0	1
*n0	n0	

← بشوف الـ n0 لما يجيها 0 وين بتروح ولما يجيها 1 وين بتروح ← n0 ← لما يجيها 0 بتروح لـ n0 وهون state البداية نفسه النهاية. لما يجيها 1 بتروح لـ empty ← يعني ما بتروح لمكان

DFA

هي نفس الحالتين ← First state الـ



← لما اجاها 1 طارحت لـ اي state بعين empty state

- لازم ارجع عليها سهم فيه 0,1 او فيه الـ alphabet الموجودة عندي حسب شو بتكون

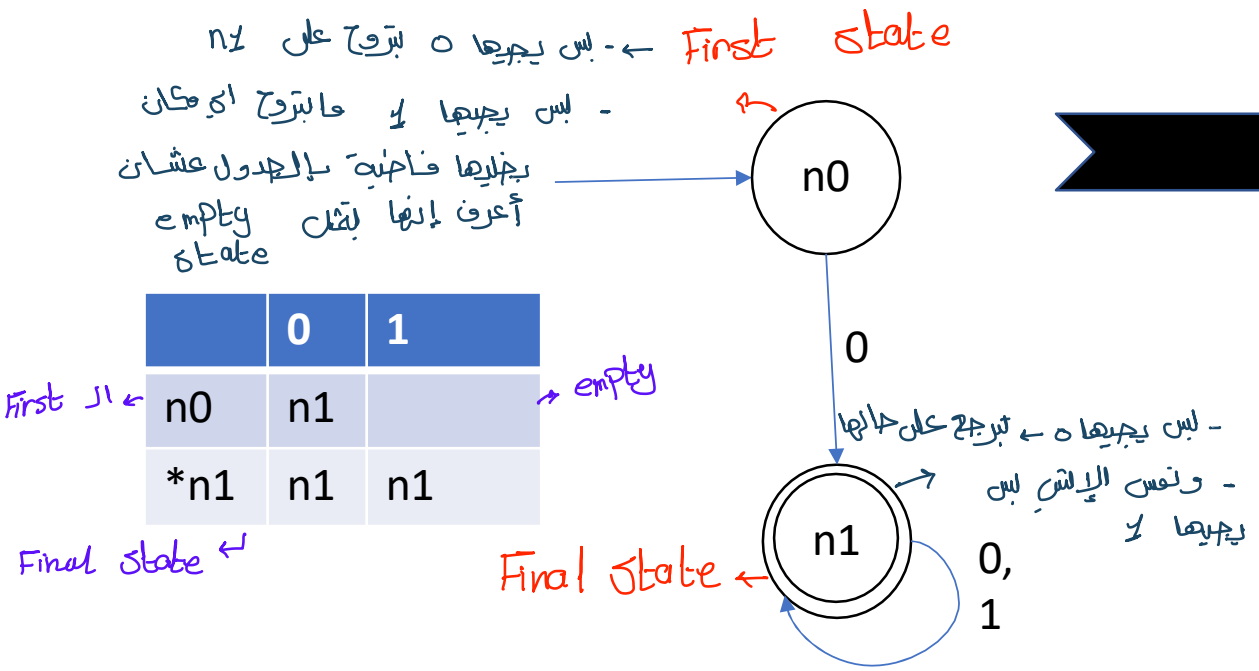
جدول DFA -

	0	1
*{n0}	{n0}	{}
{}	{}	{}

- نفس الإشي بشوف مين الـ First state و Final وكل وحدة وين بتروح
- الـ empty state حزورة ارجع عليها 0,1 دائماً"
- الـ empty state بترجل بالجدول عادي

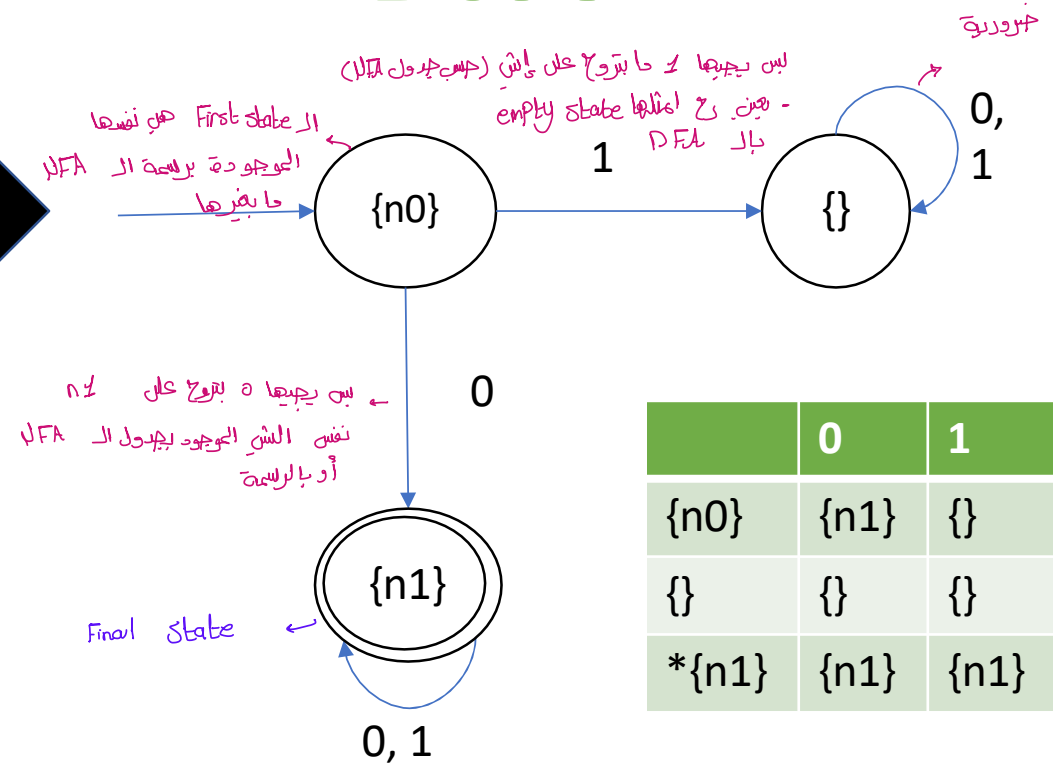
Example2: NFA with two states

NFA



	0	1
n0	n1	
*n1	n1	n1

DFA

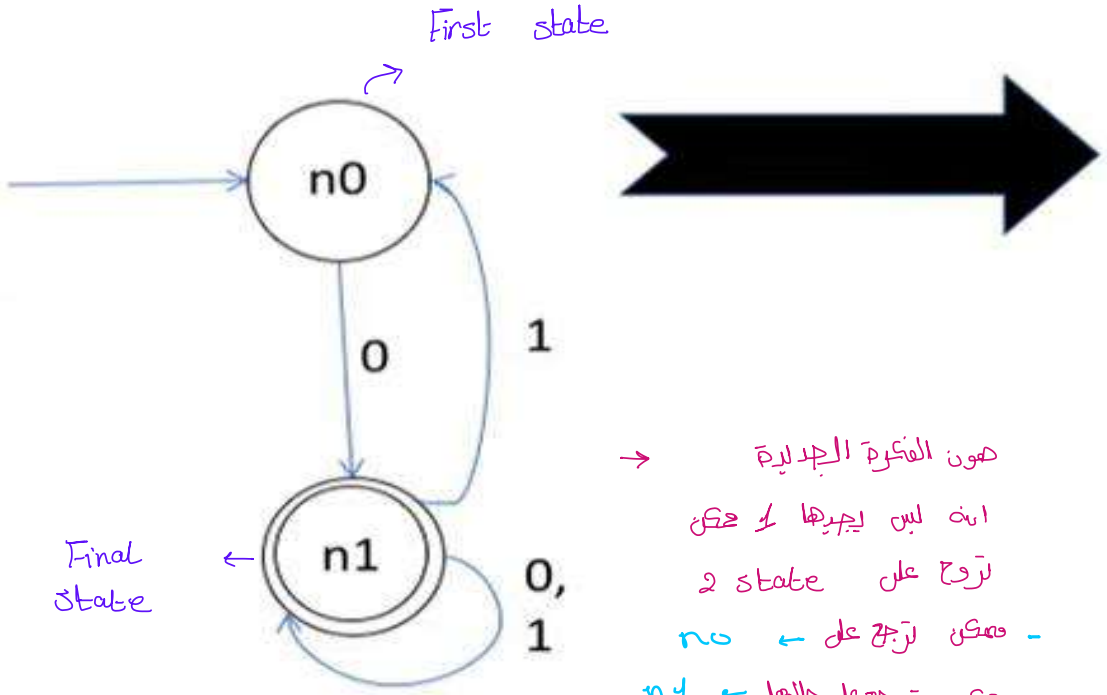


	0	1
{n0}	{n1}	{}
{}	{}	{}
*{n1}	{n1}	{n1}

Example3: NFA with 2 states

NFA

	0	1
n0	n1	
*n1	n1	n0, n1

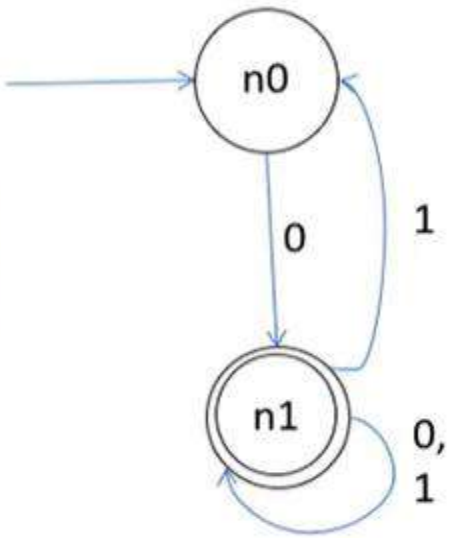


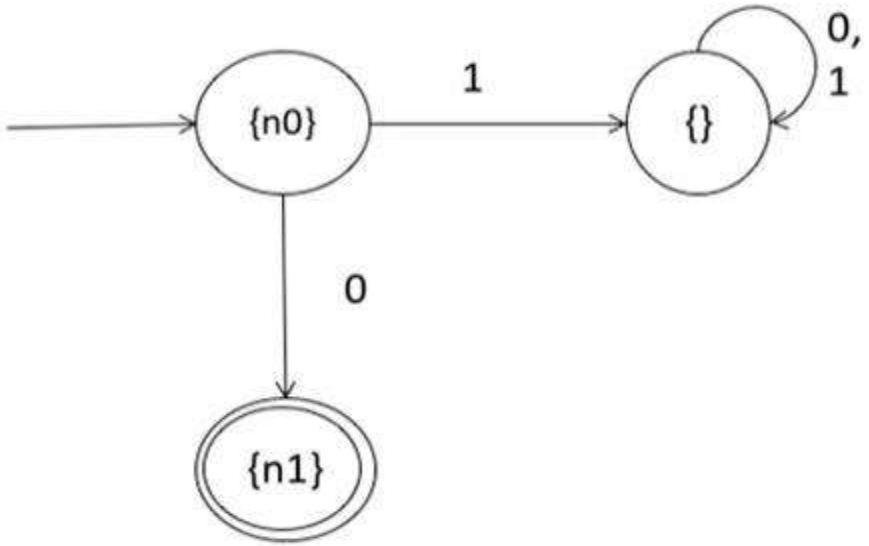
→ هون الفكرة الجديدة
انه ليس يجربها لا ممكن
ترجع على 2 state
- ممكن ترجع على n0 ← n0
- ممكن ترجع على حالها ← n1
- بجمعهم كوحدة واحدة ← n0, n1 ← بالجدول

Example3: NFA with 2 states

NFA

	0	1
n0	n1	
*n1	n1	n0, n1





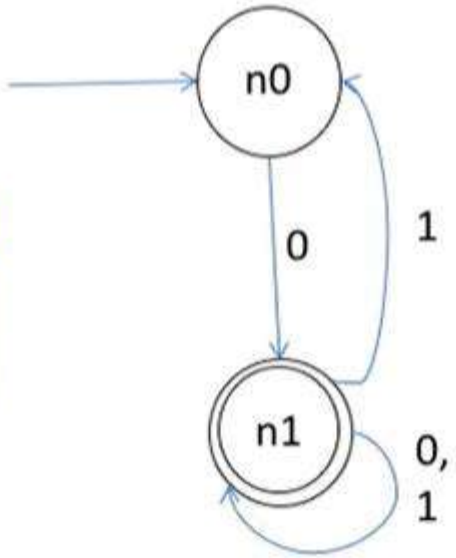
DFA

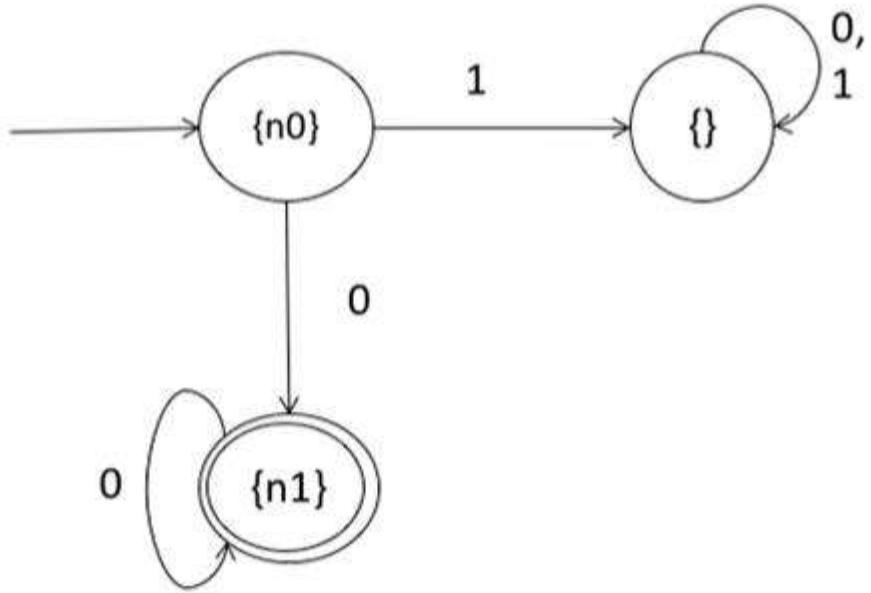
Example3: NFA with 2 states

لهد هون ذي الأفكار إلي حبل
فا في إشي جديد

NFA

	0	1
n0	n1	
*n1	n1	n0, n1



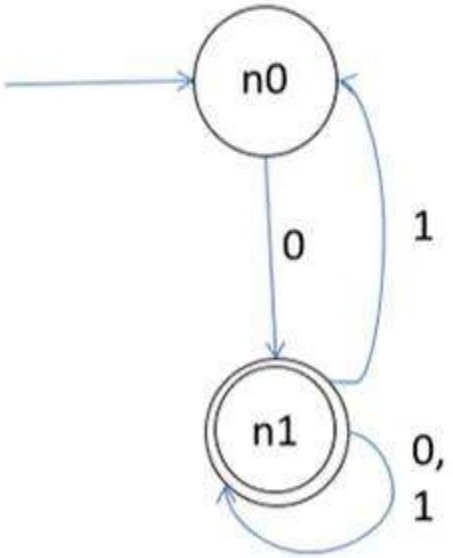


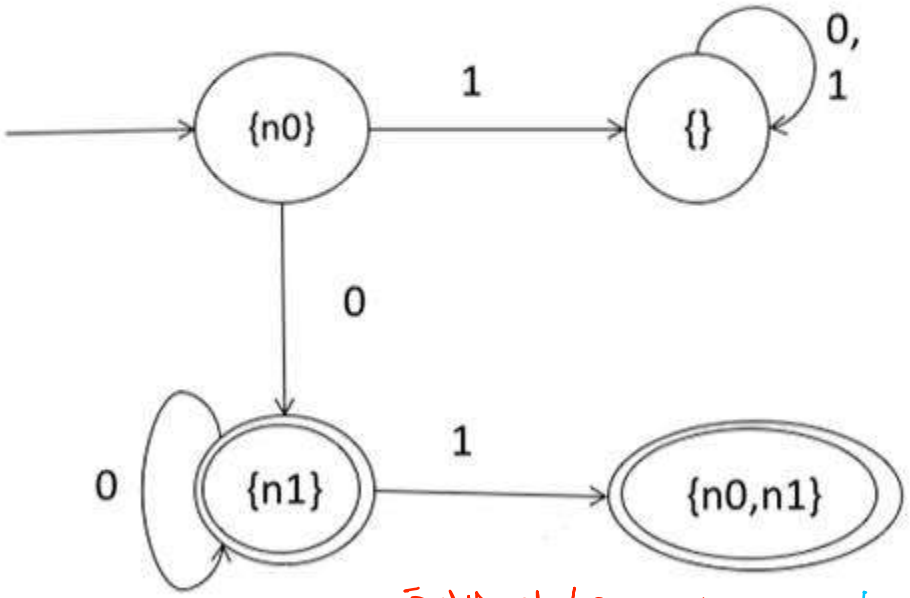
DFA

Example3: NFA with 2 states

NFA

	0	1
n0	n1	
*n1	n1	n0, n1





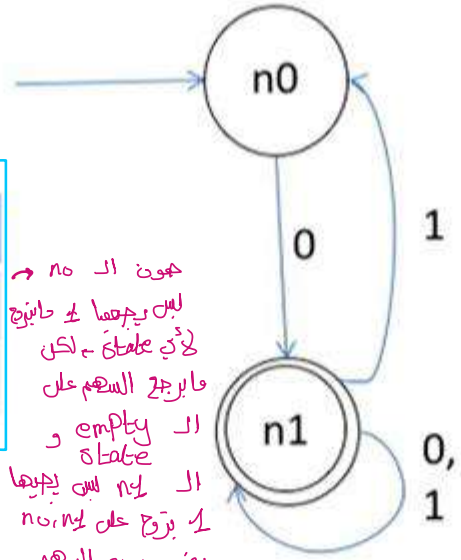
DFA

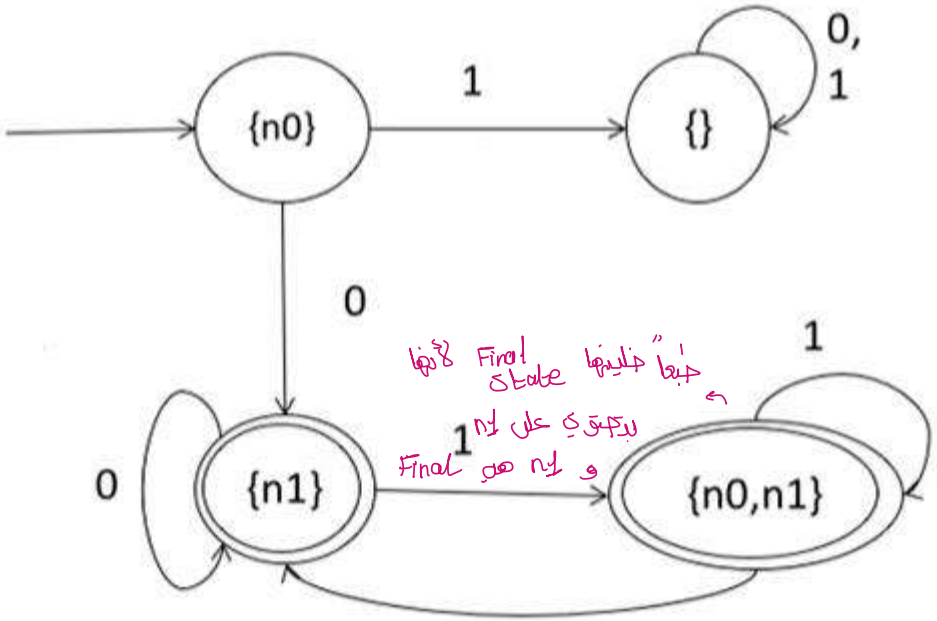
لـ - هون عملت state جديدة
عشان n1 محكن تنتقل
لـ n0,n1 مع بعض لوحدة واحدة
لما يجيها 1 رح يمثلهم بال state
ذي ما كتبتهم بالجدول

Example3: NFA with 2 states

NFA

	0	1
n0	n1	
*n1	n1	n0, n1





DFA

Example3: NFA with 2 states

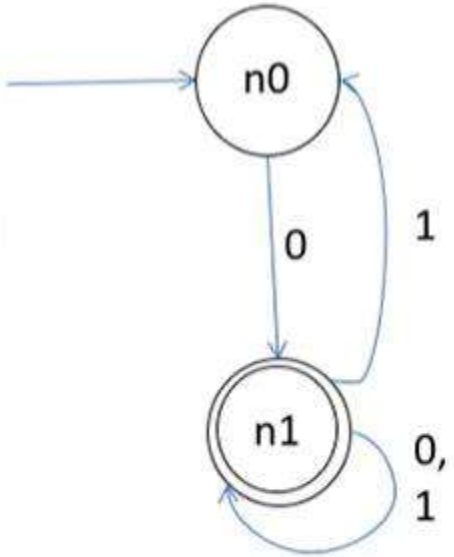
	0	1
{n0}	{n1}	{}
{}	{}	{}
*{n1}	{n1}	{n1}
*{n0, n1}	{n1}	{n0, n1}

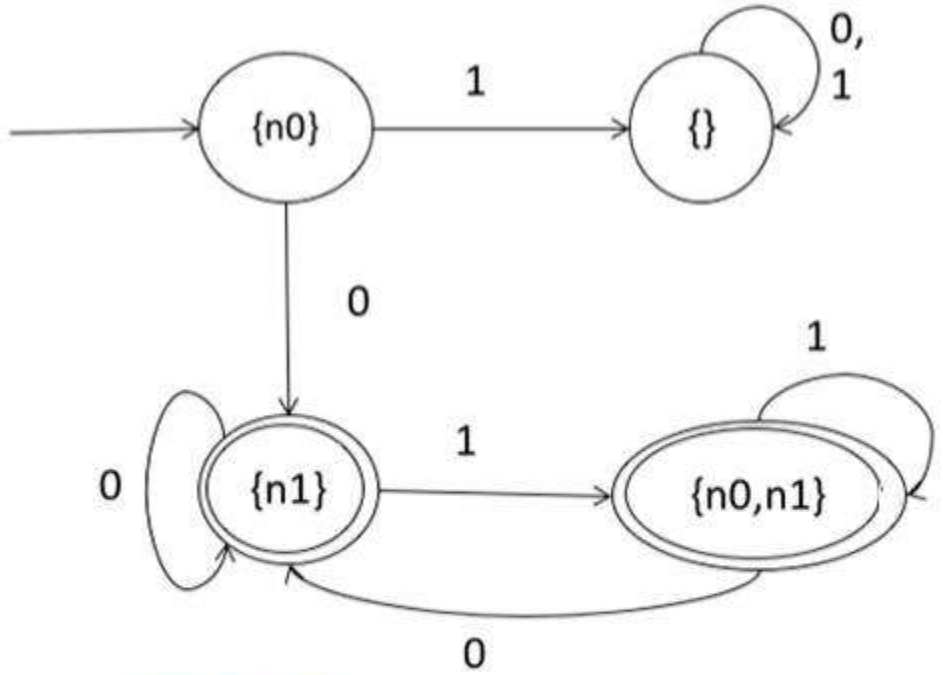
Final State

صون الحروف {n0,n1} تكون {n1} ح تتأكد من الترتيب

NFA

	0	1
n0	n1	
*n1	n1	n0, n1





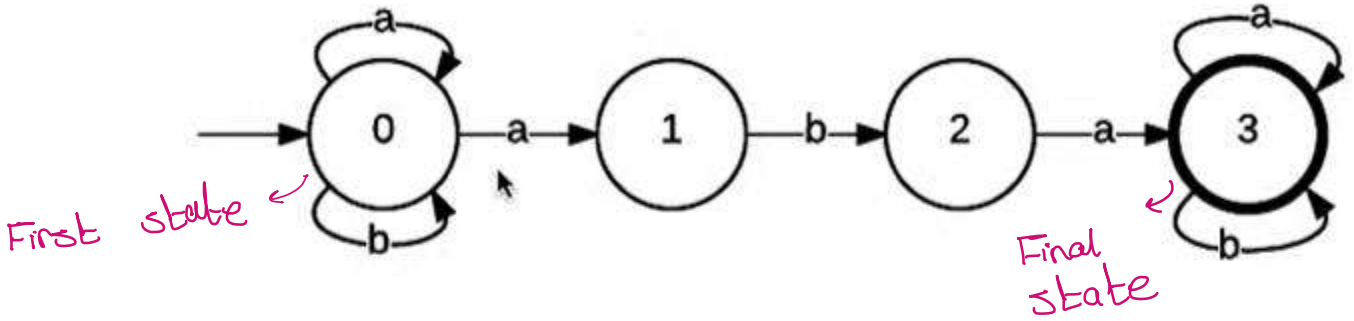
DFA

- Convert this NFA to a DFA

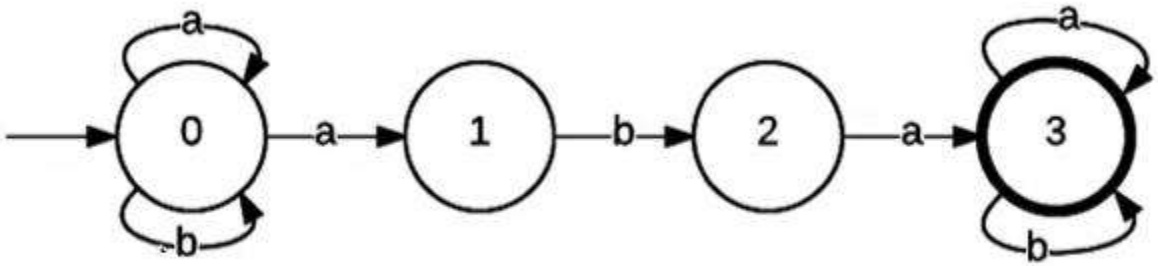
states	a	b
0	0, 1	0
1		2
2	3	
3	3	3

alphabe - اللي بتنتقل من خلالهم لـ states

- للتذكير: كل دائرة بمثل عندي state

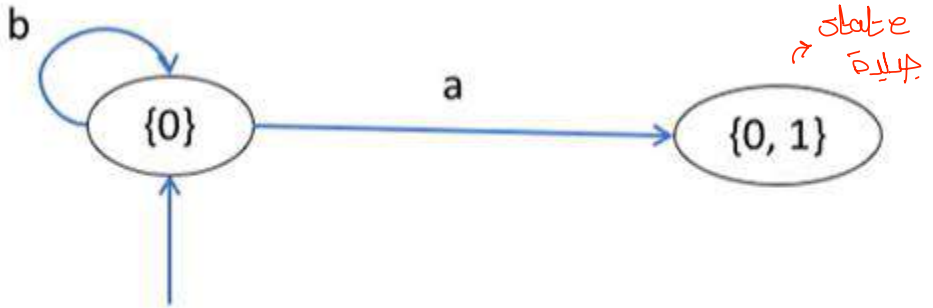


- Convert this NFA to a DFA



states	a	b
0	0, 1	0
1		2
2	3	
3	3	3

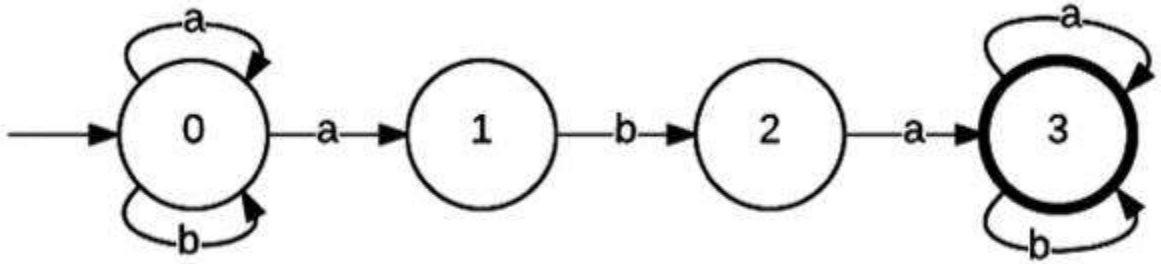
هون ح يجيب عندي حالة جديدة لما اقارن الـ {0,1} وين بروح لبس يجيبها (ب)

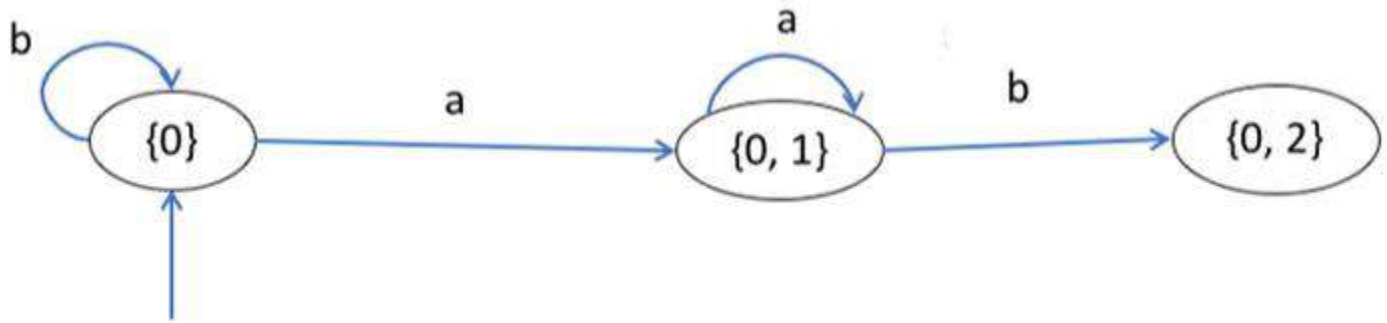


حالة جديدة

- Convert this NFA to a DFA

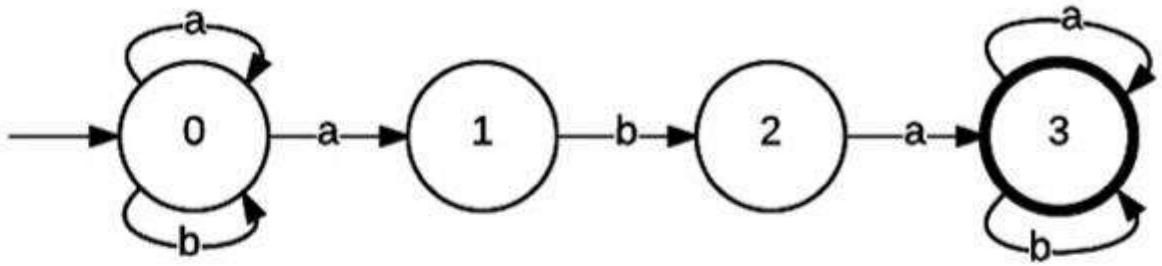
states	a	b
0	0, 1	0
1		2
2	3	
3	3	3

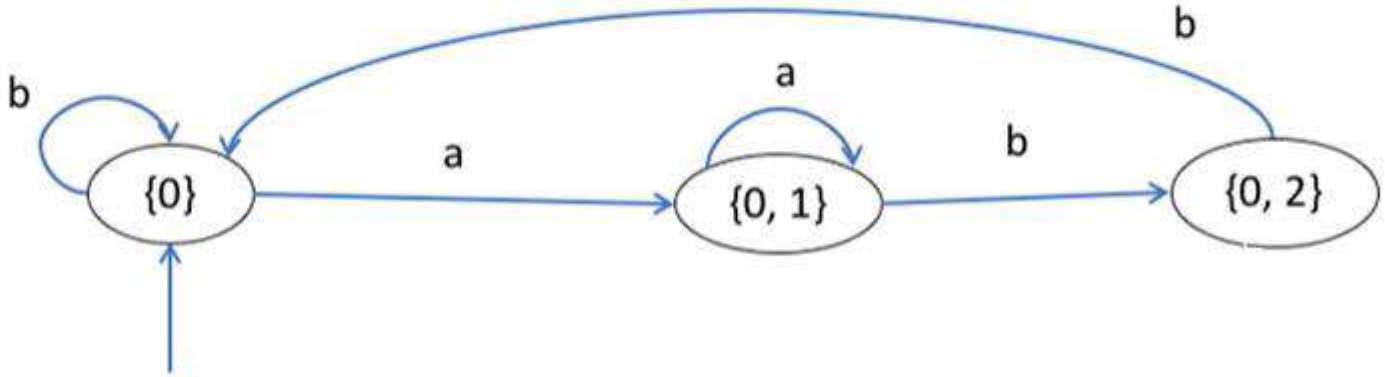




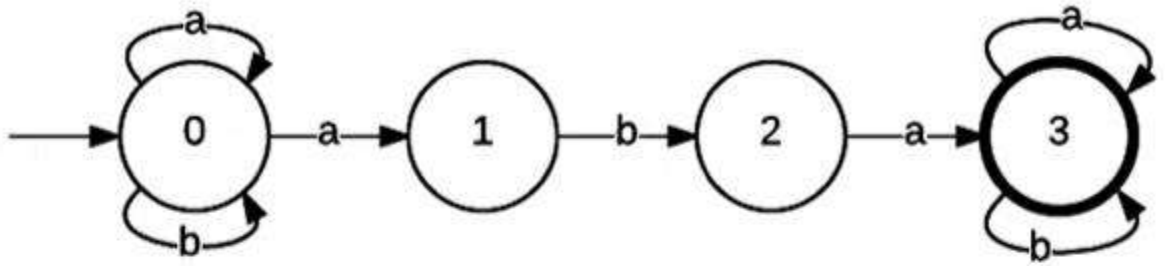
- Convert this NFA to a DFA

states	a	b
0	0, 1	0
1		2
2	3	
3	3	3





- Convert this NFA to a DFA



states	a	b
0	0, 1	0
1		2
2	3	
3	3	3

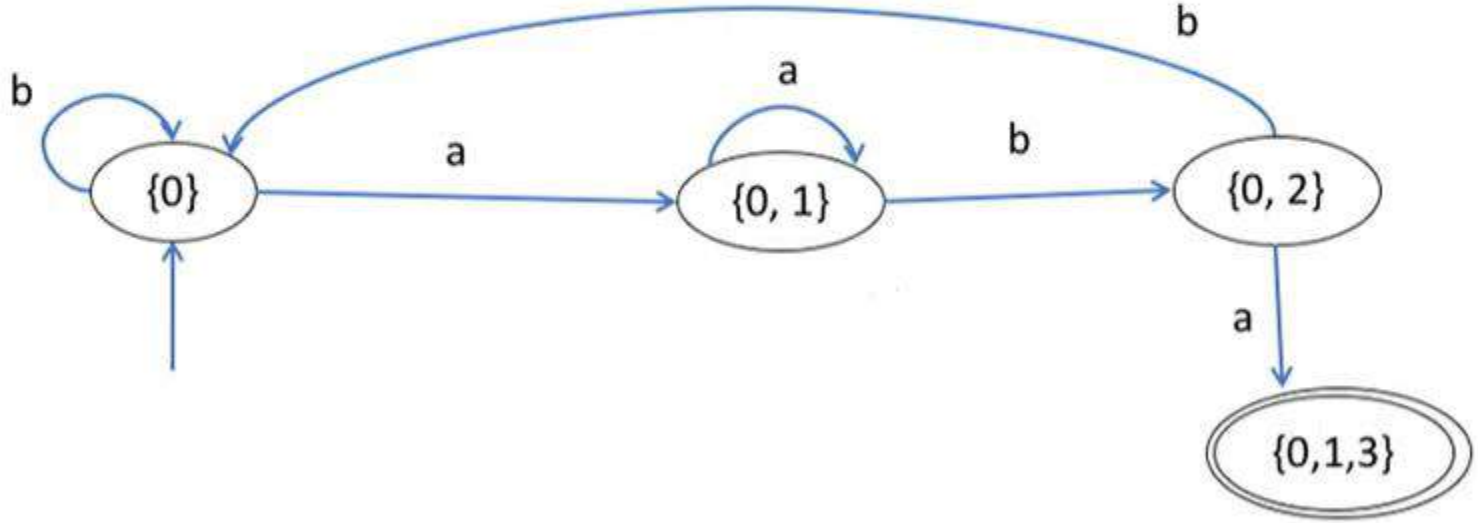
{0,1,3}

0,1,3 على ترتيب ← q بس يجيها ر ترتيب على حالها

بعض ر ترتيب على حالها

0,2,3 ← ترتيب ← ط بس يجيها ر ترتيب ←

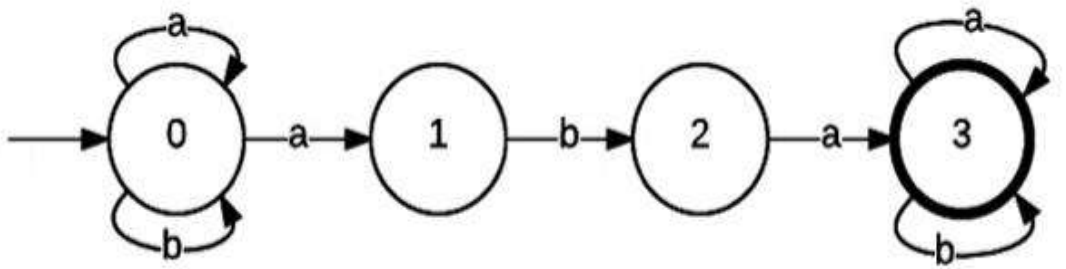
بعض ر اعل كان new state

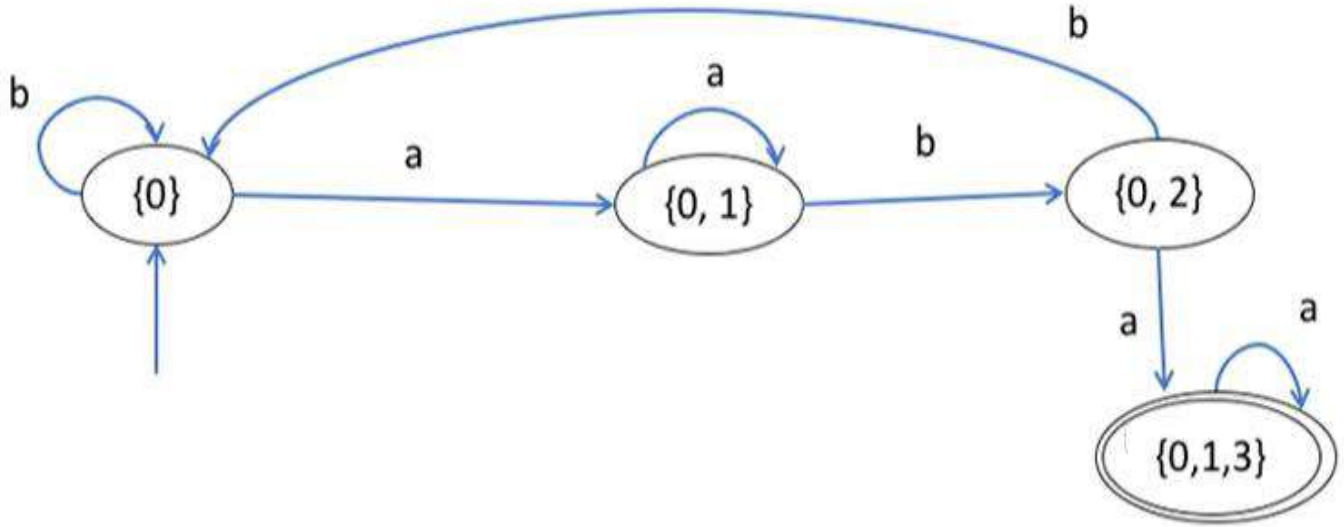


new state ← برجع بتبعها من أول

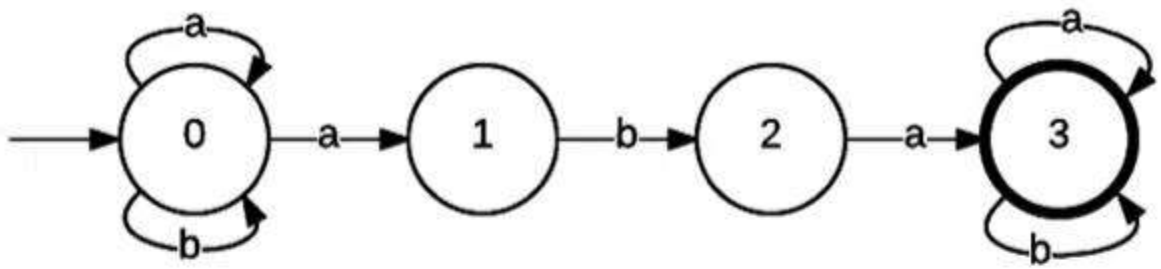
- Convert this NFA to a DFA

states	a	b
0	0, 1	0
1		2
2	3	
3	3	3





- Convert this NFA to a DFA

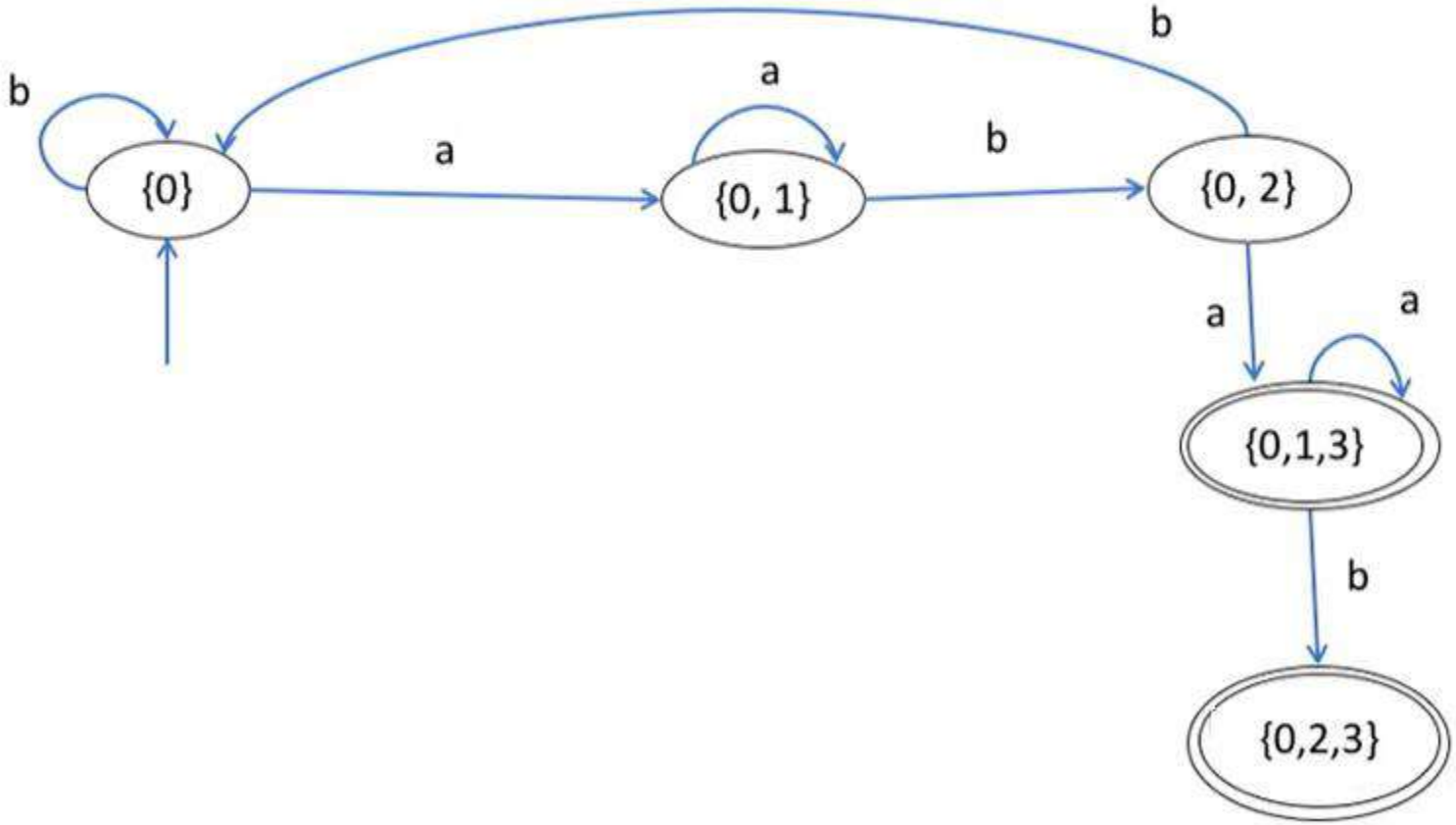


states	a	b
0	0, 1	0
1		2
2	3	
3	3	3

{0,1,3} ← ما يطلع ← {0,2,3}

{0,3} ← b " ← "

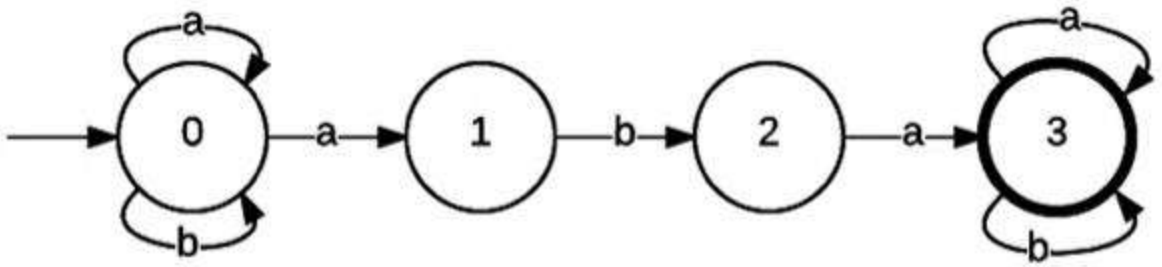
↳ new state

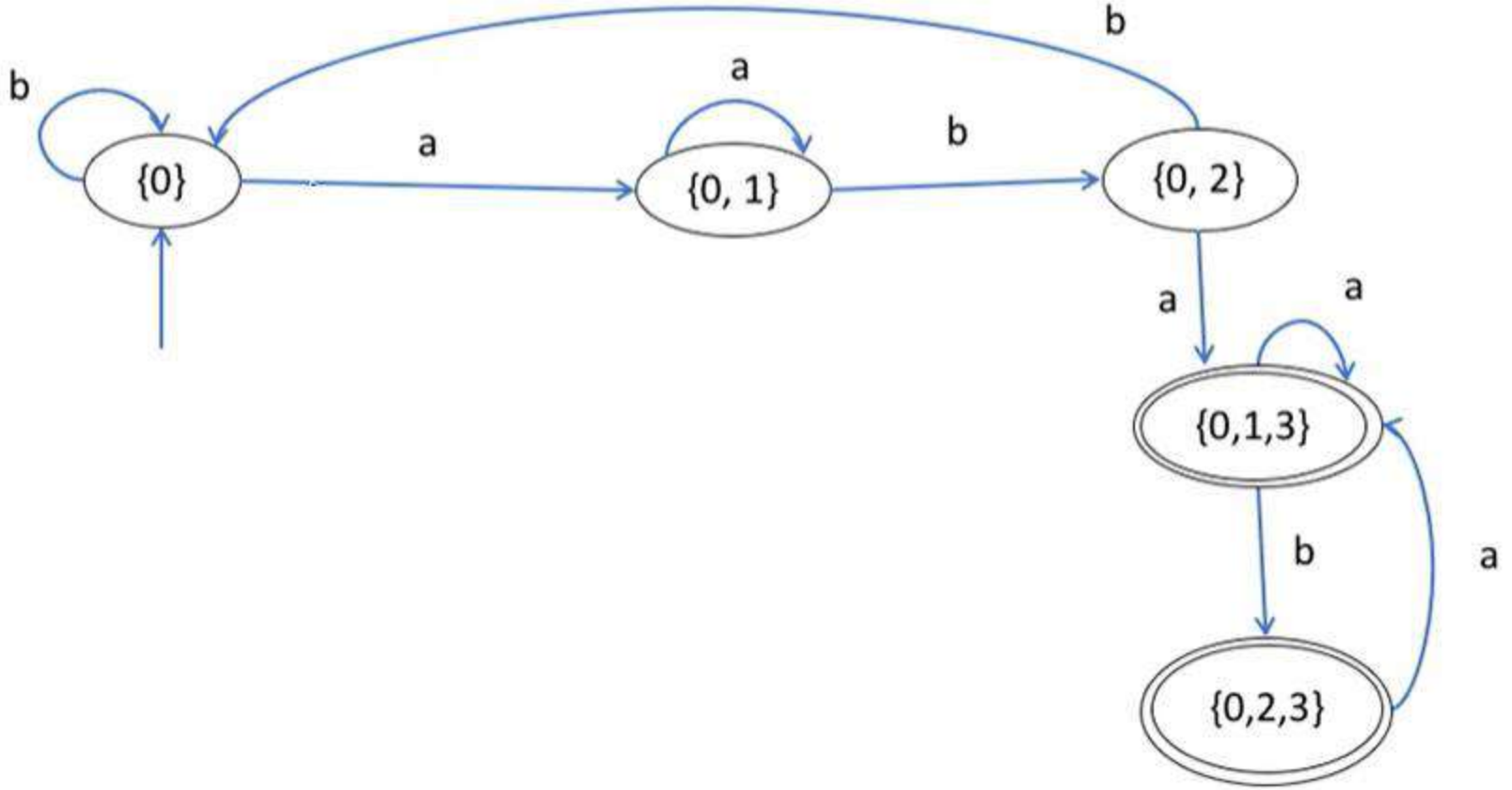


new state ↵

- Convert this NFA to a DFA

states	a	b
0	0, 1	0
1		2
2	3	
3	3	3



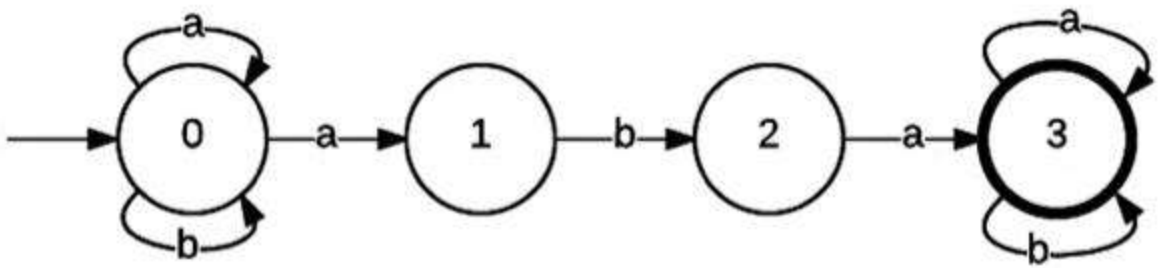


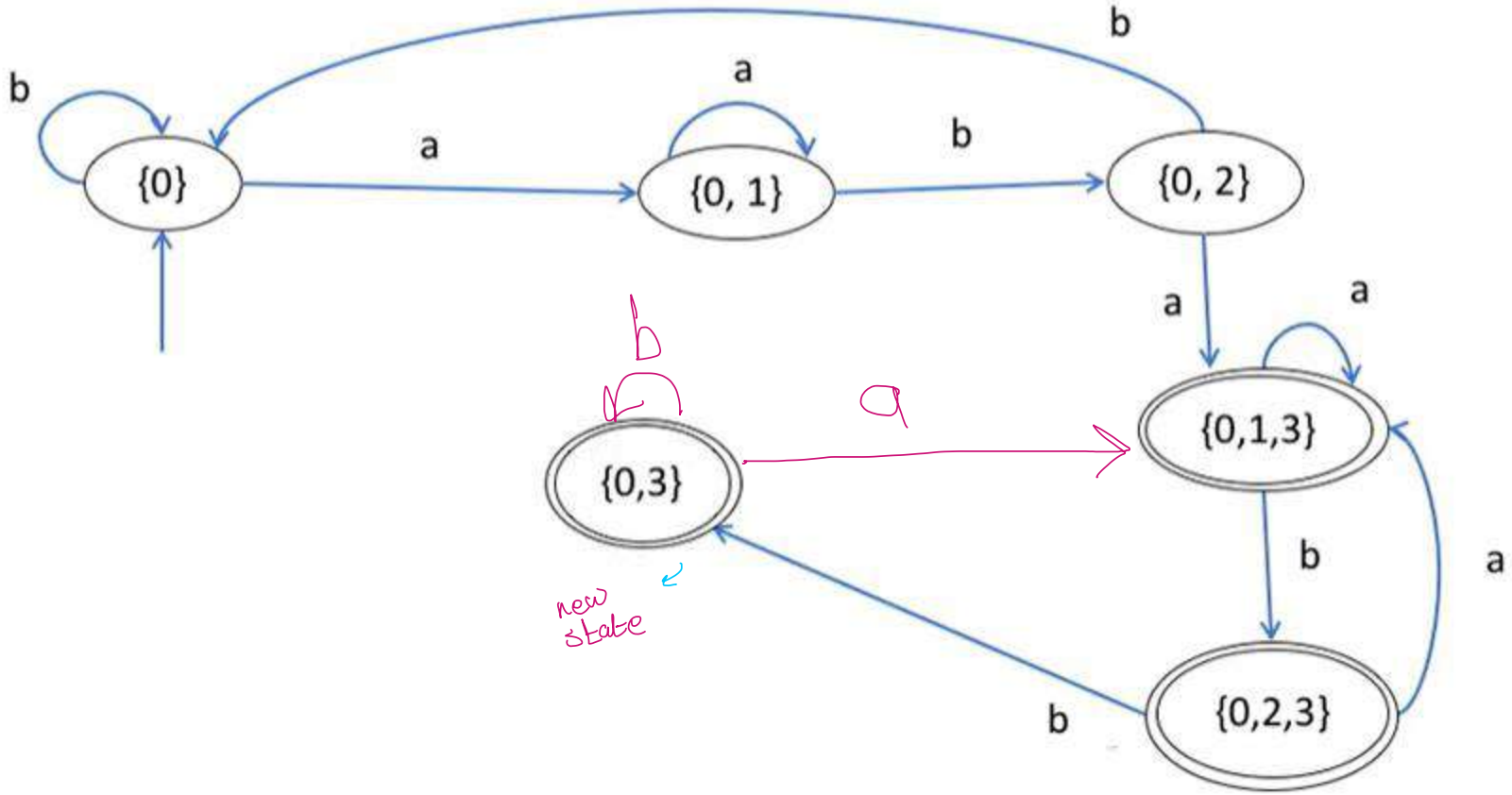
- Convert this NFA to a DFA

states	a	b
0	0, 1	0
1		2
2	3	
3	3	3

{0,1,3} ← a ← {0,3}

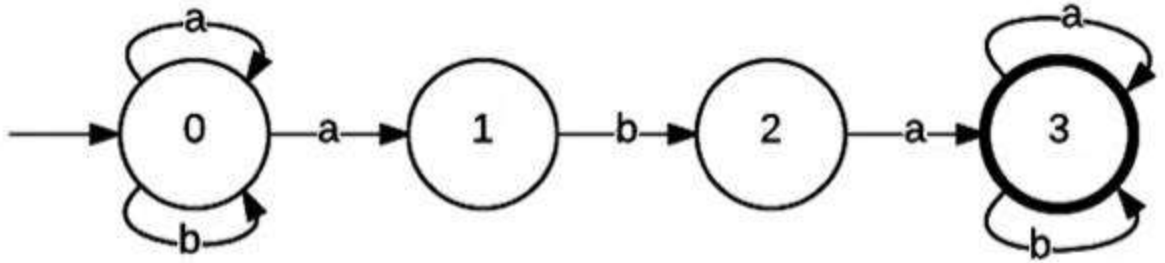
{0,3} ← b ← ,,

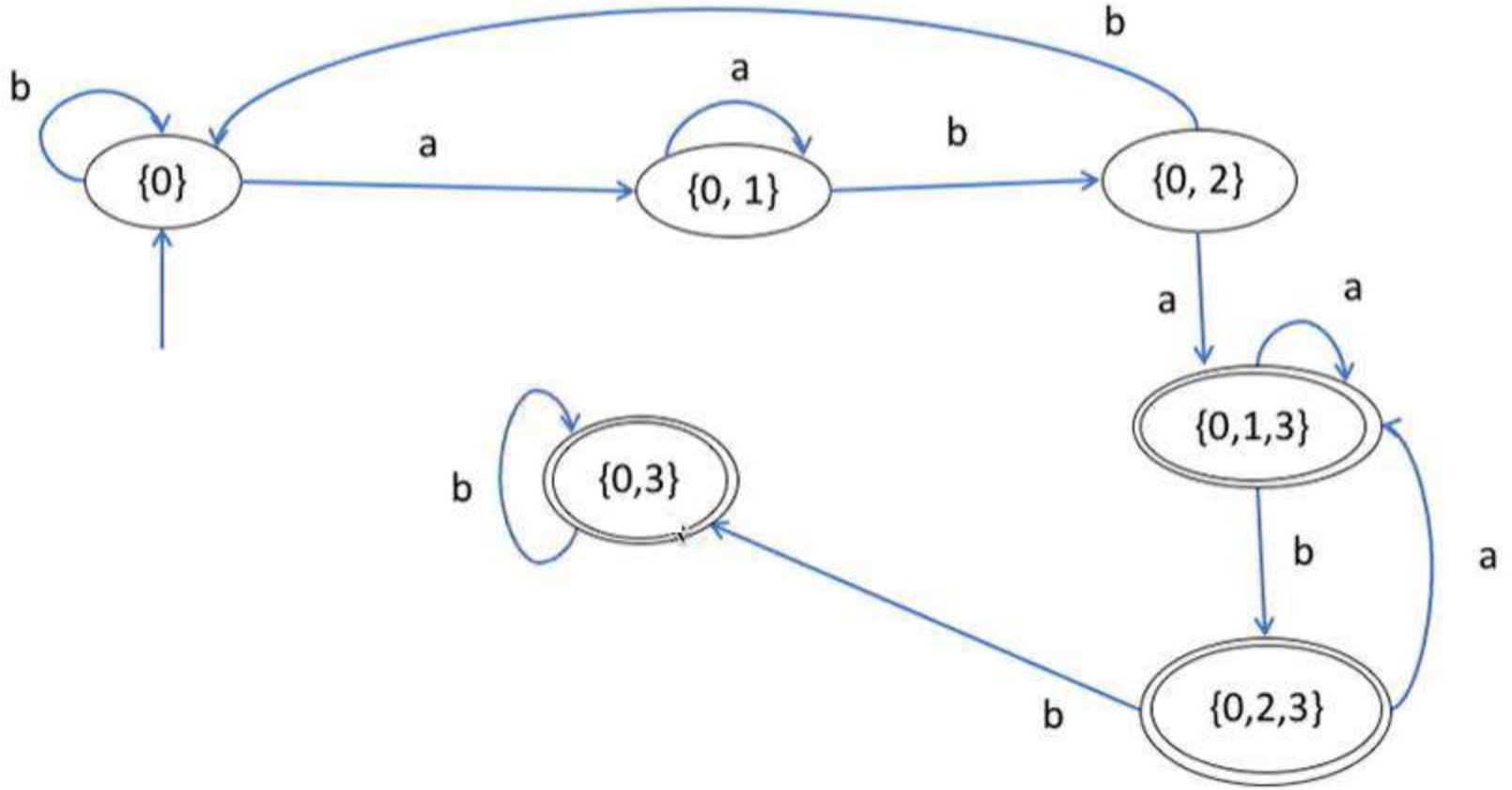




- Convert this NFA to a DFA

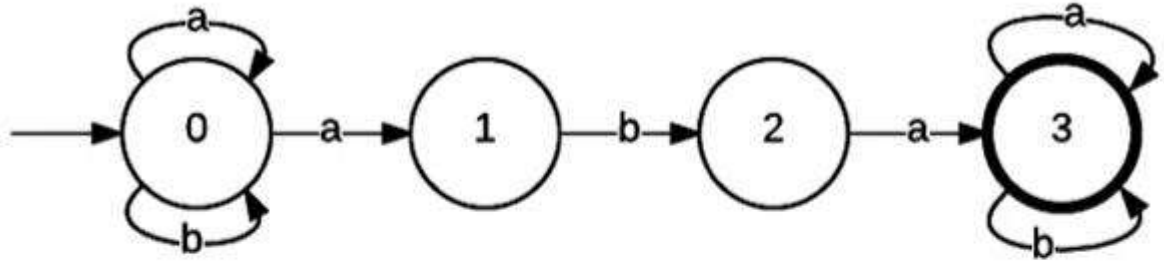
states	a	b
0	0, 1	0
1		2
2	3	
3	3	3

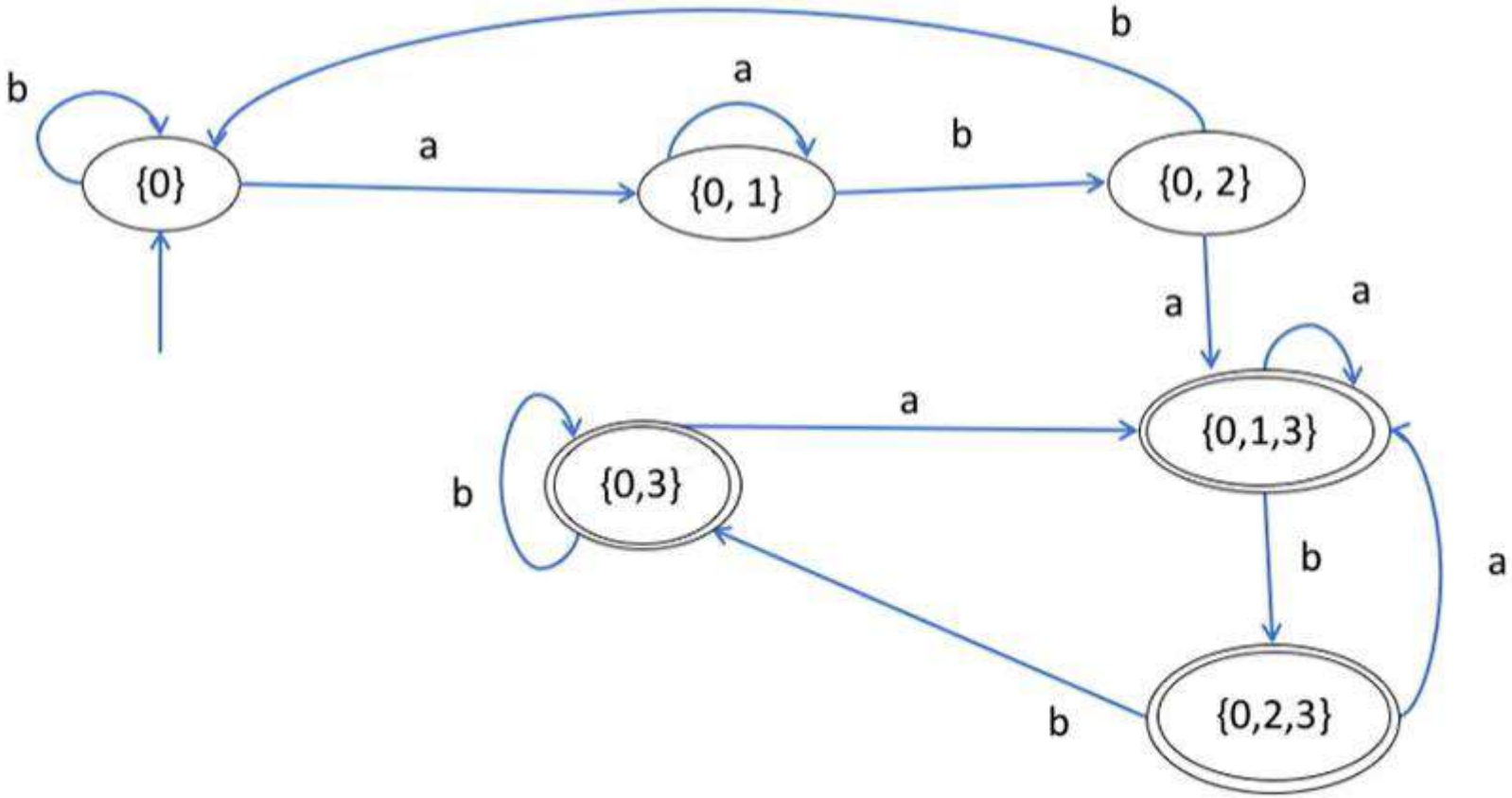




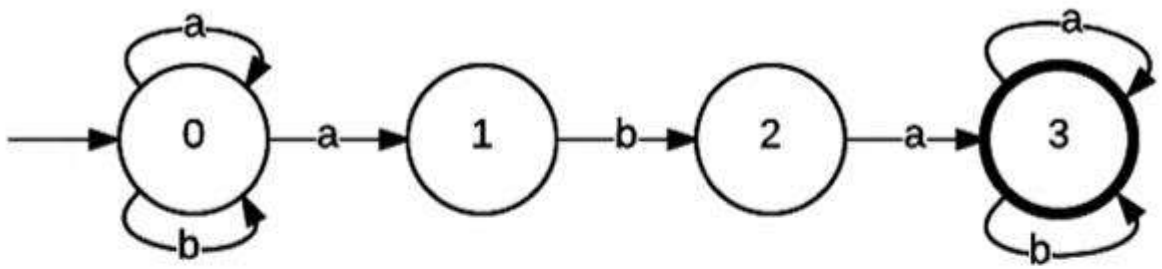
- Convert this NFA to a DFA

states	a	b
0	0, 1	0
1		2
2	3	
3	3	3





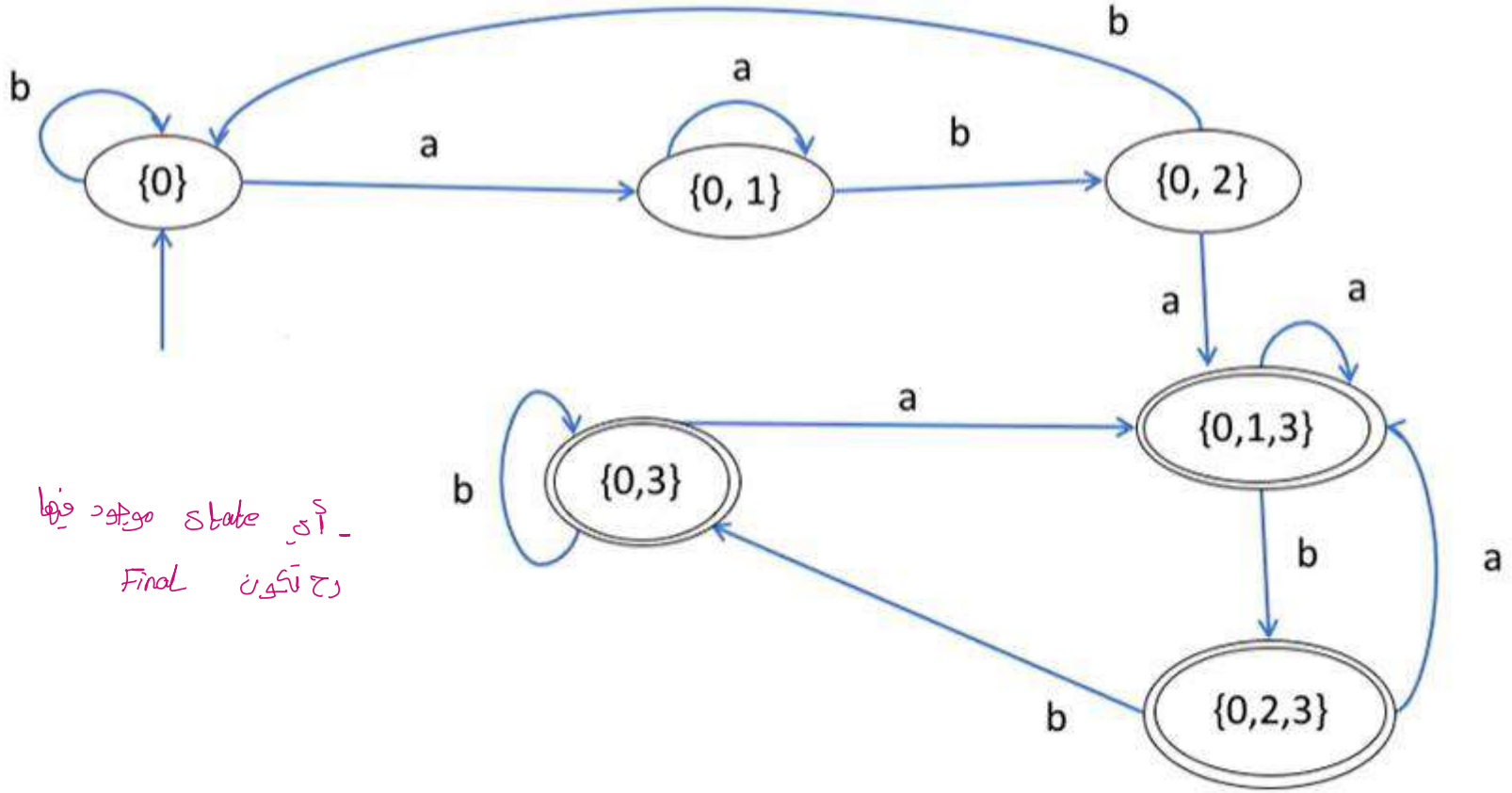
- Convert this NFA to a DFA



states	a	b
0	0, 1	0
1		2
2	3	
3	3	3

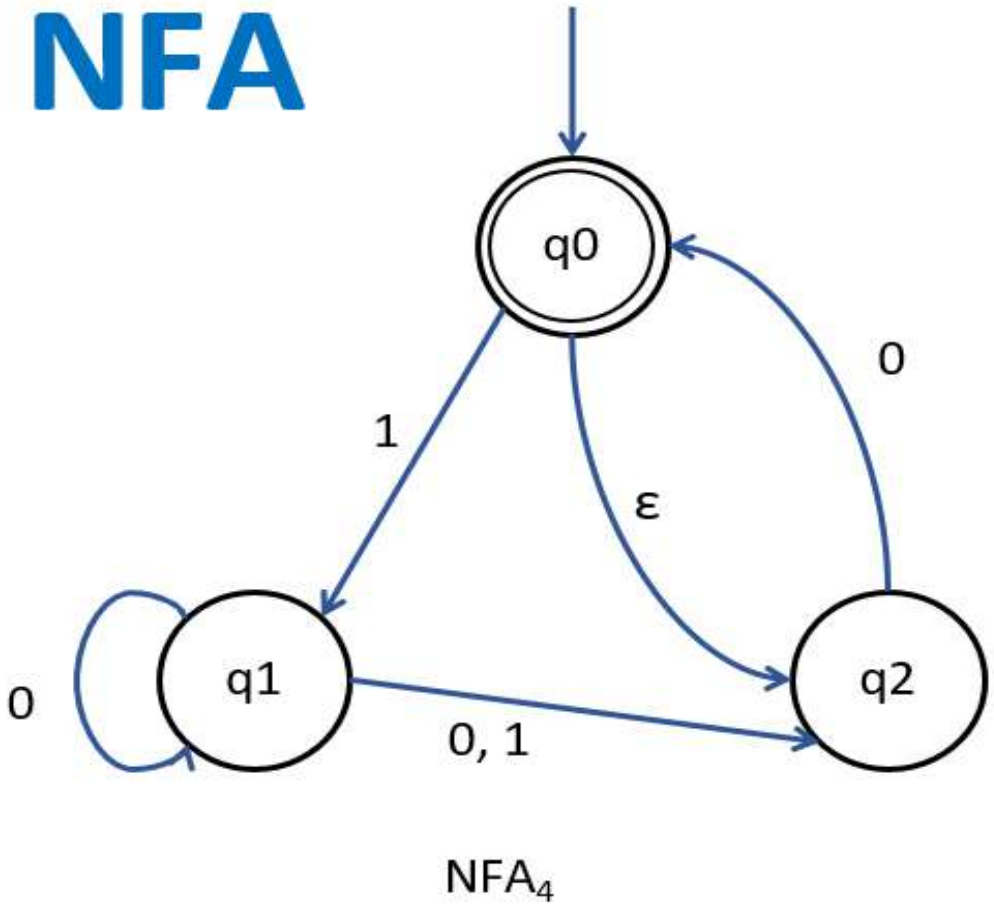
states	a	b
{0}	{0, 1}	{0}
{0, 1}	{0, 1}	{0, 2}
{0, 2}	{0, 1, 3}	{0}
*{0, 1, 3}	{0, 1, 3}	{0, 2, 3}
*{0, 2, 3}	{0, 1, 3}	{0, 3}
*{0, 3}	{0, 1, 3}	{0, 3}

Final State



3 أي state موجود فيها
Final رح تكون

Example 5: convert NFA to DFA

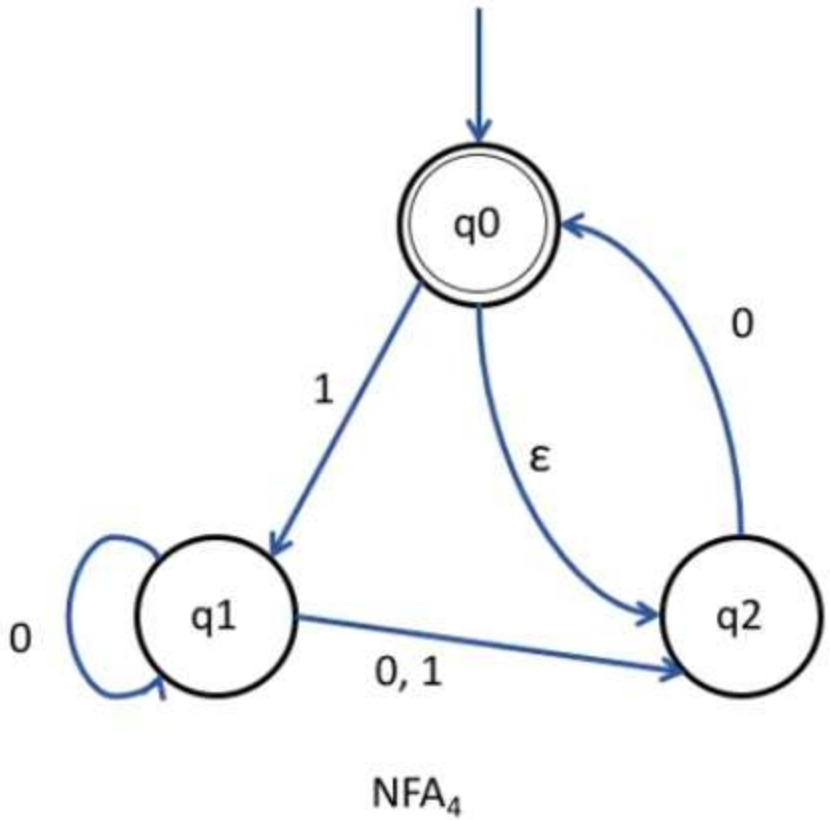


NFA_4

- State → q_0, q_1, q_2
- First state → q_0
- Final state → q_0
- alphabet → 0, 1, ε → يجيبها خذ alphabet الـ

Example 5: convert NFA to DFA

states	0	1	ε
q0		q1	q2
q1	q1,q2	q2	
q2	q0		

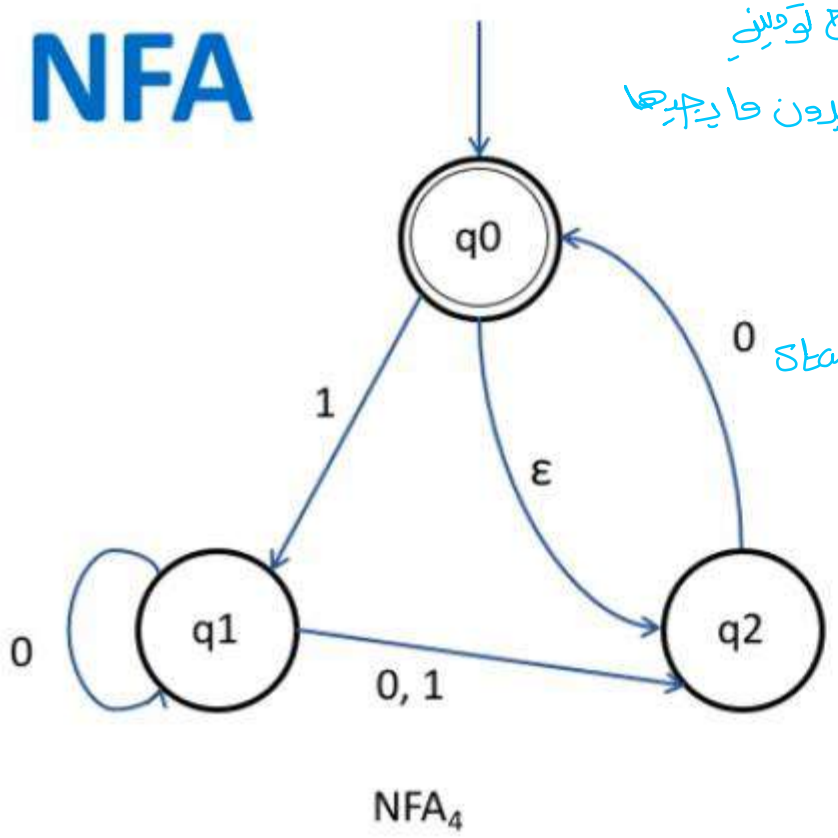


NFA_4

states	0	1	ε
q0		q1	q2
q1	q1,q2	q2	
q2	q0		

Example 5: convert NFA to DFA

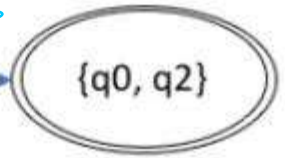
NFA



NFA_4

DFA

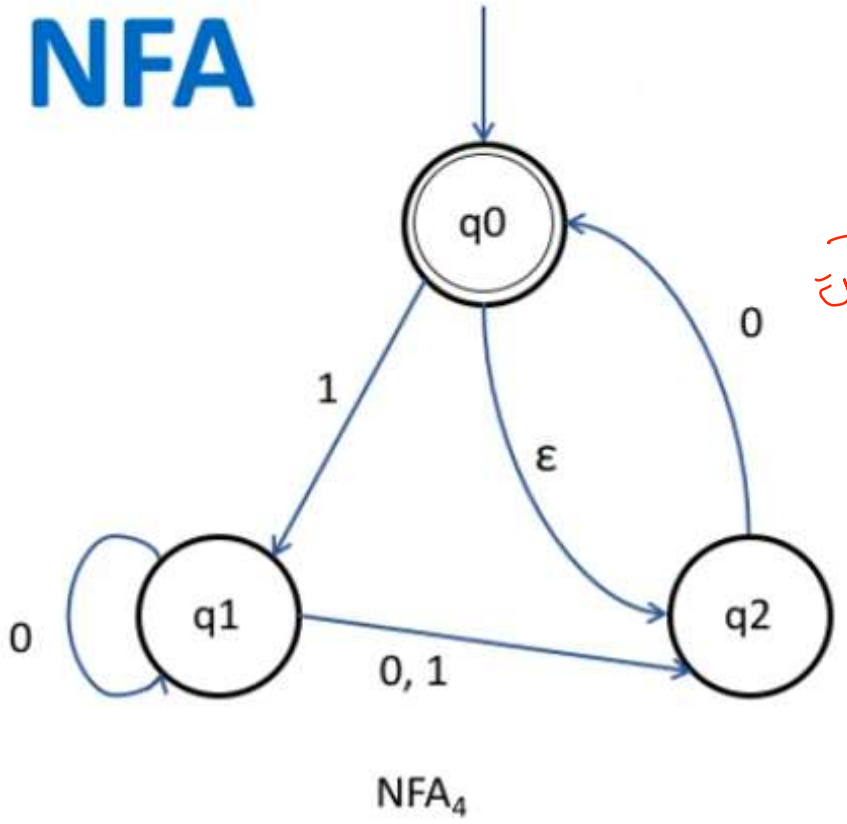
لأنه الـ q0 عندها ε رح توديني
دايركت لـ q2 (ببلاش) بدون ما يجيها
أي شي يعني
الـ ε هو حرية
جاني بودني لـ state
تالي بعدها دايركت



Example 5: convert NFA to DFA

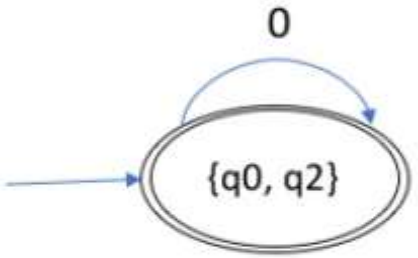
state s	0	1	ε
q0		q1	q2
q1	q1,q2	q2	
q2	q0		

NFA



NFA_4

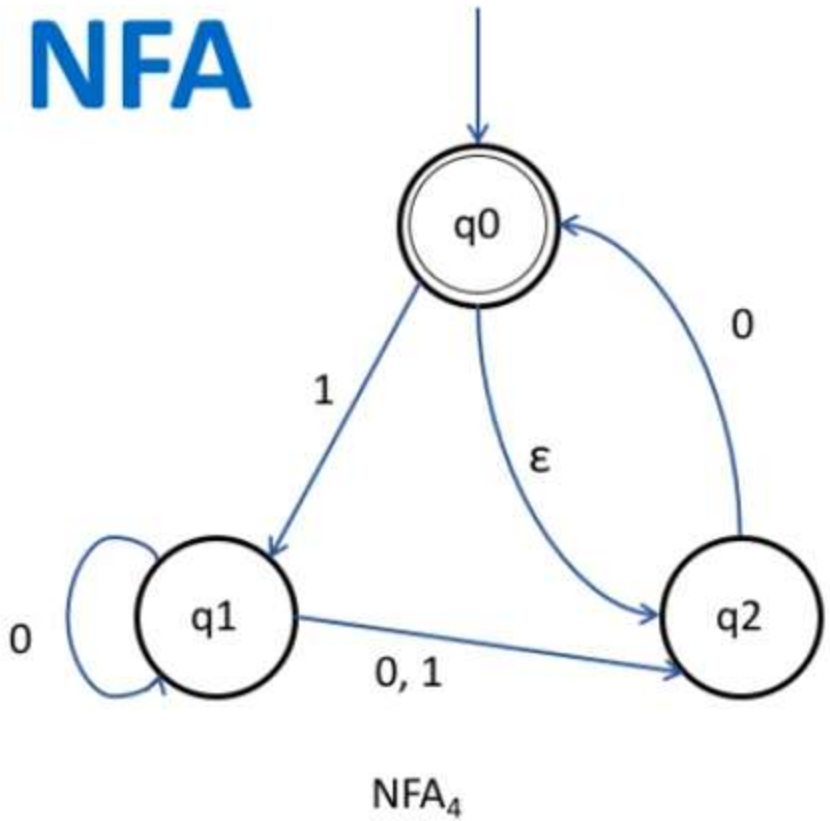
DFA



states	0	1	ε
q0		q1	q2
q1	q1,q2	q2	
q2	q0		

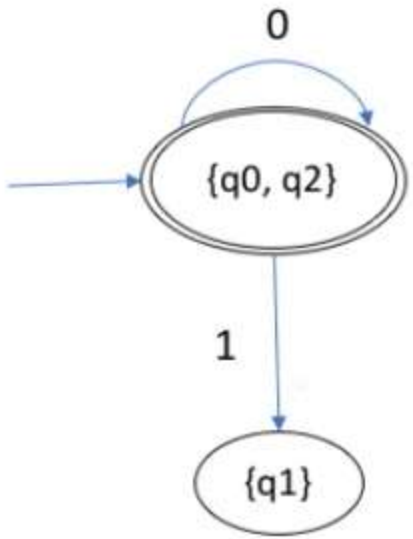
Example 5: convert NFA to DFA

NFA



NFA_4

DFA

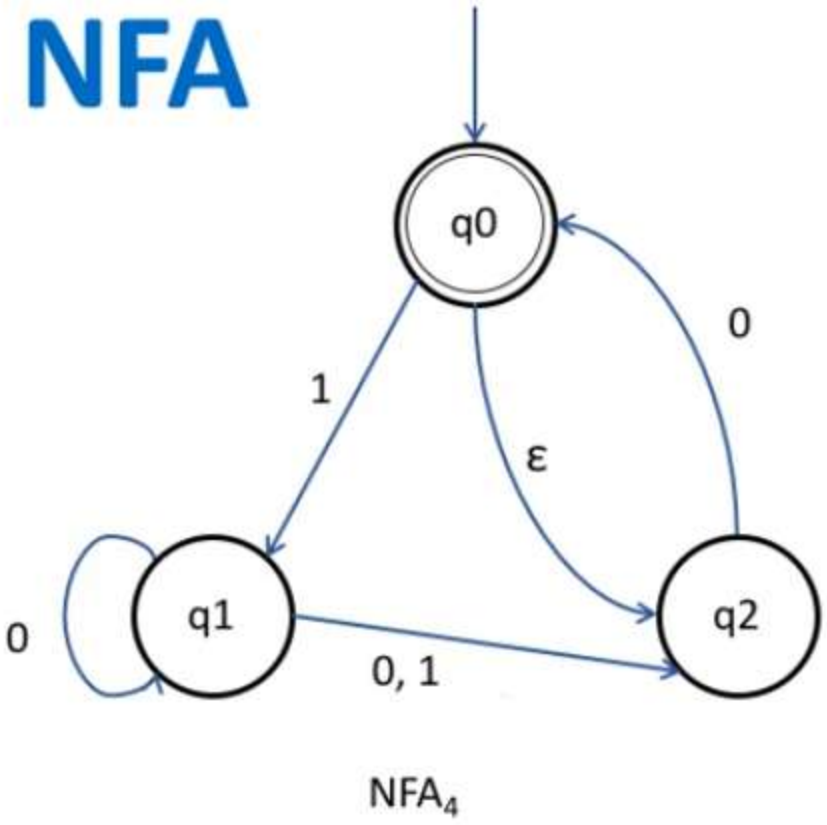


Example 5: convert NFA to DFA

states	0	1	ε
q0		q1	q2
q1	q1,q2	q2	
q2	q0		

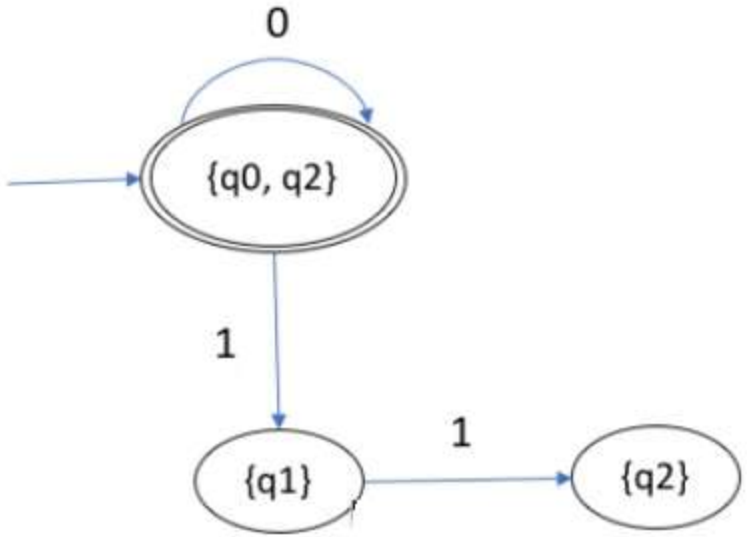
new state

NFA



NFA_4

DFA



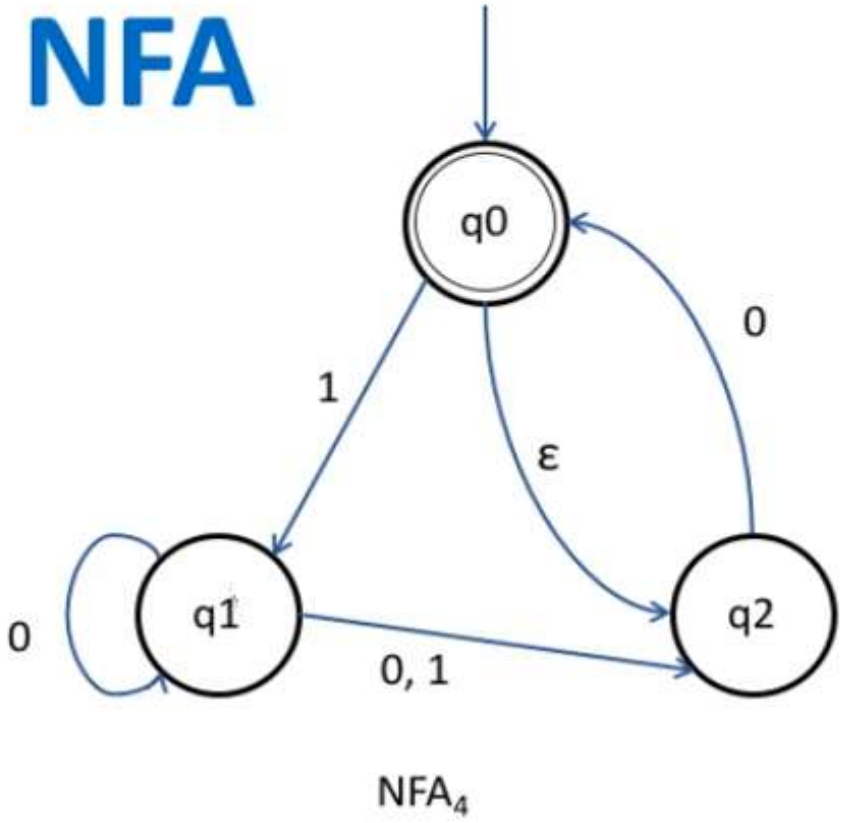
state الجديد ← {q0, q2}

Example 5: convert NFA to DFA

states	0	1	ε
q0		q1	q2
q1	q1,q2	q2	
q2	q0		

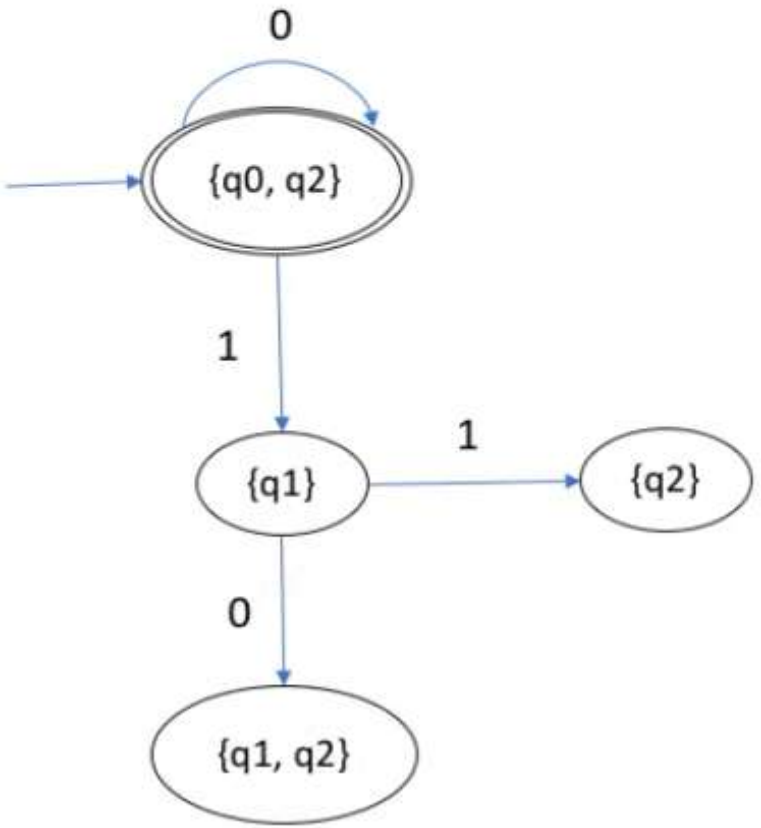
new state

NFA



NFA_4

DFA



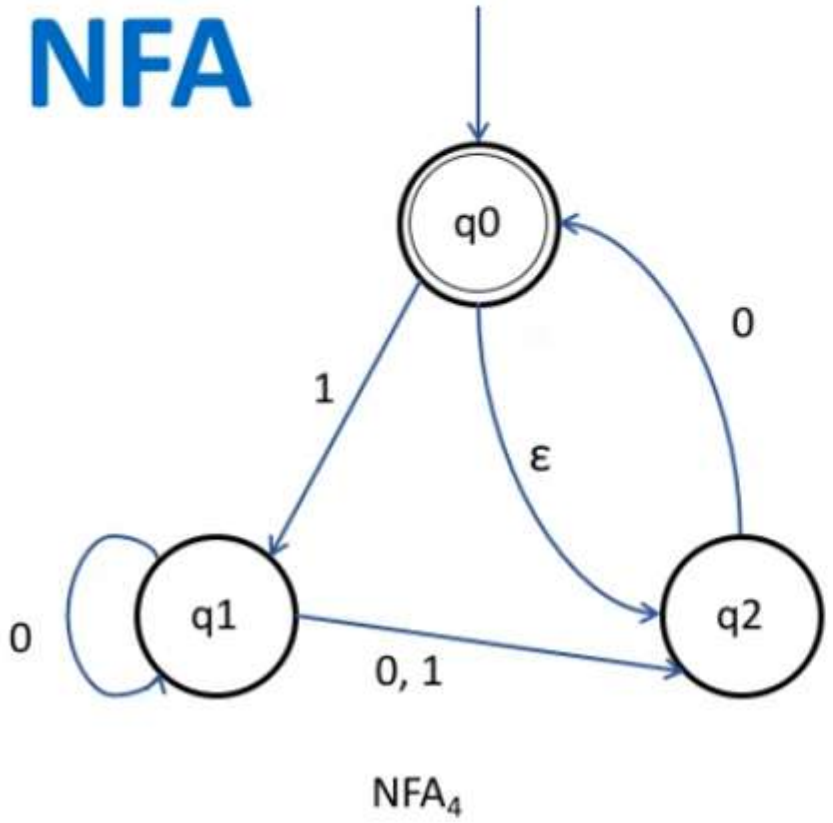
new state

Example 5: convert NFA to DFA

states	0	1	ε
q0		q1	q2
q1	q1,q2	q2	
q2 →	q0		

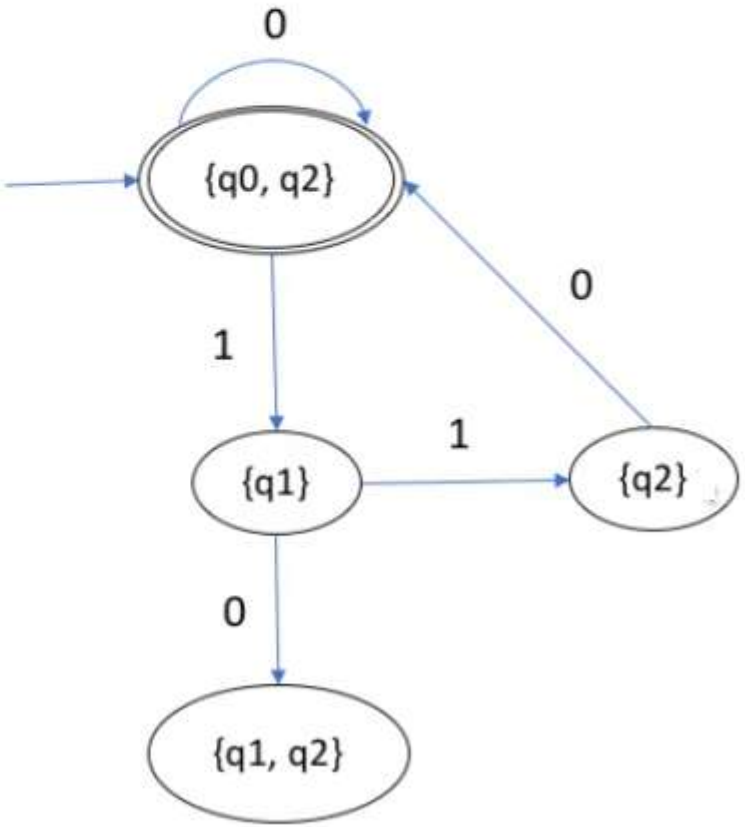
empty state

NFA



NFA_4

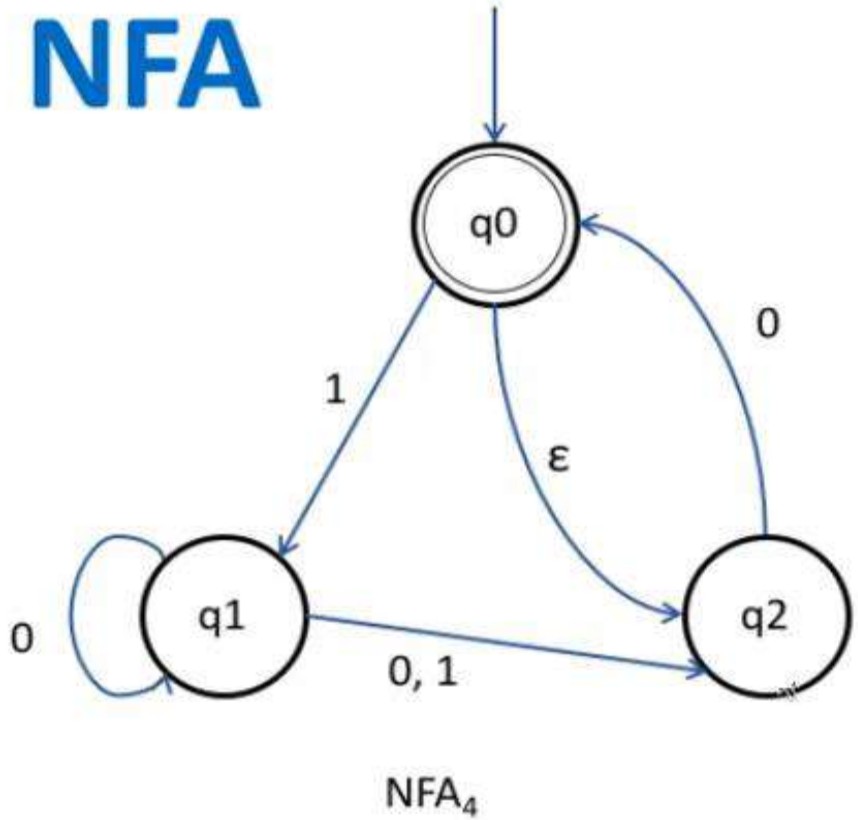
DFA



states	0	1	ε
q0		q1	q2
q1	q1,q2	q2	
q2	q0		

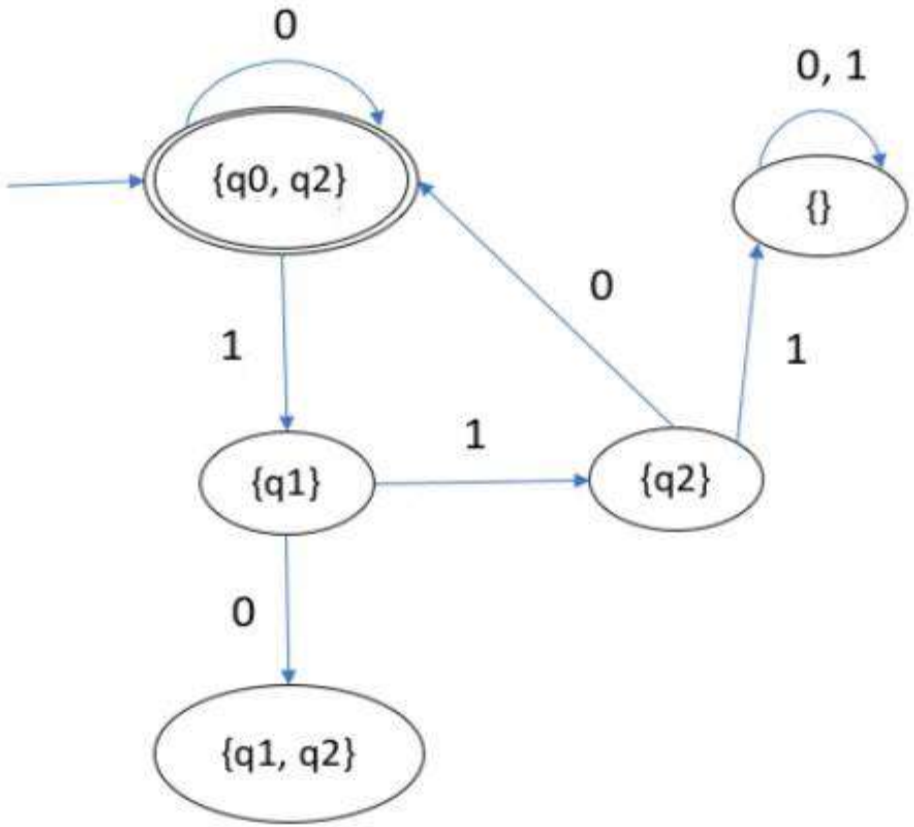
Example 5: convert NFA to DFA

NFA



NFA_4

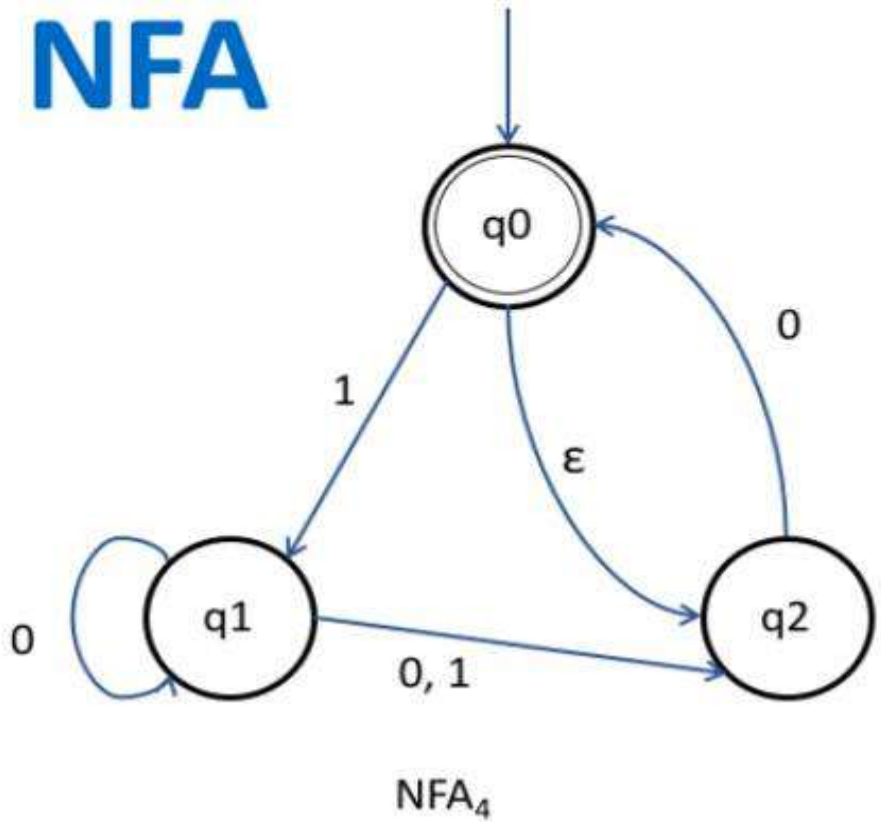
DFA



states	0	1	ε
q0		q1	q2
q1	q1,q2	q2	
q2	q0		

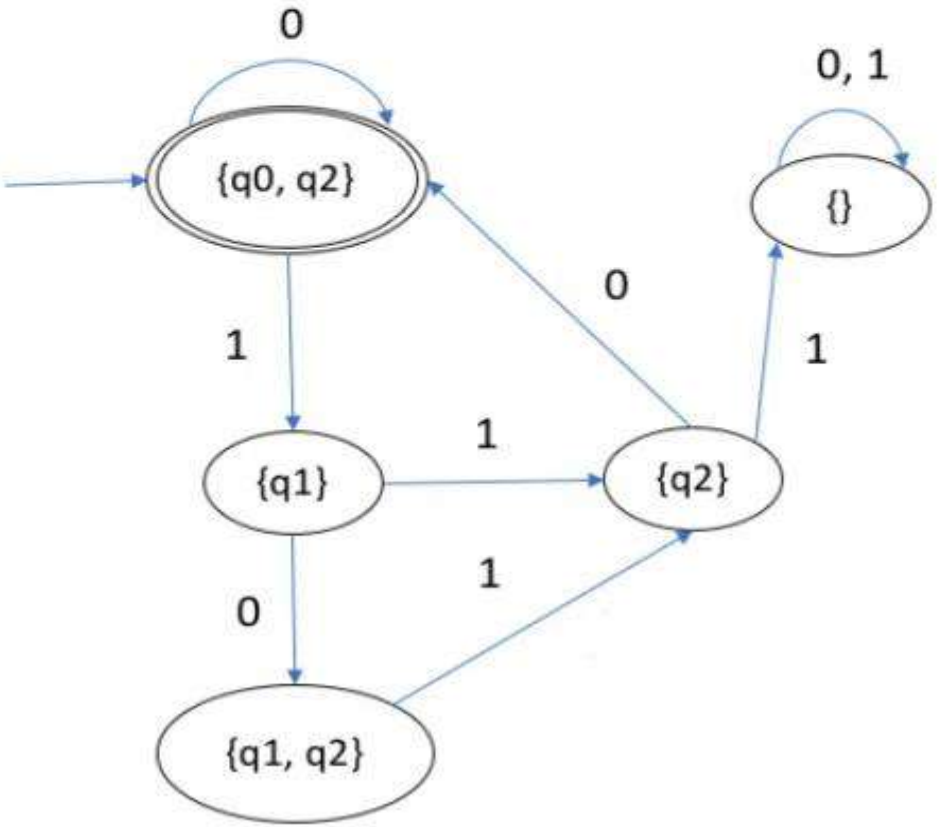
Example 5: convert NFA to DFA

NFA



NFA_4

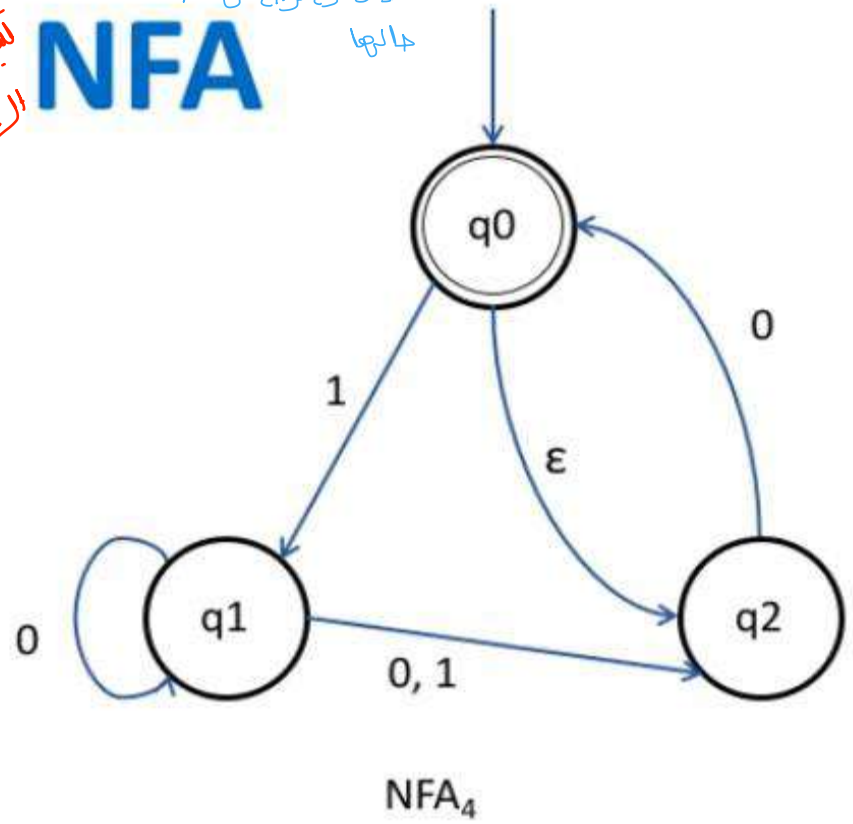
DFA



Example 5: convert NFA to DFA

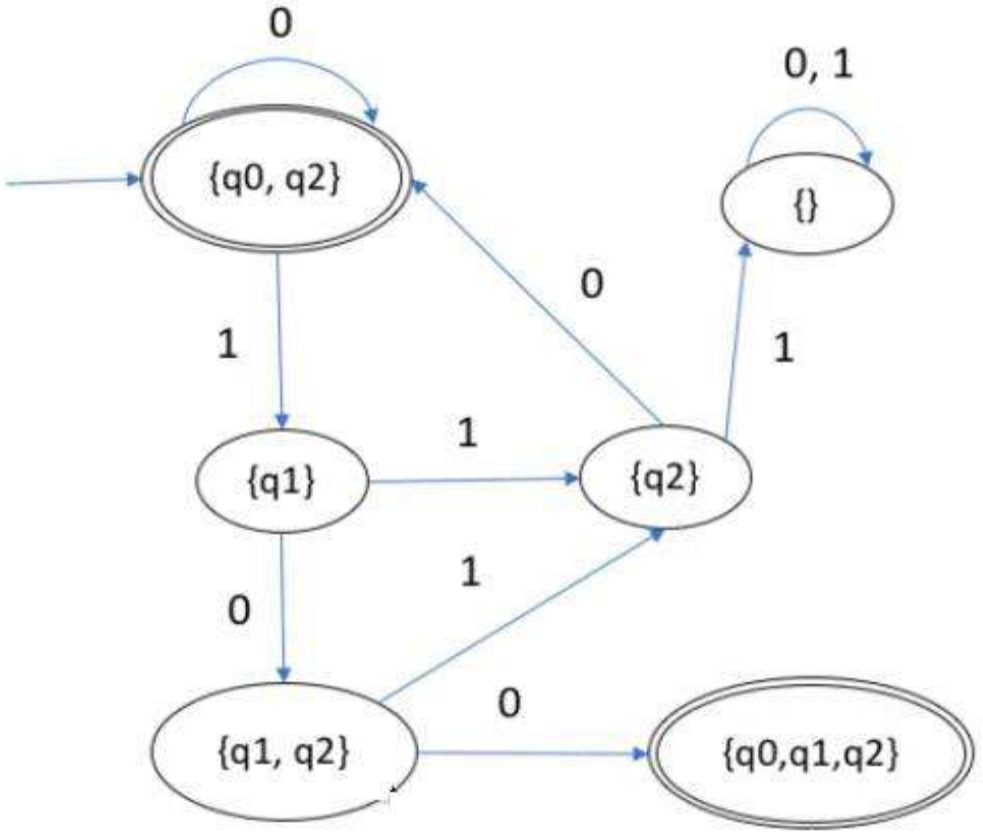
states	0	1	ε
q0		q1	q2
q1	q1,q2	q2	
q2	q0		

NFA



NFA_4

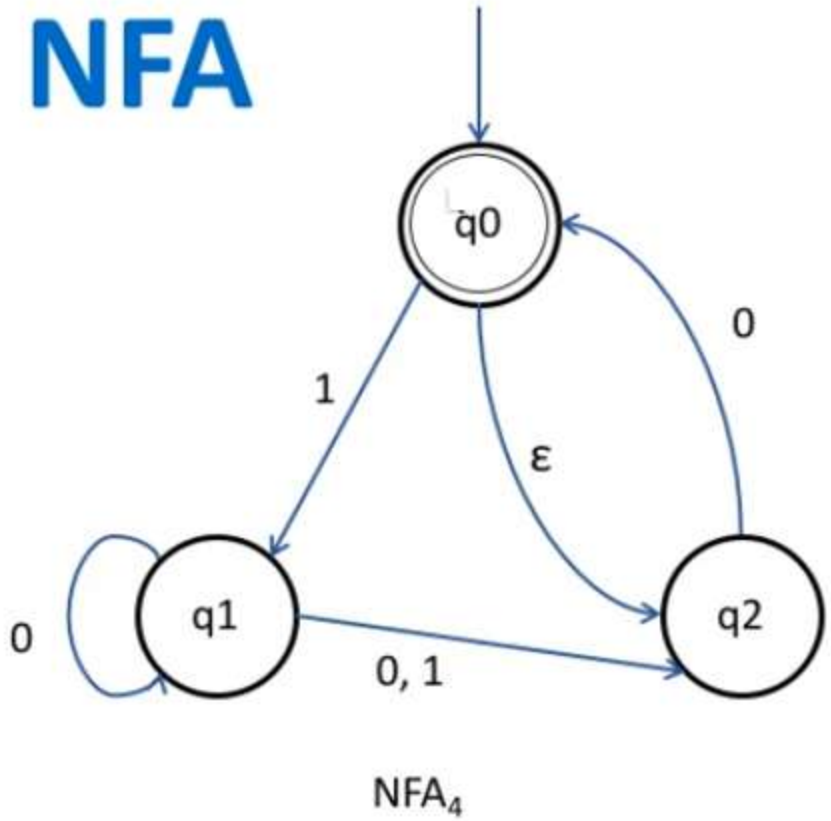
DFA



states	0	1	ε
q0		q1	q2
q1	q1,q2	q2	
q2	q0		

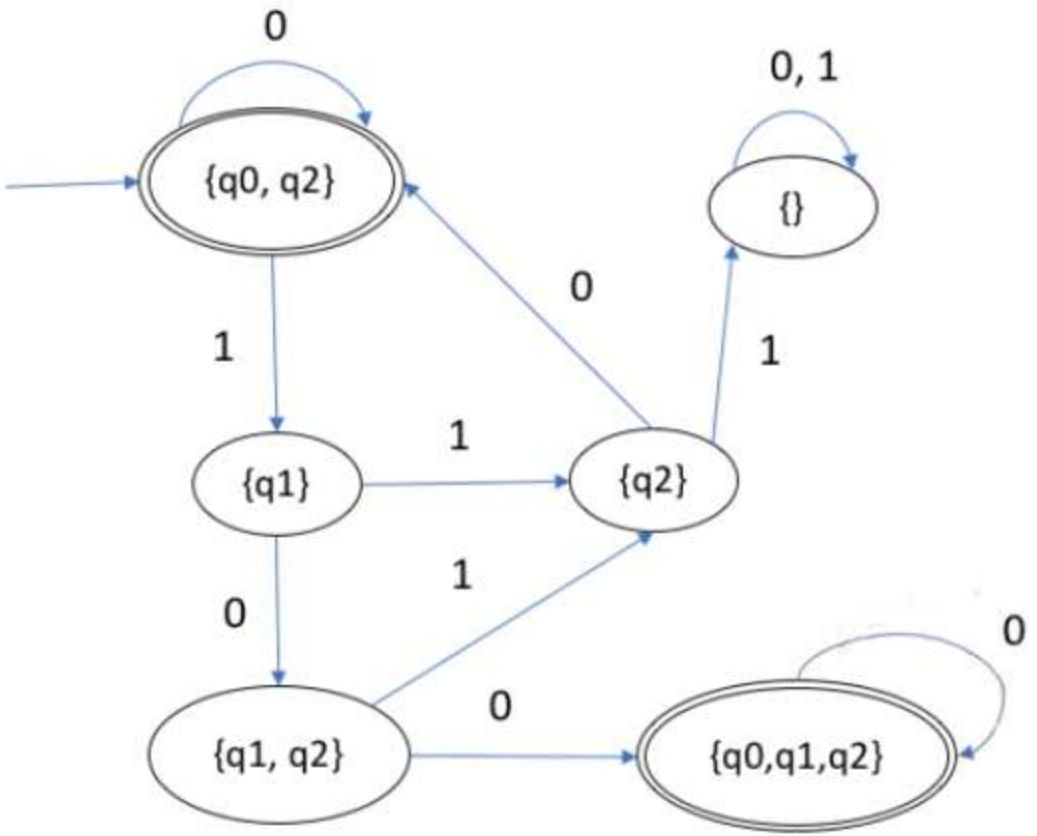
Example 5: convert NFA to DFA

NFA



NFA_4

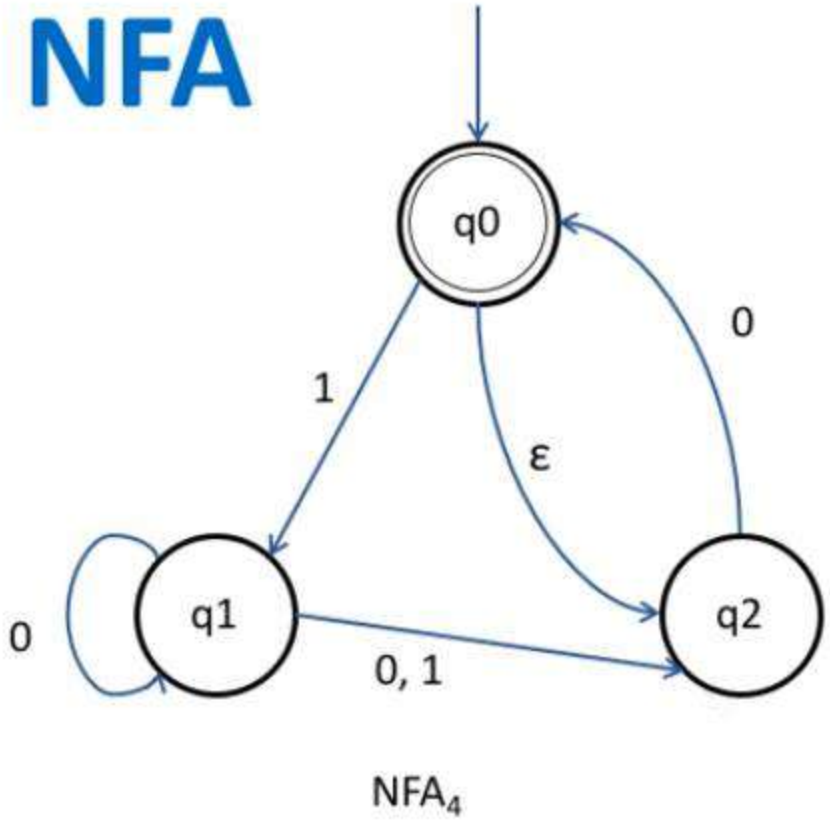
DFA



states	0	1	ε
q0		q1	q2
q1	q1,q2	q2	
q2	q0		

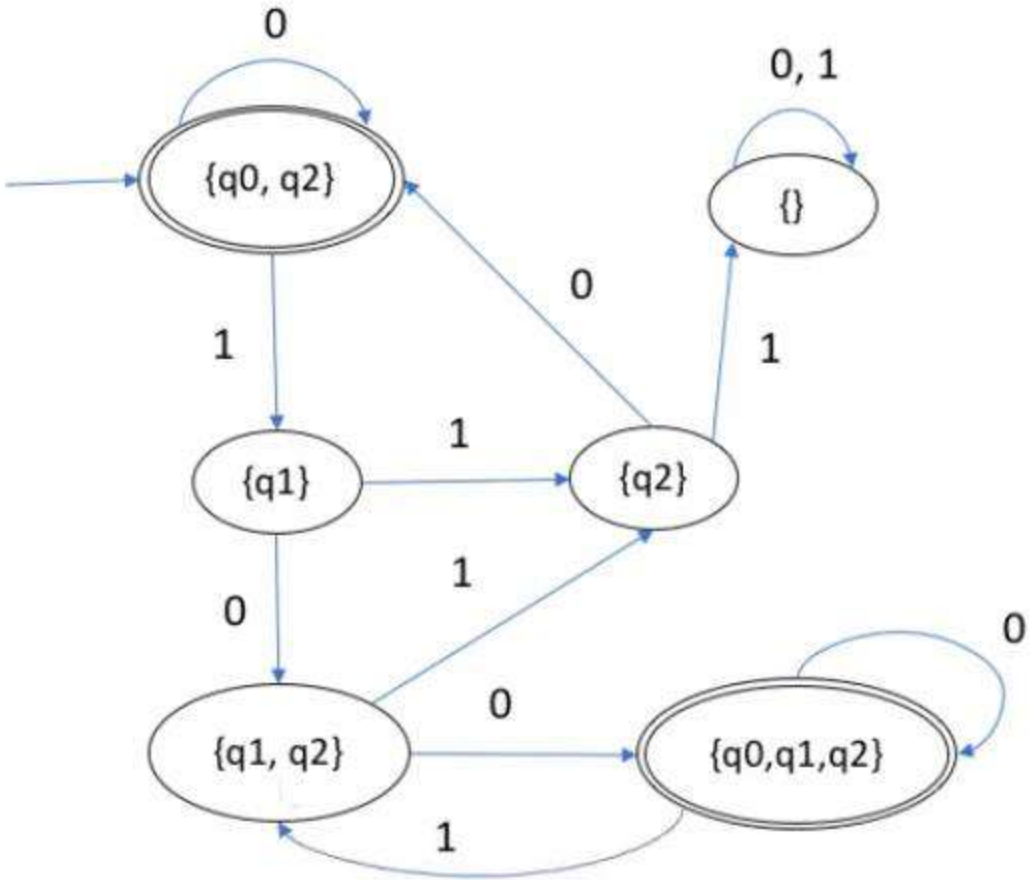
Example 5: convert NFA to DFA

NFA



NFA_4

DFA



THE END

Convert Regular Expressions(Regex) to NFA

Alphabet Σ = {a, b}. regex Σ* = (a U b)*

- Q: Design a regex for a language where:

1- All strings start with *a*:

- example strings: *ab, a, aaa, abbb, aba*
- *regex* = $a\Sigma^*$

2 All strings end with *b*:

- example strings: *b, ab, bb, bab, aaab*
- regex = Σ^*b

3 All strings start with *a* or end with *b*:

- example strings: *ab, a, b, bab, aba*
- regex = $a\Sigma^* \cup \Sigma^*b$

regex Σ* = (a U b)*

- Q: Design a regex for a language where:

4- All strings contain a single *b*:

- example strings: *b, ba, ab, aaab*
- *regex = a* b a**

5 All strings contain at least one *b*:

- example strings: *b, bb, aba, bba*
- regex = *Σ* b Σ**

6 All strings contain substring aab:

- example strings: *aab, aaba, bbaab*
- regex = *Σ* aab Σ**

- Q: Design a regex for a language where: 7- All strings

start and end with *a*:

  - example strings: *a, aa, aba, abaaaba, abbbba*
  - *regex = a Σ* a → Wrong*
  - *regex = a Σ* a U a*

8 All strings start and end with *b*:

  - example strings: *b, bb, bab, bababab, baaaab*
  - *regex = b Σ* b U b*

9 All strings start and end with the same symbol:

  - example strings: *a, b, aba, bab, babab, ababa*
  - regex: *a U b U (a Σ* a) U (b Σ* b)*

- Q: Design a regex for a language where:

10 All strings are of even length (multiple of 2):

- example strings: ε, *aa, bb, ab, ba, aabb, bbaa*
- *regex = Σ Σ = (a U b)(a U b) → Wrong doesn't gives ε*
- *regex = (Σ Σ)* = ((a U b)(a U b))**

11 All strings are of odd length (1, 3, 5, 7, 9, …):

- example strings: *b, a, bab, aaa, bbbaa*
- *regex = (Σ Σ Σ)* = ((a U b)(a U b)(a U b))* →Wrong (gives ε and doesn't gives a or b)*
- *regex = Σ* = (a U b)** → *Wrong (gives ε and if * = 2 will not gives odd number of a and b)*
- *regex = (Σ Σ)* Σ, note: (even + 1) = odd*

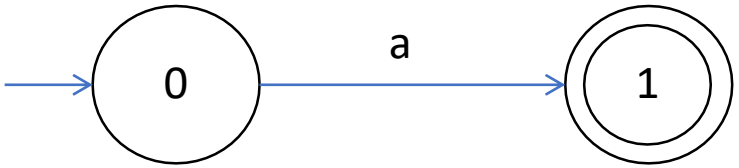
12 All strings are of length multiple of 3:

- example strings: ε, aaa, bbb, aba, bab, aaabbb
- *regex = (Σ Σ Σ)* = ((a U b)(a U b)(a U b))**

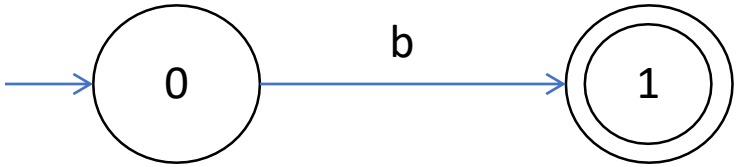
Convert regex to an NFA
Examples

Example 1&2: convert regex to an NFA

- Example 1
- regex = a



- Example 2:
- regex = b



Example 3: convert regex=**ab** to an NFA

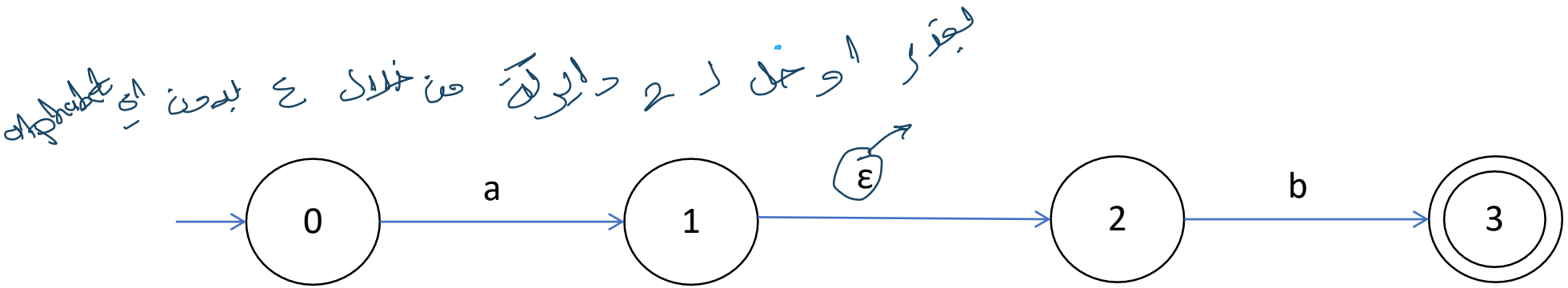
- Example 3:
- regex = ab
  - concatenation

Note:

regex = ab = (a.b)

Example 3: convert regex=**ab** to an NFA

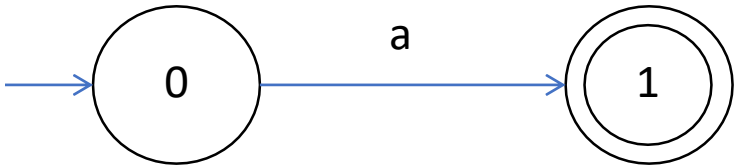
- Example 3:
- regex = ab
  - now we connect join them



Example 4: convert regex=**a*** to an NFA

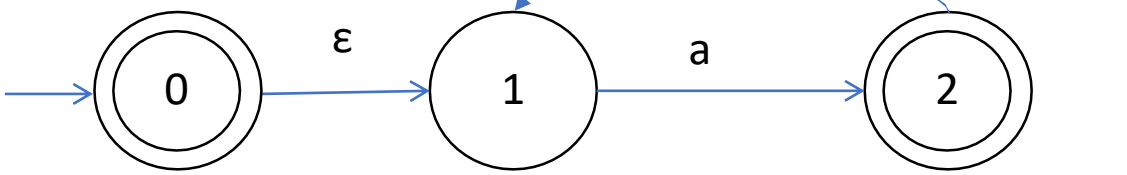
- Example 4:

- regex = a*
  - First, we show NFA for regex = a



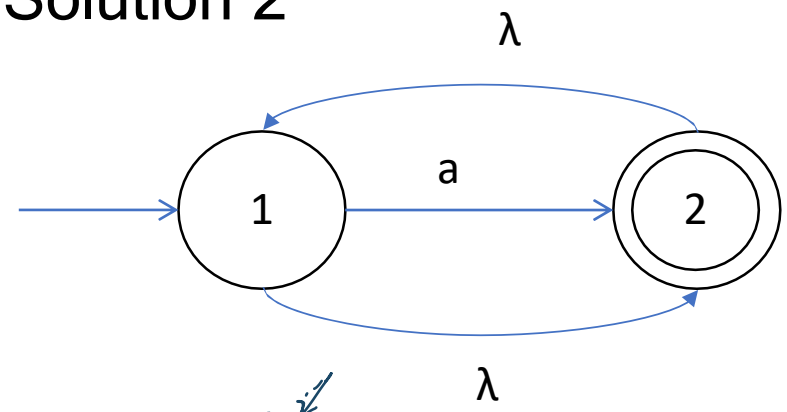
Example 4: convert regex=**a*** to an NFA

- Example 4:



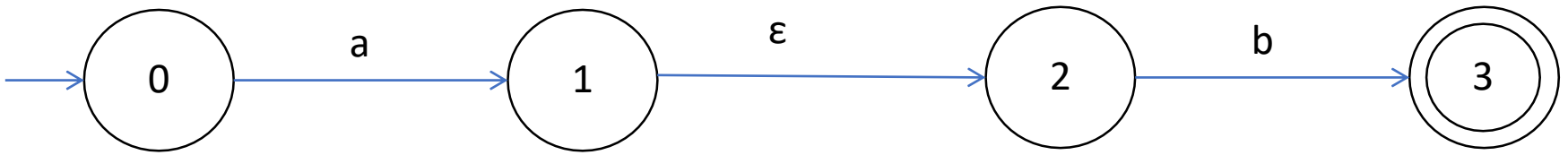
- regex = a*
  - First, we show NFA for regex = a
  - Then, we show NFA for regex = a*

Solution 2



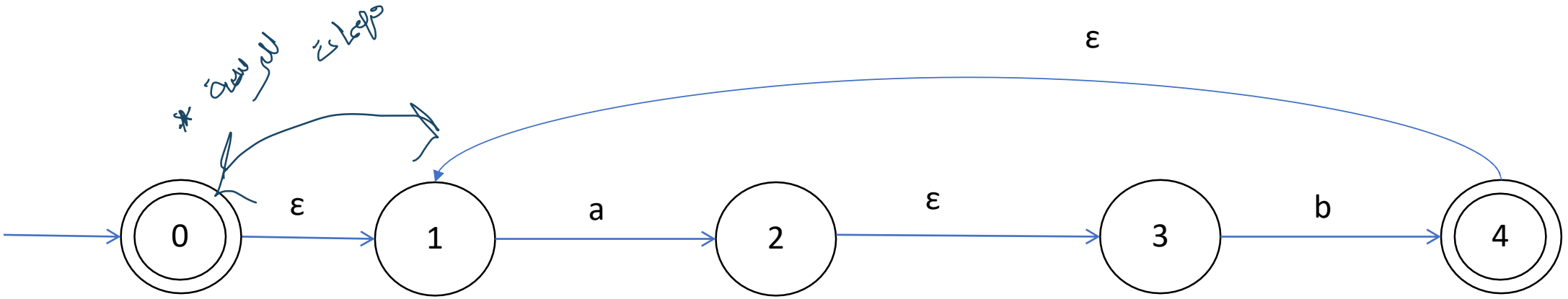
Example 5: convert regex= (ab) * to an NFA

- Example 5:
- regex = (ab)*
  - First, we show NFA for regex = **ab**



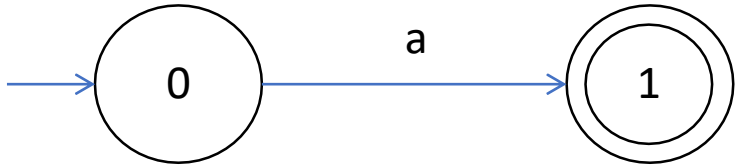
Example 5: convert regex= (ab) * to an NFA

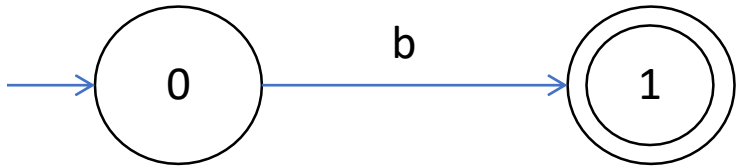
- Example 5:
- regex = (ab)*
  - First, we show NFA for regex = ab
  - Then, we show NFA for regex = (ab)*



Convert regex with union operation

- Example 6:
- regex = `(a U b)`
  - First we have two NFA's:
    - for a, and for b

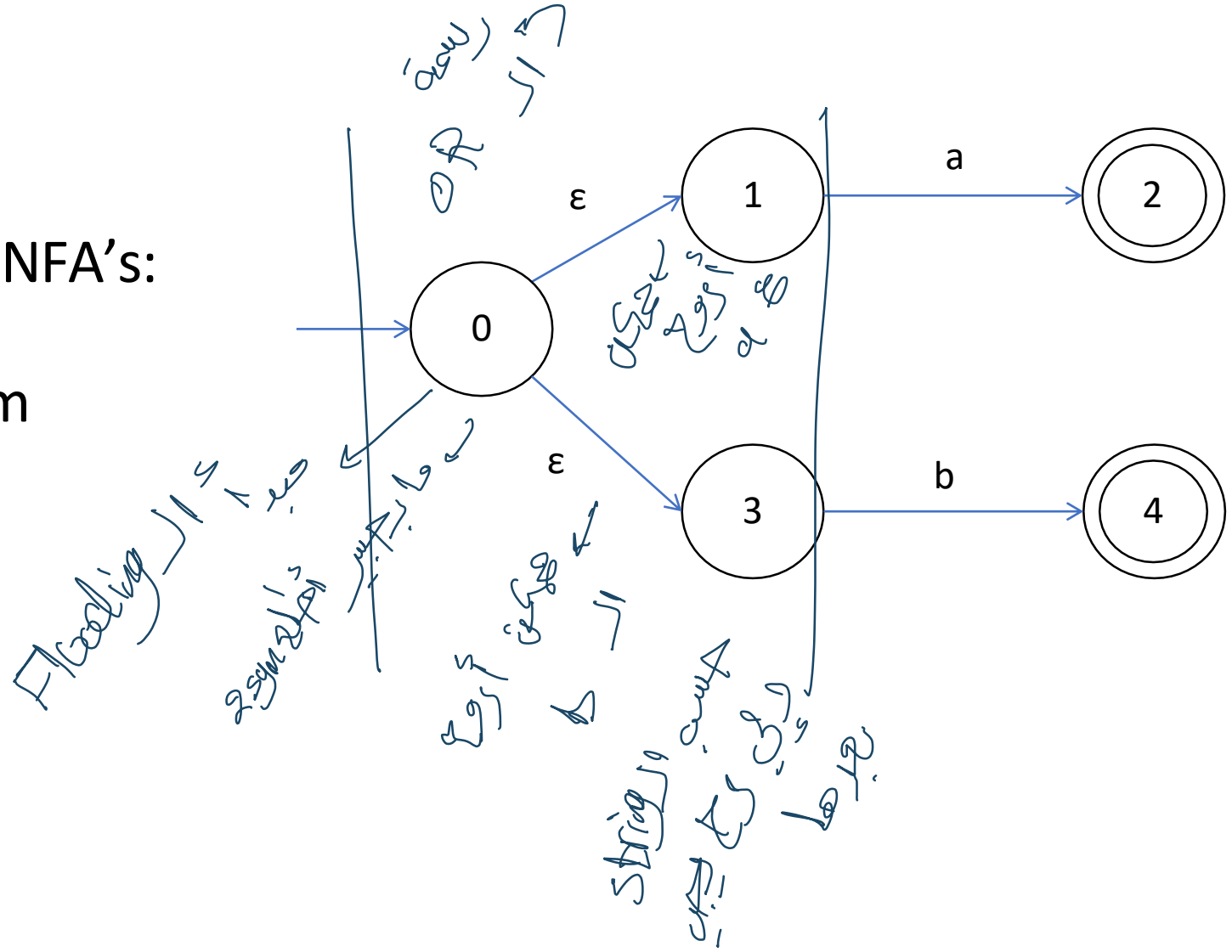




Note: (a Ub) = (a +b)

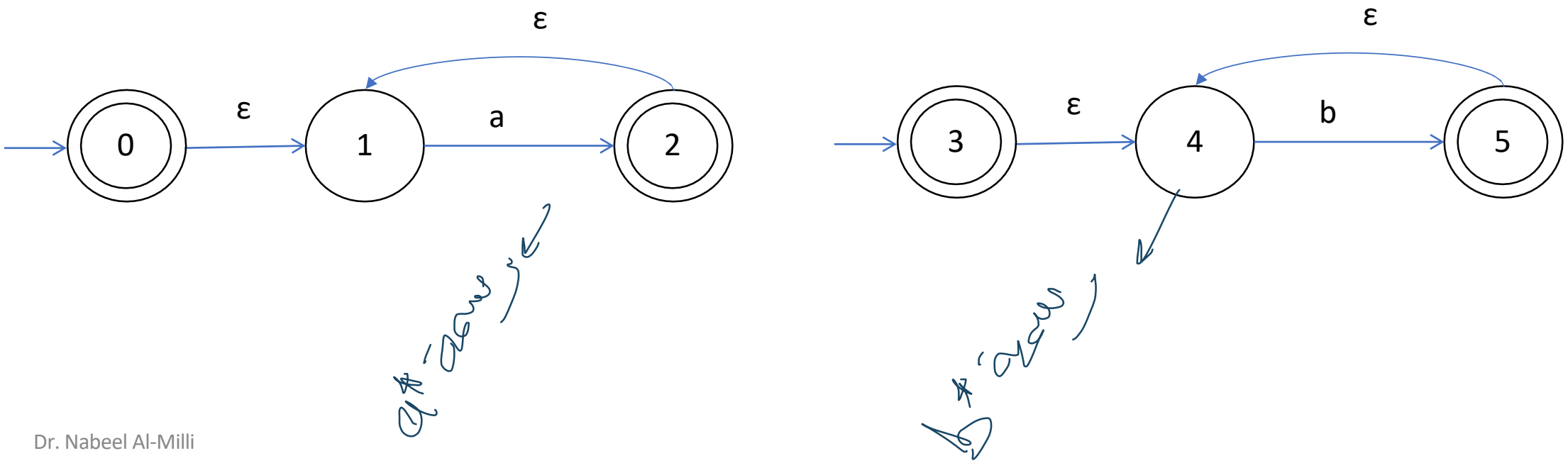
Convert regex with union operation

- Example 6:
- regex = `(a U b)`
  - First we have two NFA's:
    - for a, and for b
  - Then, we join them



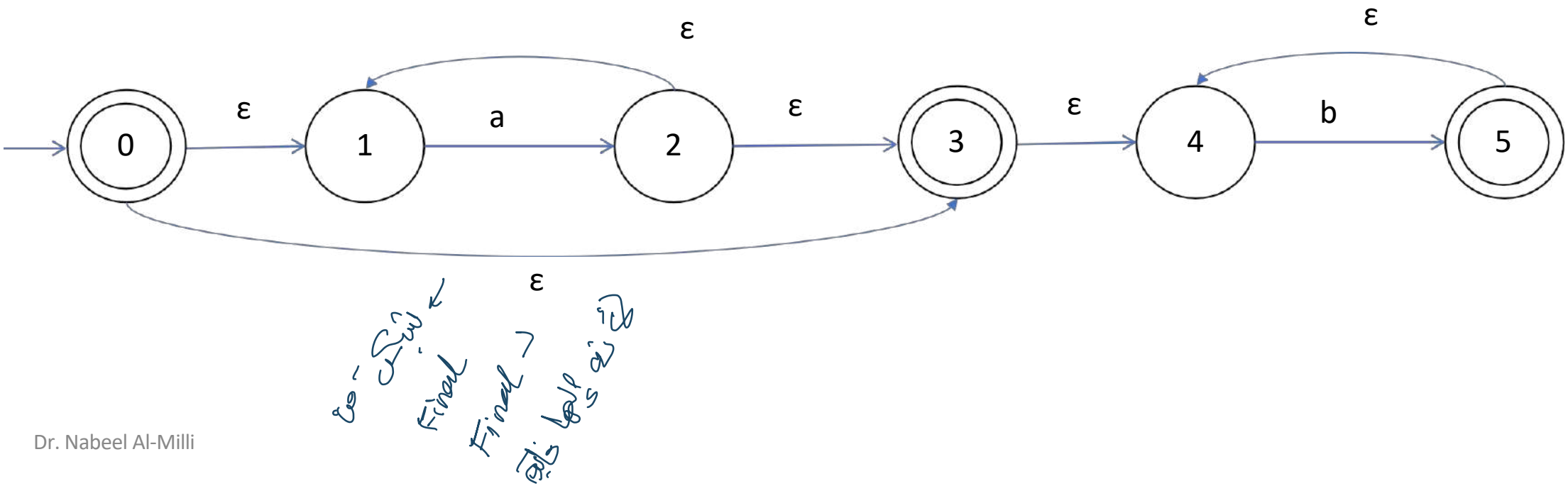
Example 7: convert **a*b*** to an NFA

- Example 7:
- regex = `a*b*`
  - we show NFA's for `a*` and `b*`



Example 7: convert **a*b*** to an NFA

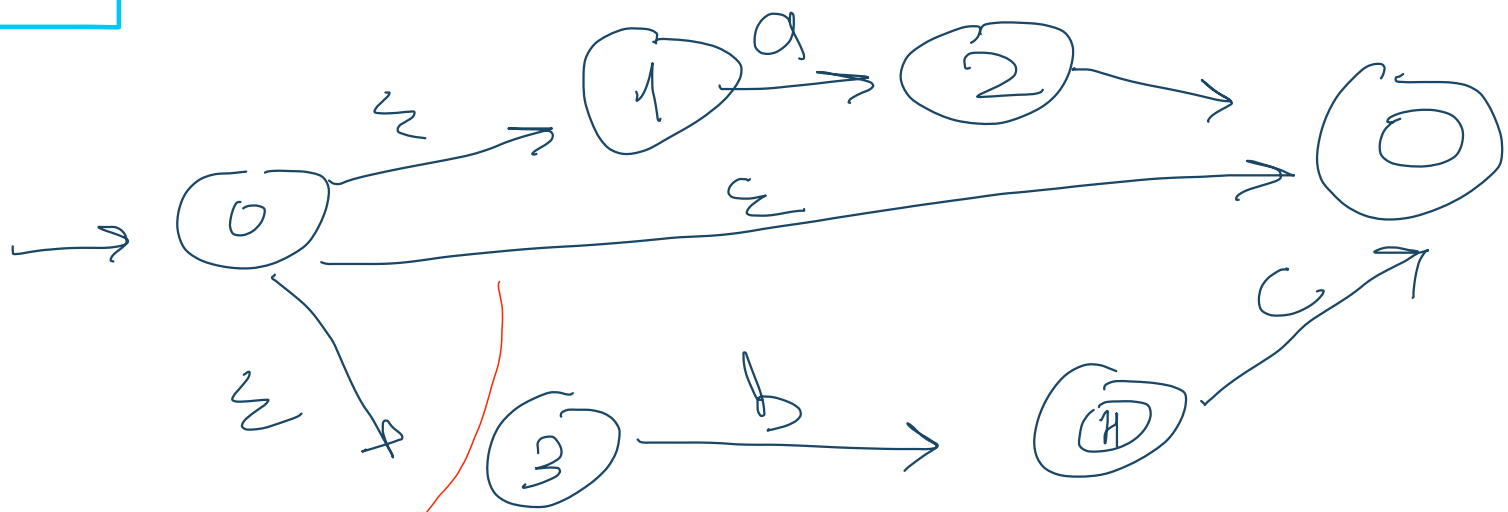
- Example 7:
- regex = `a*b*`
  - we show NFA's for **a*** and **b***
  - **e.g string = aaaaa, string2 = bbbbb**



a*ubc

THE END

$a^* \cup bc$



* عشان في حال عوضت a*
خلص يوصل لل Final وايركت
بدون ab ← لانه عندي ← U
↳ OR